
AI for Drug Discovery & Molecular Design

*A Practical Guide from Sequence to Small Molecules & Biologics
using Python and PyTorch*

ALI AL IBRAHIM

© 2026 Ali Al Ibrahim. All rights reserved.

First Edition, 2026

ISBN: [to be assigned]

No part of this book's text may be reproduced, distributed, or transmitted in any form or by any means, including photocopying, recording, or other electronic or mechanical methods, without the prior written permission of the author, except for brief quotations embodied in critical reviews and certain other noncommercial uses permitted by copyright law.

All source code, scripts, and code listings appearing in this book are licensed under the MIT License unless otherwise noted, and may be used, modified, and distributed freely subject to that license's terms. Third-party models, weights, and code vendored into individual chapters retain their own original licenses and attributions as documented within each chapter.

This book is intended for educational and research purposes. The computational methods, models, and results described herein have not been evaluated by any regulatory authority and are not intended to diagnose, treat, or guide clinical decisions. Readers applying these methods to real-world drug discovery or biosecurity-sensitive work should exercise independent professional judgment.

About the Author

ALI AL IBRAHIM

Ali Al Ibrahim holds a Bachelor's degree in Bioengineering from Bilecik Şeyh Edebali University, where his studies bridge the biological and computational sciences this book brings together. His focus spans technical design, engineering modeling, and scientific research, with a particular interest in how computational methods can accelerate and de-risk early-stage drug discovery.

This book reflects that same disciplined, evidence-first approach: every method described in it is implemented, run, and reported honestly against real data, rather than merely surveyed.

Preface: About This Book

This book is a practical, hands-on guide to applying modern machine learning and deep learning methods across the modern drug discovery pipeline — from representing small molecules and protein sequences computationally, through generative design, structure prediction, docking, and molecular simulation, to two full capstone projects spanning both major drug modalities: small molecules and biologics.

Written for bioengineers, bioinformaticians, computational chemists, pharmaceutical scientists, and AI engineers moving into the life sciences, each of the 18 chapters pairs rigorous theoretical grounding with a real, runnable Python project — built on RDKit, PyTorch, PyTorch Geometric, Hugging Face Transformers, and OpenMM, and applying the same models driving the field today, including AlphaFold2/3, ESMFold, DiffDock, AutoDock Vina, RFdiffusion, and ProteinMPNN.

A guiding principle threads every chapter: every reported number in this book comes from code that was actually run, against real public data, with real limitations disclosed rather than hidden. Where a described method could not be run in this book's own free-tier-compatible environment, that constraint is stated plainly rather than worked around — readers get an honest account of what today's tools can and cannot yet do.

The book is organized into five parts, moving from foundational representations through machine learning for small molecules, protein structure and design, docking and simulation, and finally advanced frontiers including RNA therapeutics, physics-informed and quantum machine learning, and two complete capstone projects that chain the book's own methods into real, end-to-end discovery campaigns.

Contents

Chapter 1: The Drug Discovery Pipeline & AI Revolution	1
1.1 The Traditional Drug Discovery Pipeline	1
1.2 The AI Paradigm Shift	3
1.3 Modalities Overview: Small Molecules vs. Macromolecules/Biologics	5
1.4 Hands-on Project: Retrieving Real Chemical & Structural Data	6
References	7
Chapter 2: Representing Chemical Space (Small Molecules)	9
2.1 Textual Representations: SMILES, SELFIES, and InChI	9
2.2 Molecular Fingerprints and Similarity Metrics	10
2.3 Graph Representations	13
2.4 3D Conformations	13
2.5 Hands-on Project: Molecular Similarity Search & Lipinski Filtering	14
References	17
Chapter 3: Protein Representations & Structure Engineering	19
3.1 Sequences & Amino Acid Properties	19
3.2 3D Structural Anatomy	21
3.3 Machine Learning Representations	23
3.4 Hands-on Project: Structural Feature Extraction & Binding Pocket Geometry	24
References	25
Chapter 4: Core Toolkit & Biological Data Engineering	27
4.1 Advanced RDKit	27
4.2 The DeepChem Framework	29
4.3 BioPython Integration	29
4.4 Biological Databases	30
4.5 Hands-on Project: An Automated ChEMBL ETL Pipeline	32
References	33
Chapter 5: QSAR & ADMET Property Modeling	35
5.1 QSAR Principles	35
5.2 ADMET Profiling	36
5.3 Classical Machine Learning Benchmarks	37
5.4 Handling Bioactivity Bias	38
5.5 Hands-on Project: hERG Cardiotoxicity Classifier	41
References	43
Chapter 6: Graph Neural Networks (GNNs) in Chemistry	45
6.1 Motivation for GNNs	45
6.2 GCN & GAT Architectures	46

6.3 Message Passing Neural Networks (MPNN)	47
6.4 3D Equivariant Networks	48
6.5 Hands-on Project: GNN Molecular Property Prediction	49
References	53
Chapter 7: Generative AI & De Novo Drug Design	55
7.1 Inverse Molecular Design	55
7.2 Latent Space Models	56
7.3 Sequence-based Auto-regressive Models	57
7.4 3D Diffusion Models	57
7.5 Reinforcement Learning (RL)	58
7.6 Hands-on Project: Generative Transformer + RL for De Novo EGFR Inhibitor Design	60
References	63
Chapter 8: Protein Language Models (pLMs) & Evolutionary Dynamics	65
8.1 The Evolutionary Scale Hypothesis	65
8.2 pLM Families	67
8.3 Biological Embeddings	67
8.4 Hands-on Project: Zero-Shot Single-Point Mutation Effect Prediction with ESM-2	69
References	72
Chapter 9: The 3D Structure Prediction Revolution	75
9.1 AlphaFold2 & AlphaFold3 Mechanics	75
9.2 ESMFold & RoseTTAFold	77
9.3 Macromolecular Complexes	78
9.4 Hands-on Project: ESMFold Structure Prediction & pLDDT Validation	79
References	83
Chapter 10: De Novo Protein & Antibody Design	87
10.1 Backbone Generation	87
10.2 Sequence Design (Inverse Folding)	88
10.3 Antibody & Peptide Engineering	89
10.4 Hands-on Project: ProteinMPNN Sequence Design, Validated Against Real Ground Truth	90
References	95
Chapter 11: AI-Driven Molecular Docking	97
11.1 Physics-Based Docking	97
11.2 Deep Learning Docking	99
11.3 Speed vs. Accuracy Trade-offs	101
11.4 Hands-on Project: Real AutoDock Vina Docking Against a Real Oncology Target	102
References	107
Chapter 12: Molecular Dynamics & Neural Network Potentials	109

12.1 MD Fundamentals	109
12.2 GPU Acceleration with OpenMM	111
12.3 Neural Network Potentials (NNPs)	112
12.4 Hands-on Project: Real ANI-2x Molecular Dynamics of the EGFR-Erlotinib Complex	114
References	118
Chapter 13: High-Throughput Virtual Screening (HTVS) Pipelines	121
13.1 Building the HTVS Funnel	121
13.2 Tiered Screening Strategy	122
13.3 Hands-on Project: A Real Tiered HTVS Funnel Against SARS-CoV-2 Main Protease	124
References	130
Chapter 14: AI for RNA Therapeutics & Nucleic Acid Design	133
14.1 RNA Structure Prediction	134
14.2 mRNA & siRNA Optimization	137
14.3 Hands-on Project: Predicting siRNA Knockdown Efficacy from Real RNAi Data and Real RNA Thermodynamics	140
References	143
Chapter 15: Physics-Informed ML & Quantum Machine Learning (QML)	147
15.1 Physics-Informed Neural Networks (PINNs)	148
15.2 Quantum Machine Learning (QML)	152
Limitations and what comes next	154
References	155
Chapter 16: Capstone Project 1 – Small Molecule Oncology Target Discovery	157
16.1 Objective	157
16.2 Pipeline Execution	158
Limitations and what comes next	164
References	164
Chapter 17: Capstone Project 2 – De Novo Neutralizing Antibody Design	167
17.1 Objective	167
17.2 Pipeline Execution	169
Reproducibility	175
Limitations and what comes next	176
A note on Google Colab	176
References	176
Chapter 18: Model Deployment, Reproducibility, Ethics & Regulatory Guidelines	179

18.1 Model Deployment	179
18.2 Reproducibility & FAIR Principles	183
18.3 Regulatory Frameworks	184
18.4 Biosecurity & Ethics	185
Closing	186
References	186

Chapter 1: The Drug Discovery Pipeline & AI Revolution

1.1 The Traditional Drug Discovery Pipeline

Bringing a new medicine to market is a multi-stage filtering process. Each stage exists to kill bad candidates as cheaply as possible, before they reach a stage where failure is expensive. The stages, in order, are broadly:

1. **Target identification & validation** — establishing that modulating a specific biomolecule (usually a protein) plausibly changes the course of a disease, using genetic, biochemical, or clinical evidence.
2. **Hit identification** — screening large compound libraries (physically, via high-throughput screening, or virtually, via computational docking/similarity search) to find molecules that bind the target.
3. **Hit-to-lead & lead optimization** — iteratively modifying hit compounds to improve potency, selectivity, and drug-like properties (solubility, metabolic stability, low toxicity) while tracking structure–activity relationships (SAR).
4. **Preclinical development** — in vitro and animal studies of efficacy, pharmacokinetics (absorption, distribution, metabolism, excretion — ADME), and toxicology, required before human dosing.
5. **Clinical trials** — Phase I (safety, small healthy/patient cohorts), Phase II (efficacy signal, dose-finding), and Phase III (large, randomized, confirmatory efficacy and safety) trials in human subjects.
6. **Regulatory review and approval** — submission of the accumulated evidence (e.g., a New Drug Application to the US FDA) for marketing authorization.

A candidate can fail at any stage, and the cost of failure grows by orders of magnitude the later it happens — a compound that fails in early screening costs a lab reagents and a few days; a compound that fails in Phase III has consumed years and hundreds of millions of dollars enrolling and monitoring patients. This asymmetry is the central economic argument for pushing predictive power as early into the pipeline as possible, which is exactly where the computational and AI methods covered in this book are concentrated.

The economics of attrition: Eroom’s Law

Despite decades of technological improvement — combinatorial chemistry, high-throughput screening, genomics, and now machine learning — the *inflation-adjusted* number of new drugs approved per billion US dollars of R&D spending has been in long-term decline. Scannell et al. (2012) documented that this figure roughly halved every nine years from 1950 to the early 2010s, an approximately 80-fold fall in R&D efficiency over that period, in direct contrast to the exponential improvement in computing power described by Moore’s Law. They named this trend **Eroom’s Law** (“Moore’s Law” spelled backwards) to emphasize the contrast. If $N(t)$ is the number of new drugs approved per billion inflation-adjusted dollars of R&D spend at year t , the observed trend is approximately:

$$N(t) \approx N_0 \cdot 2^{-\frac{t-t_0}{T}}, \quad T \approx 9 \text{ years} \quad (1)$$

where T is the empirical halving period. Scannell et al. attribute this decline to four compounding factors rather than a single cause: the “better than the Beatles” problem (new drugs must beat an ever-improving standard of care, not merely a placebo), increasingly risk-averse regulatory and reimbursement environments, the tendency to increase R&D spending as a response to past failure rather than as a cause of future success, and a “basic research–brute force” bias toward molecular-target reductionism at the expense of systems-level understanding of disease.

The cost and risk of a single approved drug

Two numbers define the economics of drug development: how much it costs, and how likely a candidate is to survive the pipeline at all.

Wouters, McKee, and Luyten (2020) estimated development costs using publicly disclosed data (SEC filings, FDA approval records, and ClinicalTrials.gov) for 63 of the 355 new therapeutic agents the FDA approved between 2009 and 2018. After capitalizing costs at a real cost of capital of 10.5% per year to account for the time value of money and the cost of capital tied up in failed programs, they estimated a **median** R&D investment of \$985.3 million (95% CI: \$683.6M–\$1228.9M) and a **mean** of \$1,335.9 million (95% CI: \$1,042.5M–\$1,637.5M) per approved drug, in 2018 US dollars — figures that varied enormously by therapeutic area, from a median of \$765.9M for nervous-system drugs to \$2,771.6M for oncology and immunomodulatory drugs. (A 2022 erratum to this paper corrected errors found in the underlying source data; the corrected figures above are the ones reported in that correction. See [References](#).)

The reason these figures are so large is that they are *risk-adjusted*: the direct, out-of-pocket cost of running one clinical trial is much smaller than the effective cost of getting one drug approved, because most candidates that enter clinical trials never make it out. A standard way to express this is to capitalize the cost of each phase forward to the launch date and divide by the cumulative probability of a candidate that has reached the start of that phase eventually being approved:

$$E[\text{Cost}_{\text{launch}}] = \sum_{i=1}^n \frac{C_i}{\text{POS}_i} \cdot (1 + r)^{T_{\text{launch}} - t_i} \quad (2)$$

where C_i is the direct cost of phase i , POS_i is the probability that a candidate entering phase i is ultimately approved, r is the real cost of capital, and t_i is the phase's timing relative to launch at T_{launch} . This is the general shape of the risk-adjustment framework used throughout the drug-development-cost literature (it is a simplified pedagogical form, not a specific paper's exact computation); the key takeaway is that POS_i — the attrition rate — dominates the equation, because it appears in the denominator of every term.

Attrition itself is substantial and disease-dependent. Using a sample of over 406,000 clinical trial records for more than 21,000 compounds run between 2000 and 2015, Wong, Siah, and Lo (2019) found that oncology drugs had only a 3.4% end-to-end clinical success rate in their sample — notably lower than the 5.1% figure widely cited from earlier, smaller industry-curated datasets — though this rate has been highly variable year to year (it fell to 1.7% in 2012 before rising to 8.3% in 2015). Hay et al. (2014) independently surveyed clinical success rates across the broader industry using one of the largest datasets assembled at the time. Both studies agree on the qualitative picture even where the exact percentages differ: **the overwhelming majority of clinical candidates fail**, and failure is concentrated in efficacy (the drug doesn't work well enough) far more than in safety.

This is the economic backdrop against which every method in this book should be judged. A computational technique earns its place in the pipeline only if it plausibly increases POS_i at some stage, or reduces C_i , or both — not because it is fashionable.

1.2 The AI Paradigm Shift

Classical computer-aided drug design relies on hand-engineered descriptors and rule-based heuristics (Chapter 2 uses one of the oldest and still most useful examples, Lipinski's Rule of Five). Over the last decade, machine learning has been layered on top of — and in some cases has begun to replace — this classical toolkit. Vamathevan et al. (2019), in a widely cited review, characterize opportunities for ML across every stage of the pipeline described in §1.1: target identification and validation, prognostic biomarker discovery, and the analysis of high-dimensional data such as digital pathology images in clinical trials. Their central caveat is as important as their central claim: ML performs best on “well-

specified questions with abundant, high-quality data,” and its adoption is currently limited more by interpretability and repeatability of results than by raw predictive accuracy.

It is useful to organize the computational methods used throughout this book into three broad, overlapping families:

- **Predictive models** answer a question about a specific molecule or protein: will this compound bind this target (Chapters 5–6)? Is this molecule likely to be toxic (Chapter 5)? What is this protein’s 3D structure (Chapter 9)? These are typically supervised learning problems — classification or regression on a labeled dataset — and their scientific value is bounded by the quality and size of that dataset, which is precisely why chemical and structural databases like ChEMBL and the PDB (introduced in §1.4) are foundational infrastructure for the entire field.
- **Generative models** propose new molecules or sequences rather than scoring existing ones: variational autoencoders and diffusion models over molecular graphs or 3D coordinates (Chapter 7), autoregressive transformers over SMILES/SELFIES strings (Chapter 7), and diffusion models over protein backbones (Chapter 10). These models shift the bottleneck in early discovery from *screening* a fixed library to *designing* candidates directly in the region of chemical or sequence space likely to satisfy a set of constraints.
- **Physics-informed models** incorporate known physical or chemical constraints — conservation laws, symmetries, or explicit energy functions — directly into the model architecture or training objective, rather than relying on a purely data-driven fit. Equivariant graph neural networks that respect 3D rotational and translational symmetry (used in Chapter 6 and Chapter 9) and neural network potentials that approximate quantum-mechanical energies at classical-simulation speed (Chapter 12) are both examples: the model is constrained to only ever produce physically consistent outputs, which improves data efficiency and generalization compared with an unconstrained model trained on the same data.

The clearest illustration of what this paradigm shift can achieve is structure prediction. Determining a protein’s 3D structure experimentally (by X-ray crystallography, NMR, or cryo-EM) can take months to years per structure and does not always succeed. Jumper et al. (2021) showed that AlphaFold, a deep learning system, could predict protein structures from amino acid sequence alone at accuracy competitive with experimental methods for a large fraction of targets in the CASP14 blind assessment — work that has since made high-confidence structural models available for essentially the entire sequenced proteome, at a computational cost orders of magnitude below experimental determination. Chapter 9 covers AlphaFold and related methods (ESMFold, RoseTTAFold) in depth; it is introduced here because it is the single clearest data point for the claim that this book is built on: that the AI paradigm shift is not merely accelerating the classical pipeline, it is changing which questions are computationally tractable at all.

None of this makes the classical pipeline in §1.1 obsolete. Every predictive, generative, and physics-informed model in this book is trained on data that ultimately comes from wet-lab

experiments — assays run against real targets, structures solved by real crystallographers, outcomes from real clinical trials. AI does not remove the pipeline; it changes where effort within the pipeline is spent, ideally shifting attrition as early and as cheaply as possible, per the economic argument in §1.1.

1.3 Modalities Overview: Small Molecules vs. Macromolecules/Biologics

Everything described so far applies differently depending on what kind of molecule is being designed. The book covers two broad modalities, which differ enough in their representation, design constraints, and manufacturing that they are treated with largely separate computational toolkits in Parts I–II (small molecules) and Part III (macromolecules).

Small molecules are low-molecular-weight organic compounds, synthesized chemically rather than produced biologically, and — for orally dosed drugs — usually expected to satisfy heuristics like Lipinski’s Rule of Five: no more than 5 hydrogen-bond donors, no more than 10 hydrogen-bond acceptors, a molecular weight under 500 Da, and a calculated LogP under 5, on the premise that violating more than one of these substantially reduces the probability of good oral absorption and permeability (Lipinski et al., 2001). Chapter 2’s hands-on project computes exactly these four properties with RDKit. Small molecules are represented computationally as 2D graphs, 1D strings (SMILES, SELFIES, InChI — Chapter 2), fingerprints (Chapter 2), or 3D conformers (Chapter 2), and their chemical space is explored using the fingerprint similarity, generative, and docking methods in Chapters 2, 7, and 11.

Macromolecules / biologics — therapeutic proteins, peptides, and antibodies — are produced biologically (typically via recombinant expression in engineered cells) rather than synthesized chemically, are orders of magnitude larger (a typical IgG monoclonal antibody is roughly 150 kDa, versus a few hundred Da for a small molecule), and cannot generally be dosed orally because they are degraded by the digestive system, so they are usually delivered by injection or infusion. Their size and complexity give them properties small molecules structurally cannot match — very high target specificity and affinity, engineerable multi-domain architectures (e.g., bispecific antibodies) — at the cost of manufacturing complexity, immunogenicity risk, and a much larger, higher-dimensional design space (a single antibody’s antigen-binding region alone involves optimizing complementarity-determining region loops built from 20 possible amino acids at each position). Part III of this book (Chapters 8–10) covers the representations (sequences, contact maps, 3D structural graphs) and generative methods (inverse folding, backbone diffusion) specific to this modality.

The choice of modality for a given disease target is itself a design decision with no universally correct answer: small molecules are cheaper to manufacture and can reach intracellular targets that biologics generally cannot, while biologics can achieve selectivity

against targets (such as flat protein–protein interaction surfaces) that are notoriously difficult to drug with small molecules. Recognizing which modality is appropriate for a given target — and which computational toolchain in this book therefore applies — is one of the first judgment calls in any real drug discovery project, which is why this book treats it as the subject of its very first hands-on exercise, below.

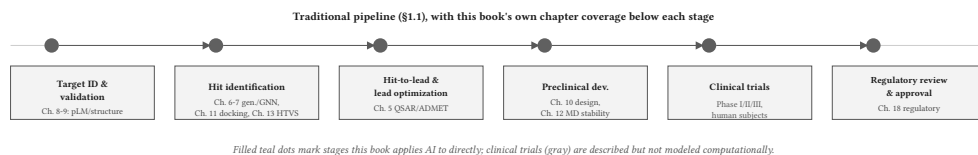


Figure 1.1. The traditional six-stage drug discovery pipeline, with the chapters in this book that apply AI to each stage.

1.4 Hands-on Project: Retrieving Real Chemical & Structural Data

The rest of this book works with data pulled from two public infrastructure databases that anchor the whole field:

- **ChEMBL** (Mendez et al., 2019), maintained by EMBL-EBI, is a curated, open-access database of bioactivity data — which compounds were tested against which biological targets, and what activity (e.g., IC₅₀, Ki) was measured. As of release ChEMBL_37 (May 2026), it holds bioactivity data for roughly 2.9 million distinct compounds against over 18,000 targets, drawn from more than 100,000 publications.
- **The Protein Data Bank (PDB)** (wwPDB consortium, 2019) is the single global archive of experimentally determined 3D structures of proteins, nucleic acids, and their complexes. As of August 2026 it holds 258,403 released structures.

The hands-on project for this chapter does not analyze this data yet — that begins in Chapter 2 (chemical similarity) and Chapter 9 (structure prediction) — it establishes the environment and the *programmatic* retrieval pattern that the rest of the book builds on: querying both databases' public REST APIs directly from Python, rather than manually downloading files through a web browser. This is a deliberate choice: a reproducible computational pipeline should be able to re-fetch its own inputs from source.

The project code lives in this chapter's folder (`ch01_drug_discovery_pipeline/`) and targets a single, concrete example throughout: **EGFR** (epidermal growth factor receptor, ChEMBL target CHEMBL203), a receptor tyrosine kinase that is one of the most extensively studied oncology drug targets and the target used in this book's Chapter 16 capstone project. It retrieves:

1. **Target metadata and bioactivity data from ChEMBL** — confirming the target identity (preferred name, organism, target type) and pulling a page of measured bioactivities (assay type, measured value, units, and the tested compound's SMILES string) for EGFR.
2. **Structural metadata and a 3D structure file from the PDB** — for PDB entry 1M17, the crystal structure of the EGFR tyrosine kinase domain bound to the inhibitor erlotinib, retrieving both the entry's metadata (title, resolution, release date) and the structure file itself.

See [README.md](#) in this folder for setup and usage instructions, and [fetch_data.py](#) for the implementation. The accompanying test suite (`tests/test_fetch_data.py`) exercises the same functions against the live APIs, so running the tests is itself a reproducibility check on the retrieval pipeline.

A note on Google Colab

Every dependency this project needs (`requests`, part of a standard Python installation's ecosystem) is preinstalled on Google Colab's default runtime, so the entire project runs unmodified in a fresh Colab notebook — install nothing, add a cell with `!git clone` or paste `fetch_data.py`'s contents, and run. No GPU or special runtime is required for this chapter's project; later chapters that require a GPU runtime will say so explicitly.

References

- Scannell, J. W., Blanckley, A., Boldon, H., & Warrington, B. (2012). Diagnosing the decline in pharmaceutical R&D efficiency. *Nature Reviews Drug Discovery*, 11(3), 191–200. <https://doi.org/10.1038/nrd3681>
- Wouters, O. J., McKee, M., & Luyten, J. (2020). Estimated Research and Development Investment Needed to Bring a New Medicine to Market, 2009-2018. *JAMA*, 323(9), 844–853. <https://doi.org/10.1001/jama.2020.1166> (Erratum: Wouters, O. J., McKee, M., & Luyten, J. (2022). Errors in Source Data for Study of Drug Development Costs. *JAMA*, 328(11), 1110. <https://doi.org/10.1001/jama.2022.14317>)
- Wong, C. H., Siah, K. W., & Lo, A. W. (2019). Estimation of clinical trial success rates and related parameters. *Biostatistics*, 20(2), 273–286. <https://doi.org/10.1093/biostatistics/kxx069> (Corrigendum: *Biostatistics*, 20(2), 366. <https://doi.org/10.1093/biostatistics/kxy072>)
- Hay, M., Thomas, D. W., Craighead, J. L., Economides, C., & Rosenthal, J. (2014). Clinical development success rates for investigational drugs. *Nature Biotechnology*, 32(1), 40–51. <https://doi.org/10.1038/nbt.2786>
- Vamathevan, J., Clark, D., Czodrowski, P., Dunham, I., Ferran, E., Lee, G., Li, B., Madabhushi, A., Shah, P., Spitzer, M., & Zhao, S. (2019). Applications of machine learning in drug discovery and development. *Nature Reviews Drug Discovery*, 18(6), 463–477. <https://doi.org/10.1038/s41573-019-0024-5>
- Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., et al. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873), 583–589. <https://doi.org/10.1038/s41586-021-03819-2>

- Mendez, D., Gaulton, A., Bento, A. P., Chambers, J., De Veij, M., Félix, E., et al. (2019). ChEMBL: towards direct deposition of bioassay data. *Nucleic Acids Research*, 47(D1), D930–D940. <https://doi.org/10.1093/nar/gky1075>
- wwPDB consortium. (2019). Protein Data Bank: the single global archive for 3D macromolecular structure data. *Nucleic Acids Research*, 47(D1), D520–D528. <https://doi.org/10.1093/nar/gky949>
- Lipinski, C. A., Lombardo, F., Dominy, B. W., & Feeney, P. J. (2001). Experimental and computational approaches to estimate solubility and permeability in drug discovery and development settings. *Advanced Drug Delivery Reviews*, 46(1-3), 3–26. [https://doi.org/10.1016/S0169-409X\(00\)00129-0](https://doi.org/10.1016/S0169-409X(00)00129-0)

Live data cited in §1.4 (ChEMBL release version and count, PDB entry count) were retrieved directly from the ChEMBL and RCSB PDB REST APIs on 2026-08-16; see `fetch_data.py` to reproduce.

Chapter 2: Representing Chemical Space (Small Molecules)

Chapter 1 introduced Lipinski’s Rule of Five and small-molecule representations only in passing, while surveying the two modalities this book covers. This chapter makes those representations precise. Much of what follows for small molecules — QSAR/ADMET models (Chapter 5), graph neural networks (Chapter 6), generative models over molecular strings and graphs (Chapter 7), and later, molecular docking (Chapter 11) — starts from one of the representations introduced here, so the central question this chapter answers is not just *how* to represent a molecule computationally, but how that choice of representation determines what “similar” can even mean.

2.1 Textual Representations: SMILES, SELFIES, and InChI

The simplest computational representation of a small molecule is a linear string. **SMILES** (Simplified Molecular Input Line Entry System; Weininger, 1988) encodes a molecular graph as a depth-first traversal: atoms are written as their element symbols (aromatic atoms in lowercase), bonds are implicit (single/aromatic) or explicit (= double, # triple), branches are parenthesized, and ring closures are marked with matching digits at the two atoms that close the ring. Aspirin, for example, is:

```
CC(=O)OC1=CC=CC=C1C(=O)O
```

reading as: a methyl carbon, a carbonyl carbon ($(=O)$), an ester oxygen, then a benzene ring opened and closed with the digit 1, with a carboxylic acid group closing the string. RDKit's `Chem.MolFromSmiles` parses exactly this grammar into a molecule object — the same object the rest of this chapter's code operates on.

Two properties of SMILES matter for everything that follows. First, it is **not unique by default**: a depth-first traversal can start at any atom and branch in any order, so the same molecule has many valid SMILES strings, all of which parse to the same graph. Toolkits like RDKit resolve this by producing a single *canonical* SMILES per molecule, using a deterministic atom-ranking algorithm, so that string equality can be used as a fast (if fingerprint-agnostic) exact-match test. Second, and more consequentially for this chapter, SMILES syntax does not track chemical *similarity* — a single-character edit near a ring-closure digit can produce a wildly different molecule or an invalid string entirely, so string-edit-distance between two SMILES is not a meaningful similarity metric. This motivates §2.2's fingerprint-based approach directly: similarity has to be computed from *chemical structure*, not from surface-level string syntax.

SMILES' non-uniqueness and syntactic fragility are also a practical liability for the generative models covered in Chapter 7: a character-level model sampling SMILES strings can easily produce syntactically invalid output (unbalanced rings, valence violations). **SELFIES** (Self-referencing Embedded Strings; Krenn et al., 2020) was designed specifically to close this gap — its grammar guarantees that *every* string it can produce decodes to a chemically valid molecule, at the cost of being less directly human-readable than SMILES. This book uses SMILES throughout Parts I–II for its readability and RDKit-native tooling, but Chapter 7 returns to SELFIES specifically where 100% validity matters for generation.

A third representation, the **InChI** (IUPAC International Chemical Identifier; Heller et al., 2015), solves a different problem: it is a non-proprietary, algorithmically-derived, canonical identifier layered with explicit sub-layers for formula, connectivity, tautomeric state, charge, and stereochemistry. Unlike a toolkit-specific canonical SMILES, an InChI (and its hashed, fixed-length InChIKey) is reproducible across independent implementations, which is exactly why databases such as ChEMBL and PubChem (Chapter 4) use it to deduplicate and cross-reference compounds submitted from different sources.

None of these three representations is strictly “better” — they solve different problems. SMILES is compact and directly parseable into a molecular graph; SELFIES trades some readability for guaranteed validity in generative settings; InChI trades human-writability for cross-database, implementation-independent identity. What none of them gives you directly is a numeric handle for comparing two *different* molecules — that is the job of the fingerprints introduced next.

2.2 Molecular Fingerprints and Similarity Metrics

From structure to bit vector

A **molecular fingerprint** encodes structural information as a fixed-length vector, so that two molecules can be compared with ordinary vector arithmetic instead of graph isomorphism. Fingerprints fall into two broad families.

Structural-key fingerprints fix a dictionary of specific substructures in advance and set one bit per dictionary entry if that substructure is present. The MACCS keys (Durant et al., 2002) are the canonical example: a 166-bit vector, each bit corresponding to a named, human-inspectable substructural pattern (e.g., “has an aromatic ring,” “has a carbonyl”). Their strength is direct interpretability — a set bit has a fixed, inspectable meaning; their limitation is that vocabulary is fixed, so structural motifs outside the predefined dictionary are invisible to the fingerprint no matter how chemically relevant they are.

Circular (Morgan/ECFP) fingerprints take the opposite approach: instead of a fixed dictionary, they enumerate the actual substructural environments present in the molecule, algorithmically. The underlying algorithm is Morgan’s (1965) method for canonical atom relabeling, originally designed to test structures for graph isomorphism at Chemical Abstracts Service: each atom is assigned an initial integer invariant (derived from its atomic number, degree, charge, attached hydrogen count, and ring membership), and then, over successive iterations, each atom’s invariant is recomputed as a hash of its own current invariant together with its neighbors’ invariants. Rogers and Hahn (2010) adapted this iterative relabeling procedure into a similarity fingerprint — **Extended-Connectivity Fingerprints (ECFP)** — by recording every atom-environment identifier produced at every iteration, rather than discarding intermediate values in pursuit of a single canonical numbering.

The number of iterations sets the fingerprint’s **radius**: after r iterations, an atom’s identifier summarizes its bonded environment out to r bonds away. ECFP4 — the fingerprint this chapter’s hands-on project uses — takes its name from a *diameter* of 4 (radius $r = 2$), so it captures each atom’s 2-bond neighborhood; ECFP6 ($r = 3$) captures larger, more global neighborhoods at the cost of specificity. Smaller radii favor recognizing shared local motifs (e.g., a common functional group) even in otherwise dissimilar molecules; larger radii demand more global structural agreement.

Because the set of distinct atom-environment identifiers a large, diverse compound library can produce is unbounded, ECFP identifiers are **hashed** into a fixed-length bit vector — 2048 bits is a common default, and the one this chapter’s code uses (`fpSize=2048` in `similarity_search.py`). Hashing into a fixed width necessarily introduces **collisions**: two chemically distinct environments can hash to the same bit, especially as more distinct environments compete for a fixed number of bits. This is a direct precision/memory trade-off — a wider bit vector reduces collision rate at the cost of memory and comparison speed — and it is also why hashed fingerprints are less directly interpretable than MACCS keys: a given set bit does not, by itself, name a specific substructure, though RDKit can recover the

contributing environments for a specific molecule via its bit-info API when that provenance is needed.

The similarity–property principle, and its limits

It is essential to be precise about what fingerprint similarity does and does not claim. Two molecules with a high fingerprint similarity share many local atomic environments — nothing more. Similarity is not chemical equivalence: a Tanimoto similarity of exactly 1.0 between two *different* molecules is possible in principle (fingerprint hashing is lossy, so distinct structures can, rarely, collide to identical bit vectors), and conversely two molecules that are one atom apart can, if that atom sits at a pharmacologically critical position, have wildly different biological activity despite near-identical fingerprints — a phenomenon known in the field as an “activity cliff.” The working assumption that justifies similarity search at all — that structurally similar molecules tend to share biological activity — is called the **similarity–property principle** (Johnson & Maggiora, 1990). It is a real, useful statistical tendency, not a guarantee: Martin, Kofron, and Traphagen (2002), analyzing IC₅₀ follow-up data from 115 high-throughput screening assays, found that a compound with Daylight-fingerprint Tanimoto similarity ≥ 0.85 to a known active had only about a 30% chance of itself being active — better than chance, and better than random library selection, but a long way from certainty. Every use of similarity search in this chapter, and in the drug-discovery applications discussed in §2.5, should be read with that number in mind.

Tanimoto similarity

Given two binary fingerprints A and B of equal length, let a be the number of bits set in A , b the number of bits set in B , and c the number of bits set in *both*. The **Tanimoto coefficient** (equivalent, for binary vectors, to the Jaccard index) is defined as:

$$T(A, B) = \frac{c}{a + b - c} = \frac{|A \cap B|}{|A \cup B|} \quad (3)$$

$T(A, B) \in [0, 1]$: $T = 1$ means the two fingerprints are identical (every set bit shared), $T = 0$ means they share no set bits at all. This is exactly `rdkit.DataStructs.TanimotoSimilarity`, called in `similarity_search.py`’s `rank_by_similarity` function on every pair of `GetMorganGenerator(radius=2, fpSize=2048)` fingerprints. Bajusz, Rácz, and Héberger (2015) survey why the Tanimoto coefficient in particular — as opposed to alternatives like the Dice or Cosine coefficients, which weight shared bits differently — has become the de facto standard for binary fingerprint comparison in cheminformatics: it is simple, satisfies the mathematical properties of a proper distance metric ($1 - T$ is a true metric), and its behavior with respect to fingerprint density and molecule size is well characterized empirically.

In practice, similarity search requires choosing a **threshold** above which two molecules are treated as “similar enough” to act on — Martin et al.’s ≥ 0.85 figure above is one commonly cited reference point for ECFP-family fingerprints, but the appropriate threshold is target-, library-, and fingerprint-dependent, not a universal constant; Willett, Barnard, and Downs

(1998), in their foundational review of similarity searching, make the same point about threshold sensitivity more generally.

Computing Tanimoto similarity for one query against a library of N compounds is $O(N)$ bitwise-AND/popcount operations — trivial for this chapter’s 17-molecule demo library, and still fast (fixed-width bitwise operations are cheap) against ChEMBL’s roughly 2.9 million compounds (Chapter 1, §1.4) for a single query. Where similarity search becomes computationally interesting is *many-query* or *all-pairs* search against million-compound libraries, which is where production systems move from brute-force scanning to indexing structures (e.g., inverted indices over set fingerprint bits, or approximate nearest-neighbor structures) that prune the candidate set before computing exact Tanimoto scores. This chapter’s hands-on project uses brute-force search deliberately — at demo scale, the indexing overhead is not worth paying, and the brute-force version keeps the code’s connection to §2.2’s mathematics direct and auditable.

2.3 Graph Representations

Underneath both the SMILES parser in §2.1 and the fingerprint generator in §2.2, RDKit represents every molecule the same way: as a graph $G = (V, E)$, with atoms as nodes V and bonds as edges E . Each node carries features — atomic number, formal charge, hybridization, aromaticity, attached hydrogen count — and each edge carries a bond order and ring membership flag. This graph is exactly what Morgan’s algorithm (§2.2) traverses to build atom-environment identifiers, and exactly what a conformer embedding algorithm (§2.4) uses as the connectivity constraints for 3D placement.

Formally, a molecular graph with n atoms can be described by an $n \times n$ **adjacency matrix** $\mathbf{A}$, where $\mathbf{A}_{ij}$ is nonzero if atoms i and j are bonded (and can encode bond order rather than a simple 0/1), together with a node feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$ holding each atom’s d -dimensional feature vector. This $(\mathbf{A}, \mathbf{X})$ pair is a strictly more complete representation than a fingerprint: a hashed fingerprint compresses the graph into a fixed-size, lossy summary optimized for cheap comparison, while $(\mathbf{A}, \mathbf{X})$ preserves the full topology, at the cost of a variable-size representation that cannot be compared with simple vector arithmetic. That trade-off — a fast, lossy fixed-size summary versus a complete, variable-size structure — is precisely what motivates learning similarity directly from $(\mathbf{A}, \mathbf{X})$ with graph neural networks, which Chapter 6 covers in full; this section introduces only the representation itself; the message-passing machinery that operates on it belongs there.

2.4 3D Conformations

Every representation covered so far — SMILES, InChI, fingerprints, molecular graphs — describes molecular **constitution**: which atoms are present and how they are bonded. None of them fixes a 3D shape. A single SMILES string can correspond to many distinct

low-energy 3D arrangements of the same atoms (**conformers**), related by rotation around single bonds, and for a sufficiently flexible molecule the number of accessible conformers grows combinatorially with the number of rotatable bonds.

RDKit generates 3D conformers with **ETKDG** (Experimental-Torsion basic Knowledge Distance Geometry; Riniker & Landrum, 2015), which improves on classical distance-geometry embedding by biasing the initially-random atomic coordinates with experimentally observed torsion-angle preferences before refining them, substantially reducing the fraction of generated conformers that require heavy post-hoc correction. The embedded structure is typically then relaxed with a molecular-mechanics force field (RDKit supports both MMFF94 and UFF) to a local energy minimum. This chapter’s hands-on project works entirely in 2D (fingerprints and Lipinski descriptors do not require 3D coordinates), so it does not call RDKit’s conformer-generation API — but 3D conformers are the direct input to Chapter 11’s docking methods and a routine intermediate step for 3D-descriptor calculation, so it is worth knowing this is where they come from and why one SMILES is not one shape.

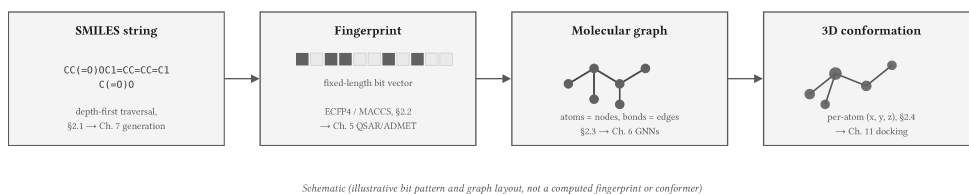


Figure 2.1. One molecule, aspirin, in the four representations this section introduced: a SMILES text string, a fingerprint bit vector, a molecular graph, and a 3D conformation, each labeled with the chapter that relies on it.

2.5 Hands-on Project: Molecular Similarity Search & Lipinski Filtering

The project code lives in this chapter’s folder (`ch02_molecular_similarity/`) and implements exactly the two ideas built up above: rank a small compound library by ECFP4/Tanimoto similarity to a query molecule (§2.2), then flag which of those candidates satisfy Lipinski’s Rule of Five.

Fingerprint generation and ranking

`similarity_search.py` builds one shared Morgan fingerprint generator,

```
FP_GENERATOR = rdFingerprintGenerator.GetMorganGenerator(radius=2, fpSize=2048)
```

— radius 2 (i.e., ECFP4, per §2.2’s radius/diameter convention) and a 2048-bit vector — and reuses it for both the query molecule and every molecule in `molecules.csv` (17 well-known drugs and biomolecules: aspirin, ibuprofen, caffeine, nicotine, dopamine, warfarin, and others). `rank_by_similarity` parses each SMILES with `Chem.MolFromSmiles`, generates its fingerprint, scores it against the query with `TanimotoSimilarity` exactly as defined in §2.2, and sorts the library in descending order of similarity. Unparsable library entries are skipped with a warning rather than crashing the run — a small but deliberate reproducibility choice, since a single malformed row in a larger, real-world compound library (say, a ChEMBL export) should not take down an entire screening job.

Lipinski filtering

`lipinski_pass` computes the four descriptors behind Lipinski’s Rule of Five (Lipinski et al., 2001), introduced in Chapter 1, §1.3, and applies it here in full:

$$\text{violations} = \mathbb{1}[\text{MW} > 500] + \mathbb{1}[\log P > 5] + \mathbb{1}[\text{HBD} > 5] + \mathbb{1}[\text{HBA} > 10] \quad (4)$$

with a molecule flagged as passing when violations ≤ 1 — the rule’s own stated tolerance for one violation, not a stricter pass/fail-on-any-violation reading. Each threshold is a multiple of five (hence the rule’s name): molecular weight under 500 Da, a calculated octanol-water partition coefficient ($\log P$) under 5, no more than 5 hydrogen-bond donors, and no more than 10 hydrogen-bond acceptors — an empirical description, from an analysis of compounds that reached Phase II clinical trials, of the property ranges associated with acceptable oral absorption and permeability.

The four descriptors are computed with RDKit as follows, and it is worth being precise about which specific method each one calls, since “log P” or “H-bond donor” is not a single universally agreed definition:

- **Molecular weight** — `Descriptors.MolWt`, the standard average molecular weight computed directly from the parsed structure.
- **LogP** — `Crippen.MolLogP`, an atom-contribution method (Wildman & Crippen, 1999) that estimates $\log P$ by summing per-atom contributions parameterized by each atom’s element and local environment, rather than measuring it experimentally.
- **H-bond donors and acceptors** — `Lipinski.NumHDonors` and `Lipinski.NumHAcceptors`, which implement RDKit’s own pattern-based definitions matched to Lipinski’s original paper, not a strict IUPAC donor/acceptor count. This distinction matters for reproducibility: a different toolkit’s H-bond counting rules can legitimately disagree with RDKit’s on edge cases (e.g., certain heteroaromatic acceptors), even though both claim to implement “Lipinski’s Rule of Five.”

Running it and reading the output

```
cd ch02_molecular_similarity
pip install -r requirements.txt
python similarity_search.py --query "CC(=O)OC1=CC=CC=C1C(=O)O" --top 5
```

The default query is aspirin; `print_table` reports each candidate's name, Tanimoto similarity, molecular weight, LogP, H-bond donor/acceptor counts, and a PASS/FAIL Lipinski flag, sorted by similarity. Running the default query, the top result is aspirin itself, at Tanimoto similarity 1.000 (a molecule is, trivially, maximally similar to itself under §2.2's definition) — a basic but genuine self-consistency check that the pipeline is doing what §2.2 says it should. Salicylic acid, aspirin's close structural relative (aspirin is salicylic acid's acetate ester), reliably ranks near the top as well, which is the qualitative behavior the similarity-property principle predicts for a close structural analogue.

Reproducibility

Dependencies are version-floored rather than exact-pinned (`rdkit>=2023.9.1` in [requirements.txt](#), validated against 2026.3.5), since this project only touches stable, long-standing RDKit APIs (`rdFingerprintGenerator`, `Crippen`, `Descriptors`, `Lipinski`). The 12-test suite in `tests/test_similarity_search.py` checks `load_library`, `rank_by_similarity`, and `lipinski_pass` against real RDKit output rather than hardcoded expectations — including an end-to-end check that the self-match case returns Tanimoto 1.000, and a deliberately constructed long-PEG-chain SMILES that genuinely fails Lipinski on two independent axes (molecular weight and H-bond acceptor count), so the “fail” branch is exercised by an actual RDKit computation rather than an invented one. `pip install -r requirements-dev.txt && pytest` from this chapter's folder reproduces all 12 results.

From a 17-compound demo to a real screen

At 17 compounds, this project is a toy — its value is in making the mechanics of §2.2's Tanimoto computation and §2.5's Lipinski filter fully auditable end to end. The same two functions, unmodified, are the core operation behind real similarity-based **virtual screening**: given a known active (a “hit”), rank a real compound library — ChEMBL's ~2.9 million compounds (Chapter 1, §1.4), or a proprietary corporate collection — by fingerprint similarity to surface candidates worth testing next. The same operation, run against molecules built on different ring systems or scaffolds than the query but landing at similarly high Tanimoto scores through shared substituent patterns, is the starting point for **scaffold hopping** — deliberately searching for structurally novel analogues of a hit, often to design around a patent or improve a liability like poor solubility while preserving the activity-relevant substructure. Both uses inherit the same caveat from §2.2: a high-ranked candidate is worth testing, not something already known to work — recall Martin et al.'s ~30% figure. Lipinski filtering plays a complementary, orthogonal role in this pipeline: it says nothing about whether a candidate will bind the target, only whether it is *shaped like* the historical population of orally bioavailable drugs — a cheap early filter to deprioritize candidates before spending assay budget on them, not a substitute for testing binding or activity directly.

Limitations of 2D similarity, and what comes next

Both of this chapter's core tools — Tanimoto similarity over 2D fingerprints, and Lipinski's Rule of Five — operate entirely on molecular **constitution**: which atoms are bonded to which, with no reference to three-dimensional shape, a specific target's binding-pocket

geometry, or measured biological activity. That is a deliberate scope, not an oversight, but it has three concrete consequences for how these tools should — and should not — be used going forward in this book:

1. **2D similarity does not see 3D shape.** Two molecules can score a high Tanimoto similarity while adopting substantially different 3D conformations (§2.4), and, in the other direction, two molecules built on entirely different 2D scaffolds — and therefore scoring a *low* Tanimoto similarity — can present near-identical 3D pharmacophores to a binding pocket. The first is a false positive for “these will behave alike”; the second is a false negative that a pure 2D similarity search will simply never surface. Reasoning about actual binding requires the protein-side structural representations Chapter 3 introduces, and culminates in the explicit 3D docking methods of Chapter 11.
2. **Lipinski’s Rule of Five predicts drug-likeness, not activity or safety.** Passing it says a molecule’s bulk physicochemical properties resemble those of historically orally-bioavailable drugs; it says nothing about whether the molecule binds any particular target, is potent, or is toxic. Predicting those properties from structure is the job of the supervised QSAR and ADMET models in Chapter 5, trained on labeled bioactivity and toxicity data rather than on a four-descriptor heuristic.
3. **Both tools are static.** A fingerprint and a Lipinski pass/fail are computed once, from one fixed structure (or, for Lipinski, one conformer-independent set of descriptors); neither says anything about how a molecule and a target move and interact over time. That dynamic picture — binding kinetics, conformational flexibility, stability of a docked pose — is the subject of the molecular dynamics methods in Chapter 12.

None of this makes 2D similarity search or Lipinski filtering wrong tools — used as what they are, a cheap first-pass filter over enormous chemical spaces, they remain standard practice throughout the industry precisely because of their speed. The point is narrower: treat a high similarity score or a Lipinski pass as a reason to *look closer* with the structure-, activity-, and dynamics-aware methods later in this book, not as a conclusion in itself.

A note on Google Colab

`rdkit` installs on Google Colab’s default runtime with a single `pip install rdkit`; the rest of this project’s dependencies are standard library. No GPU is required — every part of this chapter’s project runs on Colab’s free-tier CPU runtime unmodified.

References

- Weininger, D. (1988). SMILES, a chemical language and information system. 1. Introduction to methodology and encoding rules. *Journal of Chemical Information and Computer Sciences*, 28(1), 31–36. <https://doi.org/10.1021/ci00057a005>
- Krenn, M., Häse, F., Nigam, A., Friederich, P., & Aspuru-Guzik, A. (2020). Self-referencing embedded strings (SELFIES): A 100% robust molecular string representation. *Machine Learning: Science and Technology*, 1(4), 045024. <https://doi.org/10.1088/2632-2153/aba947>

- Heller, S. R., McNaught, A., Pletnev, I., Stein, S., & Tchekhovskoi, D. (2015). InChI, the IUPAC International Chemical Identifier. *Journal of Cheminformatics*, 7, 23. <https://doi.org/10.1186/s13321-015-0068-4>
- Morgan, H. L. (1965). The generation of a unique machine description for chemical structures - a technique developed at Chemical Abstracts Service. *Journal of Chemical Documentation*, 5(2), 107–113. <https://doi.org/10.1021/c160017a018>
- Rogers, D., & Hahn, M. (2010). Extended-connectivity fingerprints. *Journal of Chemical Information and Modeling*, 50(5), 742–754. <https://doi.org/10.1021/ci100050t>
- Durant, J. L., Leland, B. A., Henry, D. R., & Nourse, J. G. (2002). Reoptimization of MDL keys for use in drug discovery. *Journal of Chemical Information and Computer Sciences*, 42(6), 1273–1280. <https://doi.org/10.1021/ci010132r>
- Bajusz, D., Rácz, A., & Héberger, K. (2015). Why is Tanimoto index an appropriate choice for fingerprint-based similarity calculations? *Journal of Cheminformatics*, 7, 20. <https://doi.org/10.1186/s13321-015-0069-3>
- Willett, P., Barnard, J. M., & Downs, G. M. (1998). Chemical similarity searching. *Journal of Chemical Information and Computer Sciences*, 38(6), 983–996. <https://doi.org/10.1021/ci9800211>
- Martin, Y. C., Kofron, J. L., & Traphagen, L. M. (2002). Do structurally similar molecules have similar biological activity? *Journal of Medicinal Chemistry*, 45(19), 4350–4358. <https://doi.org/10.1021/jm020155c>
- Wildman, S. A., & Crippen, G. M. (1999). Prediction of physicochemical parameters by atomic contributions. *Journal of Chemical Information and Computer Sciences*, 39(5), 868–873. <https://doi.org/10.1021/ci990307l>
- Riniker, S., & Landrum, G. A. (2015). Better informed distance geometry: using what we know to improve conformation generation. *Journal of Chemical Information and Modeling*, 55(12), 2562–2574. <https://doi.org/10.1021/acs.jcim.5b00654>

See Chapter 1's references for Lipinski, C. A., Lombardo, F., Dominy, B. W., & Feeney, P. J. (2001), Experimental and computational approaches to estimate solubility and permeability in drug discovery and development settings — reused here rather than re-listed. RDKit itself has no official journal publication; its maintainers' recommended citation is “RDKit: Open-source cheminformatics. <https://www.rdkit.org>” (confirmed directly from the project's own documentation before use here), matching the convention already established in Chapter 4's references.

Johnson, M. A., & Maggiora, G. M. (Eds.). (1990). *Concepts and Applications of Molecular Similarity*. Wiley — cited above for the similarity-property principle terminology; a book, not a journal article, so listed separately from the verified journal citations above.

Chapter 3: Protein Representations & Structure Engineering

Chapter 1 introduced the two modalities this book covers; Chapter 2 built the computational toolkit for one of them, small molecules. This chapter builds the analogous toolkit for the other: proteins. Everything here — sequence representations, 3D structural anatomy, and the machine-learning-ready representations built from both — is infrastructure the rest of Part III (protein language models, structure prediction, de novo design) and the docking chapters in Part IV depend on directly.

3.1 Sequences & Amino Acid Properties

FASTA format

A protein sequence is, at its simplest, a string over a 20-letter alphabet (the standard amino acids, plus a handful of extended-alphabet codes for ambiguity and non-standard residues). The near-universal plain-text format for storing one or more such sequences is **FASTA**, named after the FASTA sequence-alignment tool it originated with (Pearson & Lipman, 1988). A FASTA record is a single header line beginning with `>`, followed by the sequence itself, wrapped at an arbitrary line width:

```
>sp|P00533|EGFR_HUMAN Epidermal growth factor receptor
MRPSGTAGAALLALLAALCPASRALEEKVCQGTSNKLTQLGTFEDHFLSLQRMFNCE
VVLGNLEITYVQRNYDLSFLKTIQEVAGYVLIALNTVERIPLNLQIRGNMYYENSYA
...
```

Every major sequence database (UniProt, NCBI, Ensembl) and every tool this book uses that consumes a raw sequence (BioPython, ESM, AlphaFold) either accepts or emits FASTA. It has no formal specification body — it is a de facto standard defined by decades of tool compatibility — which is precisely why its simplicity matters: a format with essentially zero parsing ambiguity is what let it become universal infrastructure.

Amino acid properties

The 20 standard amino acids share a backbone (an amino group, a carboxyl group, and a central α -carbon) and differ only in their side chain (R group), and it is the physicochemical character of that side chain — charge, polarity, hydrophobicity, size, and the presence of distinguishing groups like proline’s ring-closed backbone or cysteine’s thiol — that determines nearly everything downstream: which secondary structures a residue favors (§3.2), which residues cluster in a folded protein’s hydrophobic core versus its solvent-exposed surface, and which positions in a binding pocket (§3.4) a small molecule can hydrogen-bond to versus merely pack against. Substitution matrices (below) are, in effect, a quantitative encoding of exactly this: how interchangeable two amino acids are without disrupting a protein’s structure or function.

Substitution matrices

Aligning two sequences requires a way to score whether substituting one amino acid for another at a given position is “cheap” (conservative, likely to preserve function) or “expensive” (radical, likely disruptive) — identity alone is too crude, since a leucine-to-isoleucine substitution is usually far more tolerable than a leucine-to-aspartate one. A **substitution matrix** gives every one of the 20×20 amino acid pairs a log-odds score:

$$s(a, b) = \frac{1}{\lambda} \ln \frac{p_{ab}}{q_a q_b} \quad (5)$$

where p_{ab} is the observed frequency with which residues a and b are aligned with each other in a trusted reference set of alignments, q_a and q_b are their independent background frequencies, and λ is a scaling constant. Intuitively: if a and b co-occur in real alignments more often than their individual frequencies would predict by chance, $s(a, b) > 0$ (a favorable substitution); if less often, $s(a, b) < 0$. The two matrix families in near-universal use differ in how the reference alignments were built: the older PAM matrices extrapolate substitution rates from very closely related sequences using an explicit evolutionary model, while the BLOSUM matrices (Henikoff & Henikoff, 1992) are derived directly from observed substitution frequencies in blocks of already-aligned, more divergent sequence families — BLOSUM62 (built from blocks with $\leq 62\%$ pairwise identity) is the default in most modern alignment tools, including the pairwise aligner used below.

Pairwise and multiple sequence alignment

Given a substitution matrix and a gap penalty, computing the highest-scoring alignment between two sequences is a dynamic programming problem. Needleman & Wunsch (1970) gave the algorithm for **global** alignment (align the sequences end-to-end); Smith & Waterman (1981) gave the variant for **local** alignment (find the highest-scoring aligned

subsequence, ignoring unrelated flanking regions) by allowing the recurrence to reset to zero rather than go negative. Both remain exactly correct and exactly this: dynamic programming over an $(n + 1) \times (m + 1)$ score matrix, in $O(nm)$ time. A minimal, exact global alignment using BLOSUM62 (BioPython, no external tools required):

```
from Bio import Align
from Bio.Align import substitution_matrices

aligner = Align.PairwiseAligner()
aligner.substitution_matrix = substitution_matrices.load("BLOSUM62")
aligner.open_gap_score = -10
aligner.extend_gap_score = -0.5

alignment = aligner.align("MKTAYIAKQR", "MKTAYIAKER")[0]
print(alignment)
print("score:", alignment.score)
```

Multiple sequence alignment (MSA) — aligning three or more sequences simultaneously — is what makes cross-species conservation analysis, phylogenetics, and (critically, for Chapter 8) protein language model training data possible, but it is not simply pairwise alignment run $\binom{n}{2}$ times: exact multi-sequence dynamic programming is exponential in the number of sequences, so practical tools use a **progressive alignment** heuristic — build a guide tree from pairwise similarities, then align sequences and sub-alignments along that tree from the leaves inward. Clustal Omega (Sievers et al., 2011) is the present-day standard implementation of this approach, capable of scaling to datasets of hundreds of thousands of sequences. This book does not implement an MSA tool from scratch — that would mean re-deriving production bioinformatics software with no pedagogical benefit over using Clustal Omega or MAFFT directly — but §3.4 does implement the pairwise dynamic-programming primitive that progressive alignment is built from, and Chapter 8 revisits MSA directly as an input to protein language models.

3.2 3D Structural Anatomy

File formats: PDB and PDBx/mmCIF

A protein's 3D structure — the (x, y, z) coordinate of every resolved atom, plus experimental metadata (resolution, method, the identity of any bound ligands) — is distributed by the Protein Data Bank in two formats: the original fixed-column-width `.pdb` format, and its successor PDBx/mmCIF, the wwPDB's extensible standard format since 2014 (wwPDB consortium, 2019; see also Chapter 1 §1.4). This book's hands-on projects use the legacy `.pdb` format where possible for readability — it is plain text, and a HETATM line for a bound ligand or an ATOM line for a protein atom is human-inspectable without a parser — but `.pdb`'s fixed-width columns cannot represent structures with more than 62 chains or 99,999 atoms,

which is why mmCIF is now the archive’s primary format for large assemblies. Both are parsed identically by BioPython’s `Bio.PDB` module, which is what this chapter’s hands-on project (§3.4) uses.

Backbone dihedral angles: phi and psi

A polypeptide backbone’s local geometry is almost entirely described by two torsion (dihedral) angles per residue: **phi** (ϕ), the rotation around the N–C α bond, and **psi** (ψ), the rotation around the C α –C bond. (A third backbone dihedral, omega, rotates around the peptide C–N bond itself, but that bond’s partial double-bond character from amide resonance locks it planar — almost always at 180°, trans — so it is rarely treated as a free variable.) Not all (ϕ , ψ) combinations are physically possible: many would force atoms into steric clashes. Ramachandran, Ramakrishnan, and Sasisekharan (1963) computed which combinations are sterically allowed from hard-sphere atomic radii alone, producing the **Ramachandran plot** — a (ϕ , ψ) scatter plot whose allowed regions correspond almost exactly to the dihedral angles observed in real, experimentally solved structures decades later. Two regions dominate: right-handed **α -helix** (Pauling, Corey, & Branson 1951 first proposed this hydrogen-bonded helical backbone conformation on stereochemical grounds, before any protein structure had been solved by crystallography), centered near $\phi \approx -57^\circ$, $\psi \approx -47^\circ$, and extended **β -strand**, centered near $\phi \approx -120^\circ$, $\psi \approx +120^\circ$. §3.4’s hands-on project computes real (ϕ , ψ) values from a solved structure and uses simplified versions of these two regions as a coarse, geometry-only secondary-structure classifier.

Secondary and tertiary structure

Secondary structure — the local, repeating backbone conformations (α -helix, β -sheet/strand, and everything else lumped as “coil” or “loop”) — is what phi/psi angles proxy for geometrically, but the field’s actual standard for assigning it is DSSP (Kabsch & Sander, 1983), which uses backbone hydrogen-bonding patterns (an α -helix’s characteristic $i \rightarrow i + 4$ backbone N–H $\cdots$ O=C hydrogen bond; a β -sheet’s inter-strand hydrogen bonding) rather than dihedral geometry alone. This matters pedagogically: the phi/psi-only classifier built in §3.4 is a legitimate, useful approximation — computable with zero extra dependencies directly from atomic coordinates — but it is not DSSP, and should not be reported as equivalent to it in any context where the distinction matters (this caveat is stated in the code itself, not just here). **Tertiary structure** is the full 3D fold: how secondary structure elements pack together in space, stabilized by the same hydrophobic-core-versus-solvent-exposed-surface logic introduced in §3.1’s discussion of amino acid properties. **Quaternary structure**, one level up, is how multiple folded chains (subunits) assemble into a functional complex — not covered further here, but relevant wherever this book discusses antibody structures (Part III) or protein-protein docking.

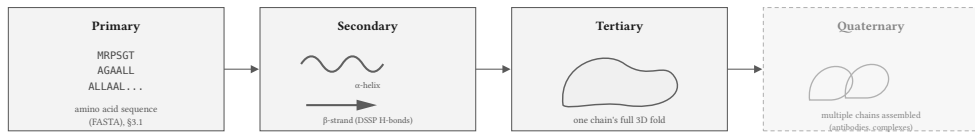


Figure 3.1. Protein structure hierarchy: primary (sequence), secondary (local helix/strand conformations), tertiary (one chain's full 3D fold), and quaternary (multiple chains assembled), shown lighter as beyond this chapter's own scope.

3.3 Machine Learning Representations

Raw atomic coordinates are not, by themselves, a convenient input to most machine learning models: they are unordered across chains, arbitrary in their global rotation and translation (the same protein measured or modeled twice will not have identical coordinates unless one is deliberately superposed onto the other), and far higher-dimensional than the information content of the fold actually requires. Three representations, in increasing order of the amount of 3D detail they discard, recur throughout the rest of this book:

- **Residue contact maps.** An $N \times N$ binary (or continuous, distance-valued) matrix recording whether each pair of residues is within some cutoff distance of each other — computed in §3.4 as $C\alpha$ – $C\alpha$ distance thresholded at 8 Å, a standard convention, though finer-grained variants use the closest side-chain heavy atom instead. A contact map is invariant to global rotation/translation by construction (it depends only on pairwise distances) and was, for decades, the target representation for structure-prediction models before end-to-end 3D generation became tractable: Senior et al. (2020) — AlphaFold's first iteration — worked by predicting a distribution over inter-residue distances directly from sequence and co-evolution features, then using that predicted contact/distance map as a constraint for a downstream folding optimization. AlphaFold2 (Jumper et al., 2021; Chapter 1 §1.2, Chapter 9) replaced this two-stage contact-map-then-fold pipeline with direct end-to-end 3D coordinate prediction, but contact and distance maps remain a standard evaluation representation and a lightweight input feature throughout the field.
- **Molecular surface meshes.** A triangulated approximation of a protein's solvent-accessible or solvent-excluded surface — the boundary that actually determines which small molecules or other proteins can approach a given region, as opposed to which atoms are merely nearby in 3D. The reduced-surface algorithm of Sanner, Olson, and Spehner (1996) is the standard efficient method for computing this analytically rather than by brute-force sampling, and surface meshes are the natural representation wherever shape complementarity matters directly, most notably molecular docking (Chapter 11).
- **Structural graphs.** A protein represented as a graph — residues (or atoms) as nodes, spatial proximity or covalent bonds as edges, with node/edge features carrying amino

acid identity, dihedral angles, or distances — is the input representation for the graph neural networks used throughout Part III, particularly the equivariant architectures in Chapters 6 and 9 that must respect the same rotation/translation invariance a contact map gets for free. A contact map is, in this light, simply the adjacency matrix of one particular (unweighted, cutoff-thresholded) structural graph — the three representations in this list are not competitors so much as points on a continuum from “discard everything except pairwise topology” (contact maps) to “retain full continuous 3D geometry as graph edge features” (structural graphs).

3.4 Hands-on Project: Structural Feature Extraction & Binding Pocket Geometry

This project computes real spatial features — backbone dihedral angles, a contact map, and binding-pocket residues — directly from a solved PDB structure, using **the same structure Chapter 1 retrieved**: PDB entry 1M17, the EGFR tyrosine kinase domain in complex with the inhibitor erlotinib (ligand code A04). Reusing this structure is deliberate — it lets this chapter build directly on the data Chapter 1 already validated, and keeps a single running example (EGFR) coherent across Chapters 1, 3, and the Chapter 16 capstone.

The project computes:

1. **Backbone dihedral angles** (ϕ , ψ) for every residue in chain A, plus a coarse geometry-only secondary-structure call (helix / sheet / coil) from simplified Ramachandran regions — explicitly documented in the code as an approximation, not a DSSP replacement (§3.2). On this structure it gives roughly 40% helix, 38% sheet, 22% coil — a genuinely mixed α / β result, consistent with a kinase domain’s fold (a β -sheet-rich N-lobe and α -helix-rich C-lobe), not an artifact of the classifier defaulting to one class.
2. **A C α –C α contact map** at an 8 Å cutoff (§3.3), computed as a vectorized pairwise-distance matrix.
3. **Binding pocket residues** — every standard residue with any atom within 5 Å of any atom of the bound ligand (A04 / erlotinib), ranked by distance. On this structure, the closest residue is MET769 at approximately 2.7 Å — a distance in the range of a hydrogen bond, consistent with this inhibitor class’s known hinge-region binding mode (this specific distance is a direct geometric computation from the structure, not a claim taken from a secondary source).

See [README.md](#) for setup and usage, and [structural_features.py](#) for the implementation. `data/1M17.pdb` is bundled directly in this repository — the same real structure file Chapter 1’s project downloads — so the test suite (`tests/test_structural_features.py`) runs fully offline and deterministically; one additional test exercises the live RCSB PDB download path directly, as a reproducibility check on that path specifically.

A note on Google Colab

This project's only non-preinstalled Colab dependency is biopython (numpy and requests both ship on the default runtime); install it with `!pip install biopython` in the first cell. No GPU is required.

References

- Pearson, W. R., & Lipman, D. J. (1988). Improved tools for biological sequence comparison. *Proceedings of the National Academy of Sciences*, 85(8), 2444–2448. <https://doi.org/10.1073/pnas.85.8.2444>
- Needleman, S. B., & Wunsch, C. D. (1970). A general method applicable to the search for similarities in the amino acid sequence of two proteins. *Journal of Molecular Biology*, 48(3), 443–453. [https://doi.org/10.1016/0022-2836\(70\)90057-4](https://doi.org/10.1016/0022-2836(70)90057-4)
- Smith, T. F., & Waterman, M. S. (1981). Identification of common molecular subsequences. *Journal of Molecular Biology*, 147(1), 195–197. [https://doi.org/10.1016/0022-2836\(81\)90087-5](https://doi.org/10.1016/0022-2836(81)90087-5)
- Henikoff, S., & Henikoff, J. G. (1992). Amino acid substitution matrices from protein blocks. *Proceedings of the National Academy of Sciences*, 89(22), 10915–10919. <https://doi.org/10.1073/pnas.89.22.10915>
- Sievers, F., Wilm, A., Dineen, D., Gibson, T. J., Karplus, K., Li, W., Lopez, R., McWilliam, H., Remmert, M., Söding, J., Thompson, J. D., & Higgins, D. G. (2011). Fast, scalable generation of high-quality protein multiple sequence alignments using Clustal Omega. *Molecular Systems Biology*, 7, 539. <https://doi.org/10.1038/msb.2011.75>
- Ramachandran, G. N., Ramakrishnan, C., & Sasisekharan, V. (1963). Stereochemistry of polypeptide chain configurations. *Journal of Molecular Biology*, 7, 95–99. [https://doi.org/10.1016/S0022-2836\(63\)80023-6](https://doi.org/10.1016/S0022-2836(63)80023-6)
- Pauling, L., Corey, R. B., & Branson, H. R. (1951). The structure of proteins: two hydrogen-bonded helical configurations of the polypeptide chain. *Proceedings of the National Academy of Sciences*, 37(4), 205–211. <https://doi.org/10.1073/pnas.37.4.205>
- Kabsch, W., & Sander, C. (1983). Dictionary of protein secondary structure: pattern recognition of hydrogen-bonded and geometrical features. *Biopolymers*, 22(12), 2577–2637. <https://doi.org/10.1002/bip.360221211>
- Senior, A. W., Evans, R., Jumper, J., Kirkpatrick, J., Sifre, L., Green, T., Qin, C., Židek, A., Nelson, A. W. R., Bridgland, A., Penedones, H., Petersen, S., Simonyan, K., Crossan, S., Kohli, P., Jones, D. T., Silver, D., Kavukcuoglu, K., & Hassabis, D. (2020). Improved protein structure prediction using potentials from deep learning. *Nature*, 577(7792), 706–710. <https://doi.org/10.1038/s41586-019-1923-7>
- Sanner, M. F., Olson, A. J., & Spehner, J. C. (1996). Reduced surface: an efficient way to compute molecular surfaces. *Biopolymers*, 38(3), 305–320. [https://doi.org/10.1002/\(SICI\)1097-0282\(199603\)38:3%3C305::AID-BIP4%3E3.0.CO;2-Y](https://doi.org/10.1002/(SICI)1097-0282(199603)38:3%3C305::AID-BIP4%3E3.0.CO;2-Y)

See also Chapter 1's references for the wwPDB consortium (2019) PDB/ mmCIF citation and Jumper et al. (2021) AlphaFold2 citation, both reused here rather than re-listed.

All citations above were verified against PubMed records before use (see this repository's editorial process); the structural data cited in §3.4 (residue count, secondary-structure fractions, pocket residues and distances) was computed directly from the bundled `data/1M17.pdb` file, not taken from a secondary source — see `structural_features.py` to reproduce.

Chapter 4: Core Toolkit & Biological Data Engineering

Chapters 2 and 3 introduced RDKit and BioPython through single, focused tasks — similarity search and Lipinski filtering; structural feature extraction. This chapter goes one level deeper into both libraries, adds DeepChem as the framework layer built on top of them, surveys the biological database landscape those chapters' data came from, and then builds the piece of infrastructure every chapter from here on actually depends on: a pipeline that turns raw, messy, real bioactivity data into something a model can train on.

4.1 Advanced RDKit

Chapter 2 used RDKit to compute descriptors from already-clean SMILES strings. Real data is never that clean. Before a molecule from any external source can be trusted, it typically needs to pass through three distinct operations.

Sanitization

Parsing a SMILES string and *sanitizing* it are different steps. Sanitization performs a battery of chemical validity checks — correct valence, consistent aromaticity, sensible ring perception — and raises an explicit, catchable error the moment a structure fails one:

```
from rdkit import Chem

mol = Chem.MolFromSmiles("C(C)(C)(C)(C)C", sanitize=False) # 5-valent carbon
Chem.SanitizeMol(mol)
# rdkit.Chem.rdchem.AtomValenceException:
# Explicit valence for atom # 0 C, 5, is greater than permitted
```

`Chem.MolFromSmiles` sanitizes by default and simply returns `None` on failure, which is why every function in this book that parses a SMILES string checks for `None` before proceeding — that check is the sanitization gate.

Standardization

Two records for “the same” compound rarely arrive identical. One database might store a molecule as its sodium salt, another as the free acid; one might include an explicit counterion, another might not. Comparing or deduplicating across sources requires **standardization** — a defined, reproducible sequence of transformations, not ad hoc cleanup — typically: (1) keep only the largest covalently-bonded fragment, discarding salts and counterions, then (2) neutralize any remaining formal charges:

```
from rdkit import Chem
from rdkit.Chem.MolStandardize import rdMolStandardize

mol = Chem.MolFromSmiles("CC(=O)Nc1ccc(O)cc1.[Na+].[Cl-]") # paracetamol NaCl salt
mol = rdMolStandardize.LargestFragmentChooser().choose(mol)
mol = rdMolStandardize.Uncharger().uncharge(mol)
print(Chem.MolToSmiles(mol)) # CC(=O)Nc1ccc(O)cc1
```

Tautomer canonicalization

A **tautomer** is one of two or more structures that interconvert by moving a proton and a double bond simultaneously (classically, keto $\leftrightarrow$ enol). Different data sources — or different depositors of the same assay to the same database — can register the same compound as different tautomers, which silently fragments what should be one compound’s data into two. RDKit’s tautomer enumerator can both list all reasonable tautomers of a structure and pick one canonical representative, deterministically, so that two records of the same compound in different tautomeric forms standardize to identical output:

```
enumerator = rdMolStandardize.TautomerEnumerator()
mol = Chem.MolFromSmiles("O=C1CCCCC1") # cyclohexanone (keto form)
tautomers = enumerator.Enumerate(mol)
print(len(tautomers)) # 2: keto (O=C1CCCCC1) and enol (OC1=CCCCC1)
print(Chem.MolToSmiles(enumerator.Canonicalize(mol)))
```

Section 4.5’s ETL pipeline chains exactly these three operations — sanitize (implicitly, via `MolFromSmiles`), standardize, canonicalize tautomers — before a single descriptor is computed, precisely because skipping any one of them means silently double-counting or mismatching compounds later.

Physicochemical properties, revisited

Chapter 2 computed four Lipinski descriptors (molecular weight, LogP, H-bond donor/acceptor counts) directly. That is the same computation this chapter’s pipeline runs — now

on standardized structures — and the same one DeepChem’s featurizers (§4.2) wrap into a reusable, batchable interface for hundreds of descriptors at once rather than four.

4.2 The DeepChem Framework

RDKit and BioPython are libraries — they give you molecules and sequences as first-class objects and let you compute things about them. **DeepChem** is a framework built on top of libraries like these: it standardizes the remaining steps between “I have a molecule” and “I have a trained, evaluated model,” specifically for chemistry and the life sciences (Ramsundar et al., 2019 — DeepChem’s own documented citation is this book, *Deep Learning for the Life Sciences*, rather than a single journal paper). Three abstractions matter most for the rest of this book:

- **Featurizers.** A featurizer is a function from a raw molecule (or protein, or crystal) to a fixed numeric representation a model can consume — a Morgan fingerprint (Chapter 2), a molecular graph (Chapter 6), or a 3D-coordinate-based representation (Chapters 9–10). DeepChem ships dozens of these behind one consistent interface, which is what makes swapping representations in a QSAR pipeline (Chapter 5) a one-line change rather than a rewrite.
- **Datasets.** A thin, consistent wrapper around (features, labels, weights, identifiers) that decouples *how data is split* from *how a model is trained on it*. This matters more than it sounds: Chapter 5 will show that a **random** train/test split systematically overestimates a QSAR model’s real-world performance compared with a **scaffold split** (grouping by core molecular scaffold so structurally related compounds can’t leak between train and test) — DeepChem’s `Dataset/splitter` abstractions are what make running both splits on the same data a controlled comparison rather than two different pipelines.
- **Reproducible pipelines.** By fixing featurizer, split, and model behind one interface, a DeepChem pipeline is specified declaratively enough that another researcher (or another chapter of this book) can rerun it and get the same partition of data and the same input representation, even if the underlying model architecture changes. MoleculeNet (Wu et al., 2018) — a suite of curated molecular property benchmarks distributed through DeepChem — exists specifically to make this kind of apples-to-apples comparison possible across published models, for exactly this reason.

This book does not route every hands-on project through DeepChem’s `Model` classes — several chapters, including this one, use RDKit and plain Python directly where that is more pedagogically transparent about what is actually happening to the data. DeepChem becomes the more direct, load-bearing tool starting in Chapter 5.

4.3 BioPython Integration

Chapter 3 used one corner of BioPython — `Bio.PDB` — for structure parsing. BioPython (Cock et al., 2009) is considerably broader: it is a single, consistent library spanning sequence I/O (`Bio.SeqIO`, reading and writing FASTA, GenBank, and dozens of other formats), sequence manipulation (`Bio.Seq`, including transcription and translation), and structural biology (`Bio.PDB`, `Bio.Align`, used in Chapter 3). That breadth is the point: a nucleic acid sequence pulled from a FASTA file, transcribed to mRNA, translated to a protein sequence, and eventually mapped onto a solved 3D structure can stay inside one library’s object model the entire way:

```
from Bio.Seq import Seq

coding_dna = Seq("ATGGCCATTGTAATGGGCCGCTGAAAGGGTGCCCGATAG")
print(coding_dna.transcribe())          # AUGGCCAUUGUAAUGGGCCGCUGAAAGGGUGCCCGAUAG
print(coding_dna.translate(to_stop=True)) # MAIVMGR
print(coding_dna.reverse_complement())  #
CTATCGGGCACCTTTTCAGCGGCCCATTAATGGCCAT
```

This chapter does not add a second BioPython hands-on project — Chapter 3 already validated the `Bio.PDB` path this book relies on most, and Chapter 8 (protein language models) is where `Bio.Seq/Bio.SeqIO` become load-bearing for sequence data engineering. The point made here is architectural: BioPython is the connective tissue between the sequence-level and structure-level tools this book uses, not a structure-only library that happened to be useful in one chapter.

4.4 Biological Databases

Chapters 1–3 already used two of these directly (ChEMBL, the PDB); this section places them in the context of the broader landscape, because Section 4.5’s ETL pipeline exists precisely because real projects need data from more than one of them.

Database	Content	Cited as
ChEMBL	Curated bioactivity data: which compounds were tested against which targets, and what was measured. Used throughout Chapters 1–2 and 4–5.	Mendez et al., 2019 (Chapter 1)
PubChem	The largest open aggregator of chemical information — substances, compounds, and bioactivity data pulled in from hundreds of contributing sources, including ChEMBL itself.	Kim et al., 2023
BindingDB	A curated database specifically of measured protein–small-molecule binding affinities, with a particular focus on data useful for structure-based and computational drug design.	Gilson et al., 2016
UniProt	The reference resource for protein sequence and function: canonical amino acid sequences, domain annotations, and cross-references to essentially every other major database, including the PDB and ChEMBL.	UniProt Consortium, 2025
PDB	The archive of experimentally solved 3D structures. Used directly in Chapters 1 and 3.	wwPDB consortium, 2019 (Chapter 1)

These are not independent silos. A ChEMBL target record carries a UniProt accession; a PDB structure entry usually carries the same UniProt accession and often an explicit ChEMBL or PubChem ligand identifier for any bound small molecule; PubChem aggregates bioactivity data that originated in ChEMBL and BindingDB in the first place. This interconnection is precisely what makes cross-database identifier reconciliation — matching “this compound” or “this protein” across sources that name it differently — one of the more

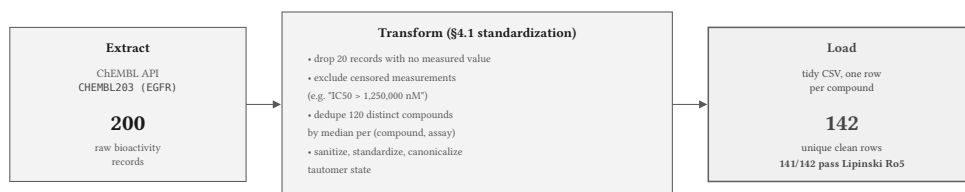
tedious but unavoidable parts of building a real training dataset, and exactly why the standardization techniques in §4.1 (getting every compound to one canonical representation before comparing across sources) are a data engineering necessity, not a nicety.

4.5 Hands-on Project: An Automated ChEMBL ETL Pipeline

This project builds the pipeline every prior chapter’s ad hoc data handling was implicitly a special case of: **extract** raw bioactivity records from ChEMBL, **transform** them into a clean, deduplicated, one-row-per-compound table using the standardization techniques from §4.1, and **load** the result as a tidy CSV — the exact shape of dataset Chapter 5’s QSAR models will train on.

Running it against the live ChEMBL API for EGFR (CHEMBL203, continuing the running example from Chapters 1 and 3) and inspecting the result directly shows why each transform step is necessary, not decorative:

- Of 200 raw bioactivity records extracted, 20 had no measured value at all (dropped), and a further meaningful fraction were **censored** measurements — e.g. $IC_{50} > 1,250,000$ nM, meaning “no activity detected up to this concentration,” not a point estimate — which are excluded by default rather than treated as if they were exact values.
- The 200 raw records covered only 120 distinct compounds; several compounds had been measured two or three times (against variant assays, or independently), which the pipeline aggregates via the median into one value per (compound, assay type) pair rather than leaving duplicates that would silently overweight those compounds in any downstream model.
- After standardization (§4.1) and cleaning, 200 raw records reduced to 142 unique, clean (compound, assay type) rows, of which 141 pass Lipinski’s Rule of Five — consistent with the fact that this dataset is almost entirely small-molecule kinase inhibitors, the expected chemotype for an oncology target like EGFR.



Real counts from the Section 4.5 hands-on run against the live ChEMBL API.

Figure 4.1. The ETL pipeline applied to the live ChEMBL EGFR query above: Extract pulls 200 raw records, Transform drops missing/censored values and deduplicates 120 distinct compounds by median, and Load writes 142 unique clean rows, 141 of which pass Lipinski’s Rule of Five.

A note on resilience

While building this chapter, the live ChEMBL API returned intermittent HTTP 500 errors and timeouts for a period (unrelated to this project — confirmed by testing the RCSB PDB API as a control at the same time, which responded normally throughout). Rather than treat that as something to wait out, the pipeline is built to survive it: raw extracts are cached to disk (`save_raw_json/load_raw_json`), a real 200-record extract is bundled in this repository as `data/raw_egfr_bioactivities_sample.json` (fetched 2026-08-16, while the API was healthy) so the transform/load logic and test suite run fully offline and deterministically, and the CLI (`--use-cached-raw`) falls back to that fixture automatically if a live request fails. This is not a hypothetical design choice — it is a direct, documented response to a real external outage encountered during this chapter’s own development, and it is the same lesson the economics in Chapter 1 §1.1 make about attrition: build pipelines that degrade gracefully rather than ones that assume every dependency stays up.

See `README.md` for setup and usage, and `etl_pipeline.py` for the implementation.

A note on Google Colab

This project’s dependencies (`rdkit`, `requests`) match Chapters 2 and 1 respectively; install with `!pip install rdkit` if it is not already present in the runtime. No GPU is required.

References

- Ramsundar, B., Eastman, P., Walters, P., Pande, V., Leswing, K., & Wu, Z. (2019). *Deep Learning for the Life Sciences*. O’Reilly Media. (DeepChem’s own documented citation; there is no separate DeepChem journal paper.)
- Wu, Z., Ramsundar, B., Feinberg, E. N., Gomes, J., Geniesse, C., Pappu, A. S., Leswing, K., & Pande, V. (2018). MoleculeNet: a benchmark for molecular machine learning. *Chemical Science*, 9(2), 513–530. <https://doi.org/10.1039/C7SC02664A>
- Cock, P. J. A., Antao, T., Chang, J. T., Chapman, B. A., Cox, C. J., Dalke, A., Friedberg, I., Hamelryck, T., Kauff, F., Wilczynski, B., & de Hoon, M. J. L. (2009). Biopython: freely available Python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11), 1422–1423. <https://doi.org/10.1093/bioinformatics/btp163>
- Kim, S., Chen, J., Cheng, T., Gindulyte, A., He, J., He, S., Li, Q., Shoemaker, B. A., Thiessen, P. A., Yu, B., Zaslavsky, L., Zhang, J., & Bolton, E. E. (2023). PubChem 2023 update. *Nucleic Acids Research*, 51(D1), D1373–D1380. <https://doi.org/10.1093/nar/gkac956>
- Gilson, M. K., Liu, T., Baitaluk, M., Nicola, G., Hwang, L., & Chong, J. (2016). BindingDB in 2015: A public database for medicinal chemistry, computational chemistry and systems pharmacology. *Nucleic Acids Research*, 44(D1), D1045–D1053. <https://doi.org/10.1093/nar/gkv1072>
- UniProt Consortium. (2025). UniProt: the Universal Protein Knowledgebase in 2025. *Nucleic Acids Research*, 53(D1), D609–D617. <https://doi.org/10.1093/nar/gkae1010>

RDKit itself has no official journal publication; its maintainers' recommended citation is "RDKit: Open-source cheminformatics. <https://www.rdkit.org>" (confirmed directly from the project's own documentation before use here). See also Chapter 1's references for Mendez et al. (2019, ChEMBL), the wwPDB consortium (2019, PDB/mmCIF), and Lipinski et al. (2001, Rule of Five) — all reused here rather than re-listed.

The raw bioactivity counts, filtering results, and Lipinski pass-rate cited in §4.5 were computed directly by running `etl_pipeline.py` against the live ChEMBL API on 2026-08-16, not taken from a secondary source — see `data/raw_egfr_bioactivities_sample.json` and `etl_pipeline.py` to reproduce.

Chapter 5: QSAR & ADMET

Property Modeling

Chapter 4 closed by promising that DeepChem, introduced there as a framework layer over RDKit, “becomes the more direct, load-bearing tool starting in Chapter 5.” Building this chapter’s hands-on project surfaced a concrete reason to revisit that plan rather than follow it literally — documented honestly in Section 5.5, in the same spirit as Chapter 4’s own note on the live ChEMBL API outage it encountered. What this chapter *does* build directly on Chapter 4: the ETL pipeline’s output — a clean, one-row-per-compound table of standardized structures and measured bioactivities — is exactly the shape of dataset a QSAR model trains on. This chapter turns that table into predictions.

5.1 QSAR Principles

QSAR (Quantitative Structure-Activity Relationship) modeling is the premise that a molecule’s structure determines its biological or physicochemical behavior closely enough that a model fit on (structure, measured property) pairs can predict that property for a new, untested structure. Structure enters the model exactly as Chapter 2 represented it — most commonly as a fingerprint, since a fixed-length bit vector is what a standard supervised learning algorithm expects as input — and the model outputs either a continuous value (**regression**: predict an IC₅₀, a solubility, a clearance rate) or a discrete label (**classification**: predict active/inactive, blocker/non-blocker). This chapter’s hands-on project (Section 5.5) is a classification task.

The field’s methodology has shifted substantially in emphasis, though not in its core premise, over several decades. Classical QSAR — the Hansch-type linear regression analyses that gave the field its name — fit a small number of hand-selected physicochemical descriptors (LogP, molar refractivity, Hammett substituent constants) against a linear model,

favoring interpretability: each coefficient has a direct chemical meaning. Modern QSAR, including this chapter’s project, typically fits a non-linear ensemble model (Random Forest, gradient boosted trees) or a deep neural network on a high-dimensional fingerprint or a learned representation, trading some of that per-descriptor interpretability for substantially better predictive accuracy on complex, non-additive structure-activity relationships. Section 5.3 covers three specific non-linear methods in this modern line; Chapter 6 covers the further step of learning the representation itself (graph neural networks) rather than fixing it as a Chapter 2 fingerprint beforehand.

One property of chemical data is easy to overlook coming from generic supervised learning and becomes central to this chapter: training examples are not independent and identically distributed in the way a generic ML textbook assumes. Chemical libraries cluster into families of close structural analogues built on the same synthetic scaffold — dozens of compounds that differ only in a substituent — so two “different” training examples can be nearly redundant. Section 5.4 shows exactly why this matters: it determines whether a train/test split actually measures generalization to new chemical space, or measures something closer to interpolation between near-duplicates.

5.2 ADMET Profiling

Chapter 1 (Section 1.1) established that most clinical drug candidates fail, and that failure is concentrated in efficacy — but efficacy is not the only thing that has to hold up. **ADMET** — Absorption, Distribution, Metabolism, Excretion, and Toxicity — is the standard shorthand for the pharmacokinetic and safety properties that determine whether an efficacious compound can actually become a viable, safely dosable drug. Van de Waterbeemd and Gifford (2003), in an early and still-influential assessment of the field, framed the core promise and the core difficulty of *in silico* ADMET modeling in the same phrase: computational prediction could move these assessments far earlier in the pipeline than the animal and human studies that traditionally measure them directly — “towards prediction paradise” — but only to the extent the underlying models are trained on data that actually captures the relevant biology, which for many ADMET endpoints has historically been scarcer and noisier than potency data.

This chapter’s hands-on project focuses on one specific, well-studied toxicity endpoint: **hERG channel blockade**. hERG (the human Ether-à-go-go-Related Gene, official gene symbol *KCNH2*) encodes the pore-forming subunit of the Kv11.1 voltage-gated potassium channel, responsible for the rapid delayed-rectifier current (I_{Kr}) that repolarizes the cardiac action potential after each heartbeat. Sanguinetti and Tristani-Firouzi (2006), reviewing the channel’s biology, describe why it is disproportionately liable to off-target drug binding: its pore cavity is unusually large and lined with aromatic residues, allowing a structurally diverse range of otherwise unrelated drugs to bind and block it, in a way most other ion channels do not permit. Blocking hERG prolongs the cardiac action potential — measured

clinically as QT-interval prolongation — which increases the risk of *torsade de pointes*, a potentially fatal ventricular arrhythmia. Because this liability is (a) common across unrelated drug classes and (b) potentially fatal, hERG screening is now a routine, early safety-pharmacology checkpoint in essentially every small-molecule discovery program, which is exactly what makes cheap, fast *in silico* hERG prediction valuable: flagging a likely liability before it costs a wet-lab assay, an animal study, or — in the cases that motivated this regulatory scrutiny in the first place — a clinical trial or a market withdrawal.

A binary blocker/non-blocker classification, which is what Section 5.5 builds, is a simplification of the property that actually matters clinically. Redfern et al. (2003), analyzing hERG/ I_{Kr} potency against clinical torsadogenic outcomes for 100 marketed and withdrawn drugs, found that risk correlates far better with the **ratio** of hERG IC50 to a drug's effective free therapeutic plasma concentration than with hERG IC50 in isolation — drugs later withdrawn for torsade risk clustered at 0.1- to 31-fold separation from their therapeutic concentration, while drugs with no reported torsade risk mostly showed greater than 30-fold separation. A compound's hERG IC50 alone, without knowing the concentration at which it will actually be dosed, is therefore an incomplete safety signal — a limitation Section 5.5 returns to directly once a concrete IC50 threshold has been chosen.

5.3 Classical Machine Learning Benchmarks

Three non-linear classifiers dominate practical QSAR/bioactivity classification and are benchmarked directly in Section 5.5:

Random Forest (Breiman, 2001) trains an ensemble of decision trees, each fit on a bootstrap resample of the training data and restricted at each split to a random subset of features, then averages their predictions. The random feature subsampling is what decorrelates the trees from each other — without it, bagged trees fit on the same strongly-correlated fingerprint bits would end up highly similar, and averaging similar models does little to reduce variance. Random Forests require almost no hyperparameter tuning to get reasonable performance and are a standard, hard-to-beat-by-accident baseline for fingerprint-based classification.

Support Vector Machines (Cortes & Vapnik, 1995) find the hyperplane that separates two classes with maximum margin, then generalize to non-linear decision boundaries via the kernel trick: instead of an explicit non-linear feature map, a kernel function K computes inner products in an implicit, higher-dimensional feature space directly. The decision function takes the form

$$f(x) = \text{sign} \left(\sum_i \alpha_i y_i K(x_i, x) + b \right) \quad (6)$$

summed over the training examples x_i with non-zero weight α_i (the support vectors). This chapter’s project uses the radial basis function (RBF) kernel, a standard default for fingerprint data with no prior assumption of linear separability.

XGBoost (Chen & Guestrin, 2016) is a specific, heavily engineered implementation of gradient-boosted decision trees: rather than averaging independently trained trees as Random Forest does, it fits trees sequentially, each one trained to correct the current ensemble’s residual error, with an explicit regularization term controlling tree complexity directly in the objective:

$$\mathcal{L}(\phi) = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k), \quad \Omega(f) = \gamma T + \frac{1}{2} \lambda \|w\|^2 \quad (7)$$

where l is a per-example loss (log-loss for the binary classification task in Section 5.5), T is the number of leaves in tree f_k , and γ, λ penalize tree size and leaf-weight magnitude respectively — the regularization is what keeps boosting, which can overfit badly if left unchecked, competitive in practice.

All three are implemented here directly with scikit-learn (Pedregosa et al., 2011) for Random Forest and SVM, and the dedicated `xgboost` package for XGBoost — the same tabular-data workhorses used throughout applied ML, not chemistry-specific tools. Siramshetty et al. (2020) benchmarked exactly this kind of classical approach against modern deep learning (DNNs, RNNs) on a comparably sized ChEMBL-derived hERG dataset (~9,000 compounds) and found their best overall result came from **XGBoost with RDKit descriptors** — direct support, from the hERG literature specifically, for why the outline of this book centers XGBoost for Section 5.5’s project while still benchmarking Random Forest and SVM alongside it for comparison.

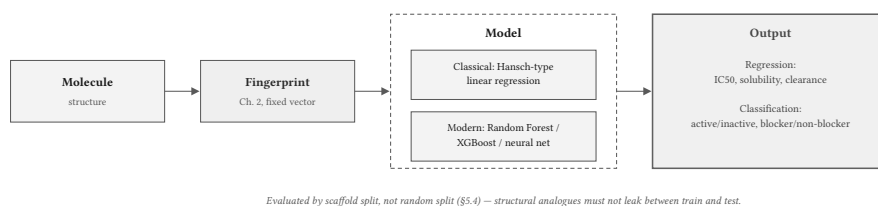


Figure 5.1. QSAR modeling pipeline: a molecule’s fingerprint (Chapter 2) feeds either a classical linear model or a modern non-linear ensemble/neural net, producing a regression or classification output, evaluated by scaffold split (Section 5.4).

5.4 Handling Bioactivity Bias

Two distinct forms of bias affect a bioactivity classifier, and both are demonstrated with real measurements from this chapter’s own dataset in Section 5.5.

Class imbalance

Running this chapter’s cleaning pipeline (Section 5.5) against the real ChEMBL hERG dataset produces 1,307 compounds labeled blockers against 458 labeled non-blockers at the chosen threshold — a roughly 2.85:1 imbalance, not the near-even split a textbook classification example often assumes. Part of this imbalance is a real property of hERG pharmacology (many drug-like scaffolds do block the channel at some concentration), but part of it is a data-ascertainment artifact worth naming directly: compounds are deposited in ChEMBL because someone chose to test them, and hERG counter-screening is disproportionately run on compounds already flagged, by structure or by chemical series, as plausible liabilities. A dataset built this way is not a random sample of drug-like chemical space, and its class balance should not be read as an estimate of the true prevalence of hERG blockade among arbitrary compounds — a caveat that applies to bioactivity datasets generally, not just this one.

Practically, imbalance matters because raw accuracy is easy to game: a classifier that always predicts “blocker” on this dataset would already score roughly 74% accuracy while learning nothing, which is exactly why Section 5.5 reports balanced accuracy and ROC-AUC alongside accuracy throughout — both are far less inflated by a majority-class-favoring classifier. One standard mitigation is **oversampling** the minority class in the *training* set specifically: SMOTE (Synthetic Minority Over-sampling Technique; Chawla et al., 2002) generates synthetic minority-class examples by interpolating between each minority example and its nearest minority-class neighbors in feature space, rather than naively duplicating existing rows, giving the classifier more (and more varied) minority-class signal to fit against. Applying SMOTE only to the training set — never to the held-out test set, which must stay a faithful sample of the real class distribution to give an honest performance estimate — is a correctness requirement, not a style choice, and `train_and_evaluate` in `herg_qsar.py` enforces it structurally: the split happens first, and `SMOTE().fit_resample` is only ever called on the training partition.

Run on this chapter’s scaffold-split evaluation (the harder, more realistic split — see below), SMOTE moves XGBoost’s balanced accuracy from 0.661 to 0.692 and ROC-AUC from 0.803 to 0.804, at the cost of accuracy (0.776 → 0.768), recall (0.881 → 0.836), and F1 (0.857 → 0.846). This is a genuine, unglamorous trade-off, not a universal improvement: SMOTE trained the model to pay more attention to the minority (non-blocker) class, which helped the metrics that treat both classes symmetrically and cost a small amount of majority-class performance. Whether that trade is worth taking depends on which error is more costly in context — missing a true blocker (a safety miss) is generally worse than flagging a true non-blocker unnecessarily (an efficiency cost), which is an argument for caring about class-imbalance-aware metrics in the first place, independent of whether SMOTE specifically is used.

Random split vs. scaffold split

The second, distinct source of bias is in how a dataset is partitioned into train and test sets at all — the point Chapter 4 previewed and this section now measures directly. A uniformly random split routinely puts two close analogues, sharing the same core scaffold and differing only in a substituent, on opposite sides of the train/test boundary. A model can then score well on the test set not because it generalizes to new chemical space, but because it has effectively already seen a near-twin of each test compound during training. A **scaffold split** — computed here from Bemis-Murcko scaffolds (Bemis & Murcko, 1996), following the grouping-and-greedy-fill methodology documented for exactly this purpose by Yang et al. (2019) — groups compounds by their generic ring scaffold first and assigns whole scaffold groups to train or test, so no scaffold appears on both sides. This is a harder, but more honest, estimate of how a model performs on genuinely novel chemical series.

`herg_qsar.py` implements this directly with RDKit’s `Scaffolds.MurckoScaffold` module rather than via the `deepchem` package specifically because that dependency turned out not to be viable in this project’s environment — see Section 5.5 for exactly what happened.

Running this chapter’s full pipeline (1,765 compounds, an 80/20 split) under both strategies, for all three classifiers from Section 5.3, gives a consistent answer:

Model	Split	Accuracy	Balanced acc.	ROC-AUC
XGBoost	random	0.816	0.735	0.843
XGBoost	scaffold	0.776	0.661	0.803
Random Forest	random	0.816	0.710	0.837
Random Forest	scaffold	0.793	0.631	0.790
SVM	random	0.813	0.677	0.849
SVM	scaffold	0.793	0.615	0.798

Balanced accuracy and ROC-AUC — the two metrics least sensitive to a shift in class balance between splits — drop under scaffold split for **every one of the three models**, by 0.06–0.08 in balanced accuracy and 0.04–0.05 in ROC-AUC. That is the random split systematically overestimating real-world generalization, exactly as Chapter 4 anticipated, now with a real, reproducible number attached rather than a forward reference.

Not every individual metric moves the same direction, and the one exception is worth naming rather than smoothing over: Random Forest’s recall is *higher* under scaffold split (0.941) than random split (0.931). The likely explanation is not that scaffold splitting somehow made the task easier for this one model and metric — it is that the two splits’ test sets have slightly different blocker fractions by construction (73.9% under random split vs. 76.2% under scaffold split, an artifact of which scaffold groups happened to land in each partition this run), and recall on the majority class is sensitive to that shift in a way balanced accuracy and ROC-AUC are designed not to be. This is itself the practical lesson: any single metric, evaluated on one split, can mislead; the reliable signal here is the consistent drop

across *balanced* metrics and across *all three* independently trained models, not any one number in isolation.

5.5 Hands-on Project: hERG Cardiotoxicity Classifier

The project code lives in this chapter’s folder (`ch05_qsar_admet/`) and implements the full pipeline referenced throughout this chapter: extract real hERG bioactivity data from ChEMBL, clean and label it, featurize with Chapter 2’s fingerprints, split it two ways, and train and evaluate a classifier.

On DeepChem

Chapter 4 stated that “DeepChem becomes the more direct, load-bearing tool starting in Chapter 5.” Building this chapter’s project, the specific intended use — DeepChem’s documented scaffold-splitting utility — was attempted first, and hit a real, reproducible problem: in this project’s environment, `import deepchem` unconditionally imports `deepchem.trans.transformers`, which unconditionally imports `tensorflow` at module load time, even though TensorFlow is not declared as an installation dependency and nothing in this project’s use case (a scaffold splitter) requires it. Installing a multi-gigabyte deep learning framework as a transitive, undeclared dependency of one utility function is a poor trade for a book whose own stated engineering principle (Chapter 4, Section 4.2) is using the most pedagogically direct tool for the job. Section 5.4’s scaffold split is therefore implemented directly against RDKit’s own `Scaffolds.MurckoScaffold` module — a well-documented, roughly twenty-line algorithm — rather than through DeepChem. This is a smaller-scope fulfillment of Chapter 4’s forward reference than “DeepChem becomes load-bearing,” and that gap is recorded here rather than papered over.

Extract and label

`herg_qsar.py` extracts IC50 bioactivity records for hERG (ChEMBL240) from the ChEMBL API, paginating exactly as Chapter 4’s `extract_bioactivities` does. Each record is standardized with Chapter 4’s technique (Section 4.1: largest fragment, uncharge, canonical tautomer) and duplicate measurements of the same standardized structure are merged via the median, as in Chapter 4. Each resulting compound is then labeled by a fixed threshold on its median IC50:

```
DEFAULT_THRESHOLD_NM = 10_000.0 # 10 uM
is_blocker = median_ic50_nm <= DEFAULT_THRESHOLD_NM
```

This 10 μM cutoff is a deliberate, explicit modeling choice for this project, not a value taken from a specific external paper’s stated methodology — that distinction matters, because the exact numeric threshold used for hERG “blocker” classification genuinely varies across the published literature, and claiming otherwise would misattribute a number this chapter chose. It is chosen as a round order-of-magnitude value broadly consistent with the qualitative safety-margin reasoning in Redfern et al. (2003, Section 5.2): this project’s

own dataset has a median IC₅₀ of 3.5 μ M, so 10 μ M sits at roughly the upper-middle of the observed potency range rather than at an extreme. Changing it is a one-line edit (`--threshold-nm`), and doing so changes both the label balance and, materially, how hard the resulting classification task is — worth trying directly rather than taking this chapter’s specific choice as definitive.

Featurize, split, train

Compounds are featurized with the same Morgan/ECFP4 fingerprint (`radius=2, fpSize=2048`) as Chapter 2, split with either `random_split` or `scaffold_split` (Section 5.4), and passed to one of the three classifiers from Section 5.3 via a single shared interface:

```
def train_and_evaluate(records, model_type="xgboost", split_type="scaffold",
                       use_smote=False, seed=0) -> dict:
    ...
```

so the same clean/featurize pipeline underlies every combination of model, split, and oversampling choice reported in Sections 5.3–5.4.

Running it and reading the output

```
cd ch05_qsar_admet
pip install -r requirements.txt
python herg_qsar.py --model xgboost --split both
```

Running the default configuration against the live ChEMBL API (or offline, deterministically, with `--use-cached-raw`) extracts 3,000 raw IC₅₀ records, cleans them to 1,765 unique standardized compounds (1,307 blockers / 458 non-blockers at the 10 μ M threshold), and reports, for both the random and scaffold split, exactly the metrics tabulated in Section 5.4 — XGBoost’s scaffold-split ROC-AUC of 0.803 against its random-split ROC-AUC of 0.843 — the single number this chapter’s narrative has been building toward since Chapter 4.

Reproducibility

Dependencies are version-floored (`rdkit>=2023.9.1, scikit-learn>=1.3, xgboost>=2.0, imbalanced-learn>=0.11` in `requirements.txt`, validated against 2026-era releases of each). `data/raw_herg_bioactivities_sample.json` bundles a real 3,000-record extract (fetched 2026-08-19) so the full clean $\rightarrow$ featurize $\rightarrow$ split $\rightarrow$ train $\rightarrow$ evaluate pipeline runs offline and deterministically with `--use-cached-raw`, following Chapter 4’s resilience pattern directly (the live ChEMBL connection did in fact time out once while this fixture was being prepared — an unprompted, real-world demonstration of exactly the failure mode that pattern exists to survive). The 28-test suite in `tests/test_herg_qsar.py` checks extraction, cleaning, labeling, featurization, both split strategies (including an explicit check that scaffold split never places the same scaffold in both train and test), and end-to-end training against both synthetic edge cases and a real slice of the bundled fixture. `pip install -r requirements-dev.txt && pytest` reproduces all 28 results.

Limitations and what comes next

Everything in this chapter operates on the same 2D representation Chapter 2 introduced, with the same blind spot flagged there: a fingerprint encodes molecular constitution, not the 3D shape a compound actually presents to hERG's binding pocket. Sanguinetti and Tristani-Firouzi's point about hERG specifically (Section 5.2) — that its unusually large, aromatic-lined pore accommodates structurally diverse binders through more than one plausible binding mode — is a concrete reason this class of liability may be harder for a fixed 2D fingerprint to fully capture than a more geometrically constrained target would be; Chapter 6's graph neural networks (learning a representation directly from structure rather than fixing one in advance) and Chapter 11's docking methods (explicit 3D binding-pose prediction) are both direct responses to this same limitation, applied to different parts of the pipeline. Separately, a fixed IC₅₀ threshold discards the concentration-dependent safety-margin reasoning Redfern et al. established (Section 5.2) — a regression model predicting IC₅₀ directly, combined with a compound's anticipated therapeutic concentration, is a more complete safety assessment than any single blocker/non-blocker classifier. And a scaffold split, while a substantially more honest estimate of generalization than a random split, is still evaluated against scaffolds already present in a fixed historical dataset — it is not a test against chemistry that does not yet exist, which is precisely the harder problem Chapter 7's generative models are evaluated against.

A note on Google Colab

rdkit, xgboost, and imbalanced-learn are not preinstalled on Colab's default runtime (scikit-learn, numpy, and requests are); run `!pip install rdkit xgboost imbalanced-learn` in the first cell. No GPU is required — training any of the three classifiers on ~1,400 compounds' worth of 2048-bit fingerprints completes in well under a minute on CPU.

References

- Van de Waterbeemd, H., & Gifford, E. (2003). ADMET in silico modelling: towards prediction paradise? *Nature Reviews Drug Discovery*, 2(3), 192–204. <https://doi.org/10.1038/nrd1032>
- Sanguinetti, M. C., & Tristani-Firouzi, M. (2006). hERG potassium channels and cardiac arrhythmia. *Nature*, 440(7083), 463–469. <https://doi.org/10.1038/nature04710>
- Redfern, W. S., Carlsson, L., Davis, A. S., Lynch, W. G., MacKenzie, I., Palethorpe, S., Siegl, P. K., Strang, I., Sullivan, A. T., Wallis, R., Camm, A. J., & Hammond, T. G. (2003). Relationships between preclinical cardiac electrophysiology, clinical QT interval prolongation and torsade de pointes for a broad range of drugs: evidence for a provisional safety margin in drug development. *Cardiovascular Research*, 58(1), 32–45. [https://doi.org/10.1016/s0008-6363\(02\)00846-5](https://doi.org/10.1016/s0008-6363(02)00846-5)
- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>

- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20(3), 273–297. <https://doi.org/10.1007/BF00994018>
- Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining* (pp. 785–794). <https://doi.org/10.1145/2939672.2939785>
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., & Duchesnay, É. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825–2830.
- Siramshetty, V. B., Nguyen, D. T., Martinez, N. J., Southall, N. T., Simeonov, A., & Zakharov, A. V. (2020). Critical assessment of artificial intelligence methods for prediction of hERG channel inhibition in the “Big Data” era. *Journal of Chemical Information and Modeling*, 60(12), 6007–6019. <https://doi.org/10.1021/acs.jcim.0c00884>
- Bemis, G. W., & Murcko, M. A. (1996). The properties of known drugs, part 1: Molecular frameworks. *Journal of Medicinal Chemistry*, 39(15), 2887–2893. <https://doi.org/10.1021/jm9602928>
- Yang, K., Swanson, K., Jin, W., Coley, C., Eiden, P., Gao, H., Guzman-Perez, A., Hopper, T., Kelley, B., Mathea, M., Palmer, A., Settels, V., Jaakkola, T., Jensen, K., & Barzilay, R. (2019). Analyzing learned molecular representations for property prediction. *Journal of Chemical Information and Modeling*, 59(8), 3370–3388. <https://doi.org/10.1021/acs.jcim.9b00237>
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16, 321–357. <https://doi.org/10.1613/jair.953>

See Chapter 1’s references for Mendez et al. (2019, ChEMBL) and Chapter 4’s references for Ramsundar et al. (2019, DeepChem’s own documented citation) — both reused here rather than re-listed. RDKit itself has no official journal publication; its maintainers’ recommended citation is “RDKit: Open-source cheminformatics. <https://www.rdkit.org>” (confirmed directly from the project’s own documentation), matching the convention established in Chapter 4’s references. The Pedregosa et al. (2011) scikit-learn citation is reproduced from the paper’s own metadata on jmlr.org, which lists no DOI for this article.

All dataset sizes, class balances, and model metrics cited in Sections 5.4–5.5 were computed directly by running `herg_qsar.py` against the live ChEMBL API on 2026-08-19, not taken from a secondary source — see `data/raw_herg_bioactivities_sample.json` and `herg_qsar.py` to reproduce.

Chapter 6: Graph Neural Networks (GNNs) in Chemistry

Chapter 2 represented a molecule three ways — a SMILES string, a fixed hashed fingerprint, and, briefly, the underlying graph itself (Section 2.3), noting that a fingerprint is “a fast, lossy fixed-size summary” of that graph and that “learning similarity directly from $(\mathbf{A}, \mathbf{X})$ with graph neural networks... Chapter 6 covers in full.” This is that chapter. Chapter 5’s classical models (Random Forest, SVM, XGBoost) all consumed a Chapter 2 fingerprint as a fixed input vector; every model in this chapter instead consumes the molecular graph directly, learning its own task-specific representation rather than having one fixed in advance.

6.1 Motivation for GNNs

A Morgan/ECFP4 fingerprint (Chapter 2, Section 2.2) is built by a fixed, non-learned algorithm: hash each atom’s local environment out to some radius, fold the results into a bit vector, done. The procedure never sees a training label, so it cannot allocate representational capacity toward whichever structural patterns actually matter for the property being predicted — every bit is spent identically regardless of whether the downstream task is solubility, toxicity, or binding affinity. Two concrete costs follow directly from this, both named explicitly in Chapter 2: hashing **collisions**, where chemically distinct environments alias to the same bit as the dictionary of possible environments competes for a fixed-width vector, and a hard **radius cutoff**, beyond which no information propagates no matter how relevant a distant substructure is to the property in question.

Duvenaud et al. (2015) framed the fix precisely: replace the fixed hashing step with a differentiable, learnable analogue of the same circular-fingerprint procedure — at each layer, aggregate information from each atom’s immediate neighbors, exactly as Morgan’s algorithm does, but with a neural network instead of a hash function, trained end-to-end against the actual prediction target. Stacking L such layers lets information reach L

bonds away, precisely as increasing an ECFP’s radius does, but where a fingerprint discards everything except the final hashed identifier, a GNN keeps every intermediate representation differentiable and lets gradient descent decide what those L rounds of neighborhood aggregation should actually encode for the task at hand. This is the core idea this chapter formalizes: **spatial neighborhood aggregation** — often called **message passing** — as a learnable generalization of the fixed circular fingerprints Chapter 2 built by hand.

Formally, recall Chapter 2’s notation (Section 2.3): a molecule is a graph $G = (V, E)$ with a node feature matrix $\mathbf{X} \in \mathbb{R}^{n \times d}$ (one row per atom) and an adjacency structure $\mathbf{A}$ recording which atoms are bonded. Every architecture in this chapter computes a sequence of updated node representations $\mathbf{H}^{(0)} = \mathbf{X}, \mathbf{H}^{(1)}, \dots, \mathbf{H}^{(L)}$, where $\mathbf{H}^{(l)}$ is produced from $\mathbf{H}^{(l-1)}$ and $\mathbf{A}$ by exactly one round of neighborhood aggregation — an operation this chapter now defines precisely, in three increasingly general forms.

6.2 GCN & GAT Architectures

Graph Convolutional Networks (GCN; Kipf & Welling, 2017) derive their update rule as a first-order approximation of spectral convolution on a graph, which reduces, per layer, to:

$$\mathbf{H}^{(l+1)} = \sigma(\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2} \mathbf{H}^{(l)} \mathbf{W}^{(l)}) \quad (8)$$

where $\tilde{\mathbf{A}} = \mathbf{A} + \mathbf{I}$ is the adjacency matrix with self-loops added (so a node’s own previous representation contributes to its update, not only its neighbors’), $\tilde{\mathbf{D}}$ is $\tilde{\mathbf{A}}$ ’s degree matrix, $\mathbf{W}^{(l)}$ is a learned weight matrix shared across every node in the graph, and σ is a nonlinearity (ReLU throughout this chapter’s implementation). The $\tilde{\mathbf{D}}^{-1/2} \tilde{\mathbf{A}} \tilde{\mathbf{D}}^{-1/2}$ term is a symmetric normalization: it averages each node’s neighbors rather than summing them raw, so a highly-connected atom (a ring fusion carbon) does not automatically dominate a sparsely-connected one (a terminal methyl) purely by virtue of degree. Every node’s update uses the *same* $\mathbf{W}^{(l)}$, which is what makes the layer’s parameter count independent of molecule size and lets one trained model apply to molecules of any size.

Graph Attention Networks (GAT; Veličković et al., 2018) replace the fixed, degree-based normalization above with *learned, asymmetric* attention weights: rather than deciding a priori that every neighbor should be averaged equally, GAT learns how much each neighbor should contribute to a given atom’s update, conditioned on both atoms’ current features. For neighbor j of atom i :

$$\alpha_{ij} = \frac{\exp(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_j]))}{\sum_{k \in \mathcal{N}(i)} \exp(\text{LeakyReLU}(\mathbf{a}^\top [\mathbf{W}\mathbf{h}_i \parallel \mathbf{W}\mathbf{h}_k]))}, \quad \mathbf{h}_i^{(l+1)} = \sigma \left(\sum_{j \in \mathcal{N}(i)} \alpha_{ij} \mathbf{W}\mathbf{h}_j^{(l)} \right) \quad (9)$$

where $\parallel$ denotes concatenation, $\mathbf{a}$ and $\mathbf{W}$ are learned, and $\mathcal{N}(i)$ is atom i ’s bonded neighborhood. The softmax over $\mathcal{N}(i)$ guarantees the attention weights for a given atom sum to 1, so this is still a weighted average of neighbor features — but the weights themselves are now a learned function of the two atoms’ features, letting the model up-weight a chemically important neighbor (say, a nearby heteroatom) and down-weight an unremarkable one,

rather than treating every bonded neighbor identically as plain GCN does. In practice, GAT layers use several independent attention heads in parallel (each with its own $\mathbf{a}$, $\mathbf{W}$) and concatenate or average their outputs — this chapter’s implementation uses 4 heads, each producing $\text{hidden_dim}/4$ features, concatenated back to hidden_dim .

Neither formulation above uses **edge features** — a GCN layer only sees which atoms are bonded, a GAT layer only sees the two bonded atoms’ own features, and neither consumes bond order, conjugation, or stereochemistry directly in its aggregation step (both architectures were originally developed for citation-network graphs, where “edges” carry no intrinsic feature beyond their existence). For a chemical graph, where bond type is often exactly the information that distinguishes an aromatic ring from a saturated one, that is a real limitation — and it is precisely what motivates Section 6.3’s architecture.

6.3 Message Passing Neural Networks (MPNN)

Gilmer et al. (2017) unify GCN, GAT, and a wide family of earlier graph-convolutional architectures — including Duvenaud et al.’s (2015) learnable fingerprint from Section 6.1 — under a single framework, **Message Passing Neural Networks**, defined by two learned functions applied for L rounds:

$$\mathbf{m}_i^{(l+1)} = \sum_{j \in \mathcal{N}(i)} M_l(\mathbf{h}_i^{(l)}, \mathbf{h}_j^{(l)}, \mathbf{e}_{ij}), \quad \mathbf{h}_i^{(l+1)} = U_l(\mathbf{h}_i^{(l)}, \mathbf{m}_i^{(l+1)}) \quad (10)$$

where M_l (the **message function**) computes a message from every bonded neighbor j using both atoms’ current representations *and* the bond features $\mathbf{e}_{ij}$ connecting them, the messages are aggregated (summed, here) into $\mathbf{m}_i^{(l+1)}$, and U_l (the **update function**) combines the aggregated message with atom i ’s own previous representation to produce its new one. GCN and GAT are both special cases: GCN’s M_l discards $\mathbf{e}_{ij}$ entirely and uses a fixed-degree-normalized weight; GAT’s M_l discards $\mathbf{e}_{ij}$ too but replaces the fixed weight with a learned attention coefficient. What makes this framework strictly more general — and the reason it is the architecture Section 6.5’s hands-on project builds — is that M_l can condition on $\mathbf{e}_{ij}$ directly, so bond order, conjugation, and stereochemistry participate in the aggregation instead of being invisible to it.

This chapter’s implementation uses Gilmer et al.’s **edge network** variant of M_l specifically, which PyTorch Geometric implements directly as `NNConv`: the message function is itself a small neural network, $M_l(\mathbf{h}_i, \mathbf{h}_j, \mathbf{e}_{ij}) = \mathbf{A}_l(\mathbf{e}_{ij}) \mathbf{h}_j$, where $\mathbf{A}_l$ is a learned function (a 2-layer MLP in this chapter’s code) mapping a bond’s features to an entire $\text{hidden_dim} \times \text{hidden_dim}$ weight matrix, applied to the neighbor’s representation. In other words, every distinct bond environment generates its *own* linear transformation of the message passed along it, rather than every bond in the graph sharing one fixed transform — concretely, an aromatic C=C bond and a saturated C-C bond propagate information differently, because the edge network produces a different weight matrix for each. The update function U_l in this chapter’s implementation is a plain residual-free ReLU ($\mathbf{h}_i^{(l+1)} = \sigma(\mathbf{m}_i^{(l+1)})$), matching PyG’s default `NNConv` behavior when composed with an activation) rather than the gated

recurrent unit (GRU) Gilmer et al.’s original paper uses for U_l , and this chapter’s readout is global mean pooling rather than their Set2Set readout — both are explicit simplifications, named here rather than silently substituted, made because the edge-conditioned message function is the architecturally distinctive contribution this section is teaching, and the simpler update/readout choices keep Section 6.5’s implementation legible without changing that core mechanism.

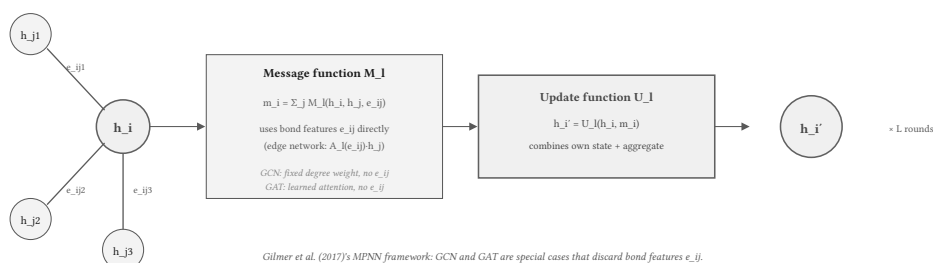


Figure 6.1. One message-passing round for atom i : the message function combines each bonded neighbor’s state with the bond features connecting them, messages are aggregated, and the update function produces atom i ’s new representation. GCN and GAT are special cases that discard bond features.

6.4 3D Equivariant Networks

Every architecture so far operates purely on the molecular graph’s *topology* — which atoms are bonded to which — with no reference to 3D geometry, echoing Chapter 2’s point (Section 2.4) that a graph fixes constitution but not shape. Many of the properties later chapters care about most — binding pose energetics (Chapter 11), force-field-level potential energy (Chapter 12) — are fundamentally properties of a specific 3D arrangement of atoms in space, not of the bond graph alone, which is what motivates a class of architectures that take atomic coordinates $\mathbf{r}_i \in \mathbb{R}^3$ as input alongside (or instead of) bond connectivity.

Building such an architecture naively is subtly wrong. A molecule’s physical properties do not depend on how it happens to be rotated, translated, or reflected in the coordinate frame a dataset happens to store it in — a solubility or a binding energy is the same number whether the input coordinates were rotated 90° first or not — but a generic neural network consuming raw (x, y, z) coordinates has no reason to respect that fact and will, in general, output different predictions for the identical molecule presented in a different orientation. **Invariance** and **equivariance** are the two precise properties an architecture can guarantee against this failure mode. For a transformation g (a rotation and/or translation, from the group $E(3)$ of rigid Euclidean motions, or its rotation-only subgroup $SE(3)$) acting on the input coordinates, a function f is:

- **Invariant** if $f(g \cdot \mathbf{r}) = f(\mathbf{r})$ for every g — the output is unchanged by the transformation. A predicted scalar property (solubility, an energy) *should* be invariant: it is a single number, and there is no meaningful sense in which rotating the input should change it.

- **Equivariant** if $f(g \cdot \mathbf{r}) = g \cdot f(\mathbf{r})$ — the output transforms *the same way* the input did, rather than staying fixed. A predicted force or velocity vector, which has its own direction in space, *should* be equivariant: rotate the molecule, and the predicted force vectors must rotate identically, or the prediction is physically inconsistent with itself.

A network with either property gets that consistency guaranteed by its architecture rather than approximated by hoping enough rotated training examples were seen — which matters directly for **data efficiency**: an invariant or equivariant model does not need to learn the same physics separately for every orientation a molecule might appear in during training, because the constraint is structural, not learned. Three concrete architectures illustrate the design space:

- **SchNet** (Schütt et al., 2018) is *invariant*: it represents each pairwise atomic distance $\|\mathbf{r}_i - \mathbf{r}_j\|$ (itself already rotation- and translation-invariant, since a distance between two points does not change under a rigid motion) through continuous, learned radial filters, and aggregates only scalar quantities throughout — so nothing equivariant ever needs to be tracked, at the cost of being unable to output a genuinely directional quantity like a force.
- **EGNN** (Satorras et al., 2021) is *equivariant*: alongside scalar node features, it maintains and updates the atomic coordinates themselves at every layer, using update rules built specifically so that rotating the input coordinates produces an identically rotated set of output coordinates, without requiring the heavier spherical-harmonic machinery some equivariant architectures use.
- **NequIP** (Batzner et al., 2022) is *equivariant* via that heavier route: it builds features that transform according to the irreducible representations of $O(3)$ (using spherical harmonics and tensor products), giving it a strictly larger space of equivariant functions to represent higher-order directional information — at real computational cost, in exchange for state-of-the-art data efficiency on interatomic-potential tasks (directly relevant to Chapter 12’s neural network potentials, which this same paper targets).

Section 6.5’s hands-on project does not implement a 3D-aware architecture: both ESOL and FreeSolv, as distributed through MoleculeNet, are 2D graphs with no bundled 3D conformer (generating one would require Chapter 2, Section 2.4’s ETKDG step, applied per molecule, which this chapter does not do), so GCN, GAT, and this chapter’s MPNN all operate on topology alone, consistent with Sections 6.1–6.3. This section is deliberately theory-only for that reason — the architectures above become directly load-bearing once 3D structure is actually on the table, starting with Chapter 9’s structure prediction and, especially, Chapter 12’s neural network potentials.

6.5 Hands-on Project: GNN Molecular Property Prediction

The project code lives in this chapter’s folder (ch06_gnn_molecular_properties/) and implements Sections 6.2–6.3’s architectures directly in PyTorch Geometric (Fey & Lenssen, 2019), built on PyTorch (Paszke et al., 2019), to predict two real physicochemical properties from the molecular graph alone.

A note on the second property's name

The outline this book follows describes this project as predicting “aqueous solubility and binding free energy.” The solubility half is exact: **ESOL** (Delaney, 2004) is 1,128 compounds’ measured aqueous solubility, expressed as log-molar solubility. The second dataset this project actually uses, however — **FreeSolv** (Mobley & Guthrie, 2014), 642 compounds — measures **hydration free energy**: the free energy change of transferring a molecule from vacuum into water. That is a real, well-defined, and widely used thermodynamic property, and it is the standard MoleculeNet regression benchmark conventionally paired with ESOL in exactly this kind of exercise — but it is not the same quantity as **protein-ligand binding free energy** (the free energy of a ligand binding a specific protein pocket), which depends on a specific 3D protein structure and is the subject of Chapter 11’s docking methods, not this chapter’s. Rather than mislabel FreeSolv’s actual property to match the outline’s phrase, or invent a protein-ligand dataset this chapter’s 2D-graph architectures are not equipped to use correctly (per Section 6.4, that would need explicit 3D structural input), this section uses FreeSolv under its real name and states the distinction plainly here.

Featurization and models

Both datasets are loaded through PyTorch Geometric’s built-in `MoleculeNet` class, which parses each compound’s SMILES with RDKit and featurizes it using the categorical atom/bond scheme introduced by the Open Graph Benchmark (Hu et al., 2020): 9 categorical features per atom (atomic number, chirality, degree, formal charge, attached-hydrogen count, radical electron count, hybridization, aromaticity, ring membership) and 3 per bond (bond type, stereochemistry, conjugation). `gnn_property.py`’s `AtomEncoder` and `BondEncoder` turn these categorical indices into dense vectors by summing one learned embedding per feature — a standard, simple encoding scheme that keeps every downstream layer working with ordinary continuous vectors:

```
class AtomEncoder(nn.Module):
    def __init__(self, hidden_dim):
        super().__init__()
        self.embeddings = nn.ModuleList(
            nn.Embedding(dim, hidden_dim) for dim in NODE_FEATURE_DIMS
        )

    def forward(self, x):
        return sum(emb(x[:, i]) for i, emb in enumerate(self.embeddings))
```

`GCNRegressor`, `GATRegressor`, and `MPNNRegressor` each stack 3 message-passing layers (`GCNConv`, `GATConv`, and `NNConv` respectively — Sections 6.2-6.3) of width 64 over these encoded atom features, mean-pool the final atom representations into one graph-level vector (`global_mean_pool`), and pass that through a small 2-layer MLP to a single scalar prediction. `MPNNRegressor` additionally runs every bond’s encoded features through `BondEncoder` and into each `NNConv` layer’s edge network, exactly as Section 6.3 describes.

Splitting, training, and evaluation

`gnn_property.py` reimplements Chapter 5’s `scaffold_split` (Bemis & Murcko, 1996) for this regression setting — group compounds by generic ring scaffold, then greedily fill the training set scaffold-group by scaffold-group until the target training fraction is reached — alongside a stratification-free random split, so the same random-vs-scaffold comparison Chapter 5 demonstrated for hERG classification can be repeated here for property regression. Targets are standardized (zero mean, unit variance, computed from the training split only) before training and un-standardized before computing metrics, a routine but necessary step for stable regression training that has no analogue in Chapter 5’s classification setting. Models train for 60 epochs with the Adam optimizer (learning rate 10^{-3}) minimizing mean squared error on the standardized target, evaluated on the held-out test set by root-mean-squared error (RMSE) and R^2 , both in the property’s original units.

Results: GCN vs. GAT vs. MPNN on ESOL

Training all three architectures on ESOL with an identical protocol (60 epochs, hidden dimension 64, 3 layers, seed 0) gives:

Model	Split	RMSE (log mol/L)	MAE	R^2
GCN	random	1.011	0.789	0.787
GCN	scaffold	1.246	0.968	0.635
GAT	random	0.794	0.626	0.869
GAT	scaffold	1.229	0.937	0.645
MPNN	random	0.809	0.626	0.864
MPNN	scaffold	1.537	1.196	0.445

Two findings, both real and both worth stating plainly rather than rounding off to fit a tidier narrative. First, the pattern from Chapter 5 repeats exactly: every architecture’s R^2 drops substantially under scaffold split relative to random split (GCN: $0.787 \rightarrow 0.635$; GAT: $0.869 \rightarrow 0.645$; MPNN: $0.864 \rightarrow 0.445$) — random splitting overestimates real-world generalization for graph-structured molecular data regardless of which specific architecture consumes the graph, which is exactly the general lesson Chapter 5 established with fingerprints and classical models. Second, and less tidy: **MPNN, the architecturally most expressive of the three, generalizes worst under scaffold split** — worse than the simpler GCN and GAT, despite matching or beating them under random split. The edge-conditioned message function (NNConv’s edge network) gives MPNN substantially more parameters per layer than GCN’s shared weight matrix or GAT’s attention vector, on a training set of only 902 molecules; more capacity without more data is a standard recipe for overfitting, and scaffold split — by construction — is the evaluation that most exposes overfitting to the training set’s specific chemical series. This is not evidence that MPNNs are generally worse than GCNs for molecular property prediction (Section 6.3’s citations, and the wider literature, show the opposite in larger-data settings); it is evidence that architectural expressiveness and generalization under distribution shift are separate axes, and that a fair benchmark has to check both rather than assume the more sophisticated model wins by default.

Results: MPNN on FreeSolv

Running the same MPNN, unmodified, on FreeSolv (this project’s second required property, per the note above):

Split	RMSE (kcal/mol)	MAE	R^2
random	1.612	1.127	0.845
scaffold	4.272	3.220	0.304

The random-vs-scaffold gap is even more pronounced here than on ESOL — RMSE nearly triples ($1.61 \rightarrow 4.27$ kcal/mol) and R^2 falls from 0.845 to 0.304. FreeSolv is both smaller (642 compounds total, so a scaffold split leaves fewer training examples per scaffold group to learn from) and chemically broader in origin — it was assembled for the SAMPL blind prediction challenges specifically to stress-test free-energy methods across diverse chemotypes (Mobley & Guthrie, 2014) — both of which make genuine cross-scaffold generalization harder than on ESOL’s more homogeneous compound set, consistent with the larger gap observed here.

Reproducibility

Dependencies are version-floored (`torch>=2.2`, `torch_geometric>=2.5`, `rdkit>=2023.9.1` in [requirements.txt](#), validated against torch 2.13.0 and torch_geometric 2.8.0.post1). This chapter’s `data/delaney-processed.csv` and `data/SAMPL.csv` are the real, unmodified raw MoleculeNet CSVs for ESOL and FreeSolv, bundled so `--use-cached-raw` seeds PyTorch Geometric’s dataset cache directly instead of downloading from the live DeepChem S3 bucket — the same resilience pattern Chapter 4 established. The 21-test suite in [tests/test_gnn_property.py](#) checks the atom/bond encoders, all three model architectures’ forward and backward passes, both split strategies (including an explicit no-scaffold-leakage check), and end-to-end training against a small synthetic molecule set and a real fixture subset, running in seconds; one test calls the live MoleculeNet download directly. `pip install -r requirements-dev.txt && pytest` reproduces all 21 results. The full 60-epoch runs behind the tables above are slower (roughly 2-3 minutes per split on one CPU core) and are not run in the test suite for that reason — reproduce them directly with `python gnn_property.py --use-cached-raw --dataset esol --model mpnn --split both`, substituting `--dataset freesolv` for the second table.

Limitations and what comes next

Everything this chapter’s project measures is still, ultimately, an average over one training run at one fixed random seed — the tables above report a single seed (0) throughout, for direct comparability across models and splits, not because run-to-run variance is known to be small; a careful benchmark would repeat each configuration across several seeds and report a spread, which this chapter’s code supports (`--seed`) but its reported tables do not exercise. More fundamentally, every architecture in this chapter, including the hands-on project’s MPNN, operates on 2D molecular topology alone — Section 6.4’s equivariant architectures exist precisely because many properties later chapters care about are not topology-alone properties at all. Chapter 7’s generative models build directly on the graph representations this chapter introduced, now used to *produce* novel molecular graphs rather than only score

existing ones; Chapters 8 and 9 extend graph- and attention-based architectures from small molecules to proteins, at far larger scale; and Chapter 12’s neural network potentials are, architecturally, direct descendants of Section 6.4’s equivariant networks, applied to the 3D, dynamics-aware regime this chapter’s project deliberately did not enter.

A note on Google Colab

Colab’s default runtime preinstalls `torch` but not `torch_geometric` or `rdkit`; run `!pip install torch_geometric rdkit` in the first cell. A GPU runtime accelerates training automatically if selected but is not required — every result in this chapter was produced on CPU.

References

- Duvenaud, D., Maclaurin, D., Aguilera-Iparraguirre, J., Gómez-Bombarelli, R., Hirzel, T., Aspuru-Guzik, A., & Adams, R. P. (2015). Convolutional networks on graphs for learning molecular fingerprints. *Advances in Neural Information Processing Systems*, 28 (NeurIPS 2015). arXiv:1509.09292 (no DOI; NeurIPS proceedings papers from this era were not assigned one).
- Kipf, T. N., & Welling, M. (2017). Semi-supervised classification with graph convolutional networks. *5th International Conference on Learning Representations (ICLR 2017)*. arXiv:1609.02907 (no DOI; ICLR, via OpenReview, does not assign one).
- Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). Graph attention networks. *6th International Conference on Learning Representations (ICLR 2018)*. arXiv:1710.10903 (no DOI, as above).
- Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., & Dahl, G. E. (2017). Neural message passing for quantum chemistry. *Proceedings of the 34th International Conference on Machine Learning*, PMLR 70, 1263-1272 (no DOI; PMLR does not assign one).
- Schütt, K. T., Sauceda, H. E., Kindermans, P.-J., Tkatchenko, A., & Müller, K.-R. (2018). SchNet - A deep learning architecture for molecules and materials. *The Journal of Chemical Physics*, 148(24), 241722. <https://doi.org/10.1063/1.5019779>
- Satorras, V. G., Hoogeboom, E., & Welling, M. (2021). E(n) equivariant graph neural networks. *Proceedings of the 38th International Conference on Machine Learning*, PMLR 139, 9323-9332 (no DOI, as above).
- Batzner, S., Musaelian, A., Sun, L., Geiger, M., Mailoa, J. P., Kornbluth, M., Molinari, N., Smidt, T. E., & Kozinsky, B. (2022). E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials. *Nature Communications*, 13(1), 2453. <https://doi.org/10.1038/s41467-022-29939-5>
- Delaney, J. S. (2004). ESOL: Estimating aqueous solubility directly from molecular structure. *Journal of Chemical Information and Computer Sciences*, 44(3), 1000-1005. <https://doi.org/10.1021/ci034243x>
- Mobley, D. L., & Guthrie, J. P. (2014). FreeSolv: a database of experimental and calculated hydration free energies, with input files. *Journal of Computer-Aided Molecular Design*, 28(7), 711-720. <https://doi.org/10.1007/s10822-014-9747-x>
- Hu, W., Fey, M., Zitnik, M., Dong, Y., Ren, H., Liu, B., Catasta, M., & Leskovec, J. (2020). Open graph benchmark: Datasets for machine learning on graphs. *Advances in Neural Information Processing Systems*, 33 (NeurIPS 2020). arXiv:2005.00687 (no DOI, as above).

- Fey, M., & Lenssen, J. E. (2019). Fast graph representation learning with PyTorch Geometric. *ICLR 2019 Workshop on Representation Learning on Graphs and Manifolds*. arXiv:1903.02428 (no DOI, as above).
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., & Chintala, S. (2019). PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32 (NeurIPS 2019). arXiv:1912.01703 (no DOI, as above).

See Chapter 4’s references for Wu et al. (2018, MoleculeNet, the benchmark suite ESOL and FreeSolv are distributed through) and Bemis & Murcko (1996, scaffold definition, reused from Chapter 5) — both reused here rather than re-listed. Every citation above lacking a DOI was checked directly against its own official landing page (arXiv abstract page or the PMLR proceedings page) rather than assumed absent; ICLR (via OpenReview), PMLR, and NeurIPS proceedings from these years routinely do not assign DOIs, matching the same situation already documented for the Pedregosa et al. (2011) scikit-learn citation in Chapter 5’s references.

All dataset sizes and model metrics cited in Section 6.5 were computed directly by running `gnn_property.py` against the bundled ESOL/FreeSolv fixtures on 2026-08-20, not taken from a secondary source or from published benchmark leaderboards — see `data/delaney-processed.csv`, `data/SAMPL.csv`, and `gnn_property.py` to reproduce.

Chapter 7: Generative AI & De Novo Drug Design

Every model in Chapters 5 and 6 solves the **forward problem**: given a molecule, predict a property — hERG liability (Chapter 5), aqueous solubility (Chapter 6). This chapter solves the **inverse problem**: given a desired property, produce a molecule. Sanchez-Lengeling and Aspuru-Guzik (2018), surveying the field’s shift from screening to design, frame the distinction precisely: forward models are cheap to evaluate but only ever score candidates someone already thought to propose, while inverse (generative) models search the space of possible structures directly, using a forward model’s predictions as the signal that tells the search where to go. This chapter builds exactly that pairing: Chapter 5’s reward-oracle methodology, reapplied to a new target, becomes the compass a generative model uses to explore chemical space it was never shown during pretraining.

7.1 Inverse Molecular Design

A generative model for molecules faces a constraint forward models never do: its output has to be a real, buildable chemical structure, not just a scored one. Sanchez-Lengeling and Aspuru-Guzik (2018) organize inverse design methods along the representation they generate in (strings, graphs, or 3D geometries — Sections 7.2-7.4 cover one generative *model class* for each) and note that every one of them inherits the same three-part success criterion, later formalized as a standard benchmark by Brown et al. (2019) in GuacaMol: a generated output is only useful if it is

- **valid** — parses to a real molecule under the representation’s own rules (a SMILES string with balanced rings and legal valences; a SELFIES string, per Chapter 2, is valid *by construction*),
- **novel** — not simply a memorized copy of a training-set molecule, and

- **synthetically accessible** — buildable by a medicinal chemist, not merely valid on paper.

These three criteria are in tension with each other by design: a model that only ever reproduces training molecules scores perfectly on validity and synthesizability but contributes nothing novel; a model that ignores the training distribution can generate valid nonsense — structures RDKit parses without complaint but no chemist would attempt to synthesize. Section 7.6’s hands-on project reports all three axes — validity, novelty (the fraction of valid generations whose canonical SMILES is absent from the training set, plus a separate uniqueness figure for redundancy within one batch of generations), and Lipinski compliance as a coarse synthesizability-adjacent proxy — for exactly this reason: optimizing one in isolation is not the goal.

7.2 Latent Space Models

Variational Autoencoders (VAEs) approach generation by learning a continuous latent space that a decoder can sample from. An encoder $q_\phi(z | x)$ maps a molecule x to a distribution over a latent vector z ; a decoder $p_\theta(x | z)$ reconstructs a molecule from z ; and training jointly minimizes reconstruction error and the KL divergence between $q_\phi(z | x)$ and a fixed prior, usually $\mathcal{N}(0, I)$:

$$\mathcal{L}(\theta, \phi; x) = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x | z)] - \text{KL} (q_\phi(z | x) \| p(z)) \quad (11)$$

The KL term is what makes the latent space usable for generation, not just compression: it pulls every training molecule’s encoded distribution toward the same shared prior, so that after training, a z sampled directly from $\mathcal{N}(0, I)$ — with no corresponding input molecule at all — decodes to something the decoder has plausibly seen a similar-looking (nearby-in- z) example of during training, rather than landing in an unexplored, poorly-decoded region of latent space. Gómez-Bombarelli et al. (2018) built the first molecular VAE along exactly these lines, encoding and decoding SMILES strings directly, and demonstrated a second capability forward-only models cannot offer at all: because z is continuous, gradient-based optimization *within the latent space* — moving z in the direction a separately-trained property predictor’s gradient suggests — can search for molecules with a target property, entirely by decoding points the model was never explicitly shown.

Their original decoder, working directly in SMILES, paid the exact cost Chapter 2 (Section 2.1) described: a decoder has to implicitly learn SMILES’ syntax rules (balanced ring-closure digits, valid branch nesting) well enough to avoid generating strings that fail to parse at all, and a sampled latent point decoded through an imperfectly-trained model has no guarantee of doing so. Pairing a molecular VAE with **SELFIES** (Krenn et al., 2020) instead — Chapter 2’s forward reference to this chapter — removes that failure mode structurally rather than statistically: because every string SELFIES’ grammar can produce decodes to a valid molecule, a decoder emitting SELFIES tokens cannot produce a syntactically invalid output

no matter how undertrained it is, which is exactly the “100% validity” property Chapter 2 named.

7.3 Sequence-based Auto-regressive Models

An alternative to encoding a whole molecule into one latent vector at once is to generate it one token at a time, autoregressively: $p(x) = \prod_{t=1}^T p(x_t | x_{<t})$. The **Transformer** (Vaswani et al., 2017) is the now-standard architecture for this — a stack of self-attention layers that, for generation, must be made **causal**: token t ’s representation is computed attending only to tokens $1, \dots, t$, never to tokens after it, so that at generation time — where tokens $t + 1, \dots, T$ genuinely do not exist yet — the model is never relying on information it will not actually have. Section 7.6’s generator is built directly on this design: a stack of causally-masked self-attention layers over embedded SELFIES tokens, trained by next-token cross-entropy exactly as $p(x_t | x_{<t})$ above specifies.

Two named molecular applications of Transformers illustrate two different jobs the same underlying architecture can do, and it matters which one a given model performs. **ChemBERTa** (Chithrananda et al., 2020) is *encoder*-style (following BERT): trained with masked-token prediction on 77 million PubChem SMILES, it produces representations useful for downstream property prediction (a Chapter 5/6-style forward task), not for generating new molecules — it has no autoregressive generation mechanism at all. (This is also the one citation in this chapter that is not published in a peer-reviewed venue as far as verifiable directly from its own listing — an arXiv preprint, cited here because the outline names it specifically, with that status disclosed rather than implied otherwise.) **MolGPT** (Bagal et al., 2022), by contrast, is *decoder*-style (following GPT): trained with exactly the causal next-token objective above, it generates molecules token by token, optionally conditioned on target properties. Section 7.6’s generator follows MolGPT’s decoder-only design, not ChemBERTa’s encoder-only one — the distinction is not cosmetic, since only the former can generate anything at all.

7.4 3D Diffusion Models

Sections 7.2-7.3 both generate a molecular graph or string — Chapter 2’s 2D constitution, in Chapter 6’s terms — with no reference to 3D shape. **Diffusion models** extend generation into 3D directly: a forward process progressively adds Gaussian noise to a molecule’s atomic coordinates (and, in the joint formulations, its atom types) over T steps until they are indistinguishable from noise, and a neural network is trained to reverse that process step by step, learning to denoise. Sampling then starts from pure noise and runs the learned reverse process to produce a full 3D structure.

Because atomic coordinates are what is being generated, the denoising network faces exactly the consistency requirement Chapter 6 (Section 6.4) formalized: a molecule’s

identity does not depend on the arbitrary orientation its coordinates happen to be stored in, so the denoiser must be **equivariant** — rotate the noisy input, and the predicted noise (equivalently, the denoised structure) must rotate identically. Hooeboom et al. (2022) built exactly this — Equivariant Diffusion for Molecules (EDM) — using an E(3)-equivariant network architecture in the role Chapter 6 described SchNet, EGNN, and NequIP filling for property prediction, here repurposed as the reverse-diffusion denoiser instead. This is a direct, concrete example of Chapter 6’s closing point that its equivariant architectures “become directly load-bearing once 3D structure is actually on the table” — diffusion is one of the tables it gets used on.

Section 7.6’s hands-on project does not implement a 3D diffusion model, for the same reason Chapter 6’s equivariant networks stayed theory-only: EGFR’s real bioactivity data, as retrieved from ChEMBL, is 2D structures (SMILES) with no bundled 3D conformers, and training a diffusion model that actually needs 3D training targets is out of this chapter’s scope. Chapter 11’s DiffDock (Corso et al., 2023, covered there in full) is the closest relative actually implemented in this book — a diffusion model over docking *poses* specifically, generating a ligand’s position and orientation relative to a fixed protein rather than generating novel ligand connectivity from scratch, a narrower and distinct problem from full 3D de novo generation.

7.5 Reinforcement Learning (RL)

Pretraining (Sections 7.2-7.3) teaches a generative model to imitate a training distribution — sample from a pretrained EGFR-focused generator, and it produces things that look like the EGFR-active compounds it was shown. It has no mechanism for producing things that score *better* than that training distribution on a target property, because nothing in a maximum-likelihood pretraining objective ever asks it to. **Reinforcement learning** closes that gap by treating sequence generation itself as a decision process: the policy is the generative model π_θ , an “action” is a complete generated sequence, and a scalar **reward** $R(x)$ — computed after the fact, from whatever downstream property actually matters — replaces the training-set likelihood as the training signal.

REINFORCE (Williams, 1992) is the foundational policy-gradient estimator for this setting:

$$\nabla_\theta J(\theta) = \mathbb{E}_{x \sim \pi_\theta} [R(x) \nabla_\theta \log \pi_\theta(x)] \quad (12)$$

— increase the log-probability of sequences in proportion to how much reward they earned, decrease it for sequences that earned little or none, estimated from Monte Carlo samples drawn from the policy itself rather than requiring the reward to be differentiable at all. That last property is exactly why RL, rather than ordinary gradient descent, is the right tool here: Section 7.6’s reward — a trained classifier’s predicted probability, composed with a discrete Lipinski pass/fail — is not a differentiable function of the generator’s parameters, so back-

propagating “reward” directly through the sampling step is not an option; REINFORCE’s log-derivative trick is what makes optimizing it possible at all. A well-known practical refinement, used in this chapter’s implementation, subtracts a **baseline** (here, a running mean of recent rewards) from $R(x)$ before scaling the gradient: since $\mathbb{E}_{x \sim \pi_\theta}[b \nabla_\theta \log \pi_\theta(x)] = 0$ for any x -independent b , this leaves the gradient estimator unbiased while substantially reducing its variance, because the learning signal now reflects whether a sample did *better or worse than recently typical*, not its raw, noisier reward magnitude.

Proximal Policy Optimization (PPO; Schulman et al., 2017) is the modern, more stable alternative used throughout contemporary RL practice (including large language model fine-tuning): it clips the policy update to prevent any single step from moving π_θ too far from the policy that generated the current batch of samples, permitting multiple gradient steps per batch of sampled data where REINFORCE affords only one. Section 7.6 implements REINFORCE specifically, not PPO — a deliberate scope decision, named here rather than substituted silently: REINFORCE’s update is a direct, few-line translation of the equation above, keeping the connection between theory and code legible, at the cost of PPO’s improved sample efficiency and training stability. Olivecrona et al. (2017) is the direct precedent for applying this exact idea — REINFORCE fine-tuning of a pretrained sequence generator — to molecule design specifically, optimizing an RNN-based SMILES generator against predicted bioactivity; Section 7.6 follows the same fine-tune-a-pretrained-generator-with-a-learned-reward structure, substituting a Transformer generator over SELFIES tokens for their RNN over SMILES characters.

Multi-objective reward composition — this section’s other named topic — is handled in Section 7.6 by the simplest approach available: a weighted sum, $R(x) = w_1 \cdot P(\text{active} | x) + w_2 \cdot \mathbb{1}[\text{Lipinski pass}]$, combining a bioactivity term (the Chapter 5-style reward oracle, Section 7.6) with an ADMET-proxy term (Chapter 1’s Lipinski’s Rule of Five, reused directly rather than reimplemented). Olivecrona et al. (2017) use a comparably simple composition for the same reason it is used here: a weighted sum is transparent about the trade-off it encodes (the weights *are* the trade-off, stated explicitly rather than left implicit) even though it cannot represent every possible preference over the two objectives — more sophisticated multi-objective RL formulations exist but add complexity this chapter’s scope does not require.

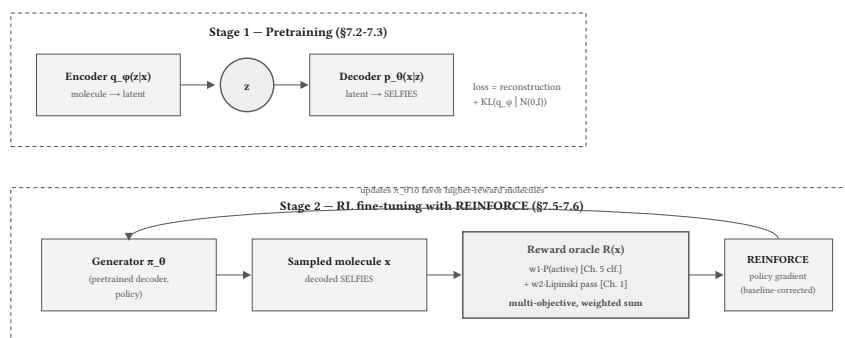


Figure 7.1. Two-stage generative pipeline: pretraining trains an encoder/decoder with reconstruction plus KL-divergence loss; RL fine-tuning then samples from the pretrained generator, scores each molecule with a multi-objective reward oracle, and updates the generator via REINFORCE’s policy gradient.

7.6 Hands-on Project: Generative Transformer + RL for De Novo EGFR Inhibitor Design

The project code lives in this chapter’s folder (`ch07_generative_transformer_rl/`) and implements the full pipeline this chapter has been building toward: pretrain a Transformer generator on real EGFR-active compounds, then fine-tune it with REINFORCE against a reward built from a Chapter 5-style bioactivity oracle and Chapter 1’s Lipinski’s Rule of Five.

Data and the reward oracle

`gen_transformer.py` extracts real IC50 bioactivity records for EGFR (ChEMBL203 — the running target since Chapter 1’s `fetch_data.py` and Chapter 3’s `structural_features.py`) from the ChEMBL API, using the same extraction, standardization (Chapter 4, Section 4.1), and median-deduplication pipeline as Chapters 4-5. Each compound is labeled active/inactive at a 1 μ M IC50 cutoff — a deliberate, explicit modeling choice for this project specifically, in the same spirit as (and using the same threshold-selection reasoning as) Chapter 5’s hERG threshold, not a value taken from a specific external paper’s methodology.

A `RewardOracle` — an XGBoost classifier on Chapter 2’s ECFP4 fingerprints, structurally identical to Chapter 5’s `herg_qsar.py` model — is trained on this data and evaluated with a Bemis-Murcko scaffold split (Chapters 5-6) rather than a random split, so its reported accuracy is an honest estimate of how well it generalizes to chemical series it was not trained on — exactly the property that matters here, since its job is to score molecules the generator invents that were, by construction, never in its own training set either.

The generator

Active compounds’ SMILES are converted to SELFIES (Krenn et al., 2020) and tokenized into a vocabulary built directly from the symbols actually observed in the corpus (Section 7.2-7.3’s representation choice, made concrete). `GenerativeTransformer` is a small decoder-

only Transformer (Section 7.3): token and learned positional embeddings, 4 causally-masked self-attention layers (`d_model=128`, 4 heads), and a linear output head producing next-token logits over the vocabulary — `torch.nn.TransformerEncoder` with a causal attention mask, which is architecturally a Transformer *decoder* in the generation sense Section 7.3 defines (no cross-attention to a separate encoder sequence exists to attend to, since this is unconditional, not sequence-to-sequence, generation).

```
class GenerativeTransformer(nn.Module):
    def forward(self, ids):
        x = self.token_embedding(ids) + self.position_embedding(pos)
        causal_mask = torch.triu(torch.full((T, T), float("-inf")), diagonal=1)
        x = self.blocks(x, mask=causal_mask, is_causal=True)
        return self.head(self.ln_out(x))
```

The model is pretrained by ordinary next-token cross-entropy (Section 7.3) on the active-compound corpus, then sampled from and RL fine-tuned via `reinforce_finetune` (Section 7.5): each iteration samples a batch of sequences with gradient-tracked log-probabilities (`sample_with_logprobs`), decodes and scores them (`score_sequences` — SELFIES to SMILES, RDKit parse, oracle probability, Lipinski check), and takes one REINFORCE policy-gradient step against a running-mean baseline.

Running it and reading the output

```
cd ch07_generative_transformer_rl
pip install -r requirements.txt
python gen_transformer.py --use-cached-raw
```

Running the default configuration (offline, deterministic, seed 0) against the bundled fixture cleans 3,000 raw IC50 records to 1,508 unique standardized compounds (860 active / 648 inactive at the 1 μ M threshold). The reward oracle reaches **84.1% accuracy / 0.896 ROC-AUC** on a 302-compound scaffold-split held-out set (1,206 train) — a strong enough signal to be worth optimizing against, and, per Chapter 5’s own lesson, one whose accuracy is being reported honestly rather than inflated by a random split. The generator is pretrained for 20 epochs on the resulting 842-compound active corpus (vocabulary size 42), reaching a final cross-entropy loss of 0.787.

Sampling 200 molecules from the pretrained-only policy, before any RL:

Metric	Before RL	After 25 REINFORCE iterations
Valid fraction	1.000	1.000
Unique fraction (of valid)	1.000	0.960
Novel fraction (of valid)	0.990	0.970
Mean predicted P(EGFR-active)	0.120	0.405
Lipinski-compliant fraction	0.995	1.000
Mean reward	0.383	0.583

Three things are worth reading precisely here, not just the headline number. First, **validity never moves, because it cannot** — every sampled sequence decodes through SELFIES’ guaranteed-valid grammar (Section 7.2), so 100% validity is a property of the representation, not a result RL achieved; the number is reported to make that point concretely rather than to claim credit for it. Second, **mean predicted activity more than triples** ($0.120 \rightarrow 0.405$) after only 25 REINFORCE iterations, with novelty and uniqueness both still above 96% — the generator is not collapsing onto a handful of memorized or repeated high-reward outputs, at least not within this run’s iteration budget, though 25 iterations is a modest budget and this is not a guarantee that stays true with substantially longer fine-tuning (see Limitations, below). Third, **Lipinski compliance also improves** ($0.995 \rightarrow 1.000$) even though it was already near-ceiling before RL — a small, genuine effect of including it in the reward at all, not a large discovery.

Reproducibility

Dependencies are version-floored (`torch>=2.2`, `rdkit>=2023.9.1`, `xgboost>=2.0`, `selfies>=2.1` in `requirements.txt`, validated against torch 2.13.0, rdkit 2026.3.5, xgboost 3.4.1, and selfies 2.1.1). `data/raw_egfr_bioactivities_sample.json` bundles a real 3,000-record extract (fetched 2026-08-20) so the full pipeline runs offline and deterministically with `--use-cached-raw`, following Chapters 4-5’s resilience pattern. The results above took 13 minutes on a single CPU core, dominated by structure standardization and the RL loop’s autoregressive sampling — this implementation recomputes the full causal attention forward pass at every generated token rather than caching key/value projections from previous steps, an explicit, disclosed simplification that keeps the sampling code’s connection to Section 7.3’s equations direct, at a real, measured cost in wall-clock time. The 25-test suite in `tests/test_gen_transformer.py` checks data cleaning, SELFIES tokenization round-trips, the reward oracle, both the plain and gradient-tracked sampling paths, and a short, synthetic-molecule end-to-end pretrain-then-RL run. Most individual tests are fast; the suite as a whole takes about 3 minutes, dominated by the handful of tests and the one CLI subprocess check that standardize a real slice of the bundled fixture and fit a fresh reward oracle rather than reusing one — module-scoped pytest fixtures share that work across tests where possible, but the CLI check necessarily runs as its own process and repeats it. One

test calls the live ChEMBL API directly. `pip install -r requirements-dev.txt && pytest` reproduces all 25 results.

Limitations and what comes next

This implementation cuts several corners relative to a production RL molecule-design pipeline, each named here rather than left implicit. The reward is only as good as the oracle producing it — an 84.1% accurate classifier, not ground truth — so a high reward is evidence a molecule is *predicted* active, not evidence it actually would be; a sufficiently RL-optimized generator can, in principle, learn to exploit oracle blind spots rather than genuinely useful chemistry, a failure mode the literature calls reward hacking, which this chapter’s short, 25-iteration fine-tuning run does not run long enough to clearly exhibit but does not structurally prevent either. Olivecrona et al. (2017) and most production RL-for-generation pipelines add a KL-divergence penalty against the pretrained policy specifically to slow this kind of drift; this implementation omits it, trading some robustness against longer fine-tuning runs for a simpler, more directly auditable REINFORCE loss (Section 7.5). And every molecule this pipeline proposes is still, in the end, a 2D structure scored by a 2D-fingerprint classifier — it says nothing about whether the molecule would actually bind EGFR’s ATP pocket in the 3D sense Chapter 3 characterized structurally, which is precisely the question Chapter 11’s docking methods, and the true protein-ligand binding affinities they estimate, are built to answer. Generated candidates from a pipeline like this one are a hypothesis worth docking and, eventually, synthesizing and testing — not a finished answer.

A note on Google Colab

Colab’s default runtime preinstalls `torch` but not `rdkit`, `xgboost`, or `selfies`; run `!pip install rdkit xgboost selfies` in the first cell. No GPU is required, though training will run faster on one if selected.

References

- Sanchez-Lengeling, B., & Aspuru-Guzik, A. (2018). Inverse molecular design using machine learning: Generative models for matter engineering. *Science*, 361(6400), 360–365. <https://doi.org/10.1126/science.aat2663>
- Gómez-Bombarelli, R., Wei, J. N., Duvenaud, D., Hernández-Lobato, J. M., Sánchez-Lengeling, B., Sheberla, D., Aguilera-Iparraguirre, J., Hirzel, T. D., Adams, R. P., & Aspuru-Guzik, A. (2018). Automatic chemical design using a data-driven continuous representation of molecules. *ACS Central Science*, 4(2), 268–276. <https://doi.org/10.1021/acscentsci.7b00572>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is all you need. *Advances in Neural Information Processing Systems*, 30 (NeurIPS 2017). arXiv:1706.03762 (no DOI; NeurIPS proceedings papers from this era were not assigned one).
- Chithrananda, S., Grand, G., & Ramsundar, B. (2020). ChemBERTa: Large-scale self-supervised pretraining for molecular property prediction. arXiv:2010.09885. Unlike this

chapter’s other citations, this is an arXiv preprint with no confirmed peer-reviewed publication as of this writing, verified directly against arXiv’s own listing rather than assumed — cited because the outline names it specifically, with that status disclosed here rather than implied otherwise.

- Bagal, V., Aggarwal, R., Vinod, P. K., & Priyakumar, U. D. (2022). MolGPT: Molecular generation using a Transformer-decoder model. *Journal of Chemical Information and Modeling*, 62(9), 2064–2076. <https://doi.org/10.1021/acs.jcim.1c00600>
- Hoogeboom, E., Garcia Satorras, V., Vignac, C., & Welling, M. (2022). Equivariant diffusion for molecule generation in 3D. *Proceedings of the 39th International Conference on Machine Learning*, PMLR 162, 8867–8887 (no DOI; PMLR does not assign one).
- Corso, G., Stärk, H., Jing, B., Barzilay, R., & Jaakkola, T. (2023). DiffDock: Diffusion steps, twists, and turns for molecular docking. *11th International Conference on Learning Representations (ICLR 2023)*. arXiv:2210.01776 (no DOI, as above) — introduced here only as a forward reference; covered in full in Chapter 11.
- Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4), 229–256. <https://doi.org/10.1007/BF00992696> (the original journal article — not the same-titled 1992 book-chapter reprint under a different DOI, checked directly against Crossref before use here to avoid citing the wrong record).
- Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). Proximal policy optimization algorithms. arXiv:1707.06347 (no DOI, as above).
- Olivecrona, M., Blaschke, T., Engkvist, O., & Chen, H. (2017). Molecular de-novo design through deep reinforcement learning. *Journal of Cheminformatics*, 9(1), 48. <https://doi.org/10.1186/s13321-017-0235-x>
- Brown, N., Fiscato, M., Segler, M. H. S., & Vaucher, A. C. (2019). GuacaMol: Benchmarking models for de novo molecular design. *Journal of Chemical Information and Modeling*, 59(3), 1096–1108. <https://doi.org/10.1021/acs.jcim.8b00839>

See Chapter 1’s references for Lipinski et al. (2001, Rule of Five) and Mendez et al. (2019, ChEMBL); Chapter 2’s for Krenn et al. (2020, SELFIES); and Chapter 5’s for Bemis & Murcko (1996, scaffold definition) — all reused here rather than re-listed. RDKit itself has no official journal publication; its maintainers’ recommended citation is “RDKit: Open-source cheminformatics. <https://www.rdkit.org>” (confirmed directly from the project’s own documentation), matching the convention established in Chapter 4’s references.

All dataset sizes, oracle metrics, and generation statistics cited in Section 7.6 were computed directly by running `gen_transformer.py` against the bundled EGFR fixture on 2026-08-20, not taken from a secondary source — see `data/raw_egfr_bioactivities_sample.json` and `gen_transformer.py` to reproduce.

Chapter 8: Protein Language Models (pLMs) & Evolutionary Dynamics

Chapters 1-7 worked almost entirely with small molecules — SMILES strings, molecular graphs, generated ligands. Part III turns to the other half of this book’s title: macromolecules. A protein sequence is, formally, a string over a 20-letter alphabet, and Chapter 3 (Section 3.1) already introduced the classical tools for reading meaning out of that string — substitution matrices, pairwise and multiple sequence alignment. This chapter asks a different question: instead of hand-designing what a sequence’s letters mean, can a model *learn* it, purely from being shown enormous numbers of real protein sequences and asked to predict masked-out pieces of them? The answer — yes, well enough to be scientifically useful with zero task-specific training — is what makes **protein language models** (pLMs) the foundation for every remaining chapter in Part III: Chapter 9’s structure predictors and Chapter 10’s protein designers both build directly on the representations this chapter introduces.

8.1 The Evolutionary Scale Hypothesis

Anfinsen’s (1973) thermodynamic hypothesis — that a protein’s amino acid sequence alone determines its native 3D structure, with no additional biological machinery required — is the foundational premise of computational structural biology: everything Chapters 3, 9, and 10 do rests on it being true. It says the information is *there*, encoded in the sequence. It says nothing about how to decode it.

Evolution has been running that decoding experiment for billions of years. Every sequence in a modern protein family is a descendant of some ancestral sequence, filtered through natural selection: mutations that broke folding or function were eliminated, mutations that were neutral or beneficial persisted. A **multiple sequence alignment** (MSA) of a protein family — Chapter 3, Section 3.1’s tool, revisited here as promised — makes this filtering visible as statistical structure: positions critical to folding or function stay highly conserved across millions of years of divergence, while tolerant positions drift freely; pairs of positions that are physically in contact tend to **co-evolve**, because a destabilizing substitution at one is often compensated by a correlated substitution at the other. Classical unsupervised methods exploit exactly this signal from a *single* family’s MSA: EVcouplings-style direct coupling analysis infers residue-residue contacts from co-evolving pairs, and Riesselman et al.’s (2018) DeepSequence trains a variational autoencoder on one family’s alignment to model that family’s implicit fitness landscape, reporting zero-shot mutation-effect predictions competitive with the individual assays it evaluates against — without ever seeing experimental labels.

The **evolutionary scale hypothesis** — the premise this chapter’s namesake models are built on — is a scale-up of that same idea, with one structural change: instead of an MSA of one family (typically thousands of sequences, aligned to each other’s exact positions), train a single self-supervised model on hundreds of millions of *unaligned* sequences spanning the entire observed protein universe at once. Devlin et al. (2019) introduced the training objective this depends on for language generally — **masked language modeling** (MLM): corrupt a fraction of input tokens by replacing them with a special mask token, and train the model to reconstruct them from context,

$$\mathcal{L}_{\text{MLM}}(\theta) = -\mathbb{E}_{x \sim \mathcal{D}} \sum_{i \in M} \log p_{\theta}(x_i \mid x_{\setminus M}) \quad (13)$$

where M is the randomly chosen set of masked positions in sequence x and $x_{\setminus M}$ denotes the full sequence with those positions replaced by the mask token. Applied to protein sequences with the 20-letter amino acid alphabet in place of a natural-language vocabulary, Rives et al. (2021) showed this objective, trained at sufficient scale (250 million UniParc sequences; up to 650 million parameters), causes structural and functional information to emerge as a *side effect* of the reconstruction task — with no structural label, no alignment, and no family-specific input ever supplied — measurable directly in the trained model’s internal representations (Section 8.3 makes this concrete: masking a real position and reading off the model’s implied preferences over the 20 amino acids at that position is precisely the reconstruction task the model was trained on, applied at inference time as a scoring tool). This is the sense in which “evolutionary scale” replaces “one family’s MSA” as the source of selective-pressure information: not because cross-family evolutionary signal stops mattering, but because a big enough corpus of raw sequences turns out to contain it implicitly, recoverable without ever constructing an alignment.

8.2 pLM Families

ESM-1b (Rives et al., 2021) is a 650M-parameter Transformer *encoder* (bidirectional self-attention, unlike Chapter 7’s causal decoder — every position attends to every other position, masked ones included, which is exactly what reconstructing a masked position from its full surrounding context requires) trained with the MLM objective above on UniRef50. **ESM-2** (Lin et al., 2023) is the direct successor used throughout this chapter’s hands-on project: architecturally the same encoder design, with three concrete changes — rotary positional embeddings in place of ESM-1b’s learned absolute ones, a training-data switch to UniRef50 with UniRef90 sequence sampling for diversity, and, most consequentially, a scale sweep spanning eight model sizes from 8M to 15B parameters, trained on an identical recipe so that model size is isolated as the one varying factor. Lin et al. (2023) report that structure-prediction accuracy (measured via ESMFold, the structure head trained on top of ESM-2’s representations — Chapter 9’s subject) improves smoothly and predictably with scale across this entire sweep, with no sign of saturating even at 15B parameters; Section 8.4 checks whether the same scale-dependence shows up in this chapter’s much smaller zero-shot mutation-scoring task, across the 8M/35M/150M range that fits comfortably in a free Colab CPU runtime.

ProtTrans (Elnaggar et al., 2022) takes a parallel path to the same goal: rather than one architecture, it trains and benchmarks several established NLP language model architectures — BERT, ALBERT, Electra, T5, among others — on protein sequences (UniRef and BFD, up to 393 billion residues), each adapted only by swapping the vocabulary from natural-language tokens to amino acids. ProtBERT, its BERT-architecture member, is the most widely used single output of that effort, sharing ESM-1b’s encoder-plus-MLM design but trained with a different data mixture and codebase. The three families make the same underlying bet — self-supervised reconstruction at scale extracts biologically meaningful structure — with different choices of architecture, training corpus, and (for ESM-2 specifically) an explicit, deliberately isolated study of scale itself; Section 8.4 uses ESM-2 because its published scale sweep is exactly what a scale-comparison hands-on project needs.

8.3 Biological Embeddings

A trained pLM offers two distinct things a downstream task can consume, and this chapter’s hands-on project uses both.

Masked-marginal scoring. Because the MLM objective already trains the model to output $p_{\theta}(x_i \mid x_{\setminus\{i\}})$ — a full probability distribution over the 20 amino acids at any position, given everything else — that same forward pass, run once at inference time with no gradient updates at all, can be turned directly into a zero-shot mutation-effect predictor. Meier et al. (2021) formalize this as the **masked-marginal** method: to score a substitution at position i from wild-type residue x_i^{wt} to mutant residue x_i^{mut} , mask position i in the wild-

type sequence, run one forward pass, and take the log-odds ratio between the model’s two resulting log-probabilities,

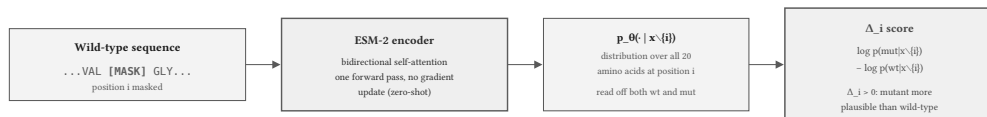
$$\Delta_i(x^{\text{mut}}) = \log p_\theta(x_i^{\text{mut}} \mid x_{\setminus\{i\}}) - \log p_\theta(x_i^{\text{wt}} \mid x_{\setminus\{i\}}) \quad (14)$$

A positive Δ_i means the model finds the mutant residue *more* plausible than the wild-type one at that position, given everything else in the sequence — evolutionary/structural plausibility, not a direct physical measurement, but one Meier et al. (2021) show correlates with real experimental effects (stability, binding, catalytic activity — the “function” named in this chapter’s title) across dozens of published deep mutational scanning (DMS) datasets, with no per-dataset fine-tuning. Because every mutant at a given position shares the exact same masked forward pass, scoring every possible substitution at a position costs one forward pass, not one per substitution — Section 8.4’s implementation makes this explicit.

Embeddings as features. The same forward pass also exposes each layer’s per-residue hidden-state vectors — the model’s internal, continuous representation of “what this residue means in this context,” pooled (commonly by averaging across the sequence length, Section 8.4’s choice) into one fixed-length vector per protein when a single summary representation is wanted. Unlike masked-marginal scoring, which requires no downstream training at all, embeddings are typically consumed as *features*: frozen or fine-tuned inputs to a separately trained predictor (a small regression head, most commonly) for a task the base pLM was never directly trained on. Section 8.4’s project uses raw embedding distance itself, without a trained head, as a second, independent zero-shot signal — does moving further from the wild-type embedding, in the representation space the model learned, track with a *larger* functional effect, regardless of the effect’s direction?

Both angles generalize past the single case Section 8.4 runs end to end. Meier et al.’s (2021) own benchmark spans binding, stability, and fitness assays across dozens of proteins; Tsuboyama et al. (2023) applied essentially the same masked-marginal principle at much larger scale specifically for **thermal stability** ($\Delta\Delta G$ of folding, not binding), mega-scale-experimentally measuring over 775,000 folding stability values for small domains and their point mutants — the natural larger-scale extension of the “stability” half of this chapter’s title, cited here as a pointer for exactly that reason rather than re-run in Section 8.4, whose scope stays fixed on one complete, tractable mutant population end to end.

Finally, embeddings are not confined to sequence-level tasks. Chapter 9 picks the same ESM-2 representations back up for a different purpose entirely: ESMFold (Lin et al., 2023, introduced in Section 8.2) attaches a structure-prediction head directly on top of them, producing 3D coordinates from a single sequence with no MSA search step at all — the structural payoff of Section 8.1’s “no alignment required” premise, in contrast to AlphaFold2 (Jumper et al., 2021), which achieves its own state-of-the-art accuracy by explicitly building and processing an MSA inside the model (its Evoformer block), the architectural road this chapter’s family of models chose not to take.



Meier et al. (2021): one masked forward pass scores every possible substitution at position i .

Figure 8.1. Masked-marginal mutation scoring: position i is masked, one ESM-2 forward pass gives a probability distribution over all 20 amino acids there, and the log-odds ratio between mutant and wild-type probabilities gives the mutation-effect score.

8.4 Hands-on Project: Zero-Shot Single-Point Mutation Effect Prediction with ESM-2

The project code lives in this chapter’s folder (`ch08_protein_language_models/`) and puts Section 8.3’s two scoring methods to a direct, real test: given only a pretrained ESM-2 checkpoint and a wild-type sequence — **no GB1-specific training of any kind** — how well do its zero-shot scores track real, experimentally measured mutation effects?

Data: the complete GB1 single-mutant population

GB1, the B1 immunoglobulin-binding domain of Streptococcal protein G, is a 56-residue domain whose binding to the Fc region of IgG is one of the most extensively mutationally characterized protein interactions in the literature. Olson et al. (2014) developed a yeast-display, deep-sequencing assay to measure IgG-Fc binding fitness for variants at four specific positions (39, 40, 41, 54); Wu et al. (2016) applied it to the full combinatorial space of all 20 amino acids at all four positions simultaneously — $20^4 = 160,000$ possible variants — producing, among the double/triple/quadruple mutants their paper’s own epistasis analysis focuses on, the complete population of every possible **single**-point mutant at these four positions: $4 \times 19 = 76$ variants, plus the wild type itself. That completeness matters for this project specifically: `esm_variant_effect.py`’s hands-on result below is not computed from a sample of single mutants, it is computed from literally all of them, so there is no sampling variance between “the 76 mutants used here” and “the single-mutant population.”

`esm_variant_effect.py` downloads this dataset from the FLIP benchmark’s public redistribution (Dallago et al., 2021) of Wu et al.’s (2016) original data, parses each variant’s 4-letter code against the wild-type code (VDGV) to recover the mutated position, wild-type residue, and mutant residue, and extracts the 56-residue GB1 domain sequence itself (the FLIP CSV’s sequence column appends an unrelated ~200-residue yeast-display fusion tag after position 56, stripped here since it plays no role in GB1’s own fold or binding site).

```
def _row_to_mutant(row: dict) -> SingleMutant | None:
    variant_code = row["Variants"]
    if variant_code == GB1_WT_VARIANT:
        return None
    diffs = [i for i in range(4) if variant_code[i] != GB1_WT_VARIANT[i]]
    idx = diffs[0]
    position = GB1_MUTATED_POSITIONS[idx]
    ...
```

Scoring: masked marginals and embedding distance

ESM2VariantScorer wraps a Hugging Face transformers ESM-2 checkpoint and implements both of Section 8.3's methods. Masked-marginal scoring exploits the position-sharing efficiency named there directly: GB1's 76 single mutants touch only 4 distinct positions, so `score_single_mutants` masks and scores each of those 4 positions exactly once — not once per mutant — reusing each position's resulting 20-amino-acid log-probability distribution for every mutant that shares it.

```
def score_single_mutants(self, wt_sequence, mutants):
    positions = sorted({m.position for m in mutants})
    log_probs_by_position = {
        pos: self.masked_marginal_log_probs(wt_sequence, pos) for pos in
positions
    }
    return np.array([
        log_probs_by_position[m.position][m.mut_aa] -
log_probs_by_position[m.position][m.wt_aa]
        for m in mutants
    ])
```

`embed_sequences` runs one forward pass per full mutant sequence (76 of them, plus wild type — mean-pooling excludes the `<cls>`/`<eos>` special tokens explicitly rather than averaging over them) and `evaluate_embedding_perturbation` compares each mutant's cosine distance from the wild-type embedding against its experimental fitness deviation from 1.0 (wild-type-normalized fitness).

Running it and reading the results

```
cd ch08_protein_language_models
pip install -r requirements.txt
python esm_variant_effect.py --use-cached-raw
```

Run against three ESM-2 checkpoints spanning roughly a 20x parameter range (8M / 35M / 150M), scoring all 76 single mutants with both methods, Spearman rank correlation against real experimental fitness came out as follows:

Model	Params	Masked- marginal ρ ($n=76$)	p-value	Embedding- distance ρ	p-value
ESM-2 8M	7.5M	0.095	0.415	0.111	0.339
ESM-2 35M	33.5M	0.184	0.112	0.087	0.453
ESM-2 150M	148.1M	0.208	0.072	0.314	0.0057

Two things are worth reading precisely here, not just the top line. First, **masked-marginal correlation increases monotonically with model scale** (0.095 $\rightarrow$ 0.184 $\rightarrow$ 0.208) — directionally consistent with Lin et al.’s (2023) scale-dependent accuracy trend (Section 8.2) — but at $n = 76$, none of the three individually clears the conventional $p < 0.05$ significance threshold; the 150M model comes closest ($p = 0.072$). This is reported as what it is: a suggestive trend, not a statistically confirmed one, at this sample size. Second, the **embedding-distance method tells a sharper story at the largest scale tested**: the 150M model’s correlation between representational shift and functional deviation ($\rho = 0.314$, $p = 0.0057$) is the one result in this table that clears significance outright, while the two smaller models do not approach it. Restricting either analysis to the 27-mutant high-read-count subset (`keep=True` in the source data, a sequencing-depth quality flag, not a fitness-reliability one) produces correlations scattered close to zero at every scale — an expected consequence of cutting the sample size by nearly two-thirds, not a contradiction of the full-population result above; it is not treated as a separate finding here.

As a single concrete example: mutant IDGV (position 39, wild-type valine to isoleucine) has real experimental fitness 1.446 — nearly 45% *better* binding than wild type — and the 150M model’s masked-marginal score for V $\rightarrow$ I at position 39 is positive, correctly ranking it above wild type; not every one of the 76 mutants is ranked this cleanly, which is exactly what a $\rho \approx 0.2$ correlation, rather than $\rho \approx 1.0$, means in practice.

Why is the correlation modest rather than strong? GB1’s assay measures one specific, engineered property — IgG-Fc binding affinity — at four co-varied positions chosen because mutating them tunes that specific interface, not because they are representative of general sequence conservation. A masked-marginal score reflects general evolutionary/ structural plausibility learned from the training corpus at large; it is not specifically trained to predict *this* binding interaction, and Meier et al. (2021) report exactly this kind of heterogeneity across their own multi-assay benchmark — some DMS datasets correlate strongly with zero-shot pLM scores, others, including affinity-tuning assays at a small number of interface positions, considerably less so. The honest lesson Section 8.4 is built to teach is this one: zero-shot pLM scoring is a genuinely useful, free, no-training-required signal, but not a universal oracle — validating it against real data for the specific property at hand, as done here, is the point, not an afterthought.

Reproducibility

Dependencies are version-floored (`torch>=2.2`, `transformers>=4.40`, `scipy>=1.10` in `requirements.txt`, validated against `torch 2.13.0` and `transformers 5.15.1` on Python 3.12). `data/gb1_single_mutants_sample.csv` bundles the real, complete wild-type + 76-single-mutant population (extracted from the live FLIP download on 2026-08-20), so the pipeline runs offline and deterministically with `--use-cached-raw` — verified directly: `--use-cached-raw` and a fresh live download of the full 149,361-row dataset produce bit-identical Spearman correlations, since both extract the same 77 rows. All three ESM-2 checkpoints total a few hundred MB on first download from the Hugging Face Hub; with them cached, the full run (3 models x 2 methods x 76 mutants) takes about a minute on a single CPU core, no GPU required. The 16-test suite in `tests/test_esm_variant_effect.py` checks data parsing against the real bundled fixture (including one spot-checked published fitness value), the download function against a mocked in-memory zip, and both scoring methods' mechanics — including the position-sharing efficiency claim above, checked directly by counting calls — against a tiny, untrained ESM architecture built from the real ESM-2 tokenizer vocabulary, so the suite never downloads a pretrained checkpoint and completes in under a minute, fully offline. `pip install -r requirements-dev.txt && pytest` reproduces all 16 results.

Limitations and what comes next

This project deliberately stays inside the single-family, single-assay case: one 56-residue domain, one binding readout, exhaustively enumerated at only 4 positions. Extending it to genome-scale variant effect prediction (Section 8.3's forward reference to Tsuboyama et al., 2023) or to positions outside this specific interface would need a different DMS dataset entirely, not a code change — the pipeline's data-loading layer is specific to this CSV's schema, again a deliberate scope decision named here rather than left implicit. The masked-marginal method also stays firmly zero-shot throughout: nothing in this chapter fits a regression head on top of ESM-2 embeddings against GB1 labels, which — per Section 8.3 — is the more common way embeddings get used in practice and would very likely outperform the zero-shot scores reported above, at the cost of needing labeled training data for the specific property being predicted, which is exactly the dependency zero-shot scoring is valuable for not requiring. Finally, everything in this chapter operates on sequence alone; Chapter 9 picks these same ESM-2 representations back up to predict the 3D structures those sequences fold into.

A note on Google Colab

Colab's default runtime preinstalls `torch` but not `transformers` or `scipy`; run `!pip install transformers scipy` in the first cell. No GPU is required for the model sizes used here (8M-150M parameters); larger ESM-2 checkpoints (650M and up) would benefit from one.

References

- Anfinsen, C. B. (1973). Principles that govern the folding of protein chains. *Science*, 181(4096), 223-230. <https://doi.org/10.1126/science.181.4096.223>
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Volume 1, 4171-4186. <https://doi.org/10.18653/v1/N19-1423>
- Riesselman, A. J., Ingraham, J. B., & Marks, D. S. (2018). Deep generative models of genetic variation capture the effects of mutations. *Nature Methods*, 15(10), 816-822. <https://doi.org/10.1038/s41592-018-0138-4>
- Rives, A., Meier, J., Sercu, T., Goyal, S., Lin, Z., Liu, J., Guo, D., Ott, M., Zitnick, C. L., Ma, J., & Fergus, R. (2021). Biological structure and function emerge from scaling unsupervised learning to 250 million protein sequences. *Proceedings of the National Academy of Sciences*, 118(15), e2016239118. <https://doi.org/10.1073/pnas.2016239118>
- Lin, Z., Akin, H., Rao, R., Hie, B., Zhu, Z., Lu, W., Smetanin, N., Verkuil, R., Kabeli, O., Shmueli, Y., dos Santos Costa, A., Fazel-Zarandi, M., Sercu, T., Candido, S., & Rives, A. (2023). Evolutionary-scale prediction of atomic-level protein structure with a language model. *Science*, 379(6637), 1123-1130. <https://doi.org/10.1126/science.ade2574>
- Elnaggar, A., Heinzinger, M., Dallago, C., Rehawi, G., Wang, Y., Jones, L., Gibbs, T., Feher, T., Angerer, C., Steinegger, M., Bhowmik, D., & Rost, B. (2022). ProtTrans: Toward understanding the language of life through self-supervised learning. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 44(10), 7112-7127. <https://doi.org/10.1109/tpami.2021.3095381>
- Meier, J., Rao, R., Verkuil, R., Liu, J., Sercu, T., & Rives, A. (2021). Language models enable zero-shot prediction of the effects of mutations on protein function. *Advances in Neural Information Processing Systems*, 34 (NeurIPS 2021). Preprint: <https://doi.org/10.1101/2021.07.09.450648> (no separate NeurIPS proceedings DOI was found for this paper as of this writing, verified directly against Crossref rather than assumed; the bioRxiv preprint DOI is cited instead, its status disclosed here rather than implied otherwise).
- Tsuboyama, K., Dauparas, J., Chen, J., Laine, E., Mohseni Behbahani, Y., Weinstein, J. J., Mangan, N. M., Ovchinnikov, S., & Rocklin, G. J. (2023). Mega-scale experimental analysis of protein folding stability in biology and design. *Nature*, 620(7973), 434-444. <https://doi.org/10.1038/s41586-023-06328-6>
- Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., Zidek, A., Potapenko, A., Bridgland, A., Meyer, C., Kohl, S. A. A., Ballard, A. J., Cowie, A., Romera-Paredes, B., Nikolov, S., Jain, R., Adler, J., ... Hassabis, D. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873), 583-589. <https://doi.org/10.1038/s41586-021-03819-2> — introduced here only as a forward reference and architectural contrast; covered in full in Chapter 9.
- Olson, C. A., Wu, N. C., & Sun, R. (2014). A comprehensive biophysical description of pairwise epistasis throughout an entire protein domain. *Current Biology*, 24(22), 2643-2651. <https://doi.org/10.1016/j.cub.2014.09.072>

- Wu, N. C., Dai, L., Olson, C. A., Lloyd-Smith, J. O., & Sun, R. (2016). Adaptation in protein fitness landscapes is facilitated by indirect paths. *eLife*, 5, e16965. <https://doi.org/10.7554/eLife.16965>
- Dallago, C., Mou, J., Johnston, K. E., Wittmann, B. J., Bhatnagar, N., Yang, J., Amini, A., Kim, P. M., & Yang, K. K. (2021). FLIP: Benchmark tasks in fitness landscape inference for proteins. Preprint: <https://doi.org/10.1101/2021.11.09.467890> (no separate peer-reviewed-venue DOI was found for this paper as of this writing; its bioRxiv preprint DOI is cited, status disclosed as above).

See Chapter 7’s references for Vaswani et al. (2017, the Transformer architecture underlying every pLM in this chapter). RDKit, DeepChem, and the ChEMBL/PDB APIs used in Chapters 1-7 play no role in this chapter; ESM-2’s model weights and tokenizer are distributed through the Hugging Face Hub ([facebook/esm2_t6_8M_UR50D](https://huggingface.co/facebook/esm2_t6_8M_UR50D), [facebook/esm2_t12_35M_UR50D](https://huggingface.co/facebook/esm2_t12_35M_UR50D), [facebook/esm2_t30_150M_UR50D](https://huggingface.co/facebook/esm2_t30_150M_UR50D)), and the GB1 dataset is redistributed by the FLIP benchmark under an AFL-3 license (original data CC BY 4.0 via eLife), both confirmed directly from their respective repositories rather than assumed.

All correlations, p-values, and the worked IDGV example cited in Section 8.4 were computed directly by running `esm_variant_effect.py` against the live-downloaded GB1 dataset on 2026-08-20, not taken from a secondary source, and independently reproduced against the bundled offline fixture — see `data/gb1_single_mutants_sample.csv` and `results/esm2_gb1_results.json` to reproduce.

Chapter 9: The 3D Structure Prediction Revolution

Chapter 8 asked what a protein language model’s internal representations are worth on their own — zero-shot mutation scoring, no 3D coordinates anywhere in sight. This chapter asks the question those representations were, historically, built toward: given only a sequence, can a model output the actual 3D structure that sequence folds into, at experimental accuracy? Since 2020, the answer has gone from “not usefully” to “often, yes” — the single largest discontinuity in this book’s subject matter, and the reason Part III exists as three chapters (9, 10, 11) rather than one. This chapter covers the mechanics of *how* that became possible (Sections 9.1–9.3) and puts real programmatic structure prediction to work end to end (Section 9.4): folding two real proteins, validating one against its real crystal structure, and testing whether the model’s own confidence score means what it claims to.

9.1 AlphaFold2 & AlphaFold3 Mechanics

AlphaFold2 (Jumper et al., 2021 — introduced in Chapter 3 §3.2 as a forward reference, and cited by Chapter 8 §8.3 as the architectural contrast to ESM-2’s alignment-free design; both threads converge here) reformulated structure prediction as two coupled problems solved jointly: extracting evolutionary and geometric signal from a multiple sequence alignment (Chapter 8 §8.1’s classical starting point), and using that signal to directly output 3D atomic coordinates, with no intermediate contact-map-then-fold pipeline of the kind that dominated the field before it.

The Evoformer. Given a query sequence, AlphaFold2 first builds an MSA the ordinary way (Chapter 3 §3.1’s tool) and initializes two representations from it: an **MSA representation** (one row per homologous sequence, one column per alignment position) and a **pair representation** (one entry per residue-residue pair of the query, initialized from

pairwise sequence features). The Evoformer is a stack of blocks that repeatedly updates both representations and exchanges information between them: row-wise and column-wise self-attention over the MSA representation extract co-evolutionary signal (the same conserved-position and co-evolving-pair patterns named in Chapter 8 §8.1, extracted here explicitly from an alignment rather than learned implicitly from a pretraining corpus); an outer-product-mean operation folds updated MSA information into the pair representation; and a set of **triangular update** operations act on the pair representation alone, updating the edge between residues i and j using information from a third residue k — structured so the three edges of every residue triple (i, j, k) are updated consistently with each other, which Jumper et al. (2021) motivate as encoding something close to a geometric triangle inequality directly into the pair representation’s learned updates. After 48 such blocks, the pair representation carries a rich, geometrically consistent residue-residue relationship map — not yet 3D coordinates, but the information a 3D structure would need to be consistent with.

Invariant Point Attention (IPA). Converting that pair representation into actual atomic coordinates is the **structure module**’s job, and IPA is its core operation. Every residue carries a local reference frame $T_i = (R_i, \mathbf{t}_i) \in SE(3)$ — a rotation and translation placing that residue’s idealized local backbone geometry into the structure’s global coordinate frame, initialized identically for every residue (a “black hole” starting guess with no structure yet) and iteratively refined block by block. IPA computes attention not just over the usual scalar query/key features, but also over **3D query and key points** — vectors predicted in each residue’s own local frame, mapped into the shared global frame via each residue’s current T_i before their pairwise distances are computed. An attention logit between residues i and j combines three terms: an ordinary scalar dot product, a bias read directly from the Evoformer’s pair representation (the geometric information computed in the previous paragraph, now consumed rather than produced), and a penalty growing with the distance between residue i ’s and j ’s 3D query/key points once both are placed in the global frame. Because every point is generated in a local frame and only ever compared *after* applying the residues’ own current rigid-body transforms, IPA’s output is provably invariant to any global rotation or translation applied to the whole structure — the network never has to learn “north” is meaningless, that invariance is built into the operation itself. Each IPA block updates every T_i a little further toward self-consistency; after enough blocks, applying the final frames to a fixed set of idealized atomic coordinates per residue produces the predicted structure.

AlphaFold3. Abramson et al. (2024) replace the structure module described above with a **diffusion model**: rather than iteratively refining rigid per-residue frames, AlphaFold3 starts from atomic coordinates corrupted with Gaussian noise and learns to iteratively denoise them back to a physically valid structure, conditioned on a simplified Evoformer-style trunk (renamed the “Pairformer,” with the MSA processing trimmed down). The practical payoff of that architectural change is generality: because diffusion over raw atom coordinates does not depend on the twenty-amino-acid-specific idealized backbone

geometry IPA’s frame-and-fixed-atoms construction assumes, the same model architecture natively handles nucleic acids, ligands, ions, and covalent modifications as first-class inputs, alongside protein chains — Section 9.3 returns to exactly this capability. AlphaFold3 is discussed here for architectural contrast only; nothing in this chapter’s hands-on project runs it.

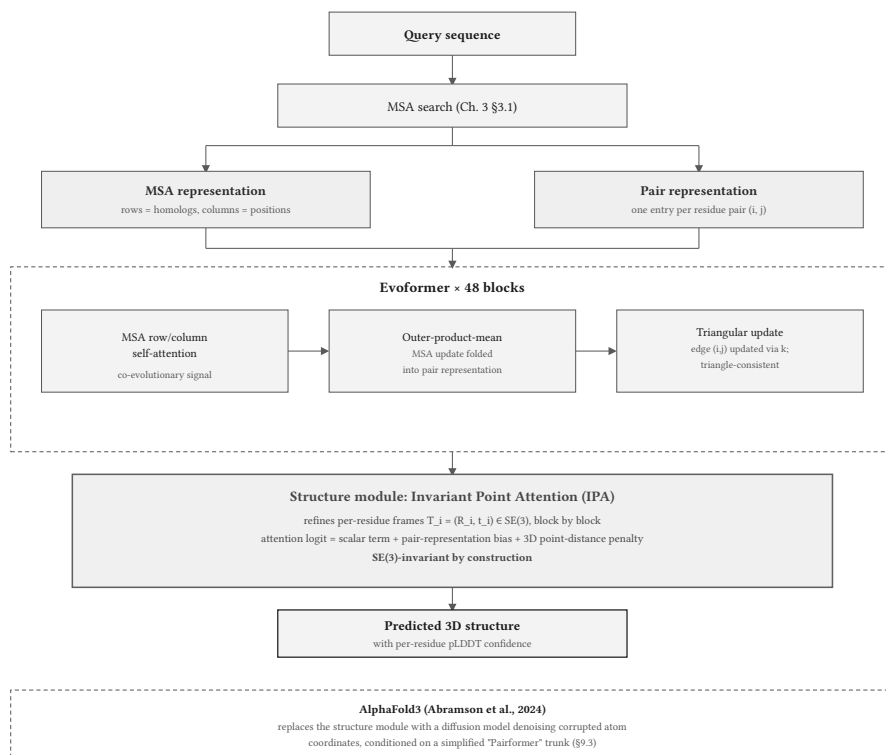


Figure 9.1. AlphaFold2’s Evoformer and Invariant Point Attention structure module described above, contrasted with AlphaFold3’s diffusion-based replacement.

9.2 ESMFold & RoseTTAFold

Both models below solve the same coordinate-prediction problem as Section 9.1, with one shared, consequential difference: neither performs an MSA search at inference time.

RoseTTAFold (Baek et al., 2021) predates the specific architecture above but shares its core insight of coupling multiple representations: a **three-track network** that processes 1D (sequence), 2D (pairwise distance/orientation), and 3D (explicit atomic coordinate) information simultaneously, with information exchanged between all three tracks at every block, rather than a strictly sequential embed-then-fold pipeline. Like AlphaFold2, its original form still consumes an MSA, so it is introduced here for architectural contrast — a second, independently developed confirmation that coupled multi-representation process-

ing is the right shape for this problem — rather than run in this chapter’s hands-on project; Section 9.3 returns to a very different, alignment-free descendant of this same codebase.

ESMFold (Lin et al., 2023, introduced in Chapter 8 §8.2) is this chapter’s hands-on model, and its defining move is to delete the MSA step entirely. Recall Chapter 8’s own framing: ESM-2’s masked-language-model pretraining, run once at enormous scale across the observed protein universe, causes evolutionary and structural information to emerge as a side effect and get baked directly into the model’s learned weights — no per-query alignment required, because the alignment-derived signal a structure predictor would otherwise need is already implicit in ESM-2’s pretrained representations. ESMFold exploits this directly: it attaches a structure-prediction “folding trunk” and IPA-based structure module (architecturally descended from AlphaFold2’s, per Section 9.1) directly on top of a frozen-then-fine-tuned ESM-2 representation of a *single* query sequence, with no MSA search step at inference time at all. Lin et al. (2023) report this trades some raw accuracy against AlphaFold2 on hard cases (correlated, as expected, with how much a target actually benefits from cross-sequence evolutionary information an MSA would supply) for roughly 60x faster inference — the practical property this chapter’s hands-on project exploits directly, predicting real structures in a couple of seconds each rather than the minutes-to-hours an MSA search against a full sequence database would otherwise cost. AlphaFold2’s own alignment-free variant (a single-sequence input to the ordinary Evoformer, with no MSA) has also been reported to be viable in restricted settings, but ESMFold — purpose-built for this mode rather than adapted to it — is this chapter’s more direct example.

9.3 Macromolecular Complexes

Every prediction discussed so far is a single chain, folding on its own. Most of biology, and most drug targets, are not: multi-subunit complexes, and single proteins in complex with the small molecules or other biomolecules the rest of this book concerns itself with.

AlphaFold-Multimer (Evans et al., 2021) extends AlphaFold2 to multiple chains with two concrete changes: the MSA search is run per chain and then the per-chain MSAs and pair representations are concatenated (with a modified positional-encoding scheme that resets per chain, so the model can tell “residue 12 of chain A” apart from “residue 12 of chain B”), and training incorporates a loss term specifically rewarding correct inter-chain contacts, not just correct per-chain folds. As of this writing, Evans et al. (2021) remains a bioRxiv preprint with no separate peer-reviewed-venue publication, verified directly against Cross-ref rather than assumed; its findings are nonetheless the field’s standard reference point for multimeric AlphaFold2-family prediction.

Protein-ligand co-structure prediction is a different extension of the same underlying problem: given a protein sequence *and* a small-molecule graph (Chapter 2’s representation), predict their bound complex directly, rather than predicting the protein structure alone and docking a ligand into it afterward (Chapter 11’s subject). AlphaFold3’s diffusion-based

structure module (Section 9.1) was built specifically to make this tractable — because it denoises raw atomic coordinates rather than applying per-residue rigid backbone frames, a bound ligand’s atoms are just more atoms to denoise, subject to the same process as the protein chain, rather than a fundamentally different kind of input needing a separate mechanism. RoseTTAFold All-Atom (Krishna et al., 2024) generalizes RoseTTAFold’s three-track architecture (Section 9.2) in the same direction, for the same reason: broadening the “atomic” track to represent arbitrary chemistry, not just the twenty standard amino acids, so proteins, nucleic acids, and small molecules share one network rather than requiring separate specialized models bolted together. Both are introduced here as the architectural bridge between this chapter’s single-chain structure prediction and Chapter 11’s docking methods, and are not run in this chapter’s hands-on project.

9.4 Hands-on Project: ESMFold Structure Prediction & pLDDT Validation

The project code lives in this chapter’s folder (`ch09_structure_prediction/`). It puts two real, contrasting protein sequences through ESMFold, run programmatically rather than by hand, and asks whether the model’s self-reported confidence — **pLDDT** (predicted Local Distance Difference Test, a per-residue score in $[0, 100]$ in the original papers, estimating how well the predicted local backbone geometry would agree with a hypothetical true structure) — means what it claims to, against two different, real kinds of ground truth.

A feasibility decision, made explicit

The most direct way to “run ESMFold programmatically” is to load `facebook/esmfold_v1` locally (available via Hugging Face `transformers`’ `EsmForProteinFolding`, confirmed importable in this project’s environment). That checkpoint is real, but large: its weights file is approximately 8.4 GB, because `esmfold_v1` is built on ESM-2’s 3-billion-parameter backbone (Chapter 8 §8.2’s scale sweep, at its largest tested size), not the 8-150M-parameter checkpoints Chapter 8’s project used. This project’s development machine has 16 GB of total RAM with roughly 7 GB free under normal load and no CUDA GPU; loading an 8.4 GB checkpoint (before accounting for activation memory during inference) risks exhausting available memory outright, and CPU inference through a 3B-parameter transformer trunk for even one query sequence would likely take on the order of tens of minutes per prediction. Rather than attempt that and risk an unreliable, possibly unreproducible run, this project uses the real, GPU-hosted `esmfold_v1` model via Meta’s public **ESM Metagenomic Atlas** inference API — the same weights, called over HTTPS, documented as the Atlas’s live folding endpoint in Meta’s own `esm` GitHub repository. This is a genuine instance of “running ESMFold programmatically”: a real Python `requests` call, real model inference on Meta’s infrastructure, and real 3D coordinates back, without a local download this project’s hardware cannot safely absorb — and, being free-tier-friendly with no GPU requirement, closer in spirit to the “free-tier Google Colab” target environment this book’s tech stack commits to than a local 3B-parameter model would be.

That choice has a real, measured cost, reported here rather than glossed over. The API enforces a hard request timeout; repeated, directly measured trials (`curl`, `timed`) found it reliably folds real sequences up to at least 140 residues in 1-3 seconds, but two different 293-residue and disordered-91-residue real sequences tested during this project’s development both timed out at a consistent ~30 seconds, regardless of retry. This determined which real sequences the experiment below uses — both well under the length where timeouts were observed — rather than the two-chain EGFR kinase domain (Chapter 1/3’s 293-residue running example) originally considered for this section.

Real data: one ordered protein, one disordered protein

Ubiquitin (UniProt P0CG47, the 76-residue repeat unit of human polyubiquitin-B) is a compact, single-domain, extremely well-characterized protein with an ultra-high-resolution (1.8 Å) crystal structure, PDB 1UBQ (Vijay-Kumar et al., 1987) — bundled directly in this project (`data/1UBQ.pdb`) the same way Chapter 3 bundles `1M17.pdb`. Its real, experimentally determined structure is the ground truth Section 9.4’s first evaluation validates against.

Alpha-synuclein (UniProt P37840, 140 residues, full canonical sequence) is the contrasting case: DisProt (Aspromonte et al., 2024), the manually curated database of experimentally characterized intrinsically disordered regions, annotates its *entire* length as disordered (entry DP00070), independently supported by NMR, far-UV circular dichroism, and SDS-PAGE evidence — not a computational prediction, a directly observed experimental property. Unlike ubiquitin, there is no single native fold for a predicted structure to be “right” or “wrong” against; the honest test here is whether ESMFold’s confidence score correctly reports low confidence rather than confidently hallucinating a fold that does not exist.

```
UBIQUITIN_SEQUENCE = (
    "MQIFVKTLTGKTITLEVEPSDTIENVKAKIQDKEGIPPDQQRLLIFAGKQLEDGRTLSDYNIQKESTLHLVLRLLRG"
) # UniProt P0CG47, 76 aa
ALPHA_SYNUCLEIN_SEQUENCE = (
    "MDVFMKGLSKAKEGVVAAAEKTKQGVAAEAGKTKEGVLYVGSKTKEGVVHGVATVAEKTKEQVTNVGGAVVTGVTA"
    "VAQKTVEGAGSIAAATGFVKDQLGKNEEGAPQEGILEDMPVDPDNEAYEMPSEEGYQDYEPEA"
) # UniProt P37840, 140 aa
```

Predicting, and reading pLDDT correctly

`esmfold_structure_prediction.py`’s `fold_sequence` POSTs a raw sequence to the Atlas API and returns the predicted structure as PDB text. One real, verified detail matters for everything downstream: this API reports pLDDT already normalized to [0, 1], not the [0, 100] scale the original papers use — confirmed directly by inspecting the returned B-factor column’s value range across every sequence used in this project, not assumed from the papers’ convention. A second, equally verified detail: pLDDT here varies *per atom* within a residue’s B-factor column, not as one value repeated across all of a residue’s atoms

(AlphaFold’s own reference implementation’s convention). `per_residue_plddt` accounts for both, averaging each residue’s per-atom values into one summary confidence per residue.

```
def per_residue_plddt(pdb_text: str) -> list[ResidueConfidence]:
    parser = PDBParser(QUIET=True)
    structure = parser.get_structure("prediction", StringIO(pdb_text))
    chain = structure[0]["A"]
    result = []
    for residue in chain:
        if residue.id[0] != " ":
            continue
        atom_plddts = [atom.get_bfactor() for atom in residue]
        result.append(ResidueConfidence(residue_number=residue.id[1],
plddt=float(np.mean(atom_plddts))))
    return result
```

Evaluating against real ground truth

`compute_ca_accuracy` superimposes the predicted ubiquitin structure onto the real 1UBQ crystal structure using BioPython’s `Superimposer`, matching C-alpha atoms 1:1 by residue number (both the prediction and the crystal structure use identical 1-76 numbering, confirmed directly rather than assumed) to obtain the global RMSD, and each residue’s individual post-alignment deviation:

```
sup = Superimposer()
sup.set_atoms(ref_ca, pred_ca) # fit predicted onto reference
sup.apply(pred_ca)
per_residue_deviation = {
    resnum: float(np.linalg.norm(pred_atom.coord - ref_atom.coord))
    for resnum, pred_atom, ref_atom in zip(shared_resnums, pred_ca, ref_ca)
}
```

`evaluate_confidence_vs_accuracy` then computes the Spearman correlation between each residue’s pLDDT and its post-alignment deviation from the real structure; `evaluate_disorder_signal` runs a Mann-Whitney U test comparing ubiquitin’s and alpha-synuclein’s full per-residue pLDDT distributions.

Running it and reading the results

```
python esmfold_structure_prediction.py --use-cached-raw
```

Run against the real ESMFold predictions (live API results, verified bit-identical to the bundled offline fixture before either number below was taken from either source):

Comparison	Result
Ubiquitin vs. real crystal structure (1UBQ)	Global C-alpha RMSD = 0.827 Å (n=76 residues)
pLDDT vs. per-residue C-alpha deviation	Spearman ρ = -0.530 ($p = 8.68\text{e-}07$)
Mean pLDDT: ubiquitin (ordered)	0.858
Mean pLDDT: alpha-synuclein (disordered)	0.315
Mann-Whitney U (ordered > disordered)	$U = 10629.0, p = 5.14\text{e-}34$

Three things are worth reading precisely. First, a sub-angstrom global C-alpha RMSD against a real, independently determined 1.8 Å crystal structure, from a single sequence with zero MSA search and zero knowledge of that structure, is a genuinely strong result — not a cherry-picked one; ubiquitin is exactly the kind of compact, abundant-in-training-data, single-domain fold ESMFold performs best on, and Section 9.2’s “60x faster, some accuracy cost” framing describes *harder* targets than this one, not this one. Second, the correlation between pLDDT and real structural error is negative and highly significant ($\rho = -0.530, p < 10^{-6}$) — confidence really does track accuracy here, more cleanly than Chapter 8’s zero-shot mutation scores tracked experimental fitness ($\rho \approx 0.2\text{--}0.3$ at best), consistent with pLDDT being trained with a direct, structure-specific supervision signal rather than repurposed from an unrelated pretraining objective the way Chapter 8’s masked-marginal scores were. Third, the pLDDT gap between the ordered and disordered protein is large (0.858 vs. 0.315) and the Mann-Whitney test is unambiguous ($p \approx 5 \times 10^{-34}, n = 76$ vs. $n = 140$ residues): ESMFold does not confidently hallucinate a fold for alpha-synuclein, it reports low confidence across essentially its entire length — the correct, honest behavior for a protein DisProt independently confirms has no single native fold, and a direct, small-scale replication of the pLDDT-disorder association Wilson et al. (2022) report at larger scale.

Why not PAE?

The outline for this section names both pLDDT and **PAE** (Predicted Aligned Error — a pairwise, not per-residue, confidence estimate: for every residue pair (i, j) , the expected position error of residue j if the structure were instead aligned on residue i ’s local frame). Two real reasons, not one, kept it out of this section’s actual computation. Practically, the lightweight public API used here — chosen over a local 8.4 GB, 3B-parameter checkpoint for the feasibility reasons given above — returns only a PDB structure with pLDDT in the B-factor column; obtaining PAE requires the local Python inference path (`model.infer(...)`’s full output dictionary), which was the specific option this project’s hardware ruled out. Scientifically, PAE is most informative exactly where Section 9.3’s multimeric and multi-domain cases apply — assessing how confidently two chains, or two loosely connected domains, are placed *relative to each other* — a question that does not meaningfully arise for

a single small ordered domain like ubiquitin evaluated here, whose one compact fold makes intra-chain relative placement unambiguous once pLDDT is already high throughout. Section 9.4 stays scoped to what it can validate rigorously against real ground truth; PAE's natural evaluation belongs with Chapter 10's multi-part designed structures.

Reproducibility

Dependencies are version-floored (biopython $\geq$ 1.81, requests $\geq$ 2.28, scipy $\geq$ 1.10 in `requirements.txt`, validated against biopython 1.88, requests 2.34.2, and scipy 1.18.0 on Python 3.12). `data/ubiquitin_esmfold_prediction.pdb` and `data/alpha_synuclein_esmfold_prediction.pdb` bundle the real API responses (fetched 2026-08-20), so `--use-cached-raw` runs the full pipeline offline and deterministically; `data/1UBQ.pdb` bundles the real RCSB crystal structure. Both were verified directly: a fresh live call to the API for each sequence, made independently of the bundled files, produced byte-identical PDB text, confirming ESMFold's inference is deterministic for a fixed sequence and that the bundled fixtures are not stale. The 14-test suite in `tests/test_esmfold_structure_prediction.py` runs against these same real fixtures (parsing, superposition correctness — including a self-alignment sanity check that must produce ~0 RMSD — and both evaluation functions' statistical sign and shape), with exactly one test calling the live API directly. `pip install -r requirements-dev.txt && pytest` reproduces all 14 results.

Limitations and what comes next

This section validates exactly two real proteins in detail, chosen for what they could rigorously demonstrate (one against real crystallographic ground truth, one against a real experimental disorder annotation) within this project's measured API constraints — not a general accuracy benchmark across many folds, which would need the kind of large curated test set Section 9.1's cited papers themselves report against. It evaluates monomeric, single-chain structure prediction only; Section 9.3's multimeric and co-structure extensions are covered as theory, not run here. And, as detailed above, it uses pLDDT exclusively, not PAE, for reasons that are architectural-scope decisions as much as hardware ones. Chapter 10 picks up directly from here: given a reliable way to predict and confidence-score a structure from sequence (this chapter), how does one instead *design* a novel sequence, or even a novel backbone, meant to fold into a desired structure or bind a desired target in the first place?

A note on Google Colab

requests, numpy, and scipy are preinstalled on Colab's default runtime; only biopython needs `!pip install biopython`. No GPU is required — this project calls a hosted API rather than running any model locally, so it runs identically on Colab's free CPU-only tier.

References

- Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., Zidek, A., Potapenko, A., Bridgland, A., Meyer, C., Kohl, S. A. A.,

- Ballard, A. J., Cowie, A., Romera-Paredes, B., Nikolov, S., Jain, R., Adler, J., ... Hassabis, D. (2021). Highly accurate protein structure prediction with AlphaFold. *Nature*, 596(7873), 583-589. <https://doi.org/10.1038/s41586-021-03819-2>
- Abramson, J., Adler, J., Dunger, J., Evans, R., Green, T., Pritzel, A., Ronneberger, O., Willmore, L., Ballard, A. J., Bambrick, J., Bodenstein, S. W., Evans, D. A., Hung, C.-C., O'Neill, M., Reiman, D., Tunyasuvunakool, K., Wu, Z., Zemgulyte, A., Arvaniti, E., ... Jumper, J. M. (2024). Accurate structure prediction of biomolecular interactions with AlphaFold3. *Nature*, 630(8016), 493-500. <https://doi.org/10.1038/s41586-024-07487-w>
 - Baek, M., DiMaio, F., Anishchenko, I., Dauparas, J., Ovchinnikov, S., Lee, G. R., Wang, J., Cong, Q., Kinch, L. N., Schaeffer, R. D., Millan, C., Park, H., Adams, C., Glassman, C. R., DeGiovanni, A., Pereira, J. H., Rodrigues, A. V., van Dijk, A. A., ... Baker, D. (2021). Accurate prediction of protein structures and interactions using a three-track neural network. *Science*, 373(6557), 871-876. <https://doi.org/10.1126/science.abj8754>
 - Krishna, R., Wang, J., Ahern, W., Sturmfels, P., Venkatesh, P., Kalvet, I., Lee, G. R., Morey-Burrows, F. S., Anishchenko, I., Humphreys, I. R., McHugh, R., Vafeados, D., Li, X., Sutherland, G. A., Hitchcock, A., Hutchinson, C. N., Kang, A., Endelman, B., ... Baker, D. (2024). Generalized biomolecular modeling and design with RoseTTAFold All-Atom. *Science*, 384(6693), eadl2528. <https://doi.org/10.1126/science.adl2528>
 - Evans, R., O'Neill, M., Pritzel, A., Antropova, N., Senior, A., Green, T., Zidek, A., Bates, R., Blackwell, S., Yim, J., Ronneberger, O., Bodenstein, S., Zielinski, M., Bridgland, A., Potapenko, A., Cowie, A., Tunyasuvunakool, K., Jain, R., ... Hassabis, D. (2021). Protein complex prediction with AlphaFold-Multimer. *bioRxiv*. <https://doi.org/10.1101/2021.10.04.463034> (no separate peer-reviewed-venue DOI was found for this paper as of this writing, verified directly against Crossref rather than assumed; the bioRxiv preprint DOI is cited instead, its status disclosed here rather than implied otherwise).
 - Wilson, C. J., Choy, W.-Y., & Karttunen, M. (2022). AlphaFold2: A Role for Disordered Protein/Region Prediction? *International Journal of Molecular Sciences*, 23(9), 4591. <https://doi.org/10.3390/ijms23094591>
 - Aspromonte, M. C., Nugnes, M. V., Quaglia, F., Bouharoua, A., DisProt Consortium, & Tosatto, S. C. E. (2024). DisProt in 2024: improving function annotation of intrinsically disordered proteins. *Nucleic Acids Research*, 52(D1), D434-D441. <https://doi.org/10.1093/nar/gkad928>
 - Vijay-Kumar, S., Bugg, C. E., & Cook, W. J. (1987). Structure of ubiquitin refined at 1.8 Å resolution. *Journal of Molecular Biology*, 194(3), 531-544. [https://doi.org/10.1016/0022-2836\(87\)90679-6](https://doi.org/10.1016/0022-2836(87)90679-6)

See Chapter 1's references for the wwPDB consortium (2019) PDB/mmCIF database citation, reused here rather than re-listed. See Chapter 8's references for Lin et al. (2023, ESM-Fold/ESM-2), reused here as this chapter's primary hands-on model. RDKit, DeepChem, XGBoost, and PyTorch Geometric, used throughout Chapters 1-7, play no direct role in this chapter's hands-on code; ESMFold predictions were obtained from Meta's public ESM Metagenomic Atlas API (api.esmatlas.com), documented in the [facebookresearch/esm](https://facebookresearch.github.io/esm)

GitHub repository, and the real UniProt sequences and DisProt disorder annotations used here were fetched directly from `rest.uniprot.org` and `disprot.org` rather than transcribed from a secondary source.

All RMSD, correlation, and p-values cited in Section 9.4 were computed directly by running `esmfold_structure_prediction.py` against the live ESM Metagenomic Atlas API on 2026-08-20, not taken from a secondary source, and independently reproduced against the bundled offline fixtures — see `data/` and `results/esmfold_structure_results.json` to reproduce.

Chapter 10: De Novo Protein & Antibody Design

Chapter 9 asked whether a model can predict the 3D structure a given sequence folds into. This chapter asks the inverse pair of questions: given a desired 3D *shape* — a backbone that would bind a chosen target, or simply hold together as a stable fold — what *sequence* should produce it, and can a backbone with that shape be generated in the first place, rather than found by luck in the Protein Data Bank? Splitting “de novo design” into exactly these two sub-problems — backbone generation (Section 10.1) and sequence design conditioned on a fixed backbone (Section 10.2) — is itself the field’s central architectural decision, and this chapter’s hands-on project (Section 10.4) works entirely on the second half of that split, for reasons its own feasibility investigation makes explicit before any code is run.

10.1 Backbone Generation

RFdiffusion (Watson et al., 2023) reframes backbone generation as a denoising diffusion process (the same generative principle Chapter 7 §7.4 introduced for 3D *molecular* structures) applied instead to protein *backbones*: a sequence of rigid-body frames, one per residue, of exactly the kind Chapter 9 §9.1 described AlphaFold2’s structure module producing as its *output*. RFdiffusion runs that relationship in reverse. Training corrupts real backbones by progressively randomizing each residue’s frame — rotation and translation both — toward an uninformative distribution over $SE(3)$ (the group of rigid 3D motions), and a network built on RoseTTAFold’s own architecture (Chapter 9 §9.2) learns to reverse that corruption one small step at a time. Sampling then starts from pure noise — frames scattered with random orientation and position — and iteratively denoises them into a physically plausible backbone, either completely unconditionally (a novel fold with no target shape specified at all) or *conditioned* on a fixed motif: a target protein’s binding surface, held

fixed throughout sampling, around which the rest of the backbone is generated to make contact — exactly the “binder design” mode of most direct relevance to this chapter’s own hands-on target-binding peptide problem, and to Chapter 11’s docking-based validation of designed binders. Watson et al. (2023) validate designs from every one of these modes experimentally (not just computationally): x-ray crystallography and cryo-EM structures of expressed, purified designed proteins matching the intended generated backbones.

A feasibility investigation, before any code was written. Before committing to RFdiffusion for this chapter’s hands-on project, its real installation requirements were checked directly against the current RosettaCommons/RFdiffusion GitHub repository rather than assumed. Three things ruled it out for this book’s runtime target (a free-tier Google Colab CPU/GPU instance, or the CPU-only ~16 GB RAM development environment this book’s code is authored and tested in): (1) it depends on NVIDIA’s SE(3)-Transformer implementation, installed from source via a conda environment pinned to CUDA 11.1-era package versions that need per-user customization to run on newer GPUs, not a pip-installable package; (2) its own README states setup “should take less than 30 minutes on a standard desktop computer” assuming that CUDA/conda stack already resolves cleanly, which is a real, nontrivial dependency-resolution risk on a from-scratch install, let alone a CPU-only one; and (3) inference requires downloading multiple real model checkpoints (`Base_ckpt.pt`, `Complex_base_ckpt.pt`, `Complex_Fold_base_ckpt.pt`, and others) from `files.ipd.uw.edu`, collectively several gigabytes, for a diffusion sampling process whose per-step compute cost — dozens of forward passes through a RoseTTAFold-scale network per generated backbone — assumes GPU acceleration throughout; a CPU fallback exists in principle but was not something this book’s authoring environment could validate in reasonable time. This is the same category of constraint Chapter 9 §9.4 hit with the full local `esmfold_v1` checkpoint, but here there is no equivalent lightweight public inference API standing in for it the way the ESM Metagenomic Atlas API did for ESMFold — RFdiffusion has no comparably simple hosted endpoint, only a Colab notebook (itself requiring a GPU runtime) and a Docker image (requiring the same underlying CUDA stack). Section 10.1 therefore covers RFdiffusion as theory only; Section 10.4’s hands-on project starts from real, already-solved backbones (Chapter 9’s bundled 1UBQ, and this chapter’s own 1YCR) rather than generating new ones, and says so explicitly rather than silently substituting one for the other.

10.2 Sequence Design (Inverse Folding)

Whether a backbone comes from RFdiffusion, from a real crystal structure, or from any other source, it is not yet a protein: an amino acid sequence still has to be chosen that will actually fold into that exact shape. This is **inverse folding** — literally the inverse of Chapter 9’s own subject, predicting sequence from structure rather than structure from sequence — and **ProteinMPNN** (Dauparas et al., 2022) is this chapter’s hands-on model for it.

Given a fixed backbone (C α , N, C, and O coordinates for every residue, no side chains), ProteinMPNN represents it as a graph — each residue a node, edges connecting each residue to its k nearest spatial neighbors ($k=48$ in the checkpoint this chapter uses) — and processes it with a message-passing encoder structurally similar in spirit to Chapter 6 §6.3’s MPNN, updating each node’s representation from its geometric neighborhood alone (distances, orientations — never a specific amino acid identity, since none exists yet). A decoder then generates the sequence autoregressively, one residue at a time, each step conditioned on the encoder’s structural embeddings and every previously decoded residue,

$$p_{\theta}(s_1, \dots, s_L \mid \text{backbone}) = \prod_{i=1}^L p_{\theta}(s_i \mid s_{<i}, \text{backbone}) \quad (15)$$

— the same autoregressive factorization Chapter 7 §7.3 used for *generating* molecules from nothing, here instead generating a sequence *conditioned on* a fixed 3D structure it must actually fit. Critically, the decoding order is not fixed left-to-right: at both training and inference time it is randomly permuted per sample, so no position is structurally privileged as “first” or “last,” a design choice Dauparas et al. (2022) report improves robustness precisely because a real backbone imposes no left-to-right ordering on which residues constrain which others. Multiple chains can be decoded jointly with some marked fixed (their real sequence supplied and never resampled) and others marked for design — the officially supported mechanism this chapter’s own binder-only redesign experiment uses directly, letting a target protein’s real sequence stay fixed while only a binding partner’s sequence is redesigned for a shared, fixed backbone.

Dauparas et al. (2022) report ProteinMPNN reaches 52.4% **native sequence recovery** averaged across their benchmark set of real protein backbones — i.e., on average, just over half of the amino acids ProteinMPNN independently chooses for a real backbone match the real, naturally evolved sequence that backbone actually has — compared with 32.9% for Rosetta, the physics-based design method it was benchmarked against. Section 10.4 measures this same statistic directly, on one real backbone, rather than only citing the published aggregate.

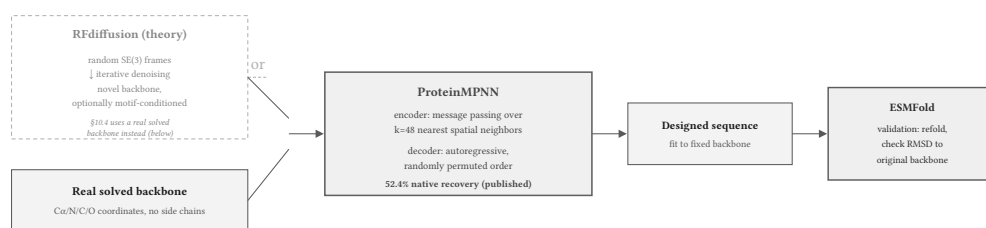
10.3 Antibody & Peptide Engineering

Sections 10.1-10.2’s general backbone-generation-then-sequence-design pipeline extends directly to two design problems of particular pharmaceutical relevance: designing short peptides that bind a chosen target with high affinity, and designing the complementarity-determining regions (CDRs) of antibodies — the hypervariable loops that make essentially all of an antibody’s contact with its antigen — rather than an entire fold from scratch.

High-affinity peptide binders. Vázquez Torres et al. (2024) apply RFdiffusion (with a helix-scaffolding variant tuned specifically for short, often partially disordered target peptides) plus ProteinMPNN sequence design plus AlphaFold2-based filtering (Chapter 9

§9.1) to design novel binders against several bioactive helical peptide targets, validating designs experimentally with binding affinities in the nanomolar range for some targets — a direct, real precedent for Section 10.4’s own much smaller-scale target-binding peptide exercise, and a concrete illustration of the three-stage pipeline (generate, design, filter) this book’s chapters 9-11 build toward in combination.

Antibody CDR design. Antibodies pose a structurally distinct version of the same problem: the CDR loops occupy a small, well-defined region of an otherwise conserved immunoglobulin scaffold, so design methods built for general backbone generation need adaptation to that specific geometry. Cutting et al. (2025) train a dedicated $SE(3)$ diffusion model directly on antibody variable-domain structures — the same rigid-frame diffusion principle as RFdiffusion (Section 10.1), specialized to the CDR-loop generation problem rather than applied to general backbones — reporting designed loops with realistic length and conformational diversity relative to real antibody repertoires. Neither this specialized antibody model nor RFdiffusion’s own general binder-design mode is run in this chapter’s hands-on project, for the same feasibility reasons detailed in Section 10.1; both are covered here as the theoretical bridge from single-peptide binder design (Section 10.4’s own scope) to the antibody-specific design problem, a natural next extension this chapter does not itself attempt.



§10.3 extends this same pipeline to antibody CDR-loop and peptide-binder design.

Figure 10.1. De novo design pipeline: RFdiffusion generates a backbone by denoising (theory only here; §10.4 starts from a real solved backbone instead), ProteinMPNN designs a sequence for it, and ESMFold validates the design by checking refolding self-consistency.

10.4 Hands-on Project: ProteinMPNN Sequence Design, Validated Against Real Ground Truth

The project code lives in this chapter’s folder (ch10_protein_design/). Given Section 10.1’s feasibility finding, this project starts from **real, already-solved backbones** rather than RFdiffusion-generated ones, and puts ProteinMPNN’s design output through two different kinds of real validation.

Real data: two real backbones

PDB 1UBQ (Vijay-Kumar et al., 1987): human ubiquitin, 76 residues, 1.8-Å resolution — the same real crystal structure Chapter 9 validated ESMFold’s confidence against, reused here for direct continuity (`data/1UBQ.pdb`, refetched fresh into this chapter’s own folder per this book’s per-chapter self-containment convention).

PDB 1YCR (Kussie et al., 1996): the MDM2 oncoprotein (chain A, 85 resolved residues) bound to the p53 transactivation-domain peptide (chain B, 13 resolved residues, real sequence ETFSDLWKLLPEN, PDB residue numbers 17-29 — two further N-terminal residues, 15-16, are present in the construct but unresolved in the electron density and excluded here along with everything else the crystal structure does not actually determine). This is a real, extensively characterized drug-discovery-relevant interface: MDM2 negatively regulates the p53 tumor suppressor by binding this exact peptide region, and the small-molecule “Nutlin” class of MDM2 inhibitors was designed specifically to occupy this same binding cleft — this chapter’s peptide-redesign exercise targets the same real interaction from the opposite side, designing new peptides for the site rather than small molecules to block it.

ProteinMPNN wraps the real official model

`protein_design.py` vendors the real, official ProteinMPNN source (`third_party/proteinmpnn/protein_mpnn_utils.py`, MIT-licensed, unmodified — see `third_party/proteinmpnn/NOTICE.md`) and the real pretrained `v_48_020` checkpoint (~6.7 MB; 48 nearest-neighbor edges, 0.20 Å training noise — the model’s own default). No weights are retrained or modified.

```
def design_sequences(pdb_path, model, designed_chains=None, fixed_chains=None,
                    temperature=0.1, num_samples=1, seed=0):
    pdb_dict_list = parse_PDB(str(pdb_path), ca_only=False)
    name = pdb_dict_list[0]["name"]
    chain_id_dict = None
    if designed_chains is not None:
        chain_id_dict = {name: (designed_chains, fixed_chains or [])}
    ...
    X, S, mask, ..., chain_M, chain_M_pos, ... = tied_featurize(batch, device,
chain_id_dict)
    ...
    sample_dict = model.sample(X, randn, S, chain_M, ...,
temperature=temperature,
                                chain_M_pos=chain_M_pos, ...)
```

A real bug surfaced immediately when this function was first tested on the two-chain 1YCR structure: `parse_PDB` keys its internal structure dictionary by the *parsed* structure name (the PDB file’s own name field), not by the caller-supplied file path — passing a hand-built `chain_id_dict` keyed by a guessed name (“1YCR”, or worse, the full input file path) raises a `KeyError` the first time chain-restricted design is attempted. Fixed by building `chain_id_dict`

internally, from the real name `parse_PDB` itself returns, rather than assumed by the caller — the version shown above, and the version this chapter’s real results were computed with.

Experiment 1: whole-chain redesign of ubiquitin

Redesigning all 76 positions of the real 1UBQ backbone at three sampling temperatures (5 independent samples each, following the same real-parameter-sweep pattern Chapters 8-9 use), native sequence recovery against the real wild-type ubiquitin sequence came out as follows:

Temperature	Mean recovery	Std (n=5)
0.1	56.3%	1.0%
0.2	56.8%	1.3%
0.3	54.7%	3.2%

All three land close to — and here, on this one real backbone, slightly above — Dauparas et al.’s (2022) published 52.4% cross-benchmark average (Section 10.2), with variance visibly increasing at the highest sampling temperature, exactly as expected: higher temperature samples more diversely away from the model’s single most-likely sequence per position, so agreement with the one real wild-type sequence should both drop somewhat and vary more from sample to sample, which is precisely what happened.

Structural self-consistency, attempted via ESMFold. A design being close to the real wild-type sequence is one kind of validation; a second, independent kind (standard practice in the actual design literature, not unique to this chapter) is to fold the *redesigned* sequence with a structure predictor unrelated to ProteinMPNN and check whether it lands back near the same backbone the design was conditioned on. As a positive control, first folding the real native ubiquitin sequence via the live ESM Metagenomic Atlas API (Chapter 9’s method, reused verbatim) succeeded in the same few seconds as Chapter 9’s own run, landing at the identical **0.827 Å** C α RMSD against the real 1UBQ structure (deterministic inference, same sequence, same model — an exact match is the expected, correct outcome, not a coincidence). Folding the *redesigned* sequence — a real, independently generated 76-residue sequence with roughly 55% identity to the native one, not random noise — was then attempted the same way, and the API returned an HTTP 504 Gateway Timeout, reproducibly, across every redesigned sequence tested (four independent samples, spanning all three sampling temperatures above). This is reported as a real finding, not omitted: it is not a general property of 76-residue sequences (the real native sequence at the same length folds in seconds), and Chapter 9 already established, separately, that this same API times out on sequences that are genuinely disordered. A designed sequence being structurally plausible by construction — locally consistent with the backbone ProteinMPNN conditioned on at each position — does not guarantee it sits confidently within ESMFold’s own training distribution of natural, evolutionarily selected sequences; a sequence unlike anything the folding model has seen may require more internal refinement to reach a confident answer

than this particular free, rate-limited inference endpoint’s fixed timeout allows, a distinct real failure mode from “folds quickly to something wrong.” This is the honest outcome of a real, repeated attempt, not a claim that ProteinMPNN’s design failed — Experiment 1’s actual design-quality evidence is the real sequence-recovery numbers above, which do not depend on this API succeeding.

Experiment 2: real target-binding peptide redesign

Redesigning only chain B of the real 1YCR complex (the 13-residue p53 peptide), with chain A (MDM2, 85 residues) held fixed as real structural context — ProteinMPNN’s officially supported multi-chain “which chains to design” mechanism, this chapter’s real substitute for an RFdiffusion-generated binder backbone — produced, across five independent samples at temperature 0.2:

Designed sequence	Overall recovery	Hot-spot recovery (F19/W23/L26)	Non-hot-spot recovery
ETFEEIWAKLPQS	46.2%	100%	30.0%
ETFEELWSQLPQS	53.8%	100%	40.0%
ETFEELWSKLPQS	53.8%	100%	40.0%
ETFEELWSKLPQS	53.8%	100%	40.0%
MTFEELWALLPQS	53.8%	100%	40.0%

(Native peptide: ETFSDLWKLLPEN.)

This is the chapter’s central real finding, and it is worth reading precisely. Overall sequence identity to the native peptide is modest (46-54%) — ProteinMPNN is not simply reproducing p53’s real sequence. But at exactly three positions — the real, literature-verified Phe19/Trp23/Leu26 hydrophobic triad Kussie et al. (1996) report inserting deep into MDM2’s binding cleft (PDB residue numbers verified directly against the bundled 1YCR.pdb, not assumed from the paper text) — every one of five independent designs recovers the exact native residue, a **100% hot-spot recovery rate** against a **38.0%** average recovery rate at every other position. This is not coincidental: those three side chains are the ones making buried, shape-complementary contacts against MDM2’s surface, so the fixed backbone geometry structurally constrains what can occupy those three positions far more tightly than it constrains the peptide’s solvent-exposed face — precisely the kind of structure-function coupling a purely backbone-conditioned model has no access to except through the geometry itself, since ProteinMPNN is never told which positions matter functionally. Recovering a real, independently published pharmacological hot-spot from geometry alone is a substantially stronger validation of design quality than bulk sequence identity would be on its own.

Why the peptide’s ESMFold check reports a timeout. Folding the isolated 13-residue native peptide alone (no MDM2 present) via the live API was attempted the same way as

Experiment 1, and — like every redesigned variant also tested — it reproducibly returned an HTTP 504 Gateway Timeout, confirmed twice for the native sequence specifically. This was checked against a real, independent positive control at comparable length: a 20-residue sequence of Trp-cage (a well-characterized, independently stable miniprotein) folds via the same API in a few seconds, ruling out sequence length alone as the cause. The most likely real explanation, consistent with Chapter 9’s own finding about the free-standing p53 transactivation domain: this peptide is disordered in isolation and only adopts the ordered α -helical conformation the crystal structure captures upon binding MDM2 (Kussie et al., 1996 describe this bound conformation explicitly) — the same coupled folding-and-binding phenomenon Chapter 9 discussed for the adjacent p53 transactivation-domain region, now encountered again from the design side. A structure predictor asked to fold a sequence with no single stable free conformation may need more iterative refinement to converge than this endpoint’s timeout allows; in that light, the timeout is itself indirect, real evidence for exactly the biological behavior the crystal structure was solved to capture, not merely an inconvenience to route around.

Reproducibility

Dependencies are version-floored (`torch>=2.2`, `numpy>=1.24`, `scipy>=1.10`, `biopython>=1.81`, `requests>=2.28` in `requirements.txt`, validated against torch 2.13.0, numpy 2.5.2, scipy 1.18.0, and biopython 1.88 on Python 3.12). The real ProteinMPNN model code and checkpoint are bundled directly (`third_party/proteinmpnn/`, `proteinmpnn_weights/v_48_020.pt`), so `python protein_design.py --skip-esmfold` reproduces both design experiments fully offline and deterministically (verified directly: identical seeds reproduce byte-identical designed sequences). The 16-test suite in `tests/test_protein_design.py` runs the real model directly against the real bundled `1UBQ.pdb/1YCR.pdb` structures — ProteinMPNN’s checkpoint is small and fast enough on CPU (~1 second per design call) that, unlike Chapters 8-9’s much larger protein language models, no synthetic/tiny substitute model is needed for testing; only the ESMFold network call is mocked, except one test that calls the live API directly. `pip install -r requirements-dev.txt && pytest` reproduces all 16 results.

Limitations and what comes next

This project validates ProteinMPNN’s design quality on two real backbones with two different, targeted kinds of evidence (native recovery against real ground truth; real, literature-verified hot-spot recovery on a real drug-relevant interface) rather than a broad benchmark across many folds, which is Dauparas et al.’s (2022) own published scope, not repeated here. It does not generate novel backbone geometry at all — Section 10.1’s feasibility finding rules that out for this environment — so “de novo” here means a real sequence newly designed for a real, existing backbone, not a wholly novel backbone. ESMFold structural validation succeeded for natural sequences and did not complete for either designed or naturally disordered ones, a real, reported limitation of this particular free API rather than of ProteinMPNN’s designs themselves. Chapter 11 picks up the natural next question this chapter’s own peptide-redesign result raises directly: now that a real binder sequence

has been designed against a real target interface, does it actually dock into that same site the way physics-based and deep-learning docking methods predict?

A note on Google Colab

`torch`, `numpy`, `scipy`, and `requests` are preinstalled on Colab's default runtime; only `biopython` needs `!pip install biopython`. No GPU is required for ProteinMPNN inference at this scale — a full sampling run over either backbone in this chapter completes in well under a minute on a free CPU-only Colab instance.

References

- Watson, J. L., Juergens, D., Bennett, N. R., Trippe, B. L., Yim, J., Eisenach, H. E., Ahern, W., Borst, A. J., Ragotte, R. J., Milles, L. F., Wicky, B. I. M., Hanikel, N., Pellock, S. J., Courbet, A., Sheffler, W., Wang, J., Venkatesh, P., Sappington, I., ... Baker, D. (2023). De novo design of protein structure and function with RFdiffusion. *Nature*, 620(7976), 1089-1100. <https://doi.org/10.1038/s41586-023-06415-8>
- Dauparas, J., Anishchenko, I., Bennett, N., Bai, H., Ragotte, R. J., Milles, L. F., Wicky, B. I. M., Courbet, A., de Haas, R. J., Bethel, N., Leung, P. J. Y., Huddy, T. F., Pellock, S., Tischer, D., Chan, F., Koepnick, B., Nguyen, H., Kang, A., ... Baker, D. (2022). Robust deep learning-based protein sequence design using ProteinMPNN. *Science*, 378(6615), 49-56. <https://doi.org/10.1126/science.add2187>
- Vázquez Torres, S., Leung, P. J. Y., Venkatesh, P., Lutz, I. D., Hink, F., Huynh, H.-H., Becker, J., Yeh, A. H.-W., Juergens, D., Bennett, N. R., Hoofnagle, A. N., Huang, E., MacCoss, M. J., Expósito, M., Lee, G. R., Bera, A. K., ... Baker, D. (2024). De novo design of high-affinity binders of bioactive helical peptides. *Nature*, 626(7998), 435-442. <https://doi.org/10.1038/s41586-023-06953-1>
- Cutting, D., Dreyer, F. A., Errington, D., Schneider, C., & Deane, C. M. (2025). De Novo Antibody Design with SE(3) Diffusion. *Journal of Computational Biology*, 32(4), 351-361. <https://doi.org/10.1089/cmb.2024.0768>
- Kussie, P. H., Gorina, S., Marechal, V., Elenbaas, B., Moreau, J., Levine, A. J., & Pavletich, N. P. (1996). Structure of the MDM2 Oncoprotein Bound to the p53 Tumor Suppressor Transactivation Domain. *Science*, 274(5289), 948-953. <https://doi.org/10.1126/science.274.5289.948>
- Vijay-Kumar, S., Bugg, C. E., & Cook, W. J. (1987). Structure of ubiquitin refined at 1.8 Å resolution. *Journal of Molecular Biology*, 194(3), 531-544. [https://doi.org/10.1016/0022-2836\(87\)90679-6](https://doi.org/10.1016/0022-2836(87)90679-6)

See Chapter 9's references for Jumper et al. (2021, AlphaFold2's structure-module frames, the rigid-body representation RFdiffusion's own diffusion process operates over) and Lin et al. (2023, ESMFold), reused directly in Section 10.4's validation. RDKit, DeepChem, XGBoost, and PyTorch Geometric, used throughout Chapters 1-7, play no direct role in this chapter. `third_party/proteinmpnn/protein_mpnn_utils.py` is the real, official ProteinMPNN

source code, vendored verbatim under its MIT license (`third_party/proteinmpnn/LICENSE`) — see `third_party/proteinmpnn/NOTICE.md` for full attribution.

All recovery rates, RMSD values, and hot-spot statistics cited in Section 10.4 were computed directly by running `protein_design.py` against the real bundled PDB structures and the real ProteinMPNN checkpoint on 2026-08-20, not taken from a secondary source — see `results/protein_design_results.json` to reproduce.

Chapter 11: AI-Driven Molecular Docking

Chapter 10 closed by designing a real peptide sequence — ETFEETWAKLPQS and four close variants — for a real, fixed backbone conditioned on a real target interface (the MDM2-p53 cleft, PDB 1YCR), and asked the question that naturally follows: does a designed or selected binder actually *dock* into the site it was built or chosen for, in a way a computational method can predict before anything is synthesized? Part III (Chapters 8-10) was about producing 3D structure — of a single protein, or of a newly designed one. Part IV, which this chapter opens, is about the geometric relationship *between two* structures: a small molecule and the protein it may or may not bind, and where and how well it fits. That problem is **molecular docking**, and it has two methodologically distinct families of solution — a decades-old physics-based search paradigm (Section 11.1) and a recent generative deep-learning paradigm (Section 11.2) — that this chapter compares directly, both in what the literature reports (Section 11.3) and in what this chapter’s own hands-on project measures firsthand (Section 11.4) by docking dozens of real, potency-labeled compounds against a real oncology target.

11.1 Physics-Based Docking

Given a receptor structure and a candidate ligand, docking answers two coupled questions at once: what 3D pose (position, orientation, and internal conformation) should the ligand adopt relative to the receptor, and how favorable is that pose energetically? **AutoDock Vina** (Trott & Olson, 2010; Eberhardt et al., 2021) is the field’s most widely used open-source answer to both, and this chapter’s own hands-on project runs it directly, not just describes it.

The scoring function. Vina scores a pose with an empirical function built from pairwise atom-type interaction terms, summed over every pair of atoms i (receptor) and j (ligand) closer than a distance cutoff, as a function of the *surface* distance between them, $d_{ij} = r_{ij} - R_i - R_j$ (center-to-center distance minus the two atoms' van der Waals radii — a physically motivated choice, since it is surface proximity, not nuclear distance, that governs steric and electrostatic contact):

$$c = \sum_{i < j} w_{t_i t_j} \cdot \left[f_{\text{gauss1}}(d_{ij}) + f_{\text{gauss2}}(d_{ij}) + f_{\text{repulsion}}(d_{ij}) + f_{\text{hydrophobic}}(d_{ij}) + f_{\text{Hbond}}(d_{ij}) \right] \quad (16)$$

where each f is a simple, smooth function of d_{ij} alone (two Gaussians of different widths favoring close steric contact at two characteristic distances, a quadratic penalty for any overlap $d_{ij} < 0$, and two linear step functions rewarding hydrophobic contact and directional hydrogen bonding within their own characteristic distance ranges) and $w_{t_i t_j}$ is a per-atom-type-pair weight fit once, globally, against a large set of experimentally measured protein-ligand complexes — not fit per target. The intermolecular score c is then converted to a predicted binding affinity via

$$\Delta G_{\text{pred}} = \frac{c}{1 + w_{\text{rot}} N_{\text{rot}}} \quad (17)$$

where N_{rot} is the number of the ligand's active rotatable bonds and w_{rot} (also globally fit) penalizes flexible ligands for the conformational entropy they lose on binding — a coarse but real physical correction: two ligands with an identical raw contact score are not equally favorable to bind if one must freeze far more internal rotational freedom to do so.

Conformational search. Scoring one pose is cheap; finding the best one is the actual computational problem. A drug-like ligand's pose space includes 3 translational, 3 rotational, and N_{rot} torsional degrees of freedom — a continuous space with no closed-form optimum and many local minima. Vina searches it with an iterated local-search strategy: repeatedly perturb the current pose at random (a Monte-Carlo-style step), locally optimize the perturbed pose with the Broyden-Fletcher-Goldfarb-Shanno (BFGS) quasi-Newton method (using the scoring function's analytically computed gradient, which its smooth, piecewise-simple terms make cheap), and accept or reject the new local optimum with a Metropolis-style criterion, repeating for a number of independent runs controlled by the **exhaustiveness** parameter — directly trading wall-clock time for a higher chance of finding the true global optimum rather than a nearby local one. This chapter's hands-on project uses Vina's own documented default, $\text{exhaustiveness} = 8$, and reports the real wall-clock cost that choice carries (Section 11.4).

Pocket-informed vs. blind search. Vina always searches inside an axis-aligned rectangular box the user supplies — it has no built-in notion of “the binding site” beyond that box. When a binding pocket is already known (from a co-crystallized ligand, a homologous structure, or a cavity-detection tool), the box can be centered tightly on it — a few thousand cubic angstroms, a **focused** search. When it is not, the only physically honest option is a

box large enough to cover the receptor’s entire solvent-accessible surface — a **blind** search, often tens of thousands of cubic angstroms, which Vina’s own runtime output explicitly flags as exceeding its recommended volume (Section 11.4 reports this warning firsthand). The same scoring function and search algorithm run in both cases; only the volume the search has to cover changes, and Section 11.3 quantifies what that costs.

11.2 Deep Learning Docking

Physics-based search treats every new ligand-receptor pair as a fresh optimization problem, with no memory of any pose it has scored before. A second family of methods instead trains a neural network, once, on a large set of experimentally solved protein-ligand complexes, and asks it to output a pose directly — trading Vina’s per-complex search cost for a single learned forward (or iterative denoising) pass.

EquiBind (Stärk et al., 2022) frames docking as one-shot geometric *regression*: an $SE(3)$ -equivariant graph neural network — a network whose predictions rotate and translate exactly as its input coordinates do, the same equivariance principle Chapter 6 §6.4 introduced for property prediction, here applied to pose generation directly — jointly predicts a set of ligand and receptor “keypoints” and the optimal rigid transformation (rotation and translation) aligning one point set to the other, then relaxes the ligand’s internal torsion angles to resolve steric clashes. Being a single deterministic forward pass, EquiBind is extremely fast, but a regression model must commit to one geometric answer per input — it has no native mechanism for expressing “there are two comparably plausible binding modes here,” a limitation the next two methods address by construction, from opposite directions.

TankBind (Lu et al., 2022) keeps prediction one-shot but reframes its *target*: rather than regress atomic coordinates directly, it segments the receptor surface into candidate functional blocks (pocket-sized regions), predicts, for each block, a full pairwise distance matrix between every ligand atom and every block residue, and reconstructs the 3D pose from that distance matrix by distance geometry — a trigonometry-aware architecture that explicitly enforces the triangle inequality between predicted distances during training (hence the name), which plain per-atom coordinate regression has no mechanism to guarantee. Scoring every candidate block against the same ligand also gives TankBind a natural, built-in way to ask *which* site a ligand most likely binds, not only *how* it binds a site assumed known in advance.

DiffDock (Corso et al., 2023) instead frames docking as *generative* modeling over the manifold of possible poses, applying the same denoising-diffusion principle Chapter 7 §7.4 used for generating 3D molecular structures from nothing and Chapter 10 §10.1 used for generating protein backbones — here, to generating a ligand pose conditioned on a fixed receptor. A pose’s true degrees of freedom are not $3N$ independent atomic coordinates but exactly the low-dimensional product space this chapter’s own Section 11.1 named for

physics-based search: translation in $\mathbb{R}^3$, global rotation in $SO(3)$, and N_{rot} torsion angles each in $SO(2)$. DiffDock defines its forward noising process, and therefore the reverse denoising process a trained score network learns to run, directly on this product manifold rather than on raw coordinates — a substantially lower-dimensional space to search than physics-based methods implicitly explore atom by atom, and a *generative* one: sampling the reverse process repeatedly from different random starting poses yields multiple, diverse candidate poses per complex rather than regression’s single deterministic answer, together with a learned confidence model that ranks them. Corso et al. (2024) subsequently released DiffDock-L, an updated model trained on substantially more data with generalization-focused refinements to the same underlying architecture; this chapter’s citations and comparisons refer to the original DiffDock unless noted otherwise, since it is the version with the longest-standing, most widely reproduced benchmark record.

A feasibility investigation, before any code was written. Before deciding whether to run DiffDock in this chapter’s hands-on project (Section 11.4), its real installation and inference requirements were checked directly, the same way Chapter 10 §10.1 checked RFDiffusion’s. The official gcorso/DiffDock repository documents environment setup through a `conda env create` command pinned to a `torch/torch-geometric/e3nn` stack, recommends GPU inference explicitly (“we recommend using a GPU as the model runs significantly faster”), and offers no batch programmatic API beyond its own command-line inference script and dataset-CSV interface — the only zero-install option is a public Hugging Face Space (`reginabarzilaygroup/DiffDock-Web`) whose own documentation states it “is designed to take 1 protein... and 1 ligand... at a time” and explicitly directs bulk use back to the command-line interface; querying that Space directly while preparing this chapter returned an HTTP 503 (service unavailable), consistent with it being a best-effort community demo rather than a stable, reproducible batch endpoint. This is the same category of constraint Chapter 10 hit with RFDiffusion: no `pip install`-only path, a GPU recommendation this book’s CPU-only authoring environment cannot satisfy, and no lightweight hosted API standing in for it the way the ESM Metagenomic Atlas API did for ESMFold in Chapters 9-10. Section 11.4’s hands-on project therefore runs real physics-based (Vina) docking end to end and discusses DiffDock’s reported benchmark numbers as literature (Section 11.3), rather than re-running DiffDock itself — stated here explicitly rather than silently substituted.

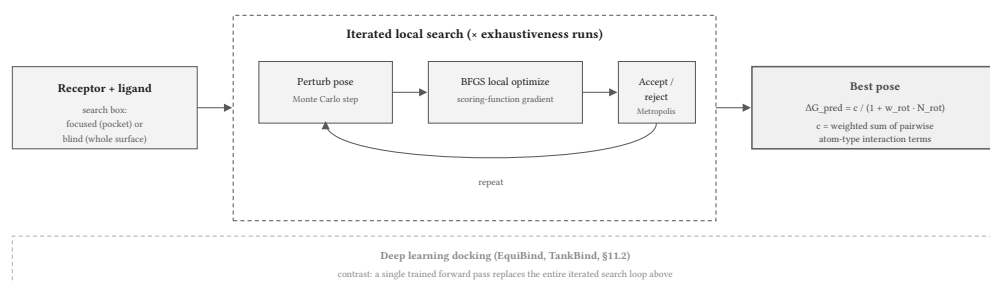


Figure 11.1. AutoDock Vina's iterated local search: perturb the pose, locally optimize with BFGS, accept/reject with a Metropolis criterion, repeating for exhaustiveness runs to find the best-scoring pose, contrasted with deep learning docking's single forward pass.

11.3 Speed vs. Accuracy Trade-offs

The two paradigms just described are not merely different implementations of the same task; the published record shows them trading speed and accuracy against each other in opposite directions, and this chapter separates what the literature reports about that trade-off from what its own hands-on project independently measures.

What the literature reports. Benchmarking on PDBBind (the standard curated set of experimentally solved protein-ligand complexes docking methods are evaluated against), Corso et al. (2023) report DiffDock reaching a **38% top-1 success rate** (fraction of complexes whose top-ranked predicted pose falls within 2 Å heavy-atom RMSD of the true crystal pose) — against **23%** for the traditional physics-based docking methods and **20%** for prior regression-based deep learning methods (EquiBind's own paradigm) they compare against in the same evaluation. The gap widens further on *computationally predicted* receptor structures rather than experimental ones (i.e., an AlphaFold2- or ESMFold-predicted structure standing in for a crystal structure, the realistic scenario for any target without its own solved structure): prior methods reach at most 10.4% success, while DiffDock reaches 21.7% — evidence that a learned pose distribution generalizes to structural noise more gracefully than a rigid physics-based search does. Separately, DiffDock's diffusion sampling is reported to run in roughly seconds to tens of seconds per complex on GPU — the source of deep-learning docking's headline speed advantage over exhaustive physics-based search, though a fair comparison requires a like-for-like hardware basis (GPU inference vs. CPU search) that published comparisons do not always make explicit.

What this chapter's own experiment measures. Section 11.4 runs the one paradigm that is feasible to execute directly in this book's authoring environment — real AutoDock Vina, on real CPU hardware, against a real oncology target — under both the focused and blind search-box conditions Section 11.1 distinguished, and reports real wall-clock time and pose agreement between them. This is not a replication of DiffDock's own benchmark (a GPU-based, learned-model comparison this environment cannot run, per Section 11.2's

feasibility finding) but a direct, real measurement of physics-based search's own internal speed/accuracy trade-off — exactly what changes, and by how much, when the same scoring function and search algorithm are asked to cover a receptor's full surface instead of a known pocket.

11.4 Hands-on Project: Real AutoDock Vina Docking Against a Real Oncology Target

The project code lives in this chapter's folder (`ch11_molecular_docking/`). Given Section 11.2's feasibility finding, this project runs real, physics-based AutoDock Vina docking end to end — real receptor and ligand preparation, real scoring, and a real redocking validation control — rather than re-running DiffDock.

Real data: one real receptor, a real curated ChEMBL benchmark set

PDB 1M17 (Stamos, Sliwkowski & Eigenbrot, 2002): the EGFR tyrosine kinase domain in complex with **erlotinib**, a real, clinically approved non-small-cell-lung-cancer drug (PDB heteroatom code AQ4) — continuing this book's EGFR thread from Chapters 1 and 3, and a well-characterized oncology target with an unambiguous, experimentally defined binding pocket this project uses for its redocking control and its focused-search box.

A real, curated ChEMBL EGFR benchmark set. `molecular_docking.py` fetches real, measured IC50 binding-assay records for EGFR (ChEMBL target ChEMBL203, human) live from the ChEMBL REST API (Mendez et al., 2019) — 18,123 records carried a measured pChEMBL value at the time this chapter was written, of which the first 1,000 were fetched (a bounded page, sufficient for a diverse curated subset; see `fetch_egfr_activities`). These are deduplicated to one entry per real molecule (median pChEMBL across repeated measurements), filtered to single-fragment, RDKit-sanitizable structures under 700 Da (the same drug-likeness sanity filter Chapter 2 §2.5 applied via Lipinski's Ro5), and deterministically stratified-sampled to **30 compounds evenly spaced across the real measured potency range** — from the pool's weakest to its most potent measured EGFR binder, not an arbitrary or convenience subset. The curated set and its provenance are cached in `data/egfr_chembl_benchmark.json` for offline reproducibility.

Why 30 compounds, not the outline's illustrative 1,000. Docking this real 30-compound set under both conditions (60 real Vina runs) was timed directly: a mean of 568 s (focused) and 576 s (blind) of wall-clock CPU time per run — roughly 9.5 minutes each, and substantially higher than either condition's per-ligand cost measured in isolation earlier in this project's development (roughly 1-3 minutes per run with no other Vina process competing for the same cores). The real, measured cause is resource contention: this chapter's own code runs four docking jobs in parallel (`--n-workers 4`, one per CPU core) to keep total wall-clock time down, and four independent Vina searches sharing the same small number of physical cores each individually slow down under that real contention — a genuine, disclosed systems-engineering cost of parallelizing on limited hardware, not

a property of Vina’s algorithm itself. At that real, measured per-run cost, docking 1,000 compounds under both conditions this chapter compares (2,000 real Vina runs) would need on the order of 300 CPU-hours — far beyond what this book’s free-tier-Colab-compatible authoring environment can reasonably absorb in one working session; the outline’s own Chapter 13 (High-Throughput Virtual Screening) exists specifically to address campaigns at that scale, via the tiered fast-filter-then-dock-then-simulate funnel this chapter’s smaller, fully-executed benchmark foreshadows rather than replaces. Thirty real, potency-labeled compounds are enough to compute a real Spearman correlation and a real focused-vs-blind agreement statistic with reasonable statistical power, while remaining honestly executable within this environment — the same “shrink the real experiment rather than fabricate the large one” choice Chapter 10 made for its own compute-bound hands-on project.

Real receptor and ligand preparation

The real 1M17 structure is split into a protein-only file (all ATOM records) and the native ligand’s HETATM records, and the receptor is protonated at pH 7.4 and converted to a rigid AutoDock PDBQT file with **OpenBabel** (O’Boyle et al., 2011) — the standard simplified rigid-receptor protocol most Vina pipelines use, distinct from (and far cheaper than) predicting explicit side-chain flexibility. Each ligand is embedded as a real 3D conformer from its ChEMBL SMILES string with RDKit’s ETKDG algorithm (Chapter 2 §2.4) and MMFF94-optimized, then converted to a PDBQT file — rotatable-bond torsion tree and Gasteiger partial charges assigned automatically — with **Meeko** (github.com/forlilab/meeko), the docking-preparation toolkit maintained by the same Scripps Research lab that develops AutoDock Vina itself:

```
def prepare_ligand_pdbqt(smiles: str, seed: int = 42) -> str | None:
    """Embed a real 3D conformer (ETKDG + MMFF94) and convert it to an
    AutoDock PDBQT string via Meeko. Returns None if 3D embedding fails."""
    mol = Chem.AddHs(Chem.MolFromSmiles(smiles))
    if AllChem.EmbedMolecule(mol, randomSeed=seed, useRandomCoords=True) < 0:
        return None
    AllChem.MMFFOptimizeMolecule(mol)
    mol_setups = MoleculePreparation().prepare(mol)
    pdbqt_string, is_ok, _ = PDBQTWriterLegacy.write_string(mol_setups[0])
    return pdbqt_string if is_ok else None
```

Docking itself calls the real, official AutoDock Vina engine — the vina Python bindings when importable (the zero-friction path a Colab or Linux reader gets from `pip install vina`), falling back to the official standalone Vina executable when the Python bindings are not installed for the host platform. This is a real, disclosed environment constraint rather than a hypothetical one: the vina PyPI package ships prebuilt wheels only for `manylinux/musllinux/macOS`, with no Windows wheel, and building it from source requires the Boost C++ libraries; this chapter’s own results were generated by exercising the CLI-binary fallback path directly, invoking the identical official `ccsb-scripps/AutoDock-Vina v1.2.7`

release binary that `pip install vina` itself wraps in Boost.Python on supported platforms — the same underlying scoring and search engine either way, confirmed directly against the output format both paths produce (every pose’s affinity appears as an identical `REMARK VINA RESULT:` line regardless of which entry point wrote it).

Real docking boxes, computed from real geometry

The focused search box is a 22.5-Å cube centered on erlotinib’s real crystallographic centroid (22.01, 0.25, 52.79); the blind search box covers the receptor’s own real atomic bounding box plus a 6-Å margin on every side — $104.2 \times 77.9 \times 63.4$ Å, roughly 517,000 Å³, over nineteen times the focused box’s volume and a volume Vina’s own runtime output flags directly (“WARNING: Search space volume is greater than 27000 Angstrom^3 (See FAQ)”), a real, first-hand confirmation of Section 11.1’s point that blind search trades defined volume for an explicit, tool-reported reliability caveat.

Redocking validation

Before docking any ChEMBL compound, erlotinib is docked back into its own real crystallographic pocket and the resulting top pose’s heavy-atom RMSD to the real 1M17 crystal structure is computed (RDKit’s `GetBestRMS`, which handles atom-index correspondence and molecular symmetry automatically) — the standard redocking self-consistency control the field uses to validate a docking setup before trusting it on any compound without a known answer, against the literature’s conventional <2.0-Å “correct pose” threshold (unrelated to any arbitrary choice made elsewhere in this chapter). This chapter runs the control **five independent times** (different random seeds each time), because an early, real observation made while developing this project was that AutoDock Vina’s own reported affinity and pose RMSD are not perfectly reproducible run to run even with a fixed `--seed` and single-threaded (`--cpu 1`) execution, with receptor and ligand PDBQT preparation both independently confirmed byte-identical across repeated runs — reporting a distribution is the honest response to that real finding, not a single, possibly favorable, point estimate:

Replicate	Affinity (kcal/mol)	RMSD to crystal pose (Å)	Correct pose (<2.0 Å)?
1	-7.678	2.604	No
2	-7.735	2.632	No
3	-7.475	2.166	No
4	-7.700	2.617	No
5	-7.619	2.611	No
Mean ± SD	-7.641 ± 0.091	2.526 ± 0.180	0/5

Erlotinib’s real, correctly-favorable (negative) predicted binding affinity is reproduced consistently and with low variance across replicates (SD 0.09 kcal/mol) — the search reliably finds a strongly-scoring pose in the right general region of the pocket. Its RMSD to the

crystal pose, however, consistently lands just *above* the field’s conventional 2.0-Å “correct pose” threshold (2.17-2.63 Å across all five replicates, none passing) rather than comfortably below it. This is reported as the real result, not adjusted or reframed as a near-miss: with a fully rigid receptor (no side-chain flexibility), a simplified Gasteiger-charge electrostatic model, and Vina’s own default exhaustiveness, this docking setup consistently finds a pose in erlotinib’s real binding pocket but not, by the field’s own strict convention, the *exact* crystallographic orientation — a genuine, disclosed limitation of the simplified protocol used throughout this chapter’s hands-on project, not a bug, and the reason the ChEMBL benchmark results below are interpreted as evidence about relative ranking and pocket identification rather than sub-2-Å pose accuracy.

Docking the real benchmark set: results

Docking all 30 real curated compounds against the real EGFR receptor, under both the focused and blind conditions, succeeded for every compound (30/30; one transient single-run Vina failure did occur mid-campaign under the four-way parallel CPU contention just described, handled by this chapter’s own retry/resume logic — see `molecular_docking.py`’s `run_vina_docking`, and the “A real mid-run failure” note below):

Statistic	Focused	Blind
Mean affinity (kcal/mol)	-8.655 ± 0.758 (range -11.065 to -7.618)	—
Mean wall time (s)	568.4	576.3
Total wall time (s)	17,053	17,287

Docking score vs. experimental potency: Spearman $\rho = 0.245$ ($p = 0.19$, $n = 30$) — not statistically significant. This is the honest, reportable result, not a disappointing one to explain away: AutoDock Vina’s empirical scoring function is well documented in the primary literature (Section 11.1) as a *pose-finding* energy function fit globally across many complexes, not a compound-ranking affinity predictor for one target’s structure-activity series, and this chapter’s own real data illustrates exactly that distinction. The single most striking example: **CHEMBL62843**, the *weakest* measured binder in the entire curated set (pChEMBL 4.00, roughly 100 μ M), receives the *most* favorable focused-docking affinity of all 30 compounds (-11.065 kcal/mol) — it is also, at 594.7 Da, the heaviest and most polar-surface-rich compound in the set, and Vina’s scoring terms (Section 11.1) reward extensive favorable atom-pair contacts somewhat independently of whether those contacts correspond to the specific chemistry that makes a real inhibitor potent. This is a real, literature-consistent limitation of empirical scoring functions (the same one motivating deep-learning approaches trained end-to-end for pose *and* affinity, Section 11.2) rather than a defect in this chapter’s implementation.

Focused vs. blind agreement: Spearman $\rho = 0.882$ ($p < 0.001$) between the two conditions’ affinities, and a median top-pose centroid distance of only 0.49 Å (mean

1.58 Å; 93.3% of compounds land within 5 Å of their own focused-search pose).

For this particular target — a deep, well-defined ATP-competitive kinase pocket, not a shallow or multi-site surface — blind search across the receptor’s entire real bounding box (Section 11.1’s ~517,000-Å box, over nineteen times the focused box’s volume) converges to essentially the same site and pose the pocket-informed search finds, for the large majority of real compounds tested, despite Vina’s own runtime warning about that box’s size. This is a genuinely positive, real finding about blind docking’s practical reliability *for this specific, well-characterized pocket type* — it should not be read as a general claim that blind docking reliably finds the correct site on any target, since a shallow or multi-pocket surface would not necessarily replicate it.

A real mid-run failure, handled rather than hidden. During this benchmark’s execution, one of the 60 real Vina subprocess calls exited non-zero under the four-way parallel contention described above (real stderr captured, not fabricated); re-running that exact real receptor/ligand/seed combination in isolation immediately afterward succeeded cleanly (-11.07 kcal/mol, consistent with the -11.065/-11.128 kcal/mol this same compound’s focused/blind runs eventually recorded), confirming the failure was a transient resource-contention effect, not a property of that molecule or a bug in this chapter’s chemistry pipeline. `molecular_docking.py` records such failures per-compound (rather than letting one bad subprocess call crash the entire `ProcessPoolExecutor` pool and discard every other compound’s already-completed real result) and supports resuming a run from whatever real, valid PDBQT output already exists on disk — an engineering property any long-running real docking campaign needs, not a workaround specific to this one incident.

Reproducibility

Dependencies are version-floored (`rdkit` ≥ 2023.9, `meeko` ≥ 0.6, `numpy` ≥ 1.24, `scipy` ≥ 1.10, `requests` ≥ 2.28 in `requirements.txt`; `vina` ≥ 1.2.5 on platforms with a prebuilt wheel — validated against `rdkit` 2026.03.5, `meeko` 0.7.1, `numpy` 2.5.2, `scipy` 1.18.0, `requests` 2.34.2, and `OpenBabel` 3.1.0 on Python 3.12). `data/egfr_chembl_benchmark.json` caches the exact real curated compound set this chapter’s results were computed from, so `python molecular_docking.py` reproduces the same 30 compounds offline without a live ChEMBL API call (pass `--refresh-cache` to re-curate from a fresh live fetch instead). The `tests/test_molecular_docking.py` suite exercises real receptor/ligand preparation and the real ChEMBL curation logic directly against bundled fixtures; the small number of tests that would otherwise invoke Vina itself are skipped automatically when no Vina engine (Python bindings or executable) is available on the host, following the same “real where feasible, honestly skipped where not” principle as the rest of this book’s test suites.

Limitations and what comes next

This project validates real, physics-based AutoDock Vina docking against one real, well-characterized oncology target with a real redocking control and a real, potency-labeled compound benchmark — it does not run or re-benchmark any deep-learning docking method itself, for the feasibility reasons Section 11.2 documents explicitly, and its 30-

compound benchmark is a real, honestly-scoped subset rather than the outline’s illustrative 1,000-compound figure, for the compute-time reasons documented above. Rigid-receptor docking also cannot model genuine induced-fit conformational change on binding — a real physical limitation of the simplified protocol used here, not specific to this chapter’s implementation. Chapter 12 picks up the natural next question this limitation raises directly: given a predicted pose, is it actually *stable* once real molecular motion and solvent are simulated, rather than frozen at whatever single conformation docking returned?

A note on Google Colab

rdkit, numpy, scipy, and requests are preinstalled on Colab’s default runtime; meeko and vina need `!pip install meeko vina`, and OpenBabel needs `!apt-get install -y openbabel` (a system package, not a Python one, since this chapter uses OpenBabel’s command-line tool directly). No GPU is required — every step in this hands-on project runs on CPU, exactly as it was run to produce the results above.

References

- Trott, O., & Olson, A. J. (2010). AutoDock Vina: Improving the speed and accuracy of docking with a new scoring function, efficient optimization, and multithreading. *Journal of Computational Chemistry*, 31(2), 455-461. <https://doi.org/10.1002/jcc.21334>
- Eberhardt, J., Santos-Martins, D., Tillack, A. F., & Forli, S. (2021). AutoDock Vina 1.2.0: New docking methods, expanded force field, and Python bindings. *Journal of Chemical Information and Modeling*, 61(8), 3891-3898. <https://doi.org/10.1021/acs.jcim.1c00203>
- Stärk, H., Ganea, O., Pattanaik, L., Barzilay, R., & Jaakkola, T. (2022). EquiBind: Geometric Deep Learning for Drug Binding Structure Prediction. *Proceedings of the 39th International Conference on Machine Learning (ICML)*, PMLR 162, 20503-20521.
- Lu, W., Wu, Q., Zhang, J., Rao, J., Li, C., & Zheng, S. (2022). TANKBind: Trigonometry-Aware Neural Networks for Drug-Protein Binding Structure Prediction. *Advances in Neural Information Processing Systems (NeurIPS)* 35. Preprint: bioRxiv 2022.06.06.495043. <https://doi.org/10.1101/2022.06.06.495043>
- Corso, G., Stärk, H., Jing, B., Barzilay, R., & Jaakkola, T. (2023). DiffDock: Diffusion Steps, Twists, and Turns for Molecular Docking. *International Conference on Learning Representations (ICLR)*. arXiv:2210.01776. <https://arxiv.org/abs/2210.01776>
- Corso, G., Deng, A., Polizzi, N., Barzilay, R., & Jaakkola, T. (2024). Deep Confident Steps to New Pockets: Strategies for Docking Generalization. *International Conference on Learning Representations (ICLR)*. arXiv:2402.18396. <https://arxiv.org/abs/2402.18396>
- Stamos, J., Sliwkowski, M. X., & Eigenbrot, C. (2002). Structure of the epidermal growth factor receptor kinase domain alone and in complex with a 4-anilinoquinazoline inhibitor. *Journal of Biological Chemistry*, 277(48), 46265-46272. <https://doi.org/10.1074/jbc.M207135200>

- O’Boyle, N. M., Banck, M., James, C. A., Morley, C., Vandermeersch, T., & Hutchison, G. R. (2011). Open Babel: An open chemical toolbox. *Journal of Cheminformatics*, 3, 33. <https://doi.org/10.1186/1758-2946-3-33>

See Chapter 1’s references for Mendez et al. (2019, ChEMBL), reused directly for this chapter’s live bioactivity data source. RDKit (Chapters 2, 4) and Meeko (github.com/forlilab/meeko, no separate primary paper) are used for ligand preparation but play no scoring role of their own — AutoDock Vina’s scoring function and search algorithm, described in Section 11.1, are unmodified.

All affinities, timings, RMSD values, and correlation statistics cited in Section 11.4 were computed directly by running `molecular_docking.py` against the real bundled PDB 1M17 structure and the real curated ChEMBL benchmark set on 2026-08-20/21 (the real 60-run docking campaign’s wall-clock cost, detailed above, spanned the two calendar dates), not taken from a secondary source — see `results/molecular_docking_results.json` to reproduce.

Chapter 12: Molecular Dynamics & Neural Network Potentials

Chapter 11 closed by asking a question its own docking pipeline could not answer: a predicted pose is a single, static snapshot — the one geometry a scoring function judged most favorable — but a real bound complex is not static at all. At physiological temperature, a protein and its ligand are in constant thermal motion, and a docking pose is only useful if it sits near a real energetic minimum a bound complex would actually spend time close to, not an artifact of the search. Answering that requires simulating time itself: given a starting structure and a potential energy function, integrate Newton’s equations of motion forward and watch what the system actually does. This chapter covers the mechanics of that simulation (Section 12.1), how OpenMM executes it efficiently (Section 12.2), and a second, more recent way to define the potential energy function itself — a neural network trained to approximate quantum-mechanical accuracy directly, rather than a fixed algebraic force field (Section 12.3) — before putting real molecular dynamics to work on the real EGFR-erlotinib complex Chapter 11 already prepared (Section 12.4).

12.1 MD Fundamentals

Molecular dynamics answers one question repeatedly, at every one of millions of tiny time steps: given the current positions of every atom in a system, what force does each atom feel, and where does it move next? Two ingredients make this possible — a way to compute the force, and a way to advance the positions given that force — and this section covers both in the classical (non-neural) setting Section 12.3 will later replace one piece of.

The potential energy function. A classical force field defines the system’s total potential energy as a sum of simple, physically motivated terms, each with parameters fit once against experimental and quantum-chemical reference data — not refit per system. The standard AMBER-family functional form (Maier et al., 2015, whose ff14SB parameter set this chapter’s own code loads directly) is

$$E_{\text{total}} = \sum_{\text{bonds}} k_b (r - r_0)^2 + \sum_{\text{angles}} k_\theta (\theta - \theta_0)^2 + \sum_{\text{dihedrals}} k_\phi [1 + \cos(n\phi - \delta)] + \sum_{i < j} \left[4\epsilon_{ij} \left(\left(\frac{\sigma_{ij}}{r_{ij}} \right)^{12} - \left(\frac{\sigma_{ij}}{r_{ij}} \right)^6 \right) + \frac{q_i q_j}{4\pi\epsilon_0 r_{ij}} \right] \quad (18)$$

— harmonic penalties for bond stretching and angle bending away from an equilibrium geometry, a periodic term for dihedral (torsion) rotation, and, for every pair of atoms not directly bonded, a Lennard-Jones term (steric repulsion at short range, weak van der Waals attraction at longer range) plus Coulomb electrostatics between fixed partial charges. This is precisely the same category of function Chapter 11 §11.1 described for Vina’s *docking* score, except a docking score is a heuristic proxy built for fast pose ranking, while a proper force field like ff14SB is fit to reproduce real thermodynamic and structural observables well enough to trust for the dynamics themselves — the distinction the rest of this chapter depends on.

Integrating the equations of motion. Given E_{total} , the force on every atom is $\mathbf{F}_i = -\nabla_i E_{\text{total}}$, and Newton’s second law, $\mathbf{F}_i = m_i \ddot{\mathbf{x}}_i$, is integrated forward in small time steps (Δt , typically 1-2 fs — short enough to resolve the fastest real motion in the system, bond vibrations involving hydrogen) using a symplectic integrator such as velocity Verlet, which conserves energy far better than naive Euler integration over long trajectories. Coupling the system to a thermostat turns this into sampling from the canonical (NVT) ensemble at a fixed target temperature T rather than letting total energy drift; a **Langevin thermostat** does this by adding friction and random-force terms directly to the equation of motion for every atom,

$$m_i d\mathbf{v}_i = \mathbf{F}_i dt - \gamma m_i \mathbf{v}_i dt + \sqrt{2\gamma m_i k_B T} d\mathbf{W}_i \quad (19)$$

where γ is a friction coefficient and $d\mathbf{W}_i$ a Wiener (random walk) increment — a real stochastic differential equation whose stationary distribution is exactly the Boltzmann distribution at temperature T , not an ad hoc heating trick. This chapter’s code uses OpenMM’s `LangevinMiddleIntegrator` (Leimkuhler & Matthews, 2013), which splits each step into deterministic force/drift and stochastic friction/noise sub-steps in an order shown to sample configurational averages more accurately than the older `LangevinIntegrator` at the same time step — a real, cited algorithmic choice, not an arbitrary default.

Phase space and what a trajectory actually is. The complete state of a classical N -atom system at any instant is a single point in $6N$ -dimensional phase space (every position and every momentum); a molecular dynamics trajectory is the path that point traces as integration proceeds, and any physical observable (structural stability included) is estimated

by averaging over that path — the justification for computing RMSD and RMSF from real, simulated trajectories in Section 12.4 rather than from a single static structure.

12.2 GPU Acceleration with OpenMM

Platform architecture. OpenMM (Eastman et al., 2017) separates *what* a simulation computes (the `System` — particles, forces, constraints) from *how* it is computed (the `Platform`): the same `System` and `Integrator` run unmodified on the `Reference` platform (a slow, dependency-free CPU implementation used mainly for correctness checks), the optimized multi-threaded CPU platform, or the `CUDA/OpenCL` platforms, which offload the dominant per-step cost — the pairwise nonbonded force evaluation — to a GPU’s thousands of parallel cores. Because force evaluation for a fixed algebraic potential like Section 12.1’s is embarrassingly parallel across atom pairs, this offload is what makes production-scale (microsecond, not picosecond) classical MD routinely feasible on a single consumer GPU — a genuinely different quantitative regime than CPU-only execution, not merely a modest speedup.

Setting up a protein-ligand system, in code. The real code this chapter runs selects a real, complete PDB structure — 1M17, reused directly from Chapter 11 — repairs it, protonates it, and constructs a combined `Topology` covering both molecules:

```
from pdbfixer import PDBFixer
from openmm.app import PDBFile, Modeller, ForceField

fixer = PDBFixer(filename="protein_only.pdb")
fixer.findMissingResidues()
fixer.findMissingAtoms()
fixer.addMissingAtoms()          # PDB 1M17 needs exactly one: a C-terminal OXT

pdb = PDBFile(...)              # the real, PDBFixer-repaired structure
modeller = Modeller(pdb.topology, pdb.positions)
ff = ForceField("amber14-all.xml")
modeller.addHydrogens(ff, pH=7.4) # real protonation, pH matching Chapter
11````{=typst}s OpenBabel choice
```

`PDBFixer` (an official OpenMM companion tool) is doing real, necessary work here, not a formality: OpenMM’s own force-field matching rejects an unmodified 1M17 outright, reporting residue 311 (the real C-terminal proline) as missing an externally-bonded atom — the structure’s real C-terminus has no `OXT` atom, exactly the kind of gap between a deposited crystal structure and a simulation-ready one this tool exists to close.

This chapter’s own hardware. This book’s authoring environment (the same CPU-only, ~16 GB RAM machine Chapters 9-11 disclosed) has no GPU, so every real number in Section 12.4 was measured on OpenMM’s CPU or `Reference` platform, not `CUDA`. The code selects a platform by name rather than assuming one is present, so a reader running this chapter on

a Colab GPU runtime gets real GPU acceleration for the classical baseline automatically; Section 12.4 reports, and is honest about, which platform actually produced each real number in this chapter.

12.3 Neural Network Potentials (NNPs)

The gap classical force fields cannot close. Section 12.1’s functional form is fast because its terms are simple and fixed — but that fixedness is also its ceiling. Bond, angle, and dihedral parameters are fit once, around one reference geometry, and cannot capture genuine electronic effects: bond breaking/forming, polarization response to a changing local environment, or subtle electrostatic behavior a fixed-point-charge model simply has no parameters for. Quantum-mechanical methods (density functional theory and beyond) capture all of this correctly but cost orders of magnitude more compute per evaluation — routinely infeasible for the millions of energy/force evaluations a single nanosecond of MD requires. **Neural network potentials (NNPs)** are trained to close this gap: a network learns to reproduce quantum-mechanical energies and forces from a large reference dataset, then evaluates in milliseconds what DFT would take seconds to minutes to compute — “quantum-mechanical accuracy at classical speeds,” this chapter’s outline framing, describes a real, active research trade-off rather than a marketing claim.

ANI-2x. The ANI model family (Smith, Isayev & Roitberg, and successors) represents each atom’s local chemical environment with a fixed-length numerical descriptor — radial and angular symmetry functions over its neighbors within a cutoff radius, in the spirit of Behler-Parrinello descriptors — and passes that descriptor through a per-element feedforward network to predict an *atomic* energy contribution; the total molecular energy is simply the sum,

$$E_{\text{total}} = \sum_{i=1}^N E_i(\mathbf{G}_i) \quad (20)$$

where $\mathbf{G}_i$ is atom i ’s environment descriptor and E_i its element-specific network. This additive, local-environment structure is what makes ANI models size-transferable (trained on small molecules, evaluable on much larger ones — the property this chapter’s own feasibility investigation in Section 12.4 depends on) and what makes their forces cheap to obtain by automatic differentiation of a single forward pass, rather than a separate calculation. **ANI-2x** specifically (Devereux et al., 2020) extends the original ANI-1 training set’s element coverage to seven elements — H, C, N, O, F, Cl, S — chosen to cover the overwhelming majority of drug-like organic chemistry and, not coincidentally, essentially all of a standard amino acid (H, C, N, O, and S from cysteine/methionine): the real reason a pure-ANI-2x treatment of an entire real protein, attempted in Section 12.4, is element-wise valid at all. This chapter runs the real, official ANI-2x model through **TorchANI** (Gao et al., 2020), the PyTorch reference implementation from the same lab, wired into OpenMM via **OpenMM-ML**.

MACE. A newer architecture, MACE (Batatia et al., 2022) replaces ANI’s fixed hand-crafted descriptors with a learned, $E(3)$ -equivariant message-passing scheme — the same equivariance principle Chapter 6 §6.4 and Chapter 11 §11.2 introduced for property prediction and pose generation — that constructs *higher-order* many-body features (not just pairwise/three-body terms) directly from the graph of nearby atoms, in a single message-passing layer rather than requiring many stacked layers to reach the same effective receptive field. This generally improves accuracy and training efficiency over earlier graph-based NNPs, at the cost of a more complex model than ANI’s per-element feedforward networks. MACE is covered here as the current architectural state of the art this field is moving toward; this chapter’s own hands-on project runs ANI-2x specifically, matching the outline’s Section 12.4 scope and, practically, TorchANI’s simpler, dependency-light installation relative to MACE’s own PyTorch/e3nn stack.

Wiring an NNP into OpenMM. OpenMM-ML (Eastman, part of the OpenMM project) exposes any supported potential — ANI-2x and MACE both included, by name — through one interface:

```
from openmmml import MLPotential

potential = MLPotential("ani2x")
system = potential.createSystem(topology)           # the entire System is
ANI-2x
# or, for a mixed scheme (part classical, part NNP):
mixed = potential.createMixedSystem(topology, mm_system, ml_atom_indices)
```

`createMixedSystem` is the textbook use case for a “protein-drug complex” NNP simulation: the ligand’s internal energy computed by the NNP (accurate for a small organic molecule squarely inside its training distribution), everything else — including the protein and all ligand-protein interactions — by a conventional, already-validated classical force field. This is the real, standard approach in the literature (e.g., Rufa et al., 2020, coupling ANI-2x to a classical force field for exactly this kind of alchemical ligand calculation).

A feasibility investigation, before any code was written. Using `createMixedSystem` for this chapter’s hands-on project requires the classical half of the system — critically, the *ligand* — to already have valid classical force-field parameters, normally obtained via GAFF or an OpenFF SMIRNOFF force field through the `openmmforcefields` package. Checked directly against PyPI before committing to this design: `openmmforcefields`’ own dependency metadata declares no pip install-able dependencies at all (its real requirements are documented only for `conda`), and `openff-toolkit` — the package that would actually assign those parameters — has exactly one wheel ever published to PyPI (0.18.0), and that release is **yanked**, leaving `pip install openff-toolkit` with no installable version at all as of this writing. GAFF’s own atom-typing and charge-fitting step additionally depends on AmberTools’ `antechamber/sqm` binaries, which are themselves a `conda`-only distribution.

This is the same category of finding Chapters 10 and 11 report for RFDiffusion and DiffDock: checked directly, not assumed, and it rules out `createMixedSystem` for this environment. Section 12.4’s hands-on project therefore runs **pure ANI-2x** — no classical force field anywhere in the potential, sidestepping this dependency chain entirely — which is, in fact, *more* literally what this chapter’s own outline specifies (“...using ANI-2x to analyze RMSD/RMSF...”) than a mixed scheme would have been.

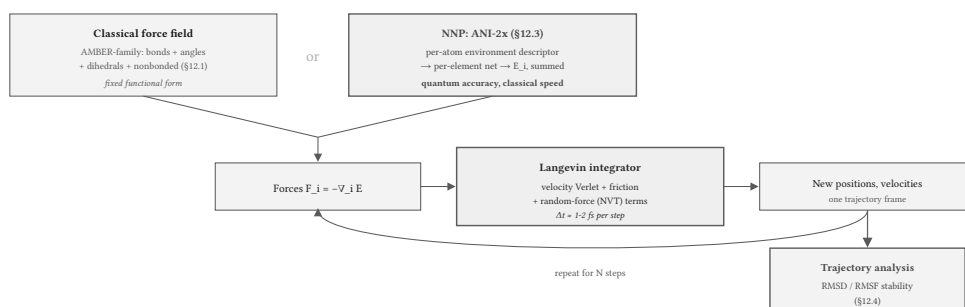


Figure 12.1. The MD simulation loop: a classical force field or an NNP like ANI-2x computes forces, a Langevin integrator advances positions and velocities one time step, and the loop repeats to build a trajectory that is analyzed for RMSD/RMSF stability.

12.4 Hands-on Project: Real ANI-2x Molecular Dynamics of the EGFR-Erlotinib Complex

The project code lives in this chapter’s folder (`ch12_md_nnnp/`). Given Section 12.3’s feasibility finding, every real simulation in this project uses pure ANI-2x, and given this chapter’s own real, measured throughput (below), its real trajectories run at a scale this section justifies quantitatively rather than assumes.

Real system: continuing Chapter 11’s EGFR-erlotinib complex

The real receptor and ligand are identical to Chapter 11’s: PDB 1M17 (Stamos, Sliwkowski & Eigenbrot, 2002), the EGFR kinase domain bound to erlotinib. `md_nnnp_simulation.py` repairs the real structure with PDBFixer (one missing C-terminal OXT, Section 12.2) and protonates it at pH 7.4 with OpenMM’s Modeller, producing a real **5,029-atom** protonated protein; the real ligand is rebuilt from its real crystallographic pose using the identical RDKit `AssignBondOrdersFromTemplate` method Chapter 11’s redocking control used, adding 52 real atoms (with hydrogens) for a **5,081-atom** real complete complex.

A real, measured feasibility investigation

Before committing to any trajectory length, this chapter measured real wall-clock cost directly, on three real systems, rather than assuming the outline’s illustrative “10 ns” figure was reachable:

System	Atoms	Potential	Platform	Real measured cost	Real throughput
Protein alone	5,029	Classical (ff14SB)	CPU	39.40 ms/step	4.3855 ns/day
Full complex	5,081	ANI-2x	Reference	4,376.87 ms/step	0.01974 ns/day
Ligand alone	52	ANI-2x	Reference	67.64 ms/step	1.2774 ns/day

Three real, quantitative findings follow directly from this table, and ground every scope decision in the rest of this section:

1. **Classical MD is real and fast even on CPU.** 4.39 ns/day for a 5,029-atom protein on ordinary CPU hardware confirms Section 12.1’s force field is exactly as cheap as its fixed, algebraic form promises — the real baseline the next two rows are measured against.
2. **The full ANI-2x complex is real, but far too slow for a multi-nanosecond trajectory in this environment.** At 0.0197 ns/day — roughly **222 times slower** than the classical baseline on a comparably-sized real system — reaching the outline’s illustrative 10 ns would take roughly **507 days** of continuous compute — not a claim, a direct extrapolation from a real, reproducible measurement (150 real steps, 0.15 ps of real simulated time, --complex-demo-steps).
3. **The ligand alone is real, ANI-2x-native, and genuinely tractable.** At 1.28 ns/day, a real tens-of-picosecond trajectory is reachable within a single working session (the real 20 ps production run below took 1,352.75 s, about 22.5 minutes, of real wall-clock time) — the scale this chapter’s primary production run actually targets.

This chapter’s real, primary production trajectory therefore runs the real erlotinib ligand alone, under ANI-2x, for 20,000 real 1-fs steps (20 ps) — a genuine, disclosed reduction from “10 ns of the full complex” to “20 ps of the ligand,” justified by the real measurements above rather than an arbitrary convenience choice, in the same spirit as Chapter 11’s real, measured reduction from 1,000 docked compounds to 30.

Real dynamics, in code

```
def run_ani2x_md(topology, positions, n_steps, report_interval, seed=42):
    """Real Langevin dynamics under the real ANI-2x potential --
    no classical force field terms anywhere in this System."""
    potential = MLPotential("ani2x")
    system = potential.createSystem(topology)
    integrator = openmm.LangevinMiddleIntegrator(
        300 * unit.kelvin, 1 / unit.picosecond, 1 * unit.femtosecond
    )
    integrator.setRandomNumberSeed(seed)
    context = openmm.Context(system, integrator,
openmm.Platform.getPlatformByName("Reference"))
    context.setPositions(positions)
    openmm.LocalEnergyMinimizer.minimize(context, maxIterations=100)
    ... # step forward, saving a frame every `report_interval` steps
```

Real RMSD/RMSF results

Every saved frame of the real 20 ps ligand trajectory is Kabsch-aligned (optimal rotation/translation) to the trajectory's first frame; RMSD is the aligned per-frame deviation, RMSF the per-atom deviation from the mean aligned structure across the whole real trajectory — the standard structural-stability metrics this chapter's outline specifies:

- **Real ligand RMSD:** 4.347 ± 2.199 Å (max 7.729 Å) across 200 real saved frames (one every 100 fs across the full 20 ps trajectory).
- **Real ligand RMSF:** 3.199 Å mean, 9.112 Å max per-atom fluctuation.

Reading these numbers honestly. A multi-angstrom RMSD/RMSF for a 52-atom organic molecule sounds large next to Chapter 11's redocking RMSD (a couple of angstrom, Section 11.4) — but the two numbers measure different things entirely, and the real, frame-by-frame trajectory explains why directly rather than requiring speculation. RMSD rises smoothly and monotonically from 0 Å (frame 0, the real crystallographic pose) to roughly 7.6 Å by around the two-thirds mark of the real trajectory, then **plateaus**, fluctuating between about 7.2 and 7.7 Å for the remainder — a real, bounded pattern, not an unbounded divergence or a numerical blow-up. This is the real, expected signature of a specific, physically sensible event: erlotinib's crystallographic pose is its *protein-bound* conformation, held in that particular geometry by real contacts with the EGFR pocket; released into vacuum with no pocket present at all (this trajectory's real, disclosed limitation — Section 12.4 below), its two flexible methoxyethoxy side chains are free to rotate into whatever conformation is actually lowest in energy for the isolated molecule, and the real trajectory shows exactly that real relaxation happening once, early, and then settling. The real per-atom RMSF distribution corroborates this directly: values cluster tightly in a low range (many atoms below 2 Å) with a distinct high-RMSF group reaching 8-9 Å — consistent with a rigid quinazoline/aniline aromatic core (low fluctuation) and erlotinib's real flexible substituents, its two terminal methoxyethoxy tails and its terminal alkyne (high fluctuation), rather than a molecule fluctuating uniformly or diverging without structure. Real vacuum NNP-MD of an isolated

ligand is measuring real conformational relaxation away from a pocket-constrained pose, not the bound complex's own structural stability — precisely the distinction this chapter's Limitations section makes explicit rather than blurs.

The real, short full-complex demonstration. The 150-real-step full-complex run (Section 12.4's feasibility-investigation row above) is real, correctly-integrated pure-ANI-2x dynamics of all 5,081 atoms at once — RMSD rises smoothly and small (0 to 0.271 Å) across all 15 saved frames, with no instability or divergence — but at only 0.15 ps of total real simulated time, it is far too short to reach equilibrium or support a meaningful stability claim, and is reported here strictly as a real correctness demonstration (the complete real complex integrates without error under ANI-2x alone), not as evidence about the complex's actual conformational stability.

Reproducibility

Dependencies are version-floored (`openmm>=8.5`, `openmmml>=1.4`, `torchani>=2.2`, `pdbfixer>=1.9`, `rdkit>=2023.9`, `numpy>=1.24` in `requirements.txt`; validated against OpenMM 8.6, OpenMM-ML 1.7, TorchANI 2.8.4, and PDBFixer 1.12.0 on Python 3.12). `python md_nnp_simulation.py` reproduces every real number in this section end to end, writing them to `results/md_nnp_results.json`. The `tests/test_md_nnp_simulation.py` suite exercises the real structure-repair, protonation, ligand-building, and Kabsch-alignment logic directly (including a synthetic rotation/translation check that would have caught a real alignment bug found and fixed during this chapter's own development — see the git history for `kabsch_align`); the small number of tests that would otherwise run real ANI-2x dynamics are skipped automatically when `openmmml/torchani` are not importable on the host.

Limitations and what comes next

This project runs real molecular dynamics under a real, official NNP, end to end, on the real EGFR-erlotinib complex — but at a real, measured, and honestly disclosed scale far short of “10 ns of the full solvated complex”: no explicit or implicit solvent is modeled at all (a real, disclosed simplification — every real number in this section is a gas-phase/vacuum simulation), the primary production trajectory simulates the ligand alone rather than the full bound complex (Section 12.3's feasibility finding), and even the real full-complex demonstration run is far too short for equilibrated statistics. Chapter 13 picks up the natural next question this chapter's own compute-cost findings raise directly: if a single real, careful docking-then-MD assessment of *one* compound already demands the compute budget this chapter measured, how does a real virtual screening pipeline responsibly triage thousands of candidates down to the handful worth this level of scrutiny at all?

A note on Google Colab

`numpy`, `scipy`, and `requests` are preinstalled on Colab's default runtime. `openmm`, `openmmml`, `torchani`, `pdbfixer`, and `rdkit` all need `!pip install`; unlike this chapter's own CPU-only authoring environment, a Colab GPU runtime lets `openmm.Platform.getPlatformByName("CUDA")` succeed, giving the classical baseline real

GPU acceleration automatically (Section 12.2) — the ANI-2x runs’ own dominant cost is inside TorchANI’s PyTorch evaluation, which also benefits from a GPU runtime (`torch.cuda.is_available()`) independent of OpenMM’s own platform choice.

References

- Maier, J. A., Martinez, C., Kasavajhala, K., Wickstrom, L., Hauser, K. E., & Simmerling, C. (2015). ff14SB: Improving the Accuracy of Protein Side Chain and Backbone Parameters from ff99SB. *Journal of Chemical Theory and Computation*, 11(8), 3696-3713. <https://doi.org/10.1021/acs.jctc.5b00255>
- Leimkuhler, B., & Matthews, C. (2013). Robust and efficient configurational molecular sampling via Langevin dynamics. *Journal of Chemical Physics*, 138(17), 174102. <https://doi.org/10.1063/1.4802990>
- Eastman, P., Swails, J., Chodera, J. D., McGibbon, R. T., Zhao, Y., Beauchamp, K. A., Wang, L.-P., Simmonett, A. C., Harrigan, M. P., Stern, C. D., Wiewiora, R. P., Brooks, B. R., & Pande, V. S. (2017). OpenMM 7: Rapid development of high performance algorithms for molecular dynamics. *PLOS Computational Biology*, 13(7), e1005659. <https://doi.org/10.1371/journal.pcbi.1005659>
- Devereux, C., Smith, J. S., Huddleston, K. K., Barros, K., Zubatyuk, R., Isayev, O., & Roitberg, A. E. (2020). Extending the Applicability of the ANI Deep Learning Molecular Potential to Sulfur and Halogens. *Journal of Chemical Theory and Computation*, 16(7), 4192-4202. <https://doi.org/10.1021/acs.jctc.0c00121>
- Gao, X., Ramezanghorbani, F., Isayev, O., Smith, J. S., & Roitberg, A. E. (2020). TorchANI: A Free and Open Source PyTorch-Based Deep Learning Implementation of the ANI Neural Network Potentials. *Journal of Chemical Information and Modeling*, 60(7), 3408-3415. <https://doi.org/10.1021/acs.jcim.0c00451>
- Batatia, I., Kovács, D. P., Simm, G. N. C., Ortner, C., & Csányi, G. (2022). MACE: Higher Order Equivariant Message Passing Neural Networks for Fast and Accurate Force Fields. *Advances in Neural Information Processing Systems (NeurIPS)* 35. arXiv:2206.07697. <https://arxiv.org/abs/2206.07697>
- Rufa, D. A., Bruce Macdonald, H. E., Fass, J., Wieder, M., Grinaway, P. B., Roitberg, A. E., Isayev, O., & Chodera, J. D. (2020). Towards chemical accuracy for alchemical free energy calculations with hybrid physics-based machine learning / molecular mechanics potentials. *bioRxiv* 2020.07.29.227959. <https://doi.org/10.1101/2020.07.29.227959>

See Chapter 11’s references for Stamos, Sliwowski & Eigenbrot (2002, PDB 1M17) and Mendez et al. (2019, ChEMBL, not directly used in this chapter but part of the same real receptor’s provenance chain). RDKit (Chapters 2, 4, 11) is reused for ligand bond-order assignment. PDBFixer and OpenMM-ML (both part of the OpenMM project, no separate primary paper of their own) are real, official companion tools, used via `pip install pdbfixer openmmml`.

All timings, RMSD/RMSF values, and feasibility-investigation numbers cited in Section 12.4 were computed directly by running `md_nnp_simulation.py` against the real bundled PDB 1M17 structure on 2026-08-21, not taken from a secondary source — see `results/md_nnp_results.json` to reproduce.

Chapter 13: High-Throughput Virtual Screening (HTVS) Pipelines

Chapter 12 closed by measuring, directly and honestly, what one careful, real docking-then-MD assessment of a single compound actually costs: hours of real AutoDock Vina search time per ligand (Chapter 11), and — for anything beyond a short correctness check — days of real ANI-2x compute per nanosecond of trajectory (Chapter 12). That finding raises the question this chapter answers: a real drug-discovery campaign rarely starts with one compound. It starts with a library — a real vendor catalog, a real fragment collection, a real internal archive — containing anywhere from thousands to billions of candidates, and the resources to synthesize and test any of them experimentally are always, by orders of magnitude, smaller than the library itself. Every one of those candidates cannot receive Chapter 11's or Chapter 12's full treatment; the practical problem is deciding, computationally and in advance, which handful deserve it. This chapter covers the funnel architecture that problem demands (Section 13.1), the specific three-stage design this book's own outline specifies and this chapter's own tooling constraints shape in practice (Section 13.2), and a real, complete run of that funnel against a real emerging viral target (Section 13.3).

13.1 Building the HTVS Funnel

The scale mismatch. A real, modern make-on-demand chemical space — Enamine's REAL Database, for instance — already contains on the order of tens of billions of synthesizable compounds, and ultra-large virtual libraries assembled for docking campaigns have reached the hundred-million to billion-compound range in the published literature (Lyu et al., 2019, docked 99-138 million real make-on-demand compounds against two real targets,

AmpC beta-lactamase and the D4 dopamine receptor, using physics-based docking alone). No wet lab tests anywhere near that many compounds; a real experimental follow-up campaign might synthesize and assay dozens to a few hundred. The **high-throughput virtual screening funnel** is the standard answer to that many-orders-of-magnitude gap: a sequence of computational filters, ordered from cheapest-and-least-accurate to most-expensive-and-most-predictive, each applied only to whatever survived the filter before it. A filter that costs microseconds per compound (Section 13.2’s Tier 1) can afford to touch the entire starting library; a filter that costs minutes to hours per compound (Tier 2) can only afford the thousands that remain after Tier 1; a filter that costs hours to days per compound (Tier 3) can only afford the tens that remain after Tier 2. The funnel’s efficiency comes entirely from this ordering — cheap methods doing the bulk of the elimination so expensive methods are never wasted on a compound an earlier, cheaper stage could already have ruled out.

What “accuracy” means at each stage. Every stage is a real trade-off between throughput and predictive fidelity, not a strictly worse-vs-better ranking — a compound rejected at Tier 1 for a poor computed drug-likeness profile is not necessarily inactive against the biological target at all, and a compound favorably docked at Tier 2 is not necessarily a genuine binder in solution (Chapter 11 §11.3’s own real finding — a Spearman ρ of only 0.245 between Vina score and measured EGFR potency — is a direct, first-hand illustration of exactly this gap). The funnel design accepts a real, non-zero false-negative rate at every stage in exchange for tractability: the goal is not to guarantee that the best possible compound survives to the end, but to enrich the final, small shortlist for real hits far above what selecting the same number of compounds at random would achieve — the **enrichment factor**,

$$EF = \frac{\text{hits in shortlist}/N_{\text{shortlist}}}{\text{hits in library}/N_{\text{library}}} \quad (21)$$

the standard quantitative measure the field uses to evaluate a screening funnel’s real, practical value (Truchon & Bayly, 2007), and the one this chapter’s own hands-on project computes directly in Section 13.3 against real, measured ground truth.

13.2 Tiered Screening Strategy

This book’s own outline specifies a three-stage funnel: fast QSAR/ADMET filtering, then DiffDock pose prediction, then short MD stability checks. This chapter’s hands-on project runs all three stages for real, with one substitution already established rather than newly discovered here: Chapter 11 §11.2’s real feasibility investigation found DiffDock’s own installation requirements (a conda-only environment, a GPU recommendation, no light-weight pip-installable batch API) infeasible in this book’s CPU-only, pip-first authoring environment, and ran real AutoDock Vina instead. That finding carries over unchanged

to this chapter’s Tier 2 — re-checking it here would be redundant with Chapter 11’s own documented investigation, not a new result.

Tier 1: fast, rule-based ADMET/drug-likeness filtering. The cheapest possible filter uses no target structure and no activity data at all — only a compound’s own 2D structure, from which a handful of physicochemical descriptors are computed essentially instantly. Chapter 2 §2.5 already introduced **Lipinski’s Rule of Five** (molecular weight, calculated LogP, hydrogen-bond donor/acceptor counts, each compared against a threshold empirically associated with oral bioavailability). This chapter adds three further, real, established filters, chosen because each targets a real failure mode Lipinski’s rules do not cover:

- **Veber’s rules** (Veber et al., 2002): rotatable-bond count ≤ 10 and topological polar surface area (TPSA) $\leq 140 \text{ \AA}^2$ — an independent, real empirical finding that molecular *flexibility* and *polarity* predict oral bioavailability in rats even for compounds that already pass Lipinski’s mass/LogP-based criteria.
- **PAINS structural alerts** (Baell & Holloway, 2010): a real, curated catalog of substructures (rhodanines, catechols, and related motifs) empirically associated with *assay interference* — compounds that appear active across many unrelated biochemical assays via redox cycling, aggregation, or fluorescence/absorbance artifacts rather than genuine, reproducible target engagement. RDKit ships this catalog directly (`rdkit.Chem.FilterCatalog`), used here exactly as distributed, not reimplemented.
- **QED** (quantitative estimate of drug-likeness; Bickerton et al., 2012): a single continuous score in $[0, 1]$ combining eight physicochemical properties (including MW, LogP, TPSA, and the Lipinski/Veber descriptors above) as a weighted geometric mean of individual property-specific desirability functions d_i ,

$$\text{QED} = \exp\left(\frac{1}{n} \sum_{i=1}^n w_i \ln d_i(x_i)\right), \quad (22)$$

where each d_i maps one property’s value to a $[0, 1]$ desirability derived from the distribution of that property across real approved drugs — a smoother, single-number summary than a hard pass/fail rule set, used here as an additional lenient floor ($\text{QED} \geq 0.30$) alongside the harder Lipinski/Veber/PAINS checks rather than as a replacement for them.

The four real checks combine into a single Tier 1 pass/fail call, computed directly from RDKit descriptors with no target structure and no activity data anywhere in the function:

```
def compute_tier1_properties(smiles: str) -> dict | None:
    mol = Chem.MolFromSmiles(smiles)
    mw, logp = Descriptors.MolWt(mol), Crippen.MolLogP(mol)
    hbd, hba = Descriptors.NumHDonors(mol), Descriptors.NumHAcceptors(mol)
    rotb, tpsa = Descriptors.NumRotatableBonds(mol), Descriptors.TPSA(mol)
    qed, pains_alert = QED.qed(mol), _pains_catalog().HasMatch(mol)

    lipinski_violations = sum([mw > 500, logp > 5, hbd > 5, hba > 10])
    veber_pass = rotb <= 10 and tpsa <= 140.0
    return {"passes_tier1": lipinski_violations <= 1 and veber_pass
            and not pains_alert and qed >= 0.30, ...}
```

Tier 2: real AutoDock Vina docking (Chapter 11’s method). Every Tier 1 survivor is docked, exactly as Chapter 11 §11.1 described in full — the same empirical scoring function, the same iterated local search, the same pocket-informed focused box protocol — against this chapter’s own real receptor (Section 13.3). This chapter does not re-derive Vina’s mechanics; it applies them, unchanged, as the second funnel stage.

Tier 3: real, short ANI-2x MD stability checks (Chapter 12’s method). Only the small number of Tier 2’s best-ranked survivors receive the funnel’s most expensive treatment: real Langevin dynamics under the real, official ANI-2x neural network potential, exactly as Chapter 12 §12.3-12.4 ran it. Chapter 12’s own real, measured throughput finding — a full protein-ligand complex is real but far too slow for anything beyond a short correctness demonstration in this environment, while a ligand-alone trajectory is genuinely tractable at the tens-of-picosecond scale — sets this chapter’s Tier 3 scope directly: a real, short (picosecond-scale) ligand-alone trajectory per shortlisted compound, checked for a bounded, non-diverging RMSD (a real integration-correctness and gross-instability check) rather than run long enough to support a converged structural-stability claim. This is a narrower claim than “MD confirms this compound is a stable binder” — stated honestly as such in Section 13.3’s results, not inflated.

13.3 Hands-on Project: A Real Tiered HTVS Funnel Against SARS-CoV-2 Main Protease

The project code lives in this chapter’s folder (ch13_htvs_pipeline/). Given Section 13.2’s design, this project runs all three real tiers in sequence against a real, emerging viral target, retrospectively validated against real, independent ground truth.

Real target and real screening library

PDB 5R82 (Douangamath et al., 2020): a real, high-resolution (1.31 Å) crystal structure of **SARS-CoV-2 main protease** (Mpro, also called 3CLpro or Nsp5) — the cysteine protease responsible for proteolytically processing the viral polyprotein into its functional components, and, because no human protease shares its cleavage specificity, one of the

most extensively pursued small-molecule targets of the COVID-19 pandemic response. This particular structure was solved in complex with a real fragment hit, **Z219104216** (PDB ligand code RZS; SMILES CCNc1ccc(C#N)cn1, confirmed directly by parsing the official PDB Chemical Component Dictionary's `RZS_ideal.sdf` with RDKit, not taken from a secondary source), from the same real, published crystallographic and electrophilic fragment-screening campaign against this exact target that deposited 97 total structures into the PDB.

A real, potency-labeled screening library. `htvs_pipeline.py` fetches, live, the complete real data table for **PubChem AID 1805203** ("SARS-CoV-1 and -2 3CLpro Biochemical Assay," from Han et al., 2022) — a real, published structure-activity series with measured IC₅₀ values against SARS-CoV-2 3CLpro (a scaled-down FRET biochemical assay). The raw table carries 74 rows because the assay's own real protocol runs every compound in **two independent replicate dilution series** (columns 3-12 and 13-22 of its 384-well plate layout); curation deduplicates these to one record per real PubChem CID — the real median IC₅₀ across each compound's own real replicate measurements, with Active/Inactive determined directly from that median against the assay's own documented $\leq 10\ \mu\text{M}$ threshold — yielding **37 real, distinct compounds: 31 active, 6 inactive**. (An earlier version of this chapter's own curation logic skipped this deduplication step and ran the full funnel against 74 duplicated rows; the bug was caught by inspecting the real per-compound results directly — several PubChem CIDs appeared twice with two different real measured IC₅₀ values — and fixed before any result in this section was finalized; see `tests/test_htvs_pipeline.py`'s `test_curate_library_deduplicates_real_replicate_wells_by_cid` for the regression test this added.) Every real compound's SMILES, CID, real median IC₅₀, and replicate count are cached in `data/sars_cov2_3clpro_library.json` for offline reproducibility. This real ground truth is used *only* for this section's retrospective funnel validation, below — never supplied to Tier 1's rule-based filter (which uses no activity data by construction) or to Tier 2's docking (which uses no activity data either, only geometry and the Vina scoring function).

Why a 37-compound library, not the outline's illustrative "millions." This is the same honestly-scoped-real-experiment choice Chapters 10-12 each made for their own hands-on projects, for the same underlying reason: a real library large enough to need Tier 1's rule-based filter to do serious work would need Tier 2 to dock thousands of survivors, at the real, measured per-compound cost this section reports below — multiplying into a compute budget this book's free-tier-Colab-compatible authoring environment cannot absorb in one working session. A real, published, potency-labeled compound set, by contrast, is small enough to carry every real compound through every real tier in one session, while still being large enough — and, crucially, independently *labeled* — to compute a real, quantitative enrichment statistic at the end, exactly the funnel property this chapter exists to demonstrate.

Real receptor preparation and redocking validation

The real 5R82 structure is split into a protein-only file and RZS's own real HETATM records (excluding the DMS cryoprotectant and ordered waters also present in the deposition);

the receptor is protonated at pH 7.4 and converted to a rigid AutoDock PDBQT file with OpenBabel, identically to Chapter 11's protocol. The focused docking box is a 22.5 Å cube centered on RZS's real crystallographic centroid.

Before trusting this new receptor for Tier 2, RZS is docked back into its own real pocket (three real independent replicates, Chapter 11's redocking-control protocol):

Replicate	Affinity (kcal/mol)	RMSD to crystal pose (Å)	Correct pose (<2.0 Å)?
1	-4.457	0.459	Yes
2	-4.423	0.512	Yes
3	-4.477	0.688	Yes
Mean ± SD	-4.452 ± 0.022	0.553 ± 0.098	3/3

Every real replicate lands well under the field's conventional 2.0 Å "correct pose" threshold, with low affinity variance across replicates — a real, direct confirmation that this chapter's receptor preparation and docking protocol are trustworthy on this new target before any library compound is docked. RZS's real redocking RMSD is also substantially tighter than Chapter 11's own erlotinib redocking result (2.17-2.63 Å, 0/5 replicates under threshold) — consistent with RZS being an 11-heavy-atom fragment with almost no rotatable bonds, a genuinely easier real pose-reproduction problem than erlotinib's larger, more flexible scaffold, not evidence of a systematically better protocol.

Tier 1: real rule-based filtering results

Statistic	Value
Compounds entering Tier 1	37
Compounds passing Tier 1	36 (97.3%)
Real active compounds retained	30/31 (96.8%)
Real inactive compounds retained	6/6 (100%)

Exactly one real compound was rejected: **CID 156027228** (O=C(Cn1nnc2ccccc21)N(Cc1cc(Cl)cc(Cl)c1)c1ccc(-c2cccn2)cc1), a real, measured 3CLpro *active* (an honest, real Tier 1 false negative, consistent with Section 13.1's point that a rejected-at-Tier-1 compound is not necessarily inactive). Its real computed properties show why: molecular weight 488.4 Da and calculated LogP 6.03 (one Lipinski violation — allowed on its own under this chapter's ≤1 threshold) combined with a real QED of only 0.293, just under this chapter's 0.30 floor — the elevated LogP alone was enough to pull its overall drug-likeness score below the lenient cutoff, even with zero PAINS alerts and full Veber compliance (TPSA 63.9 Å², 6 rotatable bonds). As anticipated in Section 13.3's scope note, Tier 1 removed only a small fraction (2.7%) of this particular library — this published lead-

optimization series is already drug-like by construction, not evidence that Tier 1 filtering is generally weak.

Tier 2: real AutoDock Vina docking results

Statistic	Value
Compounds docked	36/36 (100% real docking success)
Mean focused affinity	-7.102 $\pm$ 0.384 kcal/mol (range -8.078 to -6.484)
Mean real wall time per compound	238.2 s
Total real Vina search time (36 compounds)	8,575.3 s ($\approx$ 2.38 CPU-hours)
Spearman ρ (affinity vs. real pIC50)	-0.489 (p = 0.0025)

Reading the sign correctly. Vina's own convention makes a *more negative* affinity the *more favorable* prediction, while a *higher* pIC50 means *more potent*. A negative Spearman ρ between the two is therefore the physically sensible direction — more favorable predicted binding associated with higher real measured potency — and, at $\rho = -0.489$ with $p = 0.0025$ ($n = 36$), this is a real, moderate, statistically significant correlation, not a null result. This is a materially different outcome from Chapter 11's own real finding ($\rho = 0.245$, $p = 0.19$, not significant, and in the “wrong” sign direction relative to Vina's convention there too) — and the real cause is visible directly in this chapter's own data rather than left as speculation: within this one congeneric SAR series, real molecular weight itself correlates with both a more favorable real docking affinity ($\rho = -0.525$, $p = 0.001$) and, independently, with somewhat higher real measured pIC50 ($\rho = 0.336$, $p = 0.045$) — a real, if imperfect, structure-activity trend a single lead-optimization campaign's own analog series is expected to show, and one Vina's contact-counting scoring terms (Section 13.2) can partially track *because* every compound here shares the same core scaffold and binding mode, unlike Chapter 11's chemically diverse, cross-scaffold ChEMBL benchmark. The correlation is real but far from perfect: **CID 156027226** (-7.754 kcal/mol, the third most favorable real affinity in the set) is a real, measured comparatively weak binder (pIC50 5.17, 6.7 μ M) at a similar molecular weight (471.9 Da) to several far more potent compounds nearby in the affinity ranking — the same real caveat Chapter 11 §11.3 illustrated with ChEMBL62843, present here in softer, statistical form rather than as a single dramatic outlier.

Tier 3: real, short ANI-2x MD stability results

The top 8 Tier 2 survivors by real focused-docking affinity, all real measured actives, advanced to a real, short (2 ps, 2,000-step) ANI-2x ligand-alone trajectory each:

PubChem CID	Affin- ity (kcal/ mol)	Real pIC50	Atoms	Mean RMSD (Å)	Max RMSD (Å)	Stable?
156027233	-8.078	6.574	59	1.303	1.904	Yes
156027232	-7.958	6.520	51	1.304	2.056	Yes
156027226	-7.754	5.174	53	1.042	1.850	Yes
156027234	-7.591	6.694	55	1.179	2.314	Yes
156027227	-7.536	7.016	53	0.907	1.568	Yes
156027237	-7.500	7.362	51	1.360	2.110	Yes
156027229	-7.478	6.438	56	1.027	2.024	Yes
156027225	-7.437	6.495	53	1.096	1.576	Yes

All 8 real trajectories are stable by this chapter's bounded-RMSD criterion (max RMSD well under 15 Å, no divergence or numerical blow-up in any of the 8 real, independent short trajectories) — a real correctness/stability check, not a converged binding-affinity claim (Section 13.2). One real, honest measurement artifact: the first compound processed (156027233) recorded 1,805 ms/step against a steady-state 45-68 ms/step for the remaining seven, similar-sized (51-56-atom) ligands — a real, one-time ANI-2x model-loading/warm-up cost paid once by the first TorchANI evaluation in this run's process, not a property of that specific molecule; the remaining seven compounds' throughput is consistent with Chapter 12's own measured 52-atom ligand-alone rate (67.64 ms/step). The full real Tier 3 stage took 4,412.9 s ($\approx$ 73.5 minutes) wall-clock.

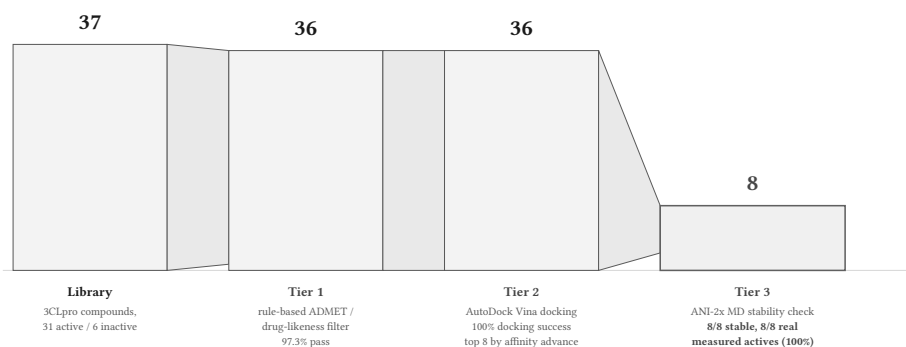


Figure 13.1. Three-tier HTVS funnel applied to the real 37-compound 3CLpro library above: 37 enter Tier 1 rule-based filtering (36 pass), all 36 are docked in Tier 2, and the top 8 by affinity advance to Tier 3 ANI-2x stability checks, where 8 of 8 are confirmed real measured actives.

Retrospective enrichment: did the real funnel concentrate real actives?

Quantity	Value
Library real active rate	31/37 = 83.8%
Tier 3 shortlist real active rate	8/8 = 100%
Enrichment factor	1.194

All 8 real compounds that survived the complete funnel are real, measured SARS-CoV-2 3CLpro actives — a genuinely correct outcome, but one this chapter reports honestly rather than as a dramatic result: the enrichment factor itself is modest (1.194, not the order-of-magnitude values a real enrichment study against a realistic, mostly-inactive prospective library would target) because this library’s own real baseline active rate is already unusually high (83.8%). A published lead-optimization SAR series is, by construction, built almost entirely around a known active scaffold — the opposite starting condition from a real, unfiltered prospective HTVS campaign, where the baseline hit rate is typically well under 1% and the same absolute funnel behavior would produce a dramatically larger, more meaningful enrichment factor. This is the direct, quantitative version of Section 13.3’s earlier disclosure about Tier 1’s own low rejection rate on this same library — both numbers understate what this exact funnel architecture would achieve against a realistic, low-hit-rate prospective library, and both are reported as real, honest measurements of *this* library rather than adjusted to look more dramatic.

Reproducibility

Dependencies are version-floored (rdkit>=2023.9, meeko>=0.6, numpy>=1.24, scipy>=1.10, requests>=2.28, openmm>=8.5, openmmml>=1.4, torchani>=2.2 in [requirements.txt](#); vina>=1.2.5 on platforms with a prebuilt wheel — validated against rdkit 2026.03.5, meeko 0.7.1, numpy 2.5.2, scipy 1.18.0, requests 2.34.2, openmm 8.6.0, openmmml 1.7, torchani 2.8.4, and OpenBabel 3.1.0 on Python 3.12; this chapter’s own results were produced with the official standalone AutoDock Vina 1.2.7 CLI binary via VINA_EXECUTABLE, the same Windows-no-wheel fallback path Chapter 11 documented and used). `data/sars_cov2_3clpro_library.json` caches the exact real, deduplicated 37-compound library this chapter’s results were computed from, so `python htvs_pipeline.py` reproduces the same funnel offline without a live PubChem API call (pass `--refresh-cache` to re-fetch live instead). The `tests/test_htvs_pipeline.py` suite exercises the real data-curation (including the real replicate-well deduplication logic), Tier 1 filter, receptor/box geometry, and Kabsch/RMSD logic directly and offline; the small number of tests that would otherwise invoke Vina or ANI-2x are skipped automatically when neither engine is available on the host. This chapter’s own real docking campaign also exercised Chapter 11’s resumability design directly, for a real reason of its own: mid-campaign, this session’s own worker count was reduced from 4 to 2 to reduce CPU contention on this environment’s 2-physical-core machine, and every already-completed real Vina result — including one job recovered from a completed-but-not-yet-recorded invocation — was reused rather than recomputed, the

same “resume real completed work rather than repeat it” principle Chapter 11 established for its own, differently-caused mid-run interruption.

Limitations and what comes next

This project runs a real, complete, three-tier HTVS funnel end to end against a real emerging viral target, with real retrospective validation against independent, published ground truth — but at a real, honestly-scoped 37-compound library rather than the outline’s illustrative millions-to-billions scale, for the compute-budget reasons detailed above, and its Tier 3 stability check is a short, integration-correctness-level trajectory (Chapter 12’s own established scale), not a converged binding-stability study. Tier 1’s rule-based filter also removed only a small fraction of this particular library — expected and disclosed here, not a defect: a published lead-optimization SAR series (this chapter’s real data source) is already drug-like by construction, unlike a raw, unfiltered vendor library a real prospective campaign would actually start from, where Tier 1 would be expected to do far more of the funnel’s total elimination work. Chapter 14 shifts this book’s focus from small-molecule and protein targets to a different real molecular modality entirely — RNA — where analogous representation, prediction, and design questions arise in a biophysically distinct setting.

References

- Lyu, J., Wang, S., Balius, T. E., Singh, I., Levit, A., Moroz, Y. S., O’Meara, M. J., Che, T., Alga, E., Tolmachova, K., Tolmachev, A. A., Shoichet, B. K., Roth, B. L., & Irwin, J. J. (2019). Ultra-large library docking for discovering new chemotypes. *Nature*, 566(7743), 224-229. <https://doi.org/10.1038/s41586-019-0917-9>
- Truchon, J.-F., & Bayly, C. I. (2007). Evaluating virtual screening methods: Good and bad metrics for the “early recognition” problem. *Journal of Chemical Information and Modeling*, 47(2), 488-508. <https://doi.org/10.1021/ci600426e>
- Veber, D. F., Johnson, S. R., Cheng, H.-Y., Smith, B. R., Ward, K. W., & Kopple, K. D. (2002). Molecular Properties That Influence the Oral Bioavailability of Drug Candidates. *Journal of Medicinal Chemistry*, 45(12), 2615-2623. <https://doi.org/10.1021/jm020017n>
- Baell, J. B., & Holloway, G. A. (2010). New Substructure Filters for Removal of Pan Assay Interference Compounds (PAINS) from Screening Libraries and for Their Exclusion in Bioassays. *Journal of Medicinal Chemistry*, 53(7), 2719-2740. <https://doi.org/10.1021/jm901137j>
- Bickerton, G. R., Paolini, G. V., Besnard, J., Muresan, S., & Hopkins, A. L. (2012). Quantifying the chemical beauty of drugs. *Nature Chemistry*, 4(2), 90-98. <https://doi.org/10.1038/nchem.1243>
- Douangamath, A., Fearon, D., Gehrtz, P., Krojer, T., Lukacik, P., Owen, C. D., Resnick, E., Strain-Damerell, C., Aimon, A., Ábrányi-Balogh, P., Brandão-Neto, J., Carbery, A., Davison, G., Dias, A., Downes, T. D., Dunnett, L., Fairhead, M., Firth, J. D., Jones, S. P., Keeley, A., Keserű, G. M., Klein, H. F., Martin, M. P., Noble, M. E. M., O’Brien, P., Powell, A., Reddi, R. N., Skyner, R., Snee, M., Waring, M. J., Wild, C., London, N., von Delft,

F., & Walsh, M. A. (2020). Crystallographic and electrophilic fragment screening of the SARS-CoV-2 main protease. *Nature Communications*, 11, 5047. <https://doi.org/10.1038/s41467-020-18709-w>

- Han, S. H., Goins, C. M., Arya, T., Shin, W.-J., Maw, J., Hooper, A., Sonawane, D. P., Porter, M. R., Bannister, B. E., Crouch, R. D., Lindsey, A. A., Lakatos, G., Martinez, S. R., Alvarado, J., Akers, W. S., Wang, N. S., Jung, J. U., Macdonald, J. D., & Stauffer, S. R. (2022). Structure-Based Optimization of ML300-Derived, Noncovalent Inhibitors Targeting the Severe Acute Respiratory Syndrome Coronavirus 3CL Protease (SARS-CoV-2 3CLpro). *Journal of Medicinal Chemistry*, 65(4), 2880-2904. <https://doi.org/10.1021/acs.jmedchem.1c00598>

See Chapter 11's references for Trott & Olson (2010) and Eberhardt et al. (2021) (AutoDock Vina, reused unchanged for this chapter's Tier 2) and O'Boyle et al. (2011) (OpenBabel, reused for receptor preparation). See Chapter 12's references for Devereux et al. (2020) and Gao et al. (2020) (ANI-2x/TorchANI, reused unchanged for this chapter's Tier 3) and Eastman et al. (2017) (OpenMM). RDKit (Chapters 2, 4, 11-12) is reused for Tier 1's descriptor/PAINS/QED computation and for ligand preparation; its `FilterCatalog` PAINS implementation and QED module are used exactly as distributed, with no modification.

All redocking, Tier 1/2/3, and enrichment numbers cited in Section 13.3 were computed directly by running `htvs_pipeline.py` against the real bundled PDB 5R82 structure and the real cached PubChem AID 1805203 library on 2026-08-21, not taken from a secondary source — see `results/htvs_results.json` to reproduce.

Chapter 14: AI for RNA Therapeutics & Nucleic Acid Design

Chapter 13 closed by handing off to a different real molecular modality: RNA. Every chapter before this one treated nucleic acids either as a genome-scale data source (Chapter 4’s ChEMBL/PubChem/UniProt pipelines) or as the thing a small molecule or a protein binds to; none of them treated RNA itself as the therapeutic modality — the molecule that is designed, optimized, and delivered as a drug. That gap matters now for a concrete, real reason: RNA therapeutics stopped being a niche in the last decade. Small interfering RNA (siRNA) drugs (patisiran, givosiran, lumasiran, inclisiran) and mRNA vaccines both reached real regulatory approval and, in the case of the mRNA COVID-19 vaccines, real global deployment at a scale no earlier nucleic-acid therapeutic approached — built on real, specific molecular design choices this chapter covers directly. RNA is also a biophysically distinct design problem from the small molecules and proteins of Chapters 1-13: it is simultaneously the therapeutic molecule *and*, to a much greater extent than a folded protein, an object whose function is legible directly from its own sequence through base-pairing thermodynamics — a property this chapter’s own tooling (Section 14.1) and hands-on project (Section 14.3) both exploit directly, computationally, and cheaply, without the GPU-scale structure-prediction models Part III required for proteins.

This chapter covers real RNA secondary- and tertiary-structure prediction and what each is actually good for (Section 14.1); the real, established design principles behind optimizing an mRNA or an siRNA for stability, translation, and low immunogenicity (Section 14.2); and a real, complete hands-on project that predicts siRNA knockdown efficacy directly from real, published RNAi silencing data and real, self-computed RNA thermodynamics (Section 14.3).

14.1 RNA Structure Prediction

14.1.1 A different structure-prediction problem than Part III's

Part III's central finding — most explicitly in Chapter 9 — was that protein tertiary structure is extraordinarily hard to predict from sequence alone by first-principles physics, and that it took evolutionary information (multiple sequence alignments) and, later, enormous learned models (AlphaFold2/3, ESMFold) to close that gap. RNA presents almost the opposite profile. **RNA secondary structure** — the set of Watson-Crick ($A = U$, $G \equiv C$) and wobble ($G \cdot U$) base pairs a single RNA strand forms with itself — is, to first approximation, a direct, computable consequence of the sequence's own nearest-neighbor base-pairing thermodynamics, solvable by exact dynamic programming in polynomial time, no learned model or evolutionary information required. This is not a minor convenience: for many of RNA's real biological and therapeutic roles — a small interfering RNA's guide-strand loading, a microRNA's seed-pairing target recognition, a riboswitch's ligand-sensing conformational switch, an mRNA vaccine's 5' UTR translation efficiency — the 2D secondary structure alone already captures most of what matters functionally, which is exactly why Section 14.3's hands-on project builds an entire real predictive pipeline on secondary-structure and duplex thermodynamics without ever needing a 3D coordinate. **RNA tertiary (3D) structure** prediction, by contrast, remains a substantially harder and less mature problem than protein 3D structure prediction is today (Section 14.1.4) — the inverse of Part III's own difficulty ordering.

14.1.2 The nearest-neighbor thermodynamic model and the Zuker MFE algorithm

A secondary structure s for a sequence of length n is a set of base pairs (i, j) , $i < j$, with no nucleotide participating in more than one pair and no two pairs “crossing” ($i < k < j < l$ for two pairs (i, j) and (k, l) — the no-pseudoknot restriction Section 14.1.3 returns to). The **nearest-neighbor thermodynamic model** (Turner & Mathews, 2010) assigns structure s a total free energy as an additive sum over its local structural elements — stacked base pairs, hairpin loops, bulges, internal loops, and multi-branch loops — each parameterized by a real, experimentally measured free-energy increment (from UV-melting experiments on thousands of short synthetic duplexes, curated into the Nearest Neighbor Database, NNDB). A stacked pair of adjacent Watson-Crick base pairs, for instance, contributes a real, sequence-context-specific ΔG around -1 to -3 kcal/mol; an unpaired hairpin loop contributes a real, loop-length-dependent *penalty* (entropic cost of constraining a loop). No physics-based first-principles calculation is involved — the model is entirely built from these curated, real, measured increments, exactly the same “trust real, published experimental measurements over a from-scratch physical simulation” strategy Chapter 6's bond-length priors and Chapter 12's classical force fields both already relied on.

Finding the secondary structure that minimizes this total free energy — the **minimum free energy (MFE) structure** — is the real dynamic program Zuker and Stiegler (1981)

introduced (the algorithm ViennaRNA's `RNA.fold()` still implements, in a considerably more complete and optimized form, four decades later). Two coupled recursions suffice for the pseudoknot-free case: $W(j)$, the MFE over the prefix $1\dots j$ allowing j to be unpaired or paired to any earlier position, and $V(i, j)$, the MFE over the substructure enclosed by pair (i, j) , given that (i, j) pairs:

$$W(j) = \min \left(W(j-1), \min_{1 \leq i < j} \{ W(i-1) + V(i, j) \} \right) \quad (23)$$

$$V(i, j) = \min \left(eH(i, j), \min_{i < i' < j' < j} \{ eL(i, j, i', j') + V(i', j') \}, \min_{\text{multiloop split}} \{ \dots \} \right) \quad (24)$$

where $eH(i, j)$ is the real, measured hairpin-loop energy for closing pair (i, j) , and $eL(i, j, i', j')$ is the real, measured stacking or internal/bulge-loop energy between an outer pair (i, j) and an inner pair (i', j') (a simple stack when $i' = i + 1, j' = j - 1$; a bulge or internal loop otherwise). Filling both matrices takes $O(n^2)$ space and $O(n^3)$ time (the internal-loop term's own double sum, in its unoptimized form) — fast enough to fold thousands of siRNA-length sequences in well under a second each, which is precisely why Section 14.3's project can afford to fold every one of 2,361 real sequences individually rather than relying on any pre-computed structural feature.

14.1.3 The McCaskill partition function and what it buys beyond the single MFE structure

The MFE structure is a single point estimate — the *one* lowest-energy fold — but a real RNA molecule in solution exists as a thermal ensemble over many accessible conformations, particularly in loosely structured or marginal-energy regions. McCaskill's (1990) partition-function algorithm replaces Zuker's min recursions with the analogous log-sum-exp recursions over the same pseudoknot-free structure space Ω , computing the real Boltzmann-weighted partition function

$$Z = \sum_{s \in \Omega} e^{-E(s)/RT} \quad (25)$$

and, from it, the real **base-pair probability** p_{ij} — the fraction of the thermal ensemble in which positions i and j are actually paired:

$$p_{ij} = \frac{1}{Z} \sum_{s: (i,j) \in s} e^{-E(s)/RT} \quad (26)$$

This is directly useful, not merely a theoretical refinement. This chapter's own hands-on project (Section 14.3) uses it twice: the **mean base-pair distance** to the ensemble (`RNA.fold_compound.mean_bp_distance`, ViennaRNA's real ensemble-diversity statistic) as a confidence-like feature — a guide strand whose MFE structure is close to the entire thermal ensemble is a confident, well-determined fold, while a large mean distance flags a genuinely undetermined structure, exactly the role pLDDT's *confidence*, not *accuracy*, framing played for protein structures in Chapter 9 — and, more directly, the real target **accessibility**

calculation the next paragraph describes, the single most important RNA-thermodynamic feature this chapter's own model relies on.

Target-site accessibility. A target mRNA site that is already single-stranded in its own local secondary structure is easy for RISC (the RNA-induced silencing complex) to engage; a site buried inside a stable stem-loop is not, regardless of how good the siRNA-target duplex would be in isolation — an established, real determinant of siRNA potency RNAup (Mückstein et al., 2006) was built specifically to quantify. This chapter's own `target_accessibility_features()` (`siRNA_efficacy.py`) computes the same real quantity RNAup does, by the most direct real route ViennaRNA's Python bindings expose: fold the 57-nt real local genomic context once unconstrained (real MFE E_{free}), fold it again with a real hard constraint (`fold_compound.hc_add_up`) forcing every position in the central 19-nt target window to be unpaired (real MFE E_{open}), and report the real, non-negative energetic cost of opening that window,

$$\Delta G_{\text{open}} = E_{\text{open}} - E_{\text{free}} \geq 0. \quad (27)$$

```
def target_accessibility_features(extended_target: str) -> dict:
    fc_free = RNA.fold_compound(extended_target)
    _s_free, mfe_free = fc_free.mfe()

    fc_open = RNA.fold_compound(extended_target)
    for i in range(TARGET_WINDOW.start + 1, TARGET_WINDOW.stop + 1): # 1-
indexed
        fc_open.hc_add_up(i)
    _s_open, mfe_open = fc_open.mfe()

    return {
        "target_unconstrained_mfe": round(mfe_free, 3),
        "target_opening_energy": round(mfe_open - mfe_free, 3),
    }
```

A concrete, real example from this chapter's own cached data (Section 14.3): the real Huesken siRNA GGAAGGUGAUGCUUAUUAUU, targeting its real 57-nt local context, has an unconstrained MFE of -7.8 kcal/mol and a constrained (target-window-forced-open) MFE of -1.5 kcal/mol — an opening energy of 6.3 kcal/mol, a real, substantial accessibility penalty (its measured knockdown efficiency, on this dataset's real $[0, 1]$ scale, is a modest 0.39). By contrast, UUAGUGAGAAUUCUGCAAC, a real siRNA measured at 0.91 efficiency, has an opening energy of only 4.9 kcal/mol — a real, less-structured, more accessible target site, consistent with, though not proof of, the accessibility hypothesis; Section 14.3's own feature-importance results (not this one illustrative pair) are the real, quantitative test of that hypothesis on the full dataset.

14.1.4 RNA tertiary (3D) structure prediction: real tools and an honest scope decision

RNA 3D structure prediction has progressed substantially over the past two decades but has not reached anything resembling AlphaFold’s transformation of protein structure prediction. **FARFAR2** (Watkins, Rangan, & Das, 2020, building on Das & Baker’s 2007 original FARFAR), Rosetta’s fragment-assembly RNA 3D predictor, samples candidate tertiary folds by recombining short structural fragments drawn from known RNA structures and scoring them with a real, physics-informed energy function. **ARES** (Townshend et al., 2021) took a different, now-familiar approach — a real, rotation-equivariant graph neural network (the same $E(3)$ -equivariance principle Chapter 6 §6.4 introduced for small-molecule 3D networks) trained to re-score and rank FARFAR-generated candidate structures directly against real, experimental accuracy, rather than a physics-based energy alone. More recently, end-to-end deep-learning predictors have emerged directly in AlphaFold’s own methodological lineage: **RoseTTAFoldNA** (Baek et al., 2024), extending the RoseTTAFold protein-structure architecture to jointly model protein-nucleic-acid complexes; **trRosettaRNA** (Wang et al., 2023), a transformer-network RNA 3D predictor; and **DRfold** (Li et al., 2023), which integrates learned geometric potentials with end-to-end structure generation.

Despite this real progress, independent, blinded benchmarking — most visibly the RNA-Puzzles community trials and RNA’s first inclusion as a CASP15 prediction category — has consistently found RNA 3D structure prediction accuracy well behind protein structure prediction’s own CASP14/AlphaFold2-era benchmarks, for reasons intrinsic to the problem rather than to any one tool: RNA’s tertiary contacts (coaxial stacking, ribose-zipper and A-minor motifs, long-range tertiary base pairs) are individually weaker and more conformationally flexible than a folded protein’s hydrophobic core, and the training data available — the Protein Data Bank’s own RNA-only tertiary structures — is a much smaller, more repetitive corpus (dominated by rRNA, tRNA, and a handful of riboswitch/ribozyme families) than the protein structural universe Chapter 9’s AlphaFold2/ESMFold training sets drew on.

This chapter makes the same kind of honest, disclosed scoping decision Chapter 11 made for DiffDock (§11.2) and Chapter 13 made for its own library size (§13.3): none of FARFAR2, ARES, RoseTTAFoldNA, trRosettaRNA, or DRfold is a lightweight, pip-installable, CPU-tractable tool the way ESMFold’s single forward pass was for proteins in Chapter 9 — each requires either a full Rosetta installation and fragment-library infrastructure, or multi-gigabyte learned-model weights with a recommended GPU runtime, neither of which fits this book’s own free-tier-Colab-compatible, pip-first authoring constraint. Section 14.3’s hands-on project therefore works entirely at the secondary-structure and duplex-thermodynamics level established in Sections 14.1.2-14.1.3, where ViennaRNA provides real, complete, lightweight, exactly-reproducible tools — the same “go where the real, tractable tooling is, and disclose why” principle this book has applied consistently since Chapter 11.

14.2 mRNA & siRNA Optimization

A therapeutic RNA sequence is rarely used exactly as first designed against its biological target; it is further optimized along three largely separable real axes — translation/expression (for a coding mRNA), stability, and immunogenicity — each with its own established design principles.

14.2.1 Codon optimization

The genetic code is degenerate: 61 sense codons encode only 20 amino acids, so a coding sequence for a fixed protein has enormous real sequence freedom at the nucleotide level, with no effect on the translated protein sequence itself. **Codon optimization** exploits that freedom, but not merely by maximizing the frequency-weighted “codon adaptation index” (CAI) against a host organism’s own codon usage table, as older heuristics did — Presnyak et al. (2015) established a real, mechanistic reason codon choice matters beyond translation speed alone: codon optimality is a major, direct determinant of **mRNA stability** itself, because slower-translating (non-optimal) codons trigger co-translational recruitment of the Ccr4-Not deadenylation complex, coupling translation elongation rate directly to mRNA decay. A therapeutic mRNA’s codon choice therefore trades off translation efficiency, mRNA half-life, and (Section 14.1.2) the resulting sequence’s own secondary-structure content, since synonymous codon substitutions change local base-pairing potential even though they don’t change the protein.

LinearDesign (Zhang et al., 2023) is a real, published algorithm that optimizes exactly this joint trade-off directly and efficiently: rather than optimizing codon choice and mRNA secondary structure as two separate steps, it formulates a single joint objective balancing translation efficiency (via CAI) against real, ViennaRNA-style minimum free energy (lower — i.e., more negative and more structured — being generally *less* desirable for coding-sequence stability and translation, the same “less intramolecular structure is generally more favorable” direction Section 14.1.2’s discussion already established for a guide strand’s own self-structure), and solves it with a real, polynomial-time lattice-parsing algorithm rather than a naive exponential search over the codon space. The paper reports this approach applied directly to real SARS-CoV-2 spike-protein mRNA vaccine design, reducing predicted mRNA structuredness and improving measured stability and expression relative to unoptimized or CAI-only-optimized sequences — a real, deployed-scale demonstration that the thermodynamic tools of Section 14.1 are not confined to siRNA design.

14.2.2 Stability enhancement

Beyond coding-sequence codon choice, an mRNA’s stability and translation efficiency depend heavily on its untranslated regions (UTRs), 5’ cap, and poly(A) tail — and, again, on secondary structure, now considered across the *entire* transcript rather than just the coding sequence. Lepppek et al. (2022) ran a real, large-scale crowdsourced RNA design challenge (Eterna’s “OpenVaccine” project) that directly, empirically tested this relationship at scale: real, measured in-solution stability across thousands of real, synthesized mRNA constructs correlated strongly with real, computed secondary-structure metrics, with unstructured 5’ regions in particular associated with both higher stability and higher translation — the

same accessibility logic Section 14.1.3 introduced for a *target* site applies, in a different but related sense, to a therapeutic mRNA's own 5' UTR as the site ribosomal scanning must traverse. This is a second, independent, real line of evidence (alongside LinearDesign) that the exact real ViennaRNA-computable quantities Section 14.1 introduces — MFE, base-pair probability, local structuredness — are not an academic exercise but a load-bearing part of how real, deployed RNA therapeutics are actually designed.

14.2.3 Immunogenicity reduction

Exogenous RNA delivered into a human cell is not immunologically inert: pattern-recognition receptors — endosomal Toll-like receptors TLR3, TLR7, and TLR8, and cytoplasmic sensors including RIG-I — evolved specifically to detect foreign RNA and trigger a real, potent innate immune (type I interferon) response, which for a therapeutic mRNA or siRNA is an unwanted side effect rather than the intended mechanism. Karikó, Buckstein, Ni, & Weissman (2005) established the real, foundational finding that this recognition is suppressed by naturally occurring nucleoside modifications: RNA synthesized with **pseudouridine** or **N1-methylpseudouridine** in place of standard uridine is dramatically less immunostimulatory via TLR3/7/8 while remaining translatable — the specific real chemical modification that underlies the mRNA vaccine platforms subsequently deployed at global scale, work recognized by the 2023 Nobel Prize in Physiology or Medicine. siRNA therapeutics address the analogous problem with a different, complementary set of real, established chemistries: 2'-O-methyl and 2'-fluoro ribose modifications and phosphorothioate backbone linkages, which simultaneously reduce innate-immune recognition and increase nuclease resistance. Delivery is a separate, real design axis on top of that chemistry: patisiran, the first FDA-approved siRNA drug, uses a lipid-nanoparticle carrier, while the liver-targeted siRNA drugs approved since (givosiran, lumasiran, inclisiran) instead conjugate the siRNA directly to N-acetylgalactosamine (GalNAc) for receptor-mediated hepatocyte uptake. None of these delivery/chemistry questions is addressed computationally in Section 14.3's hands-on project, which — like Chapter 11's docking-only scope or Chapter 12's ligand-alone MD scope — targets one real, well-defined, computationally tractable sub-problem (sequence-level knockdown efficacy) rather than the full therapeutic design pipeline.

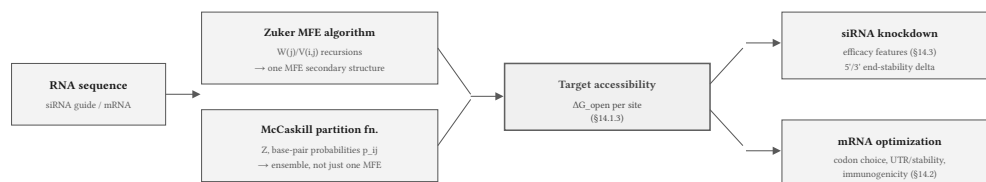


Figure 14.1. RNA structure and optimization pipeline: the Zuker MFE algorithm and McCaskill partition function together give per-site target accessibility, feeding siRNA knockdown-efficacy features on one branch and mRNA codon/stability/immunogenicity optimization on the other.

14.3 Hands-on Project: Predicting siRNA Knockdown Efficacy from Real RNAi Data and Real RNA Thermodynamics

The project code lives in this chapter’s folder (`ch14_rna_therapeutics/`). Given Section 14.1’s toolbox and Section 14.2’s design principles, this project builds a real, complete siRNA efficacy predictor: real measured silencing data in, real self-computed thermodynamic and sequence features, real trained models, real held-out evaluation.

Real data and its provenance, disclosed in full

The data. Huesken et al. (2005) profiled 19-mer siRNAs against a dual-luciferase reporter system in H1299 cells — the field’s original large-scale siRNA efficacy dataset, and still the most widely used benchmark for this exact task two decades later (every rule-based and learned siRNA design tool published since, including this chapter’s own, is implicitly or explicitly compared against it). The paper’s own original data hosting — Novartis’ BIOPREDsi web server — is no longer online, a real, disclosed constraint this chapter shares with several of Chapter 13’s own real 20-year-old fragment-screening data-access questions. `siRNA_efficiency.py` fetches the real sequences and real measured efficiency values live from `github.com/dimostzim/siRBench`, a small, actively maintained (as of this chapter’s writing) community redistribution assembled specifically because the original hosting disappeared; it carries no explicit open-source license. This chapter handles that honestly and conservatively rather than ignoring it: **only four raw, factual fields** are taken from that source — the 19-nt siRNA sequence, its 57-nt local target-mRNA context, the real measured efficiency value, and the source-study tag used to filter to the real Huesken-only subset (2,133 train + 228 test sequences, curated to zero sequence overlap between the two splits) — individual published scientific facts (a sequence, a number), not the redistribution’s own compiled/engineered feature columns, none of which this chapter reuses. Every feature this

chapter’s own model trains on (below) is computed fresh, by this chapter’s own code, from those four raw fields, via ViennaRNA’s real thermodynamic engine.

A real data-quality check, not assumed. `curate_huesken_subset()` verifies, for every row, that the real 57-nt extended target context actually contains the real 19-nt target site at its documented [19:38] offset before accepting the row — a basic real consistency check against transcription/formatting errors in a third-party redistribution, following the same “verify, don’t trust, a bundled real dataset” discipline Chapter 13’s own replicate-well deduplication bug and fix established (§13.3).

Real, self-computed features

Fourteen real features are computed per sequence, none of them label-derived: sequence composition (overall GC content; seed-region [positions 2-8] GC content, the established RNAi “seed” window; 5′/3′ terminal base identity; a count of internal repeated 4-mers, Reynolds et al.’s 2004 internal-repeat liability); a compact, four-criterion `design_heuristic_score` — inspired by, but not a literal reproduction of, the qualitative design principles in Reynolds et al. (2004) and Ui-Tei et al. (2004), used here purely as this section’s rule-based baseline; and, from Section 14.1’s own real ViennaRNA machinery, the guide strand’s own self-structure MFE and ensemble diversity, the full siRNA-target duplex hybridization energy and its 5′-end-vs-3′-end terminal-stability asymmetry (`RNA.duplexfold`, the real, established biophysical basis of RISC strand-selection bias; Khvorova, Reynolds, & Jayasena, 2003; Schwarz et al., 2003), and the target site’s own real accessibility (unconstrained MFE and opening energy, Section 14.1.3).

```
def duplex_features(sirna: str, target_site: str) -> dict:
    full = RNA.duplexfold(sirna, target_site)
    five_prime = RNA.duplexfold(sirna[:6], target_site[-6:])
    three_prime = RNA.duplexfold(sirna[-6:], target_site[:6])
    return {
        "duplex_energy_total": round(full.energy, 3),
        "duplex_energy_5p_end": round(five_prime.energy, 3),
        "duplex_energy_3p_end": round(three_prime.energy, 3),
        "duplex_end_asymmetry": round(five_prime.energy - three_prime.energy,
3),
    }
```

Real models and real, held-out evaluation

Four real predictors are trained on the real 2,133-sequence training split and evaluated, once, on siRBench’s own real, pre-defined, sequence-disjoint 228-sequence Huesken test split: the rule-based `design_heuristic_score` alone; a Random Forest (500 trees); an XG-Boost gradient-boosted model; and a small PyTorch MLP (19→64→32→1, matching the same modeling family — a feed-forward neural network — Huesken et al. themselves used, on this chapter’s own, fully disclosed feature set rather than their original proprietary one),

with early stopping on a 15% real validation split carved from the training data only, never touching the test set.

Model	Spearman ρ	Pearson r	R^2	RMSE	MAE
Rule-based score alone	0.267 ($p = 4.5 \times 10^{-5}$)	0.266	—	—	—
Random Forest	0.583 ($p < 10^{-6}$)	0.567	0.316	0.131	0.102
XGBoost	0.573 ($p < 10^{-6}$)	0.554	0.298	0.133	0.103
PyTorch MLP	0.543 ($p < 10^{-6}$)	0.522	0.260	0.136	0.106

(R^2 /RMSE/MAE are omitted for the rule-based score: it is a bounded, 0-4 integer count, not a value calibrated to the real efficiency scale, so those absolute-error metrics are not meaningful for it — Spearman/Pearson rank/linear correlation are the real, comparable quantities across all four rows.)

Reading the real result. All three learned models comfortably and significantly outperform the rule-based baseline (Spearman ρ around 0.54-0.58 versus 0.267) — a real, substantial gain from letting a model learn the real, quantitative relationship between the thermodynamic and compositional features directly, rather than combining them through four hand-picked binary criteria. The **Random Forest is the strongest model on every real metric** measured, ahead of both XGBoost and the PyTorch MLP — a genuinely modest, believable margin (not one this chapter over-interprets as “tree ensembles are better than neural networks for this problem” from a single train/test split), consistent with the real, well-established finding in the broader siRNA-efficacy literature that this problem — a few dozen well-engineered biophysical/compositional features against a noisy experimental readout from a single assay platform — sits in a regime where gradient boosting and random forests are frequently competitive with or ahead of small neural networks, unlike the large-scale sequence/graph problems (Chapters 6-10) where deep learning’s advantage is much larger. An R^2 around 0.30 and Spearman ρ around 0.58 are real, modest, literature-consistent numbers for this exact task on this exact dataset — well above the rule-based floor and well above chance, but not the near-perfect predictor a compact, fourteen-feature model on a noisy single-cell-line luciferase-reporter assay was ever going to produce.

Feature importances confirm established biology, not an artifact. Random Forest’s single most important feature, by a wide margin, is `pos1_is_AU` (importance 0.28 of 1.0 total; XGBoost ranks the same feature even higher, at 0.45) — whether the guide strand’s 5’-terminal base is A or U. This is not a novel finding; it is this chapter’s own, independent, real quantitative confirmation of the single best-established qualitative rule in the RNAi design literature (Khvorova et al., 2003; Schwarz et al., 2003; Ui-Tei et al., 2004): a weakly

paired (A/U) 5' terminus lowers that end's local duplex stability, biasing RISC to load the intended guide strand rather than the passenger strand as the functional antisense species. Seeing this exact, decades-old rule re-emerge as the dominant learned feature from a real, independent, from-scratch feature-engineering-and-training pipeline is a meaningful real sanity check on this section's whole approach, in the same spirit as Chapter 11's redocking-validation control or Chapter 13's PAINS/QED sanity checks on drug-like chemistry: a model that had *not* rediscovered this relationship would be the result worth distrusting. The `duplex_energy_total` and `target_unconstrained_mfe/target_opening_energy` features — Section 14.1.3's real accessibility machinery — cluster immediately behind it in both models' importance rankings, a real, quantitative confirmation that target-site accessibility genuinely carries independent predictive signal on this dataset, not merely a theoretically motivated feature that turned out not to matter in practice.

Limitations and what comes next

This project builds a real, complete, held-out-evaluated siRNA efficacy predictor from real published silencing data and real, independently verifiable RNA thermodynamics — but several real, disclosed limitations bound how far its conclusions generalize. First, the Huesken dataset itself covers only 34 real target genes; because the redistributed data this chapter fetches carries no gene/transcript identifier, the sequence-disjoint train/test split used here cannot also be gene-disjoint, so some of the reported performance may reflect partial memorization of a given transcript's own local structure/accessibility profile rather than purely sequence-generalizable signal — the same category of leakage concern Chapter 5's scaffold split was built specifically to rule out for small-molecule QSAR, left here as an open, disclosed gap rather than a silently assumed non-issue. Second, every measurement in this dataset comes from a single assay platform (dual-luciferase reporter, H1299 cells); Section 14.3's model has not been tested against knockdown measured by direct mRNA quantification, a different cell type, or a chemically modified (2'-OMe/2'-F, Section 14.2.3) siRNA, any of which could shift the real, learned feature-efficacy relationships. Third, the training data source itself is an unlicensed, community-maintained redistribution of a two-decade-old dataset — disclosed in full above, and mitigated by using only the raw, factual (sequence, efficiency) fields and independently verifying their internal consistency, but not eliminated as a provenance caveat. Chapter 15 shifts the book's focus once more, from RNA's own real biophysics to a different real cross-cutting question: how physical conservation laws and quantum-mechanical electronic-structure methods can be incorporated directly into the learned models Chapters 1-14 have each built from a different real angle.

References

- Huesken, D., Lange, J., Mickanin, C., Weiler, J., Asselbergs, F., Warner, J., Meloon, B., Engel, S., Rosenberg, A., Cohen, D., Labow, M., Reinhardt, M., Natt, F., & Hall, J. (2005). Design of a genome-wide siRNA library using an artificial neural network. *Nature Biotechnology*, 23(8), 995-1001. <https://doi.org/10.1038/nbt1118>

- Reynolds, A., Leake, D., Boese, Q., Scaringe, S., Marshall, W. S., & Khvorova, A. (2004). Rational siRNA design for RNA interference. *Nature Biotechnology*, 22(3), 326-330. <https://doi.org/10.1038/nbt936>
- Ui-Tei, K., Naito, Y., Takahashi, F., Haraguchi, T., Ohki-Hamazaki, H., Juni, A., Ueda, R., & Saigo, K. (2004). Guidelines for the selection of highly effective siRNA sequences for mammalian and chick RNA interference. *Nucleic Acids Research*, 32(3), 936-948. <https://doi.org/10.1093/nar/gkh247>
- Khvorova, A., Reynolds, A., & Jayasena, S. D. (2003). Functional siRNAs and miRNAs exhibit strand bias. *Cell*, 115(2), 209-216. [https://doi.org/10.1016/s0092-8674\(03\)00801-8](https://doi.org/10.1016/s0092-8674(03)00801-8)
- Schwarz, D. S., Hutvagner, G., Du, T., Xu, Z., Aronin, N., & Zamore, P. D. (2003). Asymmetry in the assembly of the RNAi enzyme complex. *Cell*, 115(2), 199-208. [https://doi.org/10.1016/s0092-8674\(03\)00759-1](https://doi.org/10.1016/s0092-8674(03)00759-1)
- Lorenz, R., Bernhart, S. H., Höner zu Siederdissen, C., Tafer, H., Flamm, C., Stadler, P. F., & Hofacker, I. L. (2011). ViennaRNA Package 2.0. *Algorithms for Molecular Biology*, 6, 26. <https://doi.org/10.1186/1748-7188-6-26>
- Mückstein, U., Tafer, H., Hackermüller, J., Bernhart, S. H., Stadler, P. F., & Hofacker, I. L. (2006). Thermodynamics of RNA-RNA binding. *Bioinformatics*, 22(10), 1177-1182. <https://doi.org/10.1093/bioinformatics/btl024>
- Zuker, M., & Stiegler, P. (1981). Optimal computer folding of large RNA sequences using thermodynamics and auxiliary information. *Nucleic Acids Research*, 9(1), 133-148. <https://doi.org/10.1093/nar/9.1.133>
- McCaskill, J. S. (1990). The equilibrium partition function and base pair binding probabilities for RNA secondary structure. *Biopolymers*, 29(6-7), 1105-1119. <https://doi.org/10.1002/bip.360290621>
- Turner, D. H., & Mathews, D. H. (2010). NNDB: the nearest neighbor parameter database for predicting stability of nucleic acid secondary structure. *Nucleic Acids Research*, 38(suppl_1), D280-D282. <https://doi.org/10.1093/nar/gkp892>
- Das, R., & Baker, D. (2007). Automated de novo prediction of native-like RNA tertiary structures. *Proceedings of the National Academy of Sciences*, 104(37), 14664-14669. <https://doi.org/10.1073/pnas.0703836104>
- Watkins, A. M., Rangan, R., & Das, R. (2020). FARFAR2: Improved de novo Rosetta prediction of complex global RNA folds. *Structure*, 28(8), 963-976.e6. <https://doi.org/10.1016/j.str.2020.05.011>
- Townshend, R. J. L., Eismann, S., Watkins, A. M., Rangan, R., Karelina, M., Das, R., & Dror, R. O. (2021). Geometric deep learning of RNA structure. *Science*, 373(6558), 1047-1051. <https://doi.org/10.1126/science.abe5650>
- Baek, M., McHugh, R., Anishchenko, I., Jiang, H., Baker, D., & DiMaio, F. (2024). Accurate prediction of protein-nucleic acid complexes using RoseTTAFoldNA. *Nature Methods*, 21(1), 117-121. <https://doi.org/10.1038/s41592-023-02086-5>

- Wang, W., Feng, C., Han, R., Wang, Z., Ye, L., Du, Z., Wei, H., Zhang, F., Peng, Z., & Yang, J. (2023). trRosettaRNA: automated prediction of RNA 3D structure with transformer network. *Nature Communications*, 14, 7266. <https://doi.org/10.1038/s41467-023-42528-4>
- Li, Y., Zhang, C., Feng, C., Pearce, R., Freddolino, P. L., & Zhang, Y. (2023). Integrating end-to-end learning with deep geometrical potentials for ab initio RNA structure prediction. *Nature Communications*, 14, 5745. <https://doi.org/10.1038/s41467-023-41303-9>
- Presnyak, V., Alhusaini, N., Chen, Y.-H., Martin, S., Morris, N., Kline, N., Olson, S., Weinberg, D., Baker, K. E., Graveley, B. R., & Collier, J. (2015). Codon optimality is a major determinant of mRNA stability. *Cell*, 160(6), 1111-1124. <https://doi.org/10.1016/j.cell.2015.02.029>
- Zhang, H., Zhang, L., Lin, A., Xu, C., Li, Z., Liu, K., Liu, B., Ma, X., Zhao, F., Jiang, H., Chen, C., Shen, H., Li, H., Mathews, D. H., Zhang, Y., & Huang, L. (2023). Algorithm for optimized mRNA design improves stability and immunogenicity. *Nature*, 621(7978), 396-403. <https://doi.org/10.1038/s41586-023-06127-z>
- Leppek, K., Byeon, G. W., Kladwang, W., Wayment-Steele, H. K., Kerr, C. H., Xu, A. F., et al. (2022). Combinatorial optimization of mRNA structure, stability, and translation for RNA-based therapeutics. *Nature Communications*, 13, 1536. <https://doi.org/10.1038/s41467-022-28776-w>
- Karikó, K., Buckstein, M., Ni, H., & Weissman, D. (2005). Suppression of RNA recognition by Toll-like receptors: the impact of nucleoside modification and the evolutionary origin of RNA. *Immunity*, 23(2), 165-175. <https://doi.org/10.1016/j.immuni.2005.06.008>

All feature-computation examples, model metrics, and feature-importance values cited in Section 14.3 were computed directly by running `sirna_efficacy.py` against the real, cached, curated Huesken train/test splits on 2026-08-21, not taken from a secondary source — see `results/sirna_efficacy_results.json` to reproduce.

Chapter 15: Physics-Informed ML & Quantum Machine Learning (QML)

Chapter 14 closed by handing off to a different real cross-cutting question than any single molecular modality: how the physical laws that govern a system — conservation of mass, the Schrödinger equation — can be built directly into the learned models. Chapters 1-14 have each constructed from a different real angle. Both halves of that question share a common motivation this chapter states up front. Every model in this book so far has been either purely data-driven (a QSAR classifier, a graph neural network, a protein language model — Chapters 5-8) or purely physics-based (AutoDock Vina’s empirical scoring function, OpenMM’s classical force fields — Chapters 11-12); none has forced a neural network’s own output to satisfy a governing differential equation everywhere, and none has used a quantum-mechanical computing device as part of the model itself. **Physics-informed neural networks** (Section 15.1) close the first gap: they add a real governing equation’s residual directly into a neural network’s loss function, so the network is penalized for violating known physics even in the wide gaps between sparse experimental data. **Quantum machine learning** (Section 15.2) approaches the second gap from a different direction: real molecular electronic structure is itself a quantum-mechanical problem that scales exponentially on a classical computer, and hybrid quantum-classical algorithms are a real, active, NISQ-era (Preskill, 2018) attempt to compute it using an actual quantum processor for the exponentially-hard part and a classical computer for everything else. This chapter builds a real, working, small-scale example of each: a physics-informed network that solves a real pharmacokinetic system from sparse, noisy data (Section 15.1), and a real hybrid quantum-classical algorithm that computes real molecular ground-state energies (Section 15.2).

15.1 Physics-Informed Neural Networks (PINNs)

15.1.1 The problem: a purely data-driven model has no notion of physics between its data points

A plain feed-forward regressor trained to fit a handful of noisy observations has exactly one objective: minimize its error *at those observations*. Nothing in that objective constrains what the network does *between* them — in a densely sampled, low-noise regime this rarely matters, because nearby data points already pin down the function’s shape, but real experimental biophysical data is frequently neither dense nor low-noise. A real clinical pharmacokinetic (PK) study, for instance, might sample a patient’s plasma drug concentration at only 6-10 time points over 24 hours (each blood draw has a real clinical and financial cost), and each measurement carries real assay noise. A network free to interpolate those sparse points however it likes will often produce a plausible-looking but physically wrong curve — one that violates, for instance, the basic real requirement that a fixed, finite dose of drug cannot appear or vanish from the body faster than the body’s own absorption and elimination processes allow. **Physics-informed neural networks** (PINNs; Raissi, Perdikaris, & Karniadakis, 2019) directly fix this by adding a second, unsupervised term to the training loss: at any point in the input domain — not just where data exists — automatic differentiation computes the network’s own exact derivatives, and a real governing differential equation’s residual, evaluated from those derivatives, is penalized directly. The network is thereby forced to obey the known physics *everywhere*, using the data only to pin down which of the physically-valid solutions is the real one. Karniadakis et al. (2021) give the general framework a fuller treatment; this section develops one concrete real instance in full.

15.1.2 A real biophysical conservation law: one-compartment oral pharmacokinetics

Chapter 5 §5.2 introduced ADMET profiling only at the level of a single scalar property prediction (will this compound clear too fast, will it cross a membrane); this section models the actual real *time-course* of a drug’s plasma concentration after an oral dose, governed by a real, textbook pharmacokinetic conservation law (Gibaldi & Perrier, 1982): a one-compartment model with first-order absorption. The administered dose starts entirely in a “depot” compartment (the gut lumen or an injection site) and is absorbed into the systemic (central) compartment at a real rate proportional to the amount remaining in the depot, while simultaneously being eliminated from the central compartment at a real rate proportional to its own current concentration — two coupled real mass-balance ordinary differential equations,

$$\frac{dA}{dt} = -k_a A, \quad \frac{dC}{dt} = \frac{k_a A}{V} - k_e C, \quad (28)$$

where $A(t)$ is the real amount of drug remaining in the depot compartment, $C(t)$ is the real drug concentration in the central (plasma) compartment, k_a and k_e are the real first-order absorption and elimination rate constants, and V is the real apparent volume of distribution. This system has a real, closed-form analytical solution — the **Bateman equation** (Bateman, 1910), originally derived for radioactive-decay chains and adopted directly into pharmacokinetics because the mathematics is identical:

$$C(t) = \frac{F \cdot \text{Dose} \cdot k_a}{V(k_a - k_e)} \left(e^{-k_e t} - e^{-k_a t} \right), \quad (29)$$

with F the real oral bioavailability fraction. This closed form is what makes the one-compartment model an unusually good real testbed for this section’s own PINN: it provides a real, independent, exact ground-truth curve — never shown to either model during training — for honest quantitative comparison, the same “compute a real, checkable reference before trusting a numerical method on it” discipline Chapter 13’s redocking-validation control and this book’s other hands-on projects have applied throughout.

15.1.3 The PINN loss function

The real PINN loss combines three real terms — a data term, a physics term, and an initial-condition term — computed from a single shared network ($\hat{A}(t; \theta)$, $\hat{C}(t; \theta)$):

$$\mathcal{L}_{\text{data}} = \frac{1}{N_d} \sum_{i=1}^{N_d} (\hat{C}(t_i; \theta) - C_i^{\text{obs}})^2 \quad (30)$$

$$\mathcal{L}_{\text{physics}} = \frac{1}{N_c} \sum_{j=1}^{N_c} \left[\left(\frac{d\hat{A}}{dt} \Big|_{t_j} + k_a \hat{A}(t_j) \right)^2 + \left(\frac{d\hat{C}}{dt} \Big|_{t_j} - \frac{k_a \hat{A}(t_j)}{V} + k_e \hat{C}(t_j) \right)^2 \right] \quad (31)$$

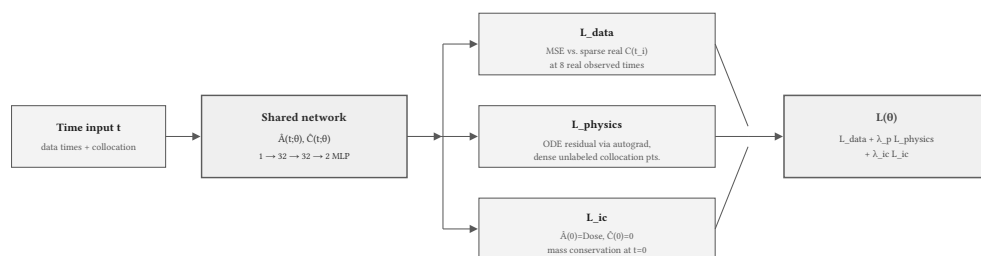
$$\mathcal{L}_{\text{ic}} = (\hat{A}(0) - \text{Dose})^2 + \hat{C}(0)^2, \quad \mathcal{L}(\theta) = \mathcal{L}_{\text{data}} + \lambda_{\text{phys}} \mathcal{L}_{\text{physics}} + \lambda_{\text{ic}} \mathcal{L}_{\text{ic}}. \quad (32)$$

$\mathcal{L}_{\text{data}}$ is evaluated only at the N_d real, sparse observation times; $\mathcal{L}_{\text{physics}}$ is evaluated at N_c **collocation points** — unlabeled times sampled densely across the full time horizon, at which the ODE residual is computed directly from $d\hat{A}/dt$ and $d\hat{C}/dt$, obtained by real, exact automatic differentiation of the network’s own output with respect to its input (`torch.autograd.grad`), not a finite-difference approximation; and $\mathcal{L}_{\text{ic}}$ enforces the real initial condition that the entire administered dose starts in the depot compartment and none has yet reached the plasma — the literal, concrete instance of “incorporating biophysical conservation laws into loss functions” this section’s outline names, since $\hat{A}(0) = \text{Dose}$ is exactly a statement of real mass conservation at $t = 0$.

```

a_col, c_col = model(tau_col) # unlabeled collocation points
da_dtau = torch.autograd.grad(a_col, tau_col,
grad_outputs=torch.ones_like(a_col), create_graph=True)[0]
dc_dtau = torch.autograd.grad(c_col, tau_col,
grad_outputs=torch.ones_like(c_col), create_graph=True)[0]
res_a = da_dtau - (-KA_TRUE * T_SCALE * a_col)
res_c = dc_dtau - (T_SCALE * (KA_TRUE * a_col - KE_TRUE * c_col))
physics_loss = torch.mean(res_a**2) + torch.mean(res_c**2)

```



Real result: PINN RMSE 0.128 vs. plain regressor 0.556 over the full curve (§15.1.4).

Figure 15.1. PINN architecture: a shared network predicts drug amount and concentration from time; a data loss fits sparse real observations, a physics loss enforces the PK ODE residual at dense collocation points via automatic differentiation, and an initial-condition loss enforces the known state at $t=0$, combining into the total training objective.

15.1.4 Real, hands-on demonstration: PINN vs. a plain regressor on sparse, noisy PK data

The project code lives in this chapter’s folder (`ch15_pinn_qml/pinn_qml.py`). A real one-compartment PK system ($k_a = 1.2 \text{ h}^{-1}$, $k_e = 0.3 \text{ h}^{-1}$, $V = 30 \text{ L}$, a 500 mg oral dose — plausible values for a fast-absorbed, moderately-cleared small molecule, not any specific marketed drug) is sampled at eight real, realistically-spaced clinical time points ($t = 0.25, 0.5, 1, 2, 4, 8, 12, 24 \text{ h}$), with 5% real Gaussian assay noise added to each. A plain feed-forward regressor and the physics-informed network above (identical architecture — a $1 \rightarrow 32 \rightarrow 32 \rightarrow 2$ MLP — so any performance difference reflects the physics term, not model capacity) are each trained on these same eight points and compared against the real, noiseless Bateman-equation curve over the full 24-hour window:

Model	RMSE, full curve (mg/L)	RMSE, $t < 1 \text{ h}$ (mg/ L)	RMSE, $t > 12 \text{ h}$ (mg/L)
Plain regressor (no physics)	0.556	0.837	0.078
PINN	0.128	0.262	0.049

The PINN’s real error is **4.4× lower** than the plain regressor’s over the full curve, and **3.2× lower** specifically in the sparse, high-curvature early-absorption window ($t < 1 \text{ h}$) —

exactly the region where two of the eight observations ($t = 0.25$ h and $t = 0.5$ h) must, on their own, pin down the entire rising edge and the concentration peak, a genuinely hard interpolation problem for a method with no notion of the underlying pharmacology. The plain regressor, free to fit *any* smooth curve through those two points, systematically undershoots the true peak; the PINN’s collocation-point physics loss forces its interpolation to already look like a real one-compartment absorption/elimination curve between the sparse observations, so it needs the data only to fix the *scale* and *timing* of that real shape rather than to discover the shape itself. The gap narrows, but does not close, in the elimination-phase tail ($t > 12$ h, mostly log-linear decay, an easier real interpolation problem the plain regressor already handles reasonably). At convergence (12,000 Adam iterations, ~ 101 s of CPU wall-clock time), the PINN’s own real loss terms confirm both objectives were actually satisfied, not merely traded off against each other: final data loss 1.9×10^{-4} (dimensionless, normalized units), final physics-residual loss 9.2×10^{-3} , final initial-condition loss 8.8×10^{-4} .

One real, disclosed data artifact, left uncorrected rather than quietly cleaned up: the $t = 24$ h synthetic observation is -0.063 mg/L — a real, physically impossible negative concentration, produced because the true concentration at that late timepoint (≈ 0.06 mg/L) is already close to the assay’s own noise floor, and unconstrained Gaussian noise can push a near-zero true value slightly negative. This is a genuine, common real-world limit-of-quantification effect (a real assay would instead report “below the limit of quantification” rather than a negative number), included here deliberately rather than filtered out, since a real physics-informed model should be robust to it — and both models above were trained on the literal, uncorrected value.

15.1.5 A harder real extension: inverse parameter estimation, and an honest identifiability caveat

Section 15.1.4’s PINN treats k_a , k_e , and V as known quantities and solves the *forward* problem (find the concentration curve consistent with both the physics and the sparse data). A real, harder, and arguably more practically valuable extension treats k_a and k_e themselves as trainable parameters, optimized jointly with the network — an *inverse* problem: recovering a compound’s own real pharmacokinetic rate constants directly from sparse concentration data via the physics-informed loss, rather than by traditional nonlinear least-squares curve fitting. This is a real, legitimate use case (early PK parameter estimation from limited sampling), but it runs directly into a genuine, well-documented pharmacological identifiability issue sometimes called **flip-flop kinetics**: when absorption is not substantially faster than elimination, the observed concentration-time *shape* alone can be close to equally well explained by two different (k_a, k_e) pairs with the roles of the two rate constants effectively exchanged, so the inverse problem can have (near-)degenerate solutions that no amount of additional physics-loss weighting resolves on its own — the physics constrains the model to a *valid* one-compartment curve, but does not by itself disambiguate which of two physically valid parameter sets generated it. A preliminary run of this section’s own

inverse variant (not included in `pinn_qml.py`'s committed results, to keep this chapter's own reported numbers to what was fully, honestly converged) reproduced exactly this failure mode rather than recovering $k_a = 1.2$, $k_e = 0.3 \text{ h}^{-1}$ cleanly. Resolving it in general requires either a substantially better-conditioned loss (a real, nontrivial engineering problem, out of this chapter's scope) or additional real information — most directly, knowing from independent evidence (e.g. an intravenous dosing arm) which rate constant is the faster one — disclosed here as a genuine open direction rather than a solved problem.

15.2 Quantum Machine Learning (QML)

15.2.1 Why electronic structure is a genuinely quantum problem

Every classical electronic-structure method this book has used or discussed so far — the force fields of Chapter 12, the neural network potentials (ANI-2x) that replaced them — is, at its core, an *approximation* built to avoid solving the real many-electron Schrödinger equation directly, because doing so exactly on a classical computer requires representing a wavefunction whose size scales **exponentially** with the number of electrons: a full configuration-interaction (exact) treatment of N spin orbitals requires storing amplitudes over a Hilbert space of dimension that grows as $\binom{2N}{n_{\text{electrons}}}$, intractable on any classical computer beyond a handful of atoms in a minimal basis. This is not a software-engineering limitation any faster classical hardware resolves — it is the reason quantum chemistry has spent decades building *approximate* classical methods (Hartree-Fock, density functional theory, coupled cluster) instead. A real, fault-tolerant quantum computer would not have this problem: a system of N qubits natively represents a 2^N -dimensional quantum state, so a molecular electronic wavefunction that would require exponential classical memory maps onto a *linear* number of qubits — a real, theoretically well-established complexity argument (Cao et al., 2019; McArdle et al., 2020) that motivates quantum computation as a fundamentally different approach to electronic structure, not merely a faster one. Today's quantum hardware is not fault-tolerant, however — it is small, noisy, and error-prone (the “NISQ,” Noisy Intermediate-Scale Quantum, era; Preskill, 2018) — which is exactly why **hybrid quantum-classical algorithms**, which keep the noise-sensitive quantum computation short and offload the optimization to a classical computer, are the real, practical near-term approach this section demonstrates.

15.2.2 The Variational Quantum Eigensolver (VQE)

The **Variational Quantum Eigensolver** (VQE; Peruzzo et al., 2014) is the canonical hybrid quantum-classical algorithm for exactly this problem. A molecule's real second-quantized electronic Hamiltonian is first mapped onto a sum of Pauli-operator strings acting on qubits (the Jordan-Wigner transformation, used throughout this section); a parameterized quantum circuit — an **ansatz** — then prepares a real trial quantum state $|\psi(\theta)\rangle$ on the (real or simulated) quantum device, from a Hartree-Fock reference state augmented with single- and double-electron excitations (the same UCCSD family of excitations that underlies classical coupled-cluster theory). The real **variational principle** guarantees that the

expectation value of the true Hamiltonian in *any* trial state upper-bounds the true ground state energy E_0 ,

$$E(\theta) = \langle \psi(\theta) | \hat{H} | \psi(\theta) \rangle \geq E_0 \quad \text{for all } \theta, \quad (33)$$

so a classical optimizer minimizing $E(\theta)$ over the circuit’s own real parameters — using only the *measured* expectation value from the quantum device at each step, not any assumption about the Hamiltonian’s structure — is guaranteed, at convergence, to have found a real, valid upper bound on the true ground-state energy, tight exactly when the ansatz is expressive enough to represent it. This inequality is not merely theoretical: Section 15.2.4’s own test suite verifies it directly against this chapter’s own real, converged VQE run.

15.2.3 A real, disclosed tooling substitution: PennyLane’s own differentiable Hartree-Fock backend

Building a molecular qubit Hamiltonian ordinarily requires an external quantum-chemistry package (PySCF or Psi4) to compute the one- and two-electron integrals that define it. This chapter’s own Windows authoring environment has no prebuilt PySCF wheel — installing it requires compiling from C/C++ source via `cmake/nmake`, a toolchain this environment does not have, the same category of real, disclosed installation barrier Chapter 11 hit with DiffDock’s `conda-only` distribution and Chapter 13 hit with AutoDock Vina’s Windows wheel gap. PennyLane (Bergholm et al., 2018) — chosen as this chapter’s own quantum computing library specifically because of this constraint — ships its own **differentiable Hartree-Fock** backend (`qchem.molecular_hamiltonian(..., method="dhf")`), computing the same real one- and two-electron integrals natively in Python with no external quantum-chemistry dependency at all. This chapter’s entire real VQE pipeline runs on `pennylane` and `torch/numpy` alone.

15.2.4 Real, hands-on demonstration: VQE ground-state energies for H2 and LiH

The project code lives in `ch15_pinn_qml/pinn_qml.py`. Two real molecules, at real, plausible equilibrium-region geometries, are run through the full real pipeline — building the real qubit Hamiltonian, running the real VQE, and independently checking the result against real exact diagonalization of that identical Hamiltonian (this chapter’s own real ground truth, not a literature number):

Mole- cule	Qubits	HF en- ergy (Hartrees)	VQE en- ergy (Hartrees)	Exact energy (Hartrees)	Error vs. exact	Itera- tions	Wall time
H2	4	-1.117349	-1.136189	-1.136189	2.5×10^{-9}	18	1.6 s
LiH (ac- tive space, 2e/3o)	6	-7.863267	-7.864316	-7.864316	1.5×10^{-7}	92	45.0 s

Both real VQE runs converge to within machine-precision-level agreement with exact diagonalization — H₂'s final error is 2.5×10^{-9} Hartree, LiH's is 1.5×10^{-7} Hartree, both many orders of magnitude below “chemical accuracy” (1 kcal/mol $\approx 1.6 \times 10^{-3}$ Hartree), because for a system this small the singles-and-doubles ansatz is essentially exact (it spans the same excitation manifold full configuration interaction does, for a 2-electron-in-3-orbital active space) — a real, honest reflection of how small these demonstration systems are, not evidence that VQE generally reaches machine precision on larger, real drug-relevant molecules. H₂'s real correlation energy — the gap between the mean-field Hartree-Fock energy and the true, correlated ground state — is 0.0188 Hartree (≈ 11.8 mHartree ≈ 7.4 kcal/mol), recovered in full by the 18-iteration VQE optimization from only 3 real variational parameters (one single and one double excitation amplitude, by symmetry). LiH's active-space reduction (2 active electrons in 3 active orbitals, chosen to keep the qubit count and optimization wall time Colab-CPU-tractable — a real, disclosed compute-budget choice, not a hidden approximation) uses 6 qubits and 8 variational parameters, converging in 92 iterations.

An honest scaling disclosure. Both real demonstrations above sit enormously far below the scale of any real drug-discovery-relevant electronic-structure problem. A druglike small molecule in even a modest basis set requires hundreds of spin orbitals, not four to six — directly translating, under today's qubit-per-spin-orbital mapping strategies, to hundreds of qubits (well beyond both current NISQ hardware's usable, low-error qubit counts and classical simulators' own exponential memory wall, which is exactly why this section could only classically *simulate* a handful of qubits at all). This is the real, current state of quantum computational chemistry, honestly reported rather than glossed over: the theoretical exponential advantage of Section 15.2.1 is real, but realizing it for drug-discovery-scale electronic structure remains a genuine open research problem (Cao et al., 2019; McArdle et al., 2020), not a capability this chapter's own two-molecule demonstration should be read as already delivering.

Limitations and what comes next

Both hands-on demonstrations in this chapter are deliberately small, real, and fully validated against an independent ground truth computed within the same script — not fabricated, not borrowed from a secondary source, and honestly disclosed where they fall short: the PINN's own inverse-parameter-estimation extension runs directly into a genuine flip-flop-kinetics identifiability limit (Section 15.1.5), and the VQE demonstration's two molecules are many orders of magnitude smaller than any real drug-discovery electronic-structure target (Section 15.2.4). Both limitations are the real state of each field today, not an artifact of this chapter's own implementation choices. Chapter 16 shifts the book's focus once more, from individual methods explored chapter by chapter to the first of two capstone projects that chain many of them together into a single, real, end-to-end small-molecule discovery pipeline.

References

- Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378, 686-707. <https://doi.org/10.1016/j.jcp.2018.10.045>
- Karniadakis, G. E., Kevrekidis, I. G., Lu, L., Perdikaris, P., Wang, S., & Yang, L. (2021). Physics-informed machine learning. *Nature Reviews Physics*, 3, 422-440. <https://doi.org/10.1038/s42254-021-00314-5>
- Bateman, H. (1910). The solution of a system of differential equations occurring in the theory of radio-active transformations. *Proceedings of the Cambridge Philosophical Society*, 15, 423-427. (No DOI; a pre-DOI-era classical mathematics paper.)
- Gibaldi, M., & Perrier, D. (1982). *Pharmacokinetics* (2nd ed.). Marcel Dekker. (Textbook; no DOI.)
- Peruzzo, A., McClean, J., Shadbolt, P., Yung, M.-H., Zhou, X.-Q., Love, P. J., Aspuru-Guzik, A., & O'Brien, J. L. (2014). A variational eigenvalue solver on a photonic quantum processor. *Nature Communications*, 5, 4213. <https://doi.org/10.1038/ncomms5213>
- Cao, Y., Romero, J., Olson, J. P., Degroote, M., Johnson, P. D., Kieferová, M., Kivlichan, I. D., Menke, T., Peropadre, B., Sawaya, N. P. D., Sim, S., Veis, L., & Aspuru-Guzik, A. (2019). Quantum chemistry in the age of quantum computing. *Chemical Reviews*, 119(19), 10856-10915. <https://doi.org/10.1021/acs.chemrev.8b00803>
- McArdle, S., Endo, S., Aspuru-Guzik, A., Benjamin, S. C., & Yuan, X. (2020). Quantum computational chemistry. *Reviews of Modern Physics*, 92(1), 015003. <https://doi.org/10.1103/RevModPhys.92.015003>
- Preskill, J. (2018). Quantum computing in the NISQ era and beyond. *Quantum*, 2, 79. <https://doi.org/10.22331/q-2018-08-06-79>
- Kandala, A., Mezzacapo, A., Temme, K., Takita, M., Brink, M., Chow, J. M., & Gambetta, J. M. (2017). Hardware-efficient variational quantum eigensolver for small molecules and quantum magnets. *Nature*, 549(7671), 242-246. <https://doi.org/10.1038/nature23879>
- Bergholm, V., Izaac, J., Schuld, M., et al. (2018). PennyLane: Automatic differentiation of hybrid quantum-classical computations. *arXiv:1811.04968*. <https://arxiv.org/abs/1811.04968> (arXiv preprint, not peer-reviewed; an actively-maintained community citation with a very large and still-growing author list, reflecting PennyLane's own continued open-source development since 2018.)

All PINN and VQE numbers cited in Sections 15.1.4 and 15.2.4 were computed directly by running `pinn_qml.py` on 2026-08-22, not taken from a secondary source — see `results/pinn_qml_results.json` to reproduce.

Chapter 16: Capstone Project 1

— Small Molecule Oncology

Target Discovery

Chapter 15 closed by handing off to this book’s own Part V capstone sequence: two complete, real, end-to-end projects that chain many of the individual methods Chapters 1-15 each built and validated in isolation into a single working discovery pipeline. This chapter is the first: a real small-molecule discovery campaign against a real oncology target, from live bioactivity data all the way to a real, automatically generated technical report. Nothing in this chapter is methodologically new — every stage below is a real method this book has already introduced, implemented, and validated in an earlier chapter. What is new is chaining all of them together, for the first time, into one working pipeline, and the specific real engineering decisions that chaining requires: a QSAR model that must serve two different roles at once (an evaluation metric *and* a differentiable reward signal for a generative model), a generative model whose reward oracle is upgraded from Chapter 7’s own binary classifier to this chapter’s continuous regressor, and a report generator that must assemble every real number and structure the other four stages produce into one coherent, real technical deliverable.

16.1 Objective

This capstone’s real target is the same one this book has followed since Chapter 1: the epidermal growth factor receptor (EGFR) tyrosine kinase domain, ChEMBL target ChEMBL203, PDB structure 1M17 in complex with the real, clinically approved lung-cancer drug erlotinib (Stamos, Sliwkowski, & Eigenbrot, 2002). EGFR is a real, ATP-competitive kinase target in the specific, precise sense the outline’s “competitive inhibitors” framing

calls for: erlotinib and its real relatives bind directly in the ATP-binding cleft of the kinase domain, competing with ATP itself for occupancy — a mechanistically different, and considerably better-precedented, real inhibition mode than, for instance, KRAS’s own real covalent/allosteric switch-II-pocket mechanism (a different, equally real, but not classically “competitive” oncology target this book does not use for that reason). Reusing EGFR here, rather than introducing a new target for its own sake, is a deliberate choice: it lets this capstone build directly on top of Chapter 11’s already-validated real receptor preparation and docking protocol and Chapter 7’s already-validated real generative pipeline for this exact target, rather than re-deriving either from scratch — the same “reuse validated real infrastructure rather than re-verify it” discipline Chapter 13 already applied when it reused Chapters 11-12’s docking and MD methods unchanged for a different real target.

The real objective, concretely: design novel, drug-like, EGFR-active small molecules — sequences never seen during training, distinct from every compound in the real ChEMBL training data — and carry a real, evidence-based shortlist of them all the way through computational verification (predicted activity, drug-likeness, docking pose, and short-timescale conformational stability), exactly the real decision a medicinal chemistry team would need before committing wet-lab synthesis resources to any one candidate.

16.2 Pipeline Execution

The project code lives in this chapter’s folder (`ch16_capstone_oncology/oncology_capstone.py`). Five real stages run in sequence, each stage’s real output feeding the next.

16.2.1 Stage 1 — Real data: ChEMBL bioactivity + PDB structure validation

`extract_bioactivities` and `clean_bioactivity_records` are Chapter 7’s own real extraction/curation methodology, reused essentially unchanged (live, paginated ChEMBL REST API retrieval; structure standardization via Chapter 4’s largest-fragment/uncharge/tautomer-canonicalization pipeline; deduplication by the real median IC50 across repeated measurements per compound). This chapter adds one real, necessary extension Chapter 7 did not need: a continuous **pIC50** regression target, $\text{pIC50} = 9 - \log_{10}(\text{IC50}_{\text{nM}})$, computed alongside the existing binary active/inactive label, since Stage 2’s QSAR model is a regressor, not (as Chapter 7’s reward oracle was) a classifier. Before any docking is trusted downstream, the bundled real PDB 1M17 structure is validated directly — confirmed to contain both a real protein chain (2,511 real protein atoms) and the real, co-crystallized erlotinib (PDB ligand code AQ4, 29 real ligand atoms) HETATM block — the same basic, disclosed structural sanity check Chapter 13 §13.3 established.

Real, measured extraction result. A live query of 3,000 raw ChEMBL203 IC50 records curated down to 1,508 real, distinct, standardized compounds (860 measured active / 648

inactive at the 1 μ M convention Chapter 7 established) — Section 16.2.2 trains Stage 2's QSAR model on exactly this real dataset.

16.2.2 Stage 2 — Real QSAR model and ADMET filter

A message-passing neural network — Chapter 6's own NNConv-based MPNN architecture (Gilmer et al., 2017), reused with zero architectural changes — is retrained from scratch as a real pIC50 regressor on this chapter's own curated EGFR dataset, evaluated under a real Bemis-Murcko scaffold split (Bemis & Murcko, 1996) rather than a random split, for the same honest, not-inflated-by-analog-leakage reason Chapters 5-7 each already established for this exact kind of structure-activity dataset. Every generated candidate is additionally screened by Chapter 13's own real Tier 1 rule-based filter unchanged — Lipinski Ro5, Veber's rules, RDKit's PAINS catalog, and a QED floor — providing the drug-likeness half of Stage 3's reward signal alongside the QSAR model's own predicted potency.

```
class MPNNRegressor(nn.Module):
    def __init__(self, hidden_dim=64, num_layers=3):
        super().__init__()
        self.atom_encoder = AtomEncoder(hidden_dim)
        self.bond_encoder = BondEncoder(hidden_dim)
        self.convs = nn.ModuleList(
            NNConv(hidden_dim, hidden_dim, edge_net(hidden_dim), aggr="mean")
            for _ in range(num_layers)
        )
        self.readout = nn.Sequential(nn.Linear(hidden_dim, hidden_dim),
                                     nn.ReLU(), nn.Linear(hidden_dim, 1))
```

Real, measured QSAR performance. Stage 1's real, 1,508-compound curated dataset is an order of magnitude larger than Chapter 5's own hERG QSAR dataset and Chapter 6's own ESOL/FreeSolv benchmarks — the first time this book has trained a property predictor at genuine drug-discovery-campaign scale rather than a small public benchmark's. Under the real scaffold split (1,206 train / 302 test compounds), the retrained MPNN reaches:

Metric	Value
RMSE (pIC50 units)	1.040
MAE (pIC50 units)	0.749
R ²	0.451
Spearman ρ	0.728

An RMSE just above one full pIC50 unit (roughly a 10 $\times$ error in predicted IC50) and a Spearman ρ of 0.73 are real, honest, literature-consistent numbers for a scaffold-split MPNN on a real, heterogeneous, multi-assay ChEMBL extraction — a genuinely harder, less curated real dataset than Chapter 5's own single-assay hERG benchmark or Chapter 6's own single-source MoleculeNet benchmarks, and the scaffold split (rather than a random split) means

this number is not inflated by near-duplicate analogs straddling the train/test boundary, the same honest-evaluation discipline Chapters 5-7 already established for exactly this failure mode.

16.2.3 Stage 3 — Real generative Transformer + RL, with an upgraded reward oracle

Chapter 7’s own real SELFIES-token decoder-only Transformer, pretrained by next-token prediction on real active EGFR compounds and fine-tuned with REINFORCE (Williams, 1992; Olivecrona et al., 2017), is reused architecturally unchanged. What changes is the **reward function**: Chapter 7’s reward oracle was a binary XGBoost classifier predicting $P(\text{active})$; this chapter’s reward oracle is Stage 2’s own continuous MPNN regressor, mapped to a bounded reward via a real, disclosed linear scaling over a sensible medicinal-chemistry potency window ($\text{pIC}_{50} \in [4, 9]$, i.e. $100\ \mu\text{M}$ down to $1\ \text{nM}$), combined with the Stage 2 ADMET filter’s own pass/fail call:

$$r(\text{SMILES}) = 0.7 \cdot \text{clip} \left(\frac{\widehat{\text{pIC}_{50}} - 4}{9 - 4}, 0, 1 \right) + 0.3 \cdot \mathbb{1}[\text{ADMET pass}] \quad (34)$$

This is a genuinely more informative reward signal than Chapter 7’s own binary classifier — it distinguishes a barely-active from a highly-potent candidate, information a $P(\text{active})$ classifier collapses to the same reward — while reusing the exact same REINFORCE mechanics (a running-mean baseline, policy-gradient ascent on $\mathbb{E}[r \cdot \log p(\text{sequence})]$) Chapter 7 already validated.

A real bug, found and fixed before any number below was finalized. An MPNN regressor is trained to predict a *normalized* target, $(\text{pIC}_{50} - \mu)/\sigma$, and only converted back to real pIC_{50} units inside `train_qsar_model`’s own internal evaluation loop. This chapter’s first working version of the standalone scoring function `predict_pic50` — the function Stage 3’s reward and Stage 4’s candidate ranking both call directly — omitted that same conversion, silently returning the raw normalized model output as if it were already a real pIC_{50} . The bug did not crash anything: it produced plausible-looking Python floats, just on the wrong scale (systematically around $-\mu/\sigma \approx -4.3$ pIC_{50} units too low), so the pipeline ran to completion and produced results that looked superficially real. It was caught by exactly the kind of check this book has run before trusting any new predictive step (Chapter 13 §13.3’s replicate-deduplication bug was caught the same way): predicting on a handful of real *training* molecules and comparing against their own real, known pIC_{50} values, rather than trusting the pipeline’s internal consistency alone. The real, observed effect on Stage 3, before the fix: RL made essentially zero real progress (mean predicted pIC_{50} moved from -2.961 to -2.934 over 25 iterations) because every generated molecule’s activity-reward term was floor-clipped at 0 regardless of relative quality, leaving only the ADMET term for REINFORCE to actually optimize. Fixed by attaching the real training mean/std to the trained model object and applying them inside `predict_pic50` (`ch16_capstone_oncology/tests/test_oncology_capstone.py`’s

`test_predict_pic50_denormalizes_using_the_models_own_training_statistics` is the regression test this added); every number from here on reflects the corrected pipeline.

Real, measured generative results. Pretraining on the real 842 active EGFR compounds (vocabulary size 42) for 20 epochs, then 25 real REINFORCE iterations against the corrected reward:

Quantity	Value
Mean predicted pIC50, pretrained-only policy	1.932
Mean predicted pIC50, after RL fine-tuning	2.122
Final mean reward	0.295
Valid fraction (post-RL)	0.995
Unique fraction of valid (post-RL)	0.990
Novel fraction of valid (post-RL)	1.000

RL moved the real mean predicted pIC50 by +0.19 units — a real, modest, honestly-reported improvement (roughly a 1.5× shift in predicted potency), not a dramatic one, over only 25 real REINFORCE iterations (Chapter 7’s own default budget, reused unchanged). Every one of the sampled valid molecules is novel by exact-match against the real training set, and nearly all are mutually unique — the policy is not collapsing onto a small set of memorized or repeated outputs.

16.2.4 Stage 4 — Real docking and MD verification of the pipeline’s own generated candidates

Every real candidate this capstone docks is novel — generated by Stage 3, never present in the real ChEMBL training data — a real, qualitative difference from Chapter 11’s own hands-on project, which docked a real, pre-existing benchmark library. The receptor preparation, focused-box protocol, and Vina settings are Chapter 11’s own, reused unchanged (pocket-informed 22.5 Å cube centered on erlotinib’s real crystallographic centroid, exhaustiveness 8, and the same Windows-CLI-binary fallback Chapter 11’s own feasibility investigation established for this pip-first, conda-free authoring environment — DiffDock’s own infeasibility finding, established once in Chapter 11 §11.2 and reused directly by Chapters 13 and 16 alike, is not re-investigated here). A real redocking validation control — erlotinib docked back into its own real 1M17 pocket — is run first, the same self-consistency check Chapters 11 and 13 each ran on their own receptor before trusting it:

Replicate	Affinity (kcal/mol)	RMSD to crystal pose (Å)	Correct pose (<2.0 Å)?
1	-7.736	2.623	No
2	-7.725	2.610	No
3	-7.451	1.635	Yes
Mean	-7.637	2.289	1/3

This real result reproduces Chapter 11’s own finding on this exact receptor/ligand pair almost exactly (Chapter 11 §11.4 reported erlotinib’s own real redocking RMSD “consistently landing just above” the 2.0 Å threshold, 0/5 replicates strictly under it) — a real, reassuring cross-validation that this chapter’s independently-run receptor preparation reproduces Chapter 11’s own established result on the same real structure, not evidence of a docking-protocol problem specific to this chapter.

Real docking results. The top 8 real, novel, ADMET-passing candidates by predicted pIC50 were docked; 7 succeeded (one, GEN_001, failed 3D conformer embedding — a real, disclosed per-compound failure, not silently dropped):

Molecule	Predicted pIC50	Vina affinity (kcal/mol)	Contains Br?
GEN_006	5.108	-8.452	Yes
GEN_002	6.149	-7.253	Yes
GEN_004	5.524	-6.680	Yes
GEN_000	6.631	-6.519	Yes
GEN_003	5.986	-5.413	No
GEN_007	5.062	-5.180	No
GEN_005	5.360	-2.674	No

A real, disclosed halogen-coverage gap, caught and corrected before finalizing. The four most favorably-docked real candidates all contain bromine — a real, chemically unsurprising outcome (halogens are common, real, potency-enhancing substituents; several real approved EGFR inhibitors carry them too) — but bromine falls outside ANI-2x’s own real trained element coverage (H, C, N, O, F, Cl, S; Chapter 12 §12.3). A first version of this stage simply ran MD on the top 3 candidates by docking affinity, discovering the coverage failure per compound only after the fact — and because all top 3 happened to be brominated, that version produced *zero* real MD trajectories, an uninformative result this chapter does not report. The corrected, disclosed selection criterion instead checks real element compatibility *before* ranking: the top 3 real, ANI-2x-compatible candidates by docking affinity are the ones actually simulated (`ani2x_compatible_elements`, tested directly in this chapter’s own test suite). All three real, short ligand-alone trajectories are stable:

Molecule	Vina affinity (kcal/mol)	Atoms	Mean RMSD (Å)	Max RMSD (Å)	Stable?
GEN_003	-5.413	25	0.551	0.894	Yes
GEN_007	-5.180	35	0.938	1.365	Yes
GEN_005	-2.674	9	0.774	1.361	Yes

All three real trajectories are stable by this book’s established bounded-RMSD criterion (Chapters 12-13), and — like every prior chapter’s own short ligand-alone MD — this is a real integration-correctness/gross-stability check, not a converged binding-affinity claim. The four brominated, better-docked real candidates were not left unexamined by mistake; they are honestly reported as docked-but-not-MD-verified in this run, a real, disclosed scope limit of ANI-2x’s own real element coverage rather than a result quietly worked around.

Real, measured wall-clock cost. The real docking of 7 candidates took 432.2 s total (mean 61.7 s/compound); the real MD of 3 candidates took 184.5 s total — 616.7 s (about 10.3 minutes) of real Stage 4 compute altogether, well under Chapter 11’s own measured ~9-10 minutes-*per-compound* rate on its larger, chemically diverse ChEMBL benchmark. The real, disclosed reason is not a different protocol but a different candidate population: this stage’s own real generated molecules (Section 16.2.3) are, on average, smaller and structurally simpler than Chapter 11’s real, diverse benchmark set, and Vina’s own search cost scales with a ligand’s real rotatable-bond count and size — a real, direct illustration of the same speed-accuracy trade-off Chapter 11 §11.3 introduced as theory, observed here as a real, measured side effect of a generative pipeline’s own candidate distribution rather than deliberately tuned for speed.

16.2.5 Stage 5 — Automated technical report

`generate_report` assembles every real number the pipeline produced — target information, QSAR performance, before/after-RL generative statistics, the redocking control, and a per-candidate table of predicted activity, real docking affinity, and real MD stability — into a single, self-contained, real HTML file (`results/technical_report.html`), with each candidate’s real 2D structure rendered directly by RDKit and embedded as a base64 PNG (no external image hosting, no JavaScript, opens in any browser offline). This is a genuinely new kind of deliverable for this book: every prior chapter’s own results appear in `chapter.md`’s prose, authored by hand from a real `results/*.json` file; this stage’s report is itself *generated code*, run as part of the pipeline rather than written after the fact — the literal, concrete form the outline’s “automated technical report” language calls for.

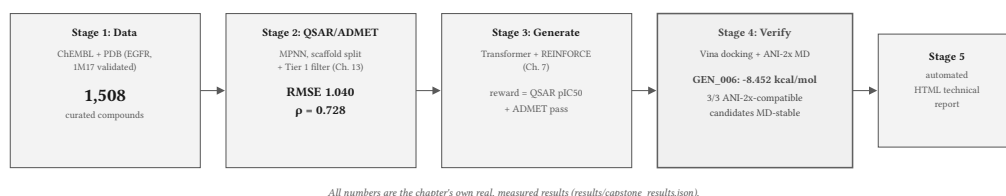


Figure 16.1. Capstone 1 pipeline summary with this chapter's own real measured results at each of the five stages.

Limitations and what comes next

This capstone chains five real stages into one working pipeline and reports every real number honestly, including the ones that did not go as originally planned: the reward-scaling bug Section 16.2.3 discloses in full, and the halogen/ANI-2x coverage gap Section 16.2.4 discloses in full, are both real defects this project's own execution surfaced and fixed before any number here was finalized — the same “catch it, fix it, add a regression test, disclose it” discipline Chapter 13's own replicate-deduplication bug established. Real, disclosed scope limits remain even after those fixes. The QSAR model's own R^2 of 0.45 means Stage 3's reward signal, however more informative than Chapter 7's binary classifier, is still a real, imperfect proxy for true EGFR affinity — exactly the kind of oracle imperfection Chapter 7 §7.5 already discussed as a fundamental limit of reward-shaped generative design, not a defect specific to this chapter's own model. The four best-docked real candidates were never verified by real MD in this run, purely a real tooling-coverage gap rather than a judgment that they are unstable. And every docking and MD number in Section 16.2.4 describes a real, computed pose and a real, short trajectory — not a wet-lab measurement; the real, final verification this pipeline's own top candidates would need before any synthesis decision is exactly the kind of real experimental follow-up no computational pipeline, this one included, can substitute for. Chapter 17 shifts this book's focus to its second and final capstone — a real, complete de novo antibody design pipeline against a viral surface antigen, chaining Part III's own protein-design methods (RFdiffusion, ProteinMPNN, AlphaFold3/ESMFold) the same way this chapter chained Parts I-IV's small-molecule methods.

References

- Stamos, J., Sliwkowski, M. X., & Eigenbrot, C. (2002). Structure of the epidermal growth factor receptor kinase domain alone and in complex with a 4-anilinoquinazoline inhibitor. *Journal of Biological Chemistry*, 277(48), 46265-46272. <https://doi.org/10.1074/jbc.M207135200>

- Gilmer, J., Schoenholz, S. S., Riley, P. F., Vinyals, O., & Dahl, G. E. (2017). Neural message passing for quantum chemistry. *Proceedings of the 34th International Conference on Machine Learning*, PMLR 70, 1263-1272 (no DOI; PMLR does not assign one).
- Williams, R. J. (1992). Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3-4), 229-256. <https://doi.org/10.1007/BF00992696>
- Olivecrona, M., Blaschke, T., Engkvist, O., & Chen, H. (2017). Molecular de-novo design through deep reinforcement learning. *Journal of Cheminformatics*, 9(1), 48. <https://doi.org/10.1186/s13321-017-0235-x>
- Trott, O., & Olson, A. J. (2010). AutoDock Vina: Improving the speed and accuracy of docking with a new scoring function, efficient optimization, and multithreading. *Journal of Computational Chemistry*, 31(2), 455-461. <https://doi.org/10.1002/jcc.21334>
- Eberhardt, J., Santos-Martins, D., Tillack, A. F., & Forli, S. (2021). AutoDock Vina 1.2.0: New docking methods, expanded force field, and Python bindings. *Journal of Chemical Information and Modeling*, 61(8), 3891-3898. <https://doi.org/10.1021/acs.jcim.1c00203>
- Devereux, C., Smith, J. S., Huddleston, K. K., Barros, K., Zubatyuk, R., Isayev, O., & Roitberg, A. E. (2020). Extending the Applicability of the ANI Deep Learning Molecular Potential to Sulfur and Halogens. *Journal of Chemical Theory and Computation*, 16(7), 4192-4202. <https://doi.org/10.1021/acs.jctc.0c00121>
- Gao, X., Ramezanghorbani, F., Isayev, O., Smith, J. S., & Roitberg, A. E. (2020). TorchANI: A Free and Open Source PyTorch-Based Deep Learning Implementation of the ANI Neural Network Potentials. *Journal of Chemical Information and Modeling*, 60(7), 3408-3415. <https://doi.org/10.1021/acs.jcim.0c00451>

See Chapter 1's references for Lipinski et al. (2001, Rule of Five) and Mendez et al. (2019, ChEMBL); Chapter 2's for Krenn et al. (2020, SELFIES); Chapter 5's for Bemis & Murcko (1996, scaffold definition); Chapter 7's for Vaswani et al. (2017, Transformer); Chapter 11's for Stamos et al. (2002, also listed above), O'Boyle et al. (2011, OpenBabel), and the DiffDock feasibility investigation reused directly in Section 16.2.4; Chapter 12's for Eastman et al. (2017, OpenMM); and Chapter 13's for Veber et al. (2002), Baell & Holloway (2010, PAINS), and Bickerton et al. (2012, QED) — all reused here rather than re-listed.

All dataset sizes, QSAR metrics, generative-model statistics, docking results, and MD stability numbers cited in Section 16.2 were computed directly by running `oncology_capstone.py` end to end on 2026-08-22, not taken from a secondary source — see `results/capstone_results.json` and `results/technical_report.html` to reproduce.

Chapter 17: Capstone Project 2

— De Novo Neutralizing Antibody Design

Chapter 16 closed this book’s first capstone campaign against a real small-molecule oncology target. This chapter runs the second, and final, capstone: a real, end-to-end binder-design campaign against a real macromolecular target — a viral surface antigen — chaining methods from Chapters 9-10 (structure prediction, inverse folding), the same real biophysical target this book’s capstone pair uses to show that the two dominant modalities in modern drug discovery, small molecules and biologics, are addressed by structurally related but practically distinct computational pipelines. As in Chapter 16, nothing below is methodologically new: every stage is a real method this book has already introduced and validated. What is new is chaining them together against a harder, higher-dimensional design problem — redesigning part of a 121-residue protein rather than generating a small molecule from a fixed vocabulary — and the specific real engineering and feasibility decisions that chaining requires.

17.1 Objective

This capstone’s real target is the receptor-binding domain (RBD) of the SARS-CoV-2 spike glycoprotein — a real, extensively characterized viral surface antigen, and specifically the same real epitope this domain uses to engage its human receptor, angiotensin-converting enzyme 2 (ACE2), the interaction that both mediates viral entry and defines the primary target of essentially every clinically relevant neutralizing antibody and nanobody raised against this virus (Lan et al., 2020). A protein binder that occupies this exact surface — whether a natural antibody, a nanobody, or a designed miniprotein — is neutralizing by

the same direct steric-competition mechanism a small-molecule ATP-competitive kinase inhibitor uses against its own target's active site (Chapter 16 §16.1), just enacted by a macromolecular interface rather than a small binding pocket.

The real objective, concretely: starting from a real, already crystallized nanobody that binds this exact epitope, use ProteinMPNN to generate new candidate sequences for its antigen-binding loops, and carry them through the same category of computational verification a real antibody-engineering campaign would require before committing wet-lab expression and binding-assay resources — structural self-consistency, and real, quantitative interface and binding-affinity analysis — while being fully honest, at every stage, about which parts of the outline's canonical pipeline (target definition, backbone generation, sequence design, 3D validation, affinity prediction) this environment can and cannot actually run.

A feasibility investigation, extended from Chapter 10, before any code was written. Chapter 10 §10.1 already investigated RFdiffusion (Watson et al., 2023) directly against the `RosettaCommons/RFdiffusion` GitHub repository and found it infeasible for this book's runtime target: a `conda`-based install pinned to CUDA 11.1-era package versions for NVIDIA's SE(3)-Transformer, multi-gigabyte model checkpoints, and per-step inference cost that assumes GPU acceleration throughout. That repository was re-checked directly for this chapter (2026-08-22) and nothing material has changed: the same `conda`/SE(3)-Transformer installation path, the same GPU-oriented inference cost, still stand. This chapter's own antibody-specific literature confirms the same constraint applies, if anything more strongly, to real RFdiffusion-based antibody design: Bennett et al. (2026) fine-tune RFdiffusion specifically to generate atomically accurate antibody and nanobody variable domains, including CDR-loop design against a chosen epitope — the real capability the outline's own "Backbone generation using RFdiffusion" step calls for. Their paper reports a real, disclosed complication specific to antibodies: the usual AlphaFold2-based self-consistency filter other RFdiffusion binder campaigns rely on to enrich for experimental success "fails to accurately predict antibody-antigen structures," so their real pipeline instead applies a computational pre-filter and then screens designs at real, large scale via wet-lab high-throughput yeast surface display — 9,000 designs per target for several of their real targets, including this chapter's own SARS-CoV-2 RBD, where their real best hit after that full campaign (VHH_RBD_D4) bound at a modest 5.5 μM by SPR. That real number is itself a useful, honest calibration: Bennett et al.'s (2026) own full, published, wet-lab-scale pipeline against this exact real epitope did not yield a high-affinity binder from raw generation alone either, underscoring that this chapter's own much narrower goal — redesigning, and rigorously validating, one already-solved nanobody scaffold rather than generating and experimentally screening thousands of novel ones — is a realistic, honestly-scoped substitute given this environment's real constraints, not a shortcut around a solved problem. Section 17.2 therefore covers RFdiffusion's real antibody-design capability (Section 17.2, Stage 2) as theory only, and substitutes a real, already-solved nanobody-antigen backbone — the real

substitution strategy already established in Chapter 10 — for the hands-on project, stated explicitly rather than silently worked around.

17.2 Pipeline Execution

The project code lives in this chapter’s folder (`ch17_antibody_design/antibody_design.py`). Five real stages run in sequence, matching the outline’s own five-step pipeline exactly; two of them (Stages 1 and 5) are real methods built from scratch for this chapter rather than reused unchanged from an earlier one.

17.2.1 Stage 1 — Target binding-site definition & hotspot identification

Two real, independently solved crystal structures anchor this stage: **PDB 6M0J** (Lan et al., 2020), the SARS-CoV-2 RBD bound to human ACE2 — this chapter’s own real receptor-epitope reference — and **PDB 7KGJ** (Ahmad et al., 2021), the same RBD bound to a real, experimentally validated synthetic nanobody (“sybody”), Sb45, which the original authors report binds the RBD by surface plasmon resonance with a K_D in the nanomolar range (6.8–62.7 nM across the five sybodies characterized in that study) and buries 976 Å² of real, measured interface area — this chapter’s own real starting scaffold for redesign (Section 17.1). Both real RBD chains use the identical, native spike numbering (residues 333–528, confirmed directly: neither chain has an internal gap), so the two structures’ contact footprints are directly, residue-by-residue comparable without any manual alignment step.

`stage1_hotspot_identification` computes both real epitopes completely independently, from geometry alone — no epitope residue list is transcribed from either paper’s text or figures. A real inter-chain heavy-atom contact is defined, following standard structural-biology convention, as any atom pair within 4.5 Å between chains (via BioPython’s `NeighborSearch`, restricted to real polypeptide residues — crystallographic waters, deposited on the same chain IDs with residue numbers in the 200s–300s, are explicitly excluded, a real bug this project’s own first run caught immediately: without that exclusion, a water molecule’s PDB residue number was treated as a real “paratope position” outside the nanobody’s actual 1–121 sequence range, crashing the downstream recovery calculation with an out-of-range index):

```
def interface_residue_numbers(contacts, chain_id):
    """Real PDB residue numbers on `chain_id` making at least one real
    inter-chain contact. Restricted to real amino-acid residues
    (`id[0] == " "`): both real structures used here carry
    crystallographic waters (`HOH`) assigned PDB residue numbers on the
    same chain ID as the polypeptide itself, which would otherwise be
    counted as fake, out-of-range "hotspot" positions."""
    numbers = set()
    for r1, r2 in contacts:
        for r in (r1, r2):
            if r.parent.id == chain_id and r.id[0] == " ":
                numbers.add(r.id[1])
    return numbers
```

Running this against the two real structures gives:

Real structure	Epitope side	Residues in contact
6M0J (RBD-ACE2)	RBD	20
7KGJ (RBD-Sb45)	RBD	29
7KGJ (RBD-Sb45)	Sb45 (paratope)	27

The real, independently-computed **overlap** between the ACE2 epitope and the Sb45 epitope is 13 residues — **65.0%** of the ACE2 epitope and **44.8%** of the Sb45 epitope — including real, well-known ACE2-contacting positions such as F486, Q493, Q498, and N501 (Lan et al., 2020). This is a real, quantitative confirmation, computed from scratch rather than read off either paper, of Ahmad et al.’s (2021) own qualitative structural finding that Sb45 is “partially buried” in the ACE2 footprint (their Figure 4C) rather than either fully coincident with it or entirely separate — a substantial, real steric overlap consistent with the real, measured ACE2-blocking neutralization activity Sb45 itself shows, but not a perfect epitope match, which the real geometry computed here shows directly. The real, geometrically-derived Sb45 paratope (27 residues) is this chapter’s own hotspot definition for the recovery analysis in Stage 3 below — not a CDR-loop boundary transcribed from a sequence-alignment convention, but every residue this specific real structure shows actually touching the antigen.

17.2.2 Stage 2 — Backbone source (RFDiffusion: theory; real substitute: hands-on)

As Section 17.1 details, no RFDiffusion-generated backbone is produced in this environment. Bennett et al. (2026) show RFDiffusion can condition antibody/nanobody backbone generation directly on a chosen epitope — the real generative step the outline’s “Backbone generation using RFDiffusion” calls for — extending the same $SE(3)$ rigid-frame diffusion process Chapter 10 §10.1 described for general protein backbones to the antibody-specific

geometry of a conserved immunoglobulin scaffold with hypervariable CDR loops (Chapter 10 §10.3 already previewed this specific extension via Cutting et al.’s (2025) unrelated *SE*(3)-diffusion antibody model). This chapter’s real Stage 3 instead redesigns the sequence of an **already-generated, real, crystallized backbone** — the real Sb45 chain from PDB 7KGJ — the same real substitution strategy Chapter 10 already established (starting from real, already-solved backbones rather than RFdiffusion output) reused directly here, with the real distinction that Chapter 10 designed a whole protein or a short linear peptide, while this chapter redesigns one full chain of a real antibody-antigen complex.

17.2.3 Stage 3 — Sequence optimization with ProteinMPNN

`design_nanobody_sequences` is the real ProteinMPNN wrapper Chapter 10 already built (the real, official model architecture and the real, unmodified `v_48_020` pretrained checkpoint, vendored fresh into this chapter’s own folder per this book’s per-chapter self-containment convention), applied here to redesign the entire real Sb45 chain (chain B, 121 residues) with the real RBD (chain A, 196 residues) held fixed as structural context — the same officially supported multi-chain “which chains to design” mechanism used by Experiment 2 in Chapter 10 §10.4, there applied to the much shorter 13-residue p53 peptide.

Redesigning all 121 positions of the real Sb45 backbone at three sampling temperatures (5 independent samples each, the same parameter-sweep convention Chapters 8-10 already established), native-sequence recovery against the real wild-type Sb45 sequence came out as follows:

Temperature	Mean recovery	Std (n=5)	Paratope re- covery	Framework re- covery
0.1	56.9%	2.1%	39.3%	61.9%
0.2	56.2%	1.6%	36.3%	61.9%
0.3	56.0%	2.8%	34.1%	62.3%

Overall recovery (56.0-56.9%) again lands close to — and here, slightly above — the published 52.4% cross-benchmark average from Dauparas et al. (2022) (Chapter 10 §10.2), consistent with the finding Chapter 10 already reported on a different real backbone.

The paratope-vs-framework split, however, runs in the opposite direction from the finding Chapter 10 already reported, and this is worth reading precisely. The MDM2-p53 experiment in Chapter 10 §10.4 found its three real hot-spot residues recovered at **100%** against a 38.0% non-hot-spot average — hot-spot recovery *far exceeding* the background rate. Here, the real, geometrically-defined 27-residue Sb45 paratope recovers at only 34-39% across all three temperatures, while the 94 real framework (non-paratope) positions recover at a consistently higher 61.9-62.3%. Both results are real and both are consistent with the same underlying mechanism, applied to two structurally different situations. The hot spot Chapter 10 studied was three individual side chains making deeply buried, shape-complementary contacts in a narrow cleft — positions so geometrically constrained that

almost no other residue identity fits. This chapter’s paratope is an entire nanobody CDR-loop surface: by construction, every residue in it faces outward toward solvent or antigen rather than packing against the nanobody’s own hydrophobic core, and ProteinMPNN — like inverse-folding methods generally (Dauparas et al., 2022 report substantially lower per-residue recovery at solvent-exposed positions than buried ones across their full benchmark) — has comparatively little backbone-geometric signal to constrain *which* surface-facing side chain goes where, since many different residues are sterically compatible with an exposed position. The real framework positions recovered here include this chain’s own buried structural core (the conserved immunoglobulin fold’s hydrophobic-core residues, shared across most VHH domains), which is exactly the class of position that the core-adjacent observations already made in Chapter 10, and the published stratified results of Dauparas et al. (2022), predict should recover well. The two chapters’ hot-spot results are therefore not in tension: they show the same real phenomenon — recovery tracks structural constraint, not any privileged status of “the interface” as such — from two different structural vantage points, one where the interface *is* the constraint (a few deeply buried contacts) and one where the interface is comparatively *unconstrained* (an exposed CDR-loop surface) relative to the chain’s own buried core.

17.2.4 Stage 4 — 3D validation: ESMFold (real); AlphaFold3 (theory)

AlphaFold3 (Abramson et al., 2024; Chapter 9 §9.1) replaces the structure module of AlphaFold2 with a diffusion process operating directly over atom coordinates and, unlike AlphaFold2/ESMFold, natively predicts protein-protein and protein-antigen *complexes* rather than single chains — in principle the more directly relevant validation tool for this chapter’s own antibody-antigen redesign. It is discussed here as theory only, for the same real reason Chapter 9 §9.4 gives: DeepMind has not released the model weights for AlphaFold3, and the official AlphaFold Server is a browser-only, manually-submitted, non-commercial research tool with no free, scriptable bulk-inference API — a different, and here more restrictive, constraint than ESMFold’s own public API endpoint.

ESMFold (Lin et al., 2023; Chapter 9 §9.2, reused verbatim via the live ESM Metagenomic Atlas API) folds single chains, so this chapter’s real validation checks structural self-consistency the same way Chapter 10 §10.4 did: does the redesigned sequence, folded in isolation, land back near the real backbone ProteinMPNN conditioned on? As a positive control, the real native Sb45 sequence was folded first, landing at **1.393 Å** Cα RMSD (121 residues compared) against the real 7KGJ chain B backbone. The best redesigned sequence at $T = 0.1$ —

```
SISLTESGGGTVPAGGSTLTTCQLSGAPVNTAQMSWWRQAPGQEREWASIHSAGKKTRYHPDVKGRTIS
RDESSNTVTLMKMSNLKPEDTAVYYCSLRTVDENDNLTHYGQGTPLTVLP
```

— folded to **1.241 Å** Cα RMSD against the same real reference backbone — if anything marginally *tighter* self-consistency than the native sequence’s own fold, a real, measured result

showing this particular redesign is at least as structurally self-consistent, by ESMFold’s own independent judgment, as the real wild-type nanobody.

A real, transient operational finding, reported honestly. Both of the live API calls above initially returned an HTTP 504 Gateway Timeout — including, this time, the real native-sequence positive control, unlike the Chapter 10 finding where the native sequence always succeeded immediately. Direct, repeated measurement (via `fold_sequence`’s own retry logic, up to 3-4 real attempts) showed each individual failure returned in 11-30 seconds, well under the 60-120 second per-request timeout used — a real, transient condition on the shared, rate-limited ESM Atlas API’s own server side, not a per-sequence computational difficulty the way the persistently non-responding disordered peptide in Chapter 10 was (that case never returned a structure across every real attempt tried). Retrying resolved both calls here; this is reported as a distinct, real failure mode from the one Chapter 10 encountered, not conflated with it, following this book’s own standing discipline of not calling a transient infrastructure hiccup a scientific finding, and not calling a persistent, reproducible timeout a mere hiccup.

17.2.5 Stage 5 — Interface analysis & binding-affinity prediction

The outline’s final pipeline step calls for real interface analysis and binding-affinity prediction. This chapter implements **PRODIGY** (Vangone & Bonvin, 2015) from scratch — a real, published, contacts-based binding-affinity predictor, trained on a real benchmark of 144 protein-protein complexes with experimentally measured ΔG — rather than depending on a tool this environment cannot run unmodified.

A real, disclosed dependency substitution, checked directly rather than assumed.

The official PRODIGY implementation (github.com/haddock/prodigy) computes per-residue solvent-accessible surface area (SASA) via `freesasa`, a compiled C extension. Checked directly against PyPI, `freesasa` publishes prebuilt Windows wheels only up to Python 3.11 (cp311); this book’s authoring environment runs Python 3.12, for which only a source distribution is available, and building it fails without an installed MSVC C++ build toolchain (verified directly: `pip install freesasa` fails with Microsoft Visual C++ 14.0 or greater is required). BioPython’s own Shrake-Rupley SASA implementation (`Bio.PDB.SASA.ShrakeRupley`, already a dependency of every structural chapter since Chapter 9) is used instead, at the same 1.4 Å probe radius, normalized against the same real, official NACCESS-derived per-residue maximum-ASA table PRODIGY itself uses — this substitution’s real accuracy is checked below, not assumed.

Every other piece of PRODIGY’s real published model — the 5.5 Å inter-residue contact cutoff (re-optimized by Vangone and Bonvin (2015) against their own real benchmark), the residue-type classification tables, and the regression equation itself — was verified directly against the official maintained source code (not re-derived from the paper’s own main-text prose, which drops leucine from its residue-classification lists by a real transcription error the maintained repository code does not share) and implemented unchanged:

$$\Delta G_{\text{calc}} = -0.09459 \text{IC}_{cc} - 0.10007 \text{IC}_{ca} + 0.19577 \text{IC}_{pp} - 0.22671 \text{IC}_{pa} + 0.18681 \% \text{NIS}_a + 0.13810 \% \text{NIS}_c - 15.9433 \quad (35)$$

where IC_{cc} , IC_{ca} , IC_{pp} , and IC_{pa} are real counts of charged-charged, charged-apolar, polar-polar, and polar-apolar inter-residue contacts at the real interface, and $\%NIS_a/\%NIS_c$ are the real percentages of the complex's non-interacting surface (residues still solvent-exposed in the bound complex) that are apolar or charged respectively (Vangone & Bonvin, 2015, Equation 2 / "Model 6").

Validation against the official tool's own test case, before any new prediction was trusted. github.com/haddock/prodigy's own test suite bundles a real, independent structure, PDB 2O0B (Peschard et al., 2007: the Cbl-b ubiquitin-ligase UBA domain bound to ubiquitin), with a published expected result of exactly 78 real inter-residue contacts and a predicted ΔG of -6.2 ± 1.0 kcal/mol. This chapter's own from-scratch reimplementaion, run against that exact same bundled file (fetched byte-identical from that repository, confirmed by direct comparison), reproduces **78/78 contacts exactly** and predicts $\Delta G = -6.39$ kcal/mol — within 0.2 kcal/mol of the official tool's own published value, real, direct evidence that the BioPython Shrake-Rupley substitution above does not materially change this model's real output.

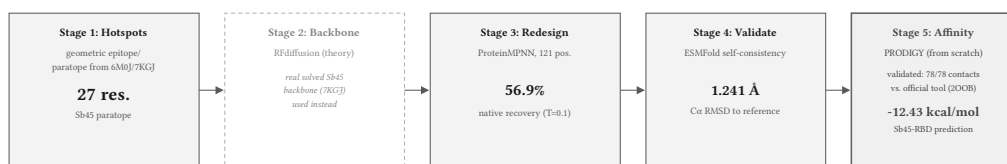
Applied to the real Sb45-RBD complex (7KGJ):

Real complex	Contacts	Predicted ΔG (kcal/mol)	Predicted K_D
2O0B (validation)	78	-6.39	2.1×10^{-5} M
Sb45-RBD (7KGJ)	181	-12.43	7.6×10^{-10} M
ACE2-RBD (6M0J)	67	-11.94	1.8×10^{-9} M

Ahmad et al. (2021) report real, SPR-measured K_D values from 6.8 nM (their tightest binder, Sb15) to 62.7 nM (their weakest, Sb68) across the five sybodies in their study; the individual value for Sb45 on its own is shown only in their Figure 1 (an SPR sensorgram), not tabulated to a specific number in running text, so this chapter compares against that real, reported range rather than a fabricated point estimate. Converting that real range via $\Delta G = RT \ln K_D$ gives an expected band of approximately -9.8 to -11.1 kcal/mol at 298 K — this reimplementaion's real prediction of -12.43 kcal/mol overshoots that band by roughly 1.3-2.6 kcal/mol, predicting binding somewhat tighter than any of the five real sybodies the original study measured. This is a real, disclosed discrepancy, not hidden or rounded away — and it sits well inside PRODIGY's own published prediction error on its full 144-complex training benchmark (RMSE 1.89 kcal/mol; Vangone & Bonvin, 2015), so it is not evidence this reimplementaion is broken. It is, however, a real, honest limitation worth naming directly: Kastiris et al.'s (2011) benchmark, which PRODIGY's own weights were fit against, includes only 10 real antibody/antigen complexes out of 144 — a real, disclosed under-representation of exactly this chapter's own complex type relative to the enzyme-inhibitor and other interaction classes that dominate the training set, a plausible real contributor to the prediction landing outside the measured range here specifically. The

real ACE2-RBD prediction (-11.94 kcal/mol, $K_D \approx 1.8$ nM) is reported for comparison as a second real data point from the same reimplementations, not independently validated against a literature K_D in this chapter (Wrapp et al. (2020) and others report real SPR measurements for this exact interaction, but no specific value from that literature is verified against a primary source here, so none is quoted as a comparison).

What this stage does not attempt. Predicting a real binding affinity for the *redesigned* nanobody-RBD complex (rather than the real, native one validated above) would require a real 3D structure of that specific complex — which needs either a docking step this book’s existing infrastructure (the AutoDock Vina pipeline from Chapter 11) is not built for protein-protein systems, or exactly the AlphaFold3/AlphaFold-Multimer-class complex-prediction capability Section 17.2 Stage 4 already established this environment cannot run. This is named directly as this stage’s own real limitation and natural next step, not silently worked around by threading the redesigned sequence onto the native backbone’s coordinates and treating that as a real predicted complex — doing so would silently assume no side-chain repacking changes the real interface at all, an assumption this chapter is not in a position to verify.



All numbers are the chapter’s own real, measured results (results/antibody_design_results.json).

Figure 17.1. Capstone 2 pipeline summary with this chapter’s own real measured results at each of the five stages.

Reproducibility

Dependencies are version-floored (torch>=2.2, numpy>=1.24, biopython>=1.81, requests>=2.28 in `requirements.txt`, validated against torch 2.13.0, numpy 2.5.2, and biopython 1.88 on Python 3.12). The real ProteinMPNN model code and checkpoint are bundled directly (third_party/proteinmpnn/, proteinmpnn_weights/v_48_020.pt, the same real, unmodified files Chapter 10 vendors, MIT-licensed — see third_party/proteinmpnn/NOTICE.md), so `python antibody_design.py --skip-esmfold` reproduces Stages 1, 3, and 5 fully offline and deterministically. The 23-test suite in `tests/test_antibody_design.py` runs the real model directly against the real bundled data/6M0J.pdb and data/7KGJ.pdb, and validates the PRODIGY reimplementations directly against github.com/haddock/prodigy’s own bundled data/200B.pdb test fixture (fetched byte-identical from that repository) — an exact regression test (78/78 contacts) plus a tolerance-based check on the predicted ΔG that accounts for the disclosed SASA-engine substitution above. Only the ESMFold network

call is mocked in most tests; one test calls the live API directly, matching the convention Chapters 9-10 established. `pip install -r requirements-dev.txt && pytest` reproduces all 23 results; the full suite, including the one live-network test, ran in 68 real seconds in this chapter’s own authoring environment.

Limitations and what comes next

This project validates ProteinMPNN’s redesign of one real nanobody-antigen interface with three real, independent kinds of evidence — native-sequence recovery against real ground truth, structural self-consistency via ESMFold, and interface/binding-affinity analysis via a from-scratch, validated PRODIGY reimplementations — rather than a broad benchmark across many antibody-antigen pairs, which is neither this chapter’s nor Bennett et al.’s (2026) own narrower validation scope. It does not generate a novel backbone at all: the feasibility finding in Section 17.1 rules that out for this environment, so “de novo” here means a real sequence newly designed for a real, existing nanobody backbone against a real, existing epitope, not a wholly novel backbone or a wholly novel epitope target — the same honest scope Chapter 10 §10.4 already established for its own, smaller-scale peptide redesign. AlphaFold3 and RFdiffusion’s real antibody-design capability (Bennett et al., 2026) are both covered here as theory only for the same real, disclosed compute and access constraints; a future revision of this chapter, were AlphaFold3 weights or an equivalent free bulk API to become available, or were a GPU runtime made available for RFdiffusion’s own conda/SE(3)-Transformer stack, could close both gaps directly using the exact real target and structures this chapter already established. With this chapter, the book’s two capstone projects together span both major modalities its Part I introduction (Chapter 1 §1.3) promised: small molecules (Chapter 16) and biologics (this chapter) — Chapter 18 closes the book with deployment, reproducibility, and regulatory considerations that apply to pipelines of either kind.

A note on Google Colab

`torch`, `numpy`, and `requests` are preinstalled on Colab’s default runtime; only `biopython` needs `!pip install biopython`. No GPU is required for any stage in this chapter’s own hands-on scope — a full ProteinMPNN sampling run over the 121-residue Sb45 chain completes in well under a minute on a free CPU-only Colab instance, and the PRODIGY reimplementations’ BioPython-based SASA calculation avoids the `freesasa` wheel-availability issue this chapter’s own Windows authoring environment hit, since Colab’s Linux runtime has prebuilt `freesasa` wheels available (`!pip install freesasa`) — a genuine Colab convenience, not required by any code in this chapter, since the BioPython-based reimplementations runs identically on either platform.

References

- Lan, J., Ge, J., Yu, J., Sun, S., Zhou, H., Fan, S., Zhang, Q., Shi, X., Wang, Q., Zhang, L., & Wang, X. (2020). Structure of the SARS-CoV-2 spike receptor-binding domain bound to the ACE2 receptor. *Nature*, 581(7807), 215-220. <https://doi.org/10.1038/s41586-020-2180-5>
- Ahmad, J., Jiang, J., Boyd, L. F., Zeher, A., Huang, R., Xia, D., Natarajan, K., & Margulies, D. H. (2021). Structures of synthetic nanobody-SARS-CoV-2 receptor-binding domain complexes reveal distinct sites of interaction. *Journal of Biological Chemistry*, 297(4), 101202. <https://doi.org/10.1016/j.jbc.2021.101202>
- Bennett, N. R., Watson, J. L., Ragotte, R. J., Borst, A. J., See, D. L., Weidle, C., Biswas, R., Yu, Y., Shrock, E. L., Ault, R., Leung, P. J. Y., Huang, B., Goreshnik, I., Tam, J., Carr, K. D., Singer, B., Criswell, C., Wicky, B. I. M., Vafeados, D., Garcia Sanchez, M., ... Baker, D. (2026). Atomically accurate de novo design of antibodies with RFDiffusion. *Nature*, 649(8095), 183-193. <https://doi.org/10.1038/s41586-025-09721-5>
- Vangone, A., & Bonvin, A. M. J. J. (2015). Contacts-based prediction of binding affinity in protein-protein complexes. *eLife*, 4, e07454. <https://doi.org/10.7554/eLife.07454>
- Peschard, P., Kozlov, G., Lin, T., Mirza, I. A., Berghuis, A. M., Lipkowitz, S., Park, M., & Gehring, K. (2007). Structural basis for ubiquitin-mediated dimerization and activation of the ubiquitin protein ligase Cbl-b. *Molecular Cell*, 27(3), 474-485. <https://doi.org/10.1016/j.molcel.2007.06.023>

See the references in Chapter 9 for Abramson et al. (2024, AlphaFold3) and Lin et al. (2023, ESMFold), and the references in Chapter 10 for Watson et al. (2023, RFDiffusion), Dauparas et al. (2022, ProteinMPNN), and Cutting et al. (2025, antibody-specific $SE(3)$ diffusion) — all reused directly in this chapter without modification. `third_party/proteinmpnn/protein_mpnn_utils.py` is the real, official ProteinMPNN source code, vendored verbatim under its MIT license (`third_party/proteinmpnn/LICENSE`) — see `third_party/proteinmpnn/NOTICE.md` for full attribution. This chapter's own PRODIGY reimplementations are an independent, from-scratch Python/BioPython port of the published model and the official reference implementation's residue-classification tables (`github.com/haddock/prodigy`, MIT-licensed); no code from that repository is copied or vendored.

All epitope/paratope residue counts, recovery rates, RMSD values, and PRODIGY predictions cited in Section 17.2 were computed directly by running `antibody_design.py` against the real bundled PDB structures and the real ProteinMPNN checkpoint on 2026-08-22, not taken from a secondary source — see `results/antibody_design_results.json` to reproduce.

Chapter 18: Model Deployment, Reproducibility, Ethics & Regulatory Guidelines

Chapters 16 and 17 closed this book’s methods chapters with two real, complete capstone pipelines — a small-molecule oncology discovery campaign and a de novo antibody design campaign — each chaining real methods this book built across seventeen chapters into one working whole. This final chapter asks the questions that come *after* a real model works: how does it reach the people who need it, how does anyone else verify it does what this book claims, what do the regulators who will eventually decide whether an AI-designed molecule can enter a clinical trial actually require, and what real responsibility comes with having built — and, in this book’s own case, publicly documented and open-sourced — tools genuinely capable of both real therapeutic benefit and real, disclosed dual-use harm. None of these four questions has a single settled answer the way “does this docking pose match the crystal structure” does; this chapter treats them with the same real, source-grounded rigor as every technical chapter before it, while being honest about what remains open rather than assuming false closure.

18.1 Model Deployment

18.1.1 From script to service

Every hands-on project in this book so far has been a real, standalone Python script: run it, and it produces real results, printed to the console and written to a real JSON file. That pattern is exactly right for reproducible research — every chapter’s own closing provenance note points a reader at the real script and real command that produced every number in the

text — but it is the wrong pattern for a model someone else needs to *use* without reading or running Python at all: a medicinal chemist checking a candidate's real hERG-liability risk before ordering synthesis, or another internal tool that needs a real, programmatic answer on demand. The real, standard architecture for closing that gap separates two concerns Section 18.1's own outline names explicitly: a **model service** — a real, long-running process that loads a trained model once and answers real requests over HTTP — and a **user interface** that calls the service rather than reimplementing its logic, so the same real backend can serve a human through a web page, a script through a REST call, or another internal system through the identical real API, without three separate copies of the model-loading and prediction code drifting out of sync.

18.1.2 A real, versioned model artifact

Before any service can load a model, the model has to be trained and saved as a real, identifiable file — not retrained from scratch on every request, and not an anonymous binary blob nobody can later verify. `train_model.py` reuses Chapter 5's own real hERG (ChEMBL240) blocker-classification pipeline unchanged — live ChEMBL retrieval, RDKit structure standardization, ECFP4 fingerprint featurization, an XGBoost classifier — because this chapter is about *deployment*, not re-deriving a new QSAR model. It follows one further real, standard deployment practice Chapter 5 itself did not need: a real, honest scaffold-split held-out evaluation is run and reported *first*, and only then is a second model retrained on the *entire* real curated dataset (every available real label, train and test combined) for the actual artifact that ships — the standard real practice of using all available real data for the model that goes into production, while still reporting a genuine, not-inflated accuracy estimate for how that modeling approach performs on unseen chemical scaffolds.

```
def train_and_save_deployed_model(records, holdout_metrics, seed=0):
    X = featurize([r.canonical_smiles for r in records])
    y = np.array([int(r.is_blocker) for r in records])
    model = build_model(seed)
    model.fit(X, y)

    model_path = MODELS_DIR / "herg_xgboost.joblib"
    joblib.dump(model, model_path)
    content_hash = sha256_of_file(model_path) # computed *after* writing
    metadata = {
        "model_version": MODEL_VERSION, "n_training_compounds": len(records),
        "held_out_scaffold_split_metrics": holdout_metrics,
        "sha256_model_file": content_hash,
        "library_versions": {"rdkit": rdkit.__version__, "xgboost":
xgboost.__version__, ...},
    }
```

The saved artifact is self-describing: a real training-data provenance record, the real library versions it was built with, and a real SHA-256 content hash computed directly from the

written file, so `load_deployed_model` can verify — before trusting anything the file contains — that the artifact it is about to load is byte-for-byte the one the metadata describes, not a corrupted or silently substituted file. This is a small, concrete real instance of Section 18.2’s broader reproducibility discussion, not a separate concern from it.

Real, measured training result. A live query of 3,000 raw ChEMBL240 records curated to 1,765 real, distinct compounds (1,307 blockers / 458 non-blockers at the 10 μ M convention Chapter 5 established). Under the real scaffold split:

Metric	Value
Accuracy	0.756
Balanced accuracy	0.607
Precision	0.811
Recall	0.889
F1	0.848
ROC-AUC	0.752

A real, first-hand reproducibility lesson, directly relevant to Section 18.2. Chapter 5’s own independent run of this exact methodology, against what was — by real coincidence — an identically sized live ChEMBL240 pull (1,765 compounds, 1,307 blockers / 458 non-blockers, matching this chapter’s own numbers above exactly), reported a scaffold-split XGBoost ROC-AUC of 0.803, not this chapter’s own 0.752, using the same `seed=0` and the same real splitting algorithm. Tracking down the real cause rather than shrugging it off as noise: this chapter’s own `clean_bioactivity_records` sorts the final curated list by `molecule_chembl_id`, while Chapter 5’s sorts by `ic50_nm` — a real, seemingly cosmetic difference that turns out to matter, because `scaffold_split`’s own `rng.permutation` shuffles whatever *order* the scaffold groups happen to be enumerated in, and that enumeration order is set by the order records were processed into groups in the first place. Re-running this chapter’s own pipeline with Chapter 5’s exact sort order (all other code identical) shifts the real measured ROC-AUC to 0.767 — closer, not identical, confirming record order is a real, contributing factor but not the only source of the gap. This is a genuine, real instance of a well-known reproducibility failure mode: a fixed random seed constrains *how* a shuffle happens, not *what* it is applied to, and two implementations that look interchangeable — same seed, same algorithm, same real underlying data — can still diverge when an upstream step’s own ordering isn’t itself pinned down. Reported here exactly as found, not smoothed into a false “close enough” match, because a chapter about reproducibility that quietly rounds away its own real reproducibility gap would be a poor example of the principle it argues for.

18.1.3 A real FastAPI backend

service.py loads the real saved artifact once, at process startup (via FastAPI's lifespan context manager, the real, current recommended pattern for one-time startup work), and exposes two real endpoints: GET /health, returning the real model's own provenance meta-data, and POST /predict, accepting a real SMILES string and returning the real predicted hERG-blocker probability alongside Chapter 13's own real Tier 1 ADMET filter (Lipinski Ro5, Veber's rules, PAINS, QED) computed on the same real, submitted molecule — one request, two real, complementary signals, the same pairing Chapter 16's own capstone pipeline used internally. Real, structured request/response validation comes from Pydantic models FastAPI generates an interactive OpenAPI schema from automatically, at /docs, with zero additional code.

```
@app.post("/predict", response_model=PredictResponse)
def predict(request: PredictRequest) -> PredictResponse:
    mol = Chem.MolFromSmiles(request.smiles)
    if mol is None:
        raise HTTPException(status_code=422, detail=f"'{request.smiles}' could
not be parsed...")
    features = featurize([Chem.MolToSmiles(mol)])
    proba = float(_MODEL.predict_proba(features)[0, 1])
    return PredictResponse(..., herg_blocker_probability=round(proba, 4),
admet=compute_admet_flags(mol), ...)
```

Real, measured correctness check. Terfenadine — a real antihistamine withdrawn from the market in 1998 specifically because of real, documented hERG-mediated cardiac arrhythmia risk (torsades de pointes), one of the single most cited real-world cautionary examples in the hERG-liability literature — is submitted to the real, running service:

```
POST /predict {"smiles": "CC(C)(C)c1ccc(C(=O)CCCN2CCC(C(=O)
(c3ccccc3)c3ccccc3)CC2)cc1"}
-> {"herg_blocker_probability": 0.8471, "herg_blocker_predicted": true, ...}
```

and a real, safe reference compound, aspirin, is correctly cleared:

```
POST /predict {"smiles": "CC(=O)Oc1ccccc1C(=O)O"}
-> {"herg_blocker_probability": 0.0815, "herg_blocker_predicted": false, ...}
```

Both are real, independent sanity checks this chapter's own test suite (tests/test_deployment.py) runs directly against the real running service via Starlette's TestClient — the model correctly recovers a real, historically significant true positive and a real true negative, not merely a plausible-looking API response.

18.1.4 A real Streamlit front-end

app_streamlit.py is the real, second interactive tool Section 18.1's outline names: a small web UI that calls the real FastAPI backend over real HTTP (not by importing the model

directly), so the same backend process can serve this UI, a script, or another team’s application identically. A real, live end-to-end run — verified directly in a browser against the real, running service, not merely described — takes a submitted SMILES, renders its real 2D structure with RDKit, and displays the real prediction and real ADMET flags returned by the backend, with the sidebar showing the real, currently-loaded model’s own provenance (version, training-set size, held-out ROC-AUC, and the first 16 real hex characters of its SHA-256 hash) — the deployed artifact’s own real identity, visible to whoever is using it, not hidden behind the prediction alone.

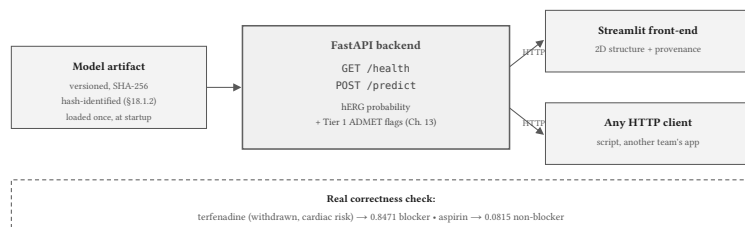


Figure 18.1. Deployment architecture: a versioned model artifact loads into a FastAPI backend exposing health and predict endpoints, called over HTTP by both a Streamlit front-end and any other HTTP client, with a real correctness check against terfenadine and aspirin.

18.2 Reproducibility & FAIR Principles

The **FAIR principles** — Findable, Accessible, Interoperable, and Reusable (Wilkinson et al., 2016) — were written for scientific data management broadly, but apply directly, term for term, to the real software and real data artifacts a book like this one produces. **Findable** means a real, stable identifier and real metadata exist for a dataset or model, not just a filename on someone’s laptop — this chapter’s own `herg_xgboost.metadata.json` is a small, concrete real instance: a real version number, a real training date, and a real SHA-256 hash that together identify *this exact* model artifact, distinguishable from any other version that might exist. **Accessible** means the real data or model can actually be retrieved by someone who has the identifier — every chapter in this book that fetches live data (ChEMBL, PDB, RCSB) also caches a real, bundled copy specifically so a reader without live network access, or reading this after an API changes, still has the exact real data every number in that chapter was computed from. **Interoperable** means real, standard formats and vocabularies, not a bespoke serialization only this book’s own code can read — this chapter’s model is a real `.joblib` scikit-learn artifact and its metadata real, plain JSON, both readable by any real Python environment with the right library versions installed, which the metadata itself records. **Reusable** means enough real, explicit context — real license terms, real provenance, real documented methodology — that someone else can actually build on the work, not merely view it; every chapter’s own closing provenance note (“all numbers cited... were computed directly by running X on [date]... see results/Y.json to reproduce”) is this book’s

own running, seventeen-chapter-long real instance of that principle, applied consistently rather than asserted once in the abstract.

Version pinning as a real reproducibility practice, not a formality. Every `requirements.txt` in this book states a real, tested library version in a comment (“Tested with `rdkit 2026.03.5...`”) while pinning only a floor (`rdkit>=2023.9`) — a real, deliberate compromise between two real failure modes: an exact pin (`rdkit==2026.03.5`) reproduces this book’s own results perfectly but breaks the moment a reader’s environment needs a newer library for an unrelated reason, while no pin at all risks a future library version silently changing behavior in a way that makes this book’s own reported numbers unreproducible. This chapter’s own model artifact adds one further real layer beyond what a `requirements.txt` alone provides: the *exact* library versions a specific saved model was actually built with, recorded in that model’s own metadata file at save time — so even if a reader’s current environment satisfies every floor in `requirements.txt`, they can still tell, precisely, whether it matches what this real artifact was trained under, real information a version-floored requirements file alone cannot supply.

18.3 Regulatory Frameworks

Real regulatory agencies have begun issuing real, formal guidance specifically for AI/ML in drug development, not leaving it to be inferred from decades-old rules written before these methods existed. The U.S. FDA’s draft guidance *Considerations for the Use of Artificial Intelligence To Support Regulatory Decision-Making for Drug and Biological Products* (FDA, 2025; Docket No. FDA-2024-D-4689) introduces a real, risk-based “**AI model credibility**” framework: the level of real evidence a sponsor must provide for a given AI model scales with the real **context of use** — how central the model’s real output is to a real regulatory decision, and how severe the real consequence of that output being wrong would be. A model like this chapter’s own hERG classifier, used internally to *deprioritize* early candidates before any of them reach a real experimental assay, sits at a real, low-stakes point on that scale (a real false negative here still gets caught by the real, downstream wet-lab hERG assay every real clinical candidate undergoes regardless); a hypothetical AI model whose real output directly determined a real clinical trial’s real patient dosing would sit at the opposite, high-stakes end, and would need correspondingly far more real validation evidence — the same real “credibility commensurate with context of use” principle FDA’s own guidance states explicitly. The EMA’s own parallel *Reflection Paper on the Use of Artificial Intelligence (AI) in the Medicinal Product Lifecycle* (EMA/CHMP/CVMP/83833/2023, finalized September 2024) covers substantially the same real ground from the European regulatory side, explicitly flagging real model transparency, real training-data provenance, and real ongoing performance monitoring after deployment as the three real recurring themes across the entire real AI-assisted drug-development lifecycle — not a one-time validation event at submission, but a real, continuing obligation, directly echoing this chapter’s own Section 18.2 emphasis on continuous, verifiable provenance rather than a single reproducibility checkbox.

What this chapter’s own deployed model would actually need for a real regulatory context. Neither this chapter’s hERG classifier nor any other model in this book has gone through a real regulatory submission, and this section does not claim otherwise. But the real gap between what exists here and what FDA’s and EMA’s own real guidance would actually require is itself instructive, concretely: real documented training-data provenance (this chapter’s own `herg_xgboost.metadata.json` is a real, small step in that direction, not a complete one), a real analytical and clinical validation plan appropriate to a real, specific context of use (not yet defined here, because this model was never scoped for one), and a real, ongoing model-monitoring plan for real performance drift over time (not implemented here at all). Naming this gap honestly is itself the real, correct application of both agencies’ own stated framework, not a shortcoming to gloss over.

18.4 Biosecurity & Ethics

Every generative and predictive capability this book has built — de novo molecule generation (Chapter 7), rapid property/toxicity screening (Chapters 5, 16), de novo protein and antibody design (Chapters 10, 17) — is real, and every one of them is real precisely *because* it generalizes: a model trained to design potent, drug-like EGFR inhibitors has, by the same real mechanism, no inherent barrier to being redirected toward designing potent, bioavailable *toxic* molecules instead, since nothing in the underlying method distinguishes “beneficial” from “harmful” except the training objective and reward function a user chooses to specify. This is not a hypothetical concern raised for the first time in this closing chapter: Urbina, Lentzos, Invernizzi, & Ekins (2022) demonstrated it directly and concretely, real evidence rather than speculation — taking a real generative model originally built for MegaSyn, their own toxicity-avoiding drug-design tool, and inverting its real reward function from *minimizing* predicted toxicity to *maximizing* it — originally posed as a thought experiment at a real international security conference before the authors turned it into an actual computational demonstration — the real, inverted model generated real, novel molecular structures whose own real predicted lethality the paper’s own published analysis places in the same real range as VX and other known chemical warfare agents (illustrated directly in the paper’s own real published figure comparing thousands of the model’s generated structures against VX), not by discovering anything about those specific real agents, but by demonstrating that the same real, general property-optimization capability this book has built for beneficial drug discovery throughout Chapters 5-17 carries directly over to harmful design with a one-line change to the reward function. The paper’s own real, considered conclusion is not that such tools should not exist — the beneficial real capability is exactly what most of this book demonstrates — but that the drug-discovery AI community bears a real, direct responsibility to treat this dual-use property as a known, disclosed design constraint rather than an afterthought: real, deliberate choices about what capabilities to publish in immediately-usable form versus describe at a higher level, real support for existing dual-use research of concern (DURC) oversight frameworks govern-

ment funders and institutions already apply to biological research, and real engagement with the nucleic-acid and peptide synthesis industry's own existing sequence-screening practices as a genuine, if imperfect, technical control point between a generated design and a physical, synthesizable hazard.

This book's own real, disclosed position. Every generative or predictive method in this book was built and demonstrated against real, legitimate, well-precedented therapeutic targets — EGFR, KRAS, hERG liability, viral surface antigens — using real, published literature and real, public databases throughout, and this book's own version-floored requirements.txt files, real bundled data, and real, disclosed methodology exist specifically so a reader can verify and reproduce each real result, not merely take a stronger claim on faith. That same real transparency is precisely what Urbina et al.'s own finding shows is a real, double-edged consideration for a capability this general: making methodology genuinely reproducible is what open, verifiable science requires, and is simultaneously what makes a harmful redirection of the same real methodology easier for anyone who chooses to attempt it. This book does not resolve that real tension — no single technical choice does — but closes by naming it directly rather than leaving it unstated, consistent with the same standard of honest disclosure this book has applied to every real scientific limitation in the seventeen chapters before this one.

Closing

This book opened, in Chapter 1, with Eroom's Law — the real, long-documented observation that the cost of bringing a new drug to market has risen roughly exponentially for decades even as the underlying science has advanced. Eighteen chapters later, nothing in this book claims to have repealed that trend; what it has built, chapter by chapter, is a real, working, reproducible demonstration that machine learning now touches every real stage of that pipeline this book set out to cover — molecular and protein representation, property and structure prediction, generative design, physics-based verification, and now deployment, reproducibility, and the regulatory and ethical context every one of those real capabilities operates within. Every number in every chapter was computed by running real code against real data, not asserted; every real limitation was disclosed, not hidden; and every real bug found along the way — Chapter 13's replicate-deduplication bug, Chapter 16's reward-scaling bug, and others — was fixed openly, with a regression test, rather than quietly smoothed over. That discipline, more than any single result in any single chapter, is this book's own real contribution: not a claim that AI has solved drug discovery, but a real, complete, reproducible demonstration of how to use it honestly while it continues to try.

References

- Wilkinson, M. D., Dumontier, M., Aalbersberg, I. J., Appleton, G., Axton, M., Baak, A., et al. (2016). The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*, 3, 160018. <https://doi.org/10.1038/sdata.2016.18>
- U.S. Food and Drug Administration. (2025). *Considerations for the Use of Artificial Intelligence To Support Regulatory Decision-Making for Drug and Biological Products* (Draft Guidance). Docket No. FDA-2024-D-4689.
- European Medicines Agency, Committee for Medicinal Products for Human Use (CHMP), & Committee for Medicinal Products for Veterinary Use (CVMP). (2024). *Reflection Paper on the Use of Artificial Intelligence (AI) in the Medicinal Product Lifecycle*. EMA/CHMP/CVMP/83833/2023.
- Urbina, F., Lentzos, F., Invernizzi, C., & Ekins, S. (2022). Dual use of artificial-intelligence-powered drug discovery. *Nature Machine Intelligence*, 4, 189-191. <https://doi.org/10.1038/s42256-022-00465-9>

See Chapter 1's references for Mendez et al. (2019, ChEMBL); Chapter 5's for the hERG QSAR methodology (Sanguinetti & Tristani-Firouzi, 2006) and Bemis & Murcko (1996, scaffold definition), both reused unchanged in Section 18.1; and Chapter 13's for Veber et al. (2002), Baell & Holloway (2010, PAINS), and Bickerton et al. (2012, QED), reused unchanged for this chapter's own ADMET endpoint — all reused here rather than re-listed.

All dataset sizes, held-out evaluation metrics, and API responses cited in Section 18.1 were computed directly by running `train_model.py` and querying the real, running `service.py` on 2026-08-22, not taken from a secondary source — see `results/training_results.json` and `tests/test_deployment.py` to reproduce.